WEB PROGRAMMING

From the Platform to Production

FOUNDATIONS EDITION

Dr. Mahyar Teymournezhad

Computer Engineering Program

Technical review: 11 November 2026

About this edition

This foundations edition contains the first three completed sessions of a comprehensive university course book: how the Web works, semantic HTML and accessibility, and professional responsive CSS. It uses the final chapter format, laboratory model, worked-answer policy, and visual system that subsequent parts follow.

The student edition includes full answers to formative exercises and quizzes. Secure summative variants remain in a separate access-controlled instructor supplement.

How to use the book

Predict before running examples, complete each laboratory checkpoint, commit your work, and attempt every exercise before reading its solution. Treat browser tools, tests, and version history as engineering evidence.

Publication information

Revised foundations edition, 28 August 2026. Author: Dr. Mahyar Teymournezhad. Copyright (c) 2026 Dr. Mahyar Teymournezhad. Distributed under the Apache License, Version 2.0; see the repository LICENSE file. The current technical baseline targets Node.js 24 LTS.

Preface

The Web is often introduced as a collection of commands: write this tag, copy that style rule, install a framework, and run a server. Those commands can produce a page, but they do not yet give a student a dependable mental model. When something fails, copied steps offer little help. This book therefore begins with smaller, more human questions. What is the browser trying to do? Which program is speaking? What information crosses the network? Which part can we observe? Only then do we name the protocol, language feature, or tool that solves the problem.

No previous knowledge of web development is assumed. A technical term is introduced in plain language, connected to a familiar situation, and then refined until it is precise enough for engineering work. Abbreviations are expanded when they first become important. Diagrams show relationships; examples make those relationships concrete; laboratories turn explanation into evidence. Advanced material is not separated from fundamentals by a sudden jump. It grows from the same ideas at a larger scale.

The continuing Campus Hub project gives the course a single thread. Students first publish a small document, then make it semantic, responsive, interactive, data-driven, secure, tested, and deployable. Each addition has a reason. The aim is not merely to finish an application, but to understand what the application asks the browser, network, server, and database to do.

For students

Predict before running an example. Keep a record of what you observed and what you inferred. Attempt the exercises before reading their answers, and compare reasoning rather than memorizing a final line of code. If a paragraph feels dense, return to its concrete example and explain it aloud in your own words.

For instructors

Each session is organized as a teachable sequence: motivating problem, concept model, prediction, demonstration, guided laboratory, independent transfer, debugging clinic, retrieval quiz, and reflection. The explanations are detailed enough for preparation and self-study, while checkpoints provide natural places for questions, live demonstrations, and formative assessment.

About the author

Dr. Mahyar Teymournezhad

Dr. Mahyar Teymournezhad is the author and curriculum designer of *Web Programming: From the Platform to Production*. He developed the book as the central reference for a university computer engineering course, with one consistent path from a student's first encounter with the Web to production-oriented full-stack engineering.

The authoring approach places durable concepts before fashionable tools. It treats accessibility, security, testing, version control, and professional explanation as everyday engineering responsibilities rather than optional final topics. Every laboratory is designed to leave observable evidence, and every worked answer explains why a solution is appropriate, what assumptions it makes, and where a different choice may be justified.

Contents

Orientation - Prepare Your Web-Programming Workstation 17

Orientation map . . . 17
Learning outcomes . . . 17
1. Understand the workstation before installing tools . . . 18
1.1 Required equipment and access . . . 18
1.2 Record the machine you are preparing . . . 19
2. Prepare the operating system and file view . . . 19
3. Install a current web browser . . . 20
4. Install and orient Visual Studio Code . . . 20
4.1 Windows . . . 20
4.2 macOS . . . 20
4.3 Linux . . . 21
4.4 Verify the editor . . . 21
5. Learn the minimum terminal model . . . 21
5.1 The project root is a place, not an abstract idea . . . 22
6. Install and configure Git . . . 23
6.1 Install Git . . . 23
6.2 Verify and identify the author . . . 23
7. Install Node.js 24 LTS and npm . . . 24
7.1 Windows and macOS installer path . . . 24
7.2 Linux path . . . 24
7.3 Optional version-manager path . . . 24
7.4 Verify Node.js and npm . . . 24
7.5 JavaScript is the course's primary programming language . . . 25
7.6 Verify JavaScript in the browser . . . 25
7.7 Verify JavaScript in Node.js . . . 26
7.8 How JavaScript develops across the course . . . 26
8. Create or obtain your remote repository before developing . . . 26
8.1 Preferred course method: create from the instructor's GitHub template . . . 27
8.2 If the instructor supplies a clone URL instead of a template . . . 30
8.3 If you already have the repository on this computer . . . 30
8.4 If the instructor provides a ZIP archive . . . 30
8.5 Open the whole repository in VS Code . . . 31
8.6 Use the same save-check-commit-push cycle for developed work . . . 31
9. Run the reproducible readiness test . . . 31
10. Start and inspect the local server . . . 32
10.1 What the book means by two terminals . . . 32
10.2 Start the server and read its output . . . 33
10.3 Open the address correctly . . . 33
10.4 Find Network in developer tools . . . 34
10.5 Stop the server deliberately . . . 35

10.6 Optional troubleshooting: npm cannot find package.json . . . 35
11. Prepare accessibility inspection tools . . . 36
12. Create and push a personal proof-of-work commit . . . 36
13. Troubleshoot from evidence . . . 37
14. Readiness record . . . 37

Orientation exercises **38**

Exercise 1 - Name the responsible tool . . . 38
Exercise 2 - Diagnose the directory . . . 38
Exercise 3 - Explain the evidence . . . 38
Exercise 4 - Protect secrets . . . 38
Exercise 5 - Assign the terminal roles . . . 39
Exercise 6 - Find an empty Network panel . . . 39
Exercise 7 - Separate connection and path failures . . . 39
Exercise 8 - One language, two environments . . . 39

Orientation worked answers **39**

Answer 1 . . . 39
Answer 2 . . . 39
Answer 3 . . . 40
Answer 4 . . . 40
Answer 5 . . . 40
Answer 6 . . . 40
Answer 7 . . . 40
Answer 8 . . . 40

Orientation retrieval quiz **41**

Quiz answers . . . 41
Orientation summary . . . 42
Orientation references . . . 42
What changes often? . . . 43

Session 1 - How the Web Works **44**

Session map . . . 44
Learning outcomes . . . 44
1. Begin with evidence . . . 44
2. Internet is not the Web . . . 45
2.1 What is a network? . . . 46
2.2 Clients and servers are roles . . . 47
2.3 HTML, CSS, and JavaScript have different jobs . . . 48
2.4 Website, web application, and API . . . 50
Checkpoint 1 - classify the roles . . . 51
3. Read a URL as structured data . . . 51

A note about the colon character . . . 52
3.1 A path is a name, not necessarily a disk location . . . 52
3.2 Query data and fragments . . . 52
3.3 Origin and same-origin reasoning . . . 53
3.4 Encoding and normalization . . . 55
Predict before running . . . 55
4. From navigation to pixels . . . 55
4.0 The protocol stack: cooperation without confusion . . . 57
4.1 Name resolution . . . 59
4.2 Connection and TLS . . . 65
4.3 HTTP exchange . . . 65
4.4 Browser processing . . . 67
5. Local file, local server, public server . . . 68
6. Developer tools are measuring instruments . . . 70
A disciplined debugging loop . . . 70
7. The smallest useful HTML document . . . 71
7.1 Elements, tags, content, and attributes . . . 71
7.2 HTML is semantic . . . 72
7.3 Document skeleton . . . 72
7.4 Parsing is error-tolerant, not error-free . . . 73
7.5 HTML, DOM, and View Source are related but distinct . . . 73
8. Git records decisions . . . 74
Repository, commit, branch, and remote . . . 74
Commit quality . . . 74

Guided laboratory - Observe and publish locally 75

Goal and acceptance criteria . . . 75
Safety and privacy . . . 75
Step 1 - Establish the baseline . . . 75
Step 2 - Add navigable content . . . 76
Step 3 - Serve through HTTP . . . 76
Step 3A - Observe name resolution . . . 76
Step 4 - Cause and diagnose a failure . . . 76
Step 5 - Verify . . . 77
Observation record . . . 77
Cleanup . . . 77

Exercises 78

Foundation . . . 78
Exercise 1 - URL anatomy . . . 78
Exercise 2 - Order the events . . . 78
Exercise 3 - Read the response . . . 78
Application . . . 78
Exercise 4 - Diagnose three observations . . . 78

Exercise 5 - Draw the boundary . . . 78
Challenge . . . 78
Exercise 6 - One page, multiple requests . . . 78
Exercise 7 - Git evidence . . . 79
Exercise 8 - Follow a DNS answer . . . 79
Exercise 9 - Source HTML and DOM . . . 79
Exercise 10 - Method properties . . . 79
Exercise 11 - Compare origins . . . 79
Exercise 12 - Diagnose by layer . . . 79
Exercise 13 - Expand, compress, and interpret IPv6 . . . 79

Worked answers **80**
Answer 1 . . . 80
Answer 2 . . . 80
Answer 3 . . . 80
Answer 4 . . . 81
Answer 5 . . . 81
Answer 6 . . . 81
Answer 7 . . . 81
Answer 8 . . . 81
Answer 9 . . . 82
Answer 10 . . . 82
Answer 11 . . . 82
Answer 12 . . . 82
Answer 13 . . . 83

Debugging clinic **83**
Diagnosis . . . 83

Retrieval quiz **84**
Quiz answers and misconception notes . . . 84
Self-check scale . . . 85

Session summary **85**
Glossary . . . 86
Primary references and further reading . . . 88
What changes often? . . . 88
Session reflection . . . 88
Learning journal notes . . . 89

Session 2 - Semantic HTML and Accessible Forms **90**
Session map . . . 90
Learning outcomes . . . 90
Before the concepts: find, edit, and view Campus Hub . . . 91

Find the correct Campus Hub file . . . 91
Use a plain-text code editor . . . 91
Make one visible change and save it . . . 92
Start the local server and open the browser output . . . 92
Read the evidence when the result differs . . . 93
1. Begin with the user task . . . 94
2. HTML is a language of meaning . . . 94
2.1 Read the syntax before choosing the element . . . 95
2.2 Document-wide meaning begins in html and head . . . 96
2.3 Structure follows the document, not its font size . . . 97
2.4 Landmarks divide the page into navigable regions . . . 97
3. Text content, links, and media . . . 98
3.1 Text elements communicate rhetorical structure . . . 98
3.2 Lists express grouping . . . 99
3.3 Link text should make sense in context . . . 99
3.4 Images require a purpose decision . . . 99
3.5 Dates, abbreviations, and data can have machine-readable values . . . 100
3.6 Audio and video require equivalent access . . . 100
3.7 Tables are for relationships in data . . . 100
4. Forms are conversations with constraints . . . 100
4.1 Choose the submission method from the operation . . . 102
4.2 Labels and descriptions remain available . . . 102
4.3 Select the narrowest appropriate native control . . . 102
4.4 Predict which controls will be submitted . . . 103
4.5 Choose the encoding, especially for files . . . 104
4.6 Autocomplete, input purpose, and data minimization . . . 104
4.7 Constraint validation is useful but incomplete . . . 105
4.8 Design error recovery before adding JavaScript . . . 105
Predict before running . . . 105
5. Three related representations . . . 106
5.1 Accessible name, description, role, and state . . . 106
5.2 Keyboard operation is an engineering test . . . 107
6. Accessibility is broader than conformance . . . 107

Guided laboratory - Build an accessible event form 109

Goal and acceptance criteria . . . 109
Step 1 - Read before editing . . . 109
Step 2 - Establish landmarks and navigation . . . 109
Step 3 - Build the form from native controls . . . 110
Step 4 - Define the submission contract . . . 110
Step 5 - Express constraints . . . 110
Step 6 - Test without a mouse . . . 110
Step 7 - Validate the source and inspect the tree . . . 110
Step 8 - Verify, explain, and commit . . . 110

Laboratory observation record . . . 110

Exercises 111

Foundation . . . 111

Exercise 1 - Element choice . . . 111

Exercise 2 - Label and name . . . 111

Exercise 3 - Alternative text . . . 111

Application . . . 111

Exercise 4 - Repair the outline . . . 111

Exercise 5 - Diagnose the inaccessible control . . . 111

Exercise 6 - Table relationships . . . 112

Challenge . . . 112

Exercise 7 - Validation boundary . . . 112

Exercise 8 - Accessibility-tree reasoning . . . 112

Exercise 9 - Predict the submitted data . . . 112

Exercise 10 - Choose a testing strategy . . . 112

Exercise 11 - Repair the syntax and semantics . . . 112

Exercise 12 - Choose controls and encoding . . . 112

Exercise 13 - Design error recovery . . . 112

Exercise 14 - Model a complex table . . . 113

Worked answers 113

Answer 1 . . . 113

Answer 2 . . . 113

Answer 3 . . . 113

Answer 4 . . . 113

Answer 5 . . . 113

Answer 6 . . . 114

Answer 7 . . . 114

Answer 8 . . . 114

Answer 9 . . . 114

Answer 10 . . . 114

Answer 11 . . . 115

Answer 12 . . . 115

Answer 13 . . . 115

Answer 14 . . . 116

Debugging clinic 116

Diagnosis . . . 116

Retrieval quiz 116

Quiz answers and misconception notes . . . 117

Self-check scale . . . 118

Session summary 118

Glossary . . . 119
Primary references and further reading . . . 120
What changes often? . . . 121
Session reflection . . . 121

Session 3 - Cascading Style Sheets, Layout, and Responsive Design 122

Session map . . . 122
Learning outcomes . . . 122
1. CSS is a language for presentation and constraints . . . 123
1.1 Read a CSS rule as a complete sentence . . . 123
1.2 Rules, at-rules, and shorthands are different structures . . . 124
1.3 Connect CSS to HTML deliberately . . . 124
1.4 The browser already has a stylesheet . . . 125
2. Selectors match; the cascade decides . . . 125
2.1 Build selector literacy before composing long selectors . . . 125
2.2 The cascade resolves competing declarations . . . 126
2.3 Specificity is a comparison, not a score to maximize . . . 128
2.4 Inheritance is not selector matching . . . 128
2.5 Custom properties carry decisions through the cascade . . . 128
3. Boxes turn declared values into geometry . . . 129
3.1 display establishes outer and inner behavior . . . 129
3.2 Content, padding, border, and margin form the box model . . . 129
3.3 Normal flow is a resilient default . . . 130
3.4 Sizing begins with intrinsic content . . . 130
3.5 Logical properties follow the writing system . . . 131
3.6 Margin collapse and overflow provide evidence . . . 131
3.7 Positioned layout changes reference and flow . . . 132
4. Values express type, scale, color, and visual hierarchy . . . 132
4.1 Units communicate relationships . . . 132
4.2 Color notation is separate from color usability . . . 136
4.3 Typography controls reading, not merely decoration . . . 136
4.4 Backgrounds, borders, and shadows must support content . . . 137
4.5 Form controls retain platform behavior . . . 137
5. Choose a layout model from the relationship . . . 138
5.1 Flexbox distributes items along a main axis . . . 138
5.2 Grid coordinates tracks in two dimensions . . . 139
5.3 Alignment is not ordering . . . 139
6. Responsive design treats context as an input . . . 139
6.1 Viewport behavior begins in HTML . . . 140
6.2 Media queries respond to the environment . . . 140
6.3 Container queries make components context-aware . . . 142
6.4 Images and embedded media . . . 143
6.5 User preferences are responsive inputs . . . 143

7. Accessible presentation survives real user conditions . . . 144
7.1 Interaction states must remain perceivable . . . 144
7.2 Target size is measured after layout . . . 144
7.3 Zoom, text spacing, and reflow are layout tests . . . 144
7.4 Hiding and visual order can change access . . . 145
7.5 Motion, contrast, and forced colors require combined evidence . . . 145
Predict before running . . . 145
8. CSS architecture should make change inexpensive . . . 145
9. Debug CSS by tracing evidence, not by guessing values . . . 146
9.1 Open the correct browser panels . . . 146
9.2 Trace horizontal overflow systematically . . . 147

Guided laboratory - Build the responsive Campus Hub shell 147

Before you begin . . . 147
Goal and acceptance criteria . . . 148
Step 1 - Inspect the unstyled document . . . 148
Step 2 - Establish tokens and global constraints . . . 149
Step 3 - Build component layouts . . . 149
Step 4 - Add a content-driven enhancement . . . 149
Step 5 - Conduct an overflow audit . . . 149
Step 6 - Verify preferences and interaction . . . 149

Exercises 150

Foundation . . . 150
Exercise 1 - Cascade decision . . . 150
Exercise 2 - Box calculation . . . 150
Exercise 3 - Calculate and choose font-relative units . . . 151
Application . . . 151
Exercise 4 - Repair overflow . . . 151
Exercise 5 - Flexbox or Grid . . . 151
Exercise 6 - Focus regression . . . 151
Challenge . . . 151
Exercise 7 - Source order . . . 151
Exercise 8 - Responsive reasoning . . . 151
Exercise 9 - Diagnose an unstyled page . . . 151
Exercise 10 - Match selectors deliberately . . . 151
Exercise 11 - Inheritance and CSS-wide keywords . . . 151
Exercise 12 - Positioning and stacking . . . 152
Exercise 13 - Media or container query . . . 152
Exercise 14 - Accessible control styling . . . 152
Exercise 15 - Developer-tools evidence plan . . . 152
Exercise 16 - Architecture review . . . 152

Worked answers 152

Answer 1 . . . 152
Answer 2 . . . 152
Answer 3 . . . 153
Answer 4 . . . 153
Answer 5 . . . 153
Answer 6 . . . 153
Answer 7 . . . 153
Answer 8 . . . 153
Answer 9 . . . 154
Answer 10 . . . 154
Answer 11 . . . 154
Answer 12 . . . 154
Answer 13 . . . 155
Answer 14 . . . 155
Answer 15 . . . 155
Answer 16 . . . 155

Debugging clinic **156**

Diagnosis . . . 156

Retrieval quiz **156**

Quiz answers and misconception notes . . . 157
Self-check scale . . . 158

Session summary **158**

Glossary . . . 159
Primary references and further reading . . . 160
What changes often? . . . 161
Session reflection . . . 161
Instructor teaching guide . . . 161
Suggested five-hour sequence . . . 161
Demonstrations that expose misconceptions . . . 162
Common misconceptions to listen for . . . 162
Accommodations and alternatives . . . 163
Formative rubric . . . 163

Session 4 - JavaScript from First Principles **164**

Session map . . . 164
Prerequisites . . . 164
Learning outcomes . . . 164

0. Prepare the computer and run your first JavaScript **165**

0.1 Understand the five tools before installing them . . . 165

0.2 Install Node.js on Windows, step by step . . . 166
0.3 Install or verify Visual Studio Code . . . 167
0.4 Open the correct terminal and prove its current directory . . . 167
0.5 Create and run the first Node.js program . . . 168
0.6 Run JavaScript in a browser . . . 169
Why are these two files separate? . . . 169
0.7 Read the first failures instead of starting over . . . 170

1. A program is an ordered explanation for a machine 171
1.1 ECMAScript and JavaScript . . . 172
1.2 Your first file, from an exact directory . . . 172
1.3 Execution order, comments, and case sensitivity . . . 172

2. Values have types; bindings have names 173
2.1 Expressions produce values . . . 174
2.2 const and let . . . 174
2.3 Strings and template literals . . . 175

3. Operators create relationships 175
3.1 Conversion must be visible . . . 175
3.2 Truthiness and short-circuiting . . . 176

4. Control flow chooses and repeats work 176
4.1 Loops repeat with a changing state . . . 176

5. Functions give a calculation a name and boundary 177
5.1 Declarations, expressions, and arrow functions . . . 178
5.2 Scope answers where a name is available . . . 178

6. Arrays model ordered collections 179
6.1 Transformations express intent . . . 179

7. Objects model named facts 179
7.1 Primitive copying and object identity . . . 180

8. Errors are evidence, not verdicts 180

9. Small tests make behavior repeatable 181

Guided laboratory - Model Campus Hub event availability 182
Goal and evidence . . . 182
Step 1 - Establish the baseline . . . 182
Step 2 - Read the event shape . . . 182
Step 3 - Write one function at a time . . . 182

Step 4 - Transform the collection . . . 182
Step 5 - Validate the boundary . . . 182
Step 6 - Compare output with code . . . 182
Step 7 - Commit evidence . . . 183

Exercises **184**

Foundation . . . 184
Exercise 1 - Values and types . . . 184
Exercise 2 - Binding or object change . . . 184
Exercise 3 - Strict comparison . . . 184
Exercise 4 - Trace a branch . . . 184
Application . . . 184
Exercise 5 - Write a total function . . . 184
Exercise 6 - Array pipeline . . . 184
Exercise 7 - Scope diagnosis . . . 184
Exercise 8 - Identity diagnosis . . . 184
Challenge . . . 184
Exercise 9 - Boundary table . . . 184
Exercise 10 - Debug a failing calculation . . . 184
Exercise 11 - Choose loop or transformation . . . 185
Exercise 12 - Test quality . . . 185
Exercise 13 - Prove the environment before debugging code . . . 185
Exercise 14 - Separate source, command, and output . . . 185
Exercise 15 - Trace browser script loading . . . 185

Worked answers **185**

Answer 1 . . . 185
Answer 2 . . . 185
Answer 3 . . . 185
Answer 4 . . . 185
Answer 5 . . . 186
Answer 6 . . . 186
Answer 7 . . . 186
Answer 8 . . . 186
Answer 9 . . . 186
Answer 10 . . . 186
Answer 11 . . . 187
Answer 12 . . . 187
Answer 13 . . . 187
Answer 14 . . . 187
Answer 15 . . . 187

Debugging clinic **187**

Symptom 1 - ReferenceError: remaining is not defined . . . 187

Symptom 2 - TypeError mentions reading a property of undefined . . . 188
Symptom 3 - The calculation concatenates digits . . . 188
Symptom 4 - A test changed another test's data . . . 188

Retrieval quiz **188**

Quiz answers and misconception notes . . . 188

Session summary **189**

Glossary . . . 190
Primary references and further reading . . . 190
What changes often? . . . 191
Instructor teaching guide . . . 191
Suggested eight-hour sequence . . . 191
Demonstrations that expose misconceptions . . . 191
Assessment and inclusion . . . 191

Appendix A - Tested laboratory reference solutions **192**

Week 01 - index.html . . . 192
Week 02 - index.html . . . 193
Week 03 - index.html . . . 195
Week 03 - styles.css . . . 196
Week 04 - events.js . . . 198
Week 04 - events.test.js . . . 199
Week 04 - hello-browser.html . . . 199
Week 04 - hello-browser.js . . . 199
Week 04 - hello.js . . . 200
Week 04 - index.html . . . 200
Week 04 - styles.css . . . 200

Appendix B - Quick commands **201**

Orientation - Prepare Your Web-Programming Workstation

Essential question: How can a student prove that a computer is ready to edit, run, inspect, test, and preserve web programs before the first programming lab?

This orientation assumes no previous experience with developer tools. Follow it from beginning to end on the computer you will bring to laboratories. Do not stop when an installer says *Finished*. An installation is complete only when you can run the tool, observe its version, serve the course page, inspect the request, run the tests, and record a Git commit.

Orientation map

Table 0.1 defines the preparation task and the evidence that demonstrates readiness.

Table 0.1 - Workstation-preparation scope and evidence

Item	Plan
Audience	Students with no assumed web-development-tool experience
Time	Approximately 90-150 minutes, depending on download speed and operating system
Required result	One working local course repository opened as a folder in the editor
Primary programming language	Modern JavaScript, first in the browser and later in Node.js on the server
Runtime	Node.js 24 Long-Term Support (LTS), including npm
Core evidence	Tool versions, passing course tests, a local HTTP response, a Network-panel observation, and a Git commit
Help threshold	Ask for laboratory support after recording the exact command, complete error text, operating system, and attempted remedy

Learning outcomes

After completing this orientation, you should be able to:

1. distinguish the editor, terminal, runtime, package manager, server, browser, developer tools, test runner, and version-control system;
2. install and verify a supported browser, Visual Studio Code, Git, Node.js 24 LTS, and npm on Windows, macOS, or Linux;
3. identify the current directory, move safely through folders, and run a command without typing the displayed prompt symbol;
4. obtain and open the course repository without losing its folder structure;
5. run the course verification suite and local development server;
6. inspect a localhost request in browser developer tools;
7. configure Git identity, create a small meaningful commit, and push it to the approved remote repository; and
8. distinguish the JavaScript language from the browser and Node.js environments that execute it; and
9. collect useful diagnostic evidence without exposing passwords, access tokens, or private student information.

1. Understand the workstation before installing tools

A web-programming workstation is a cooperating set of programs, not one large application. The **code editor** changes text files. The **terminal** provides a text interface for running commands. **Node.js** executes JavaScript outside the browser. **npm**, installed with Node.js, runs the scripts defined by a project. A local **web server** listens for HTTP requests. The **browser** requests and renders resources, while its **developer tools** expose evidence. Automated tests check stated properties. **Git** records selected versions of the project.

Figure 0.2 shows the working loop used throughout this book. Read it clockwise. Editing changes project files; a terminal command runs a Node.js-backed local server; the browser requests `localhost`; developer tools support inspection; tests check the project; and a deliberate Git commit preserves a meaningful checkpoint. The arrows describe a working process, not a claim that the tools are one program.

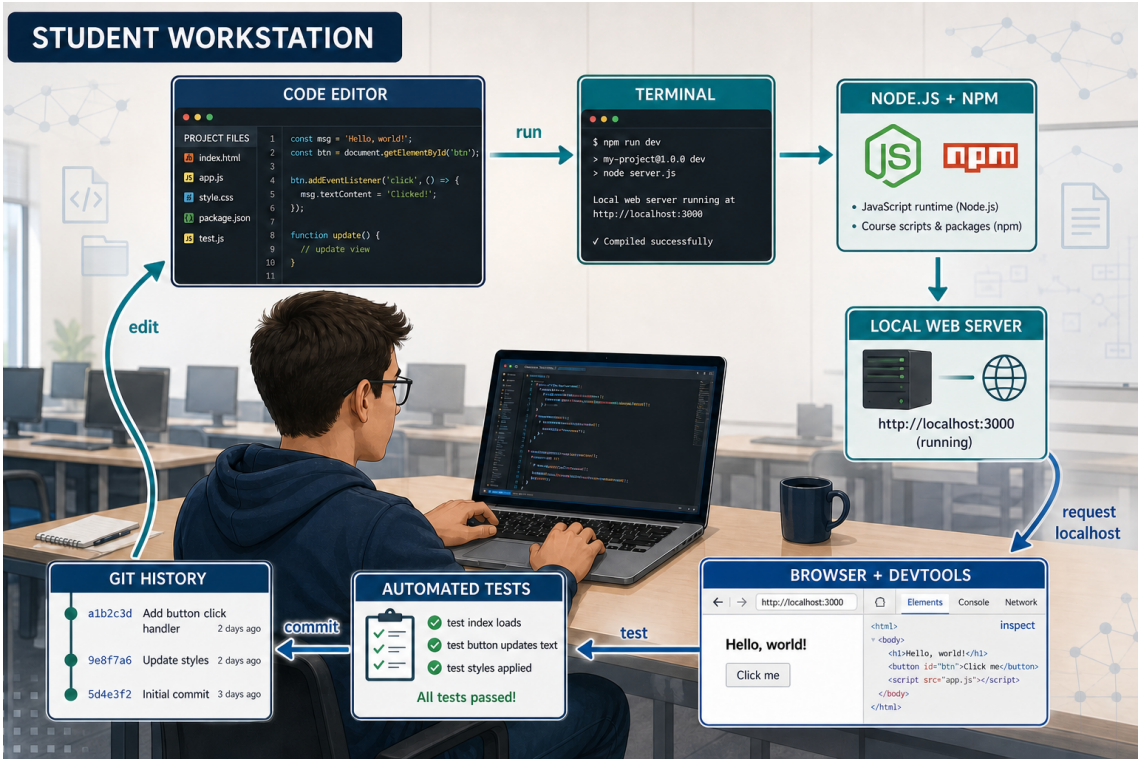


Figure 0.2. The course workstation keeps editing, command execution, the Node.js runtime, local serving, browser inspection, automated testing, and Git history as distinct but cooperating responsibilities.

1.1 Required equipment and access

Table 0.2 separates requirements from useful additions. The course does not require an expensive gaming computer, a paid editor, a public server, or a second monitor.

Table 0.2 - Equipment and access for the course

Item	Required baseline	Recommended when available
Computer	A supported 64-bit Windows, macOS, or Linux laptop or desktop with a physical keyboard	A laptop that can be brought to class and connected to an external display
Memory	8 GB RAM for the foundation laboratories	16 GB or more for later databases, browser tooling, and concurrent services
Storage	At least 10 GB free after operating-system updates	20 GB free so repositories, package caches, database files, and screenshots do not compete for space
Processor	A currently supported x64 or ARM64 processor	Four or more logical cores for smoother concurrent tools
Display	A readable display capable of browser zoom	A second display is useful but never required
Network	Reliable Internet access for installation, documentation, and repository synchronization	A wired connection for large downloads when wireless service is unstable
Input and audio	Physical keyboard; built-in pointer is sufficient	Headphones for media and screen-reader exercises; an external mouse is optional
Permissions	Permission to install approved software, or access to an administrator who can assist	A standard daily account plus separate elevation when installation requires it
Accounts	University account and access to the course repository or learning system	A Git-hosting account protected with multi-factor authentication when remote pushes are required

A tablet or phone is valuable for responsive testing, but it is not a replacement for the primary workstation. The course requires a file system, terminal, local server, developer tools, and Git. Students using a managed laboratory computer should confirm that work persists after sign-out and should push or copy approved work to durable storage before leaving.

1.2 Record the machine you are preparing

Before downloading software, record:

- operating-system name and version;
- processor architecture, normally x64 or ARM64;
- available memory and storage;
- whether the account can install applications; and
- the folder in which course work will be stored.

Use a short local path that you can find again. Examples are `C:\Users\Ada\Documents\web-course` on Windows and `/Users/ada/Documents/web-course` on macOS. Avoid placing the active repository inside a folder whose synchronization software repeatedly locks generated files. Never work directly inside an email attachment, a compressed ZIP archive, or a browser's temporary download preview.

2. Prepare the operating system and file view

1. Install pending security updates and restart the computer.
2. Confirm that the date, time, and time zone are correct. Incorrect clocks can break certificate checks, Git timestamps, and authentication.
3. Create the course-work folder in your own user account.

4. Configure the file manager to show complete filenames and extensions. A file named `index.html.txt` is a text file, not an HTML document.
5. Ensure that UTF-8 text files can be created. The repository's `.editorconfig` later asks a compatible editor to use UTF-8, Line Feed (LF) endings, two-space indentation, a final newline, and no trailing whitespace.
6. Decide where installation evidence will be stored. A plain text file named `setup-evidence.txt` inside your personal course notes is sufficient. Do not record passwords, recovery codes, or access tokens.

On Windows 11, File Explorer provides **View > Show > File name extensions**. On macOS, Finder provides **Settings > Advanced > Show all filename extensions**. Linux file managers vary; the important evidence is that you can distinguish `index.html` from `index.html.txt`.

3. Install a current web browser

Install the current stable release of Chrome, Edge, or Firefox from the browser's official site. The first browser is the course's primary inspection environment. A second browser engine is useful later for compatibility checks, but it is not required for the first laboratory.

1. Download the installer from the official publisher.
2. Check that the site uses HTTPS and that the publisher name is correct.
3. Run the installer and allow the browser to update automatically.
4. Open the browser and visit `https://example.com/`.
5. Open developer tools. On Windows and Linux, use `F12` or `Ctrl+Shift+I`. On macOS, use `Command+Option+I`.
6. Select the **Network** panel, reload the page, and confirm that a document request appears.

The exact panel layout changes between releases. Learn the evidence, not the color or screen position: request URL, method, status, response headers, timing, and initiator. Do not install a large collection of browser extensions. An extension can change requests, inject content, consume memory, or confuse a laboratory observation. Use a clean browser profile when results are surprising.

4. Install and orient Visual Studio Code

This book uses **Visual Studio Code (VS Code)** because it is available on the three supported desktop platforms. It is a text editor, not the same product as Visual Studio. Students may use another capable editor, but instructors can only guarantee that the published steps match VS Code.

4.1 Windows

1. Open the official VS Code download page.
2. Download the **User Installer** for the correct x64 or ARM64 architecture.
3. Run the installer. Keep the options that add VS Code to the `PATH` and allow a folder to be opened with Code when they are offered.
4. Close and reopen PowerShell after installation so it reads the updated `PATH`.

4.2 macOS

1. Download the macOS build from the official VS Code site. The Universal build supports both Intel and Apple silicon.
2. Open the downloaded disk image and drag `Visual Studio Code.app` into **Applications**.

3 Start VS Code from Applications.

4 Open the Command Palette with `Command+Shift+P`, run **Shell Command: Install 'code' command in PATH**, and restart Terminal.

4.3 Linux

1 Use the official `.deb` package on Debian or Ubuntu, the official `.rpm` package on Fedora or Red Hat compatible systems, or the documented package for your distribution.

2 Install through the distribution's package tool so updates can be delivered normally.

3 Start a new terminal after installation.

4.4 Verify the editor

In a new terminal, run:

```
code --version
```

The first line should be a version number. If the graphical editor opens but the command is unknown, VS Code is installed but its command-line launcher is not on `PATH`. Repeat the relevant platform step. You can continue temporarily by using **File > Open Folder**, but repair the command before later automation exercises.

Open a **folder**, not only an individual HTML file. The Explorer area should show the project tree. VS Code includes basic HTML, CSS, JavaScript, JSON, Markdown, terminal, and Git support; no extension is required for the first three sessions. Extensions are executable software. Install only an instructor-approved extension, review its publisher and permissions, and remove it if it changes course results. AI completion is optional and must follow the university's assessment and privacy policy; it is never a prerequisite for this book.

5. Learn the minimum terminal model

A terminal window displays output and accepts commands through a **shell**. Windows students use PowerShell in this book. macOS commonly uses Z shell (`zsh`), and Linux commonly uses Bash. Shell syntax differs, so use the code block labelled for your platform.

The prompt may display a path followed by `>` or `$`. Do not type the prompt symbol shown in a tutorial. Type only the command. Press Enter to run it. Commands operate relative to a **current working directory**, so confirm the directory before deleting, moving, installing, or starting a server.

Table 0.3 - Essential navigation commands

Purpose	Windows Command Prompt	Windows PowerShell	macOS or Linux shell
Show current directory	`cd`	`Get-Location`	`pwd`
List items	`dir`	`Get-ChildItem`	`ls`
Enter a folder	`cd /d "full-path"`	`Set-Location "full-path"`	`cd "full-path"`
Move to parent	`cd ..`	`Set-Location ..`	`cd ..`
Create a folder	`mkdir web-course`	`New-Item -ItemType Directory web-course`	`mkdir web-course`
Clear the display	`cls`	`Clear-Host`	`clear`
Stop a foreground server	`Ctrl+C`	`Ctrl+C`	`Ctrl+C`

Paths containing spaces must be quoted, for example `Set-Location "C:\Users\Ada\Course Work"` or `cd "/Users/ada/Course Work"`. Tab completion reduces spelling errors: type the first few characters of a filename and press Tab. The Up Arrow recalls a previous command. Before pressing Enter on an unfamiliar command, explain what it will read, change, create, or remove.

5.1 The project root is a place, not an abstract idea

Every terminal is currently *inside* one folder. A command such as `npm run ...` does not search the whole computer for the project you had in mind. npm begins from the terminal's current folder and expects the project's `package.json` there. For student work in this course, the **repository root** is the outer `web-programming-coursework-2026_Fall-STUDENT_NUMBER` folder containing `package.json`, `labs`, and `scripts`. The final part of the folder name contains the student's real university number, not the literal text `STUDENT_NUMBER`.

For a hypothetical example, suppose Command Prompt begins like this:

```
C:\Users\Student>
```

The path before `>` says that the current folder is `C:\Users\Student`. It is not the course repository. If npm were run there, it would search that directory for `package.json` and could report a path such as `C:\Users\Student\package.json`. This is a troubleshooting example, not output that should appear during the successful orientation procedure. Figure 0.3 contrasts the hypothetical wrong location with the correct repository root.

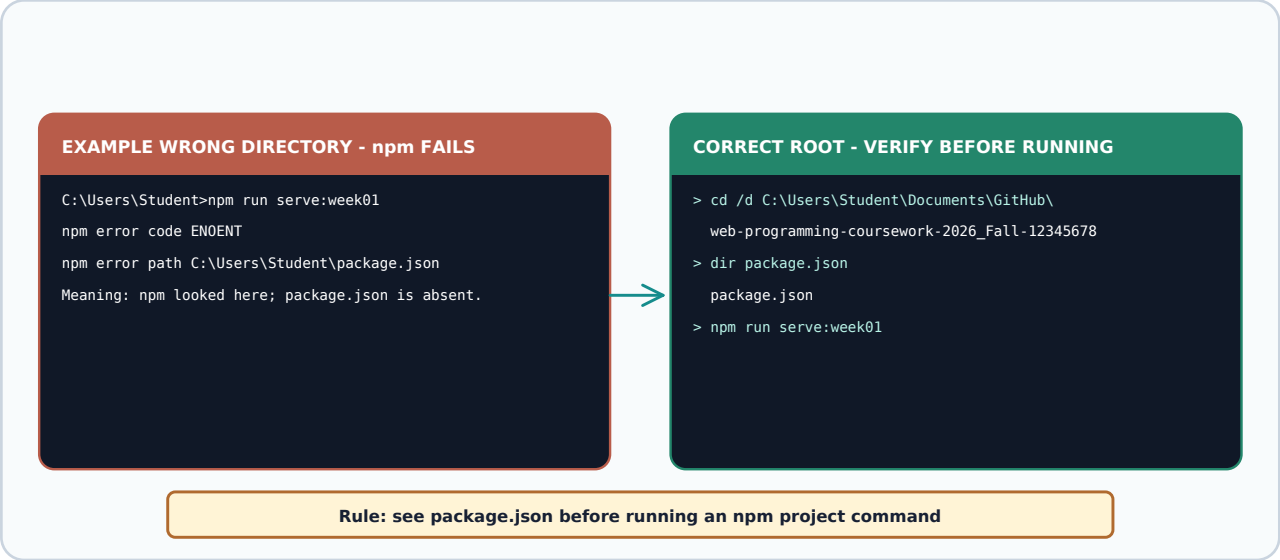


Figure 0.3. npm searches for package.json in the terminal's current project directory.

Use this four-part check before every project command:

Step 1 - Locate the project folder. In File Explorer, browse to the folder that visibly contains `package.json`. If a student named `Student` cloned the repository into the usual GitHub folder, an example location would be `C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall-12345678`. Here, `12345678` is a fictional student number. Your Windows account name, student number, and chosen clone location will be different.

Step 2 - Enter that folder. In Command Prompt, type the following command exactly. `/d` allows `cd` to change the drive as well as the folder:

```
cd /d "C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall-12345678"
```

Step 3 - Prove the location. Run `cd`, followed by `dir package.json`. The first command should print the repository path; the second should list `package.json`. If the second command says **File Not Found**, stop. Do not run npm yet.

Step 4 - Run the project command only after the proof succeeds. For example, run `npm test` or `npm run serve:week01`.

There is also a convenient visual route. Open the correct project folder in File Explorer, click its address bar, type `cmd`, and press Enter. Windows opens Command Prompt already positioned in that folder. Alternatively, right-click an empty area inside the folder and choose **Open in Terminal** when Windows offers that command. Still run `dir package.json`; visual navigation can also end in the wrong nested folder.

6. Install and configure Git

Git is the version-control system used for checkpoints, review, collaboration, and recovery. Git and a hosting service such as GitHub are related but distinct: Git works locally; a hosting service stores remote repositories and adds account, review, and collaboration features.

6.1 Install Git

- **Windows:** download Git for Windows from the official

Git installation page. The standard options are suitable for this course. PowerShell and VS Code should be allowed to find Git.

- **macOS:** run `git --version`. macOS may offer to install its Command Line

Tools. Alternatively, use the current official Git installer or an institution-approved package manager.

- **Debian or Ubuntu:** run `sudo apt update`, followed by

`sudo apt install git`.

- **Fedora:** run `sudo dnf install git`.

Use your distribution's corresponding supported command if it is not listed. Do not copy an installation command from an advertisement or unreviewed forum.

6.2 Verify and identify the author

Run:

```
git --version
```

Then configure the identity that should be stored in new commits:

```
git config --global user.name "Your Name"
git config --global user.email "your-address@example.edu"
git config --global init.defaultBranch main
```

Replace the examples with your own approved identity. If a public repository must not expose a personal email address, use the privacy address offered by the course's hosting service. Confirm the result:

```
git config --global --list
```

The repository contains `.editorconfig` rules that standardize text files. After obtaining the repository, run `git config core.autocrlf input` inside it. This is a repository-local setting that helps preserve LF line endings without changing unrelated repositories.

Git credentials are not the same as commit identity. Never place a password, personal access token, private key, or multi-factor recovery code in a source file. When remote access is introduced, use the hosting service's supported credential manager or Secure Shell (SSH) workflow and enable multi-factor authentication.

7. Install Node.js 24 LTS and npm

Node.js is a JavaScript runtime outside the browser. npm is the package manager and project-script runner installed with Node.js. The repository declares `>=24 <25` in `package.json`; its `.nvmrc` records the tested v24 release. Do not substitute an end-of-life Node.js version merely because it was already installed.

At this book's technical review date, Node.js 24 is an LTS release. Always confirm the maintained status on the official Node.js releases page before a new academic year.

7.1 Windows and macOS installer path

1. Open the official Node.js download page.
2. Select the latest available **24.x LTS** installer for the computer's x64 or ARM64 architecture.
3. Run the installer and keep npm and `PATH` integration enabled.
4. Close every old terminal window and open a new one.

7.2 Linux path

Use the installation method linked from the official Node.js download page for your distribution, or use an institution-approved Node version manager. Default distribution repositories can contain an older major version, so version verification is mandatory. Do not use an unexamined `curl ... | sudo bash` command copied from a forum.

7.3 Optional version-manager path

A version manager is useful when different projects require different Node.js majors. The manager and commands differ between Windows and POSIX systems, so it is optional rather than the common classroom baseline. If you use one, install the version recorded in `.nvmrc`, activate it in the current shell, and still run all verification commands below. A version manager does not replace Node.js; it selects which installed Node.js version the shell uses.

7.4 Verify Node.js and npm

Run:

```
node --version
npm --version
```

The Node.js output must begin with `v24.`. npm should print its own version. The two numbers are not expected to match because Node.js and npm are separate packages. If `node` works in one terminal but not another, compare the shells, restart VS Code, and inspect `PATH` before reinstalling software.

Do not use `sudo npm install -g ...` or an elevated PowerShell for ordinary course packages. This repository currently needs no third-party package installation for the foundation tests. Later chapters will use local project dependencies recorded in `package.json` and a lockfile.

7.5 JavaScript is the course's primary programming language

This course teaches web programming primarily with **JavaScript**. JavaScript is the programming language; a browser and Node.js are two different environments that can execute programs written in that language. The standardized definition of modern JavaScript is called **ECMAScript**. In normal conversation, developers say *JavaScript* when referring to programs that follow that standard together with the facilities supplied by their execution environment.

The distinction matters because the language and the environment contribute different capabilities:

- JavaScript itself supplies values, variables, operators, decisions, loops,

functions, arrays, objects, classes, promises, and modules.

- A browser adds browser-specific application programming interfaces (APIs),

including the Document Object Model (DOM), events, `fetch`, and web storage. Browser JavaScript can respond to a click and change a page.

- Node.js adds server and operating-system APIs, including files, processes,

streams, and HTTP servers. Node.js does not provide a browser page or DOM by default.

- npm is not another programming language. It installs project dependencies and

runs named scripts, while Node.js executes the JavaScript used by those tools and servers.

Figure 0.4 shows one language entering two runtimes. The shared middle is why students can transfer language knowledge from client-side code to server-side code. The different host APIs are why code cannot always be copied unchanged between the two environments.

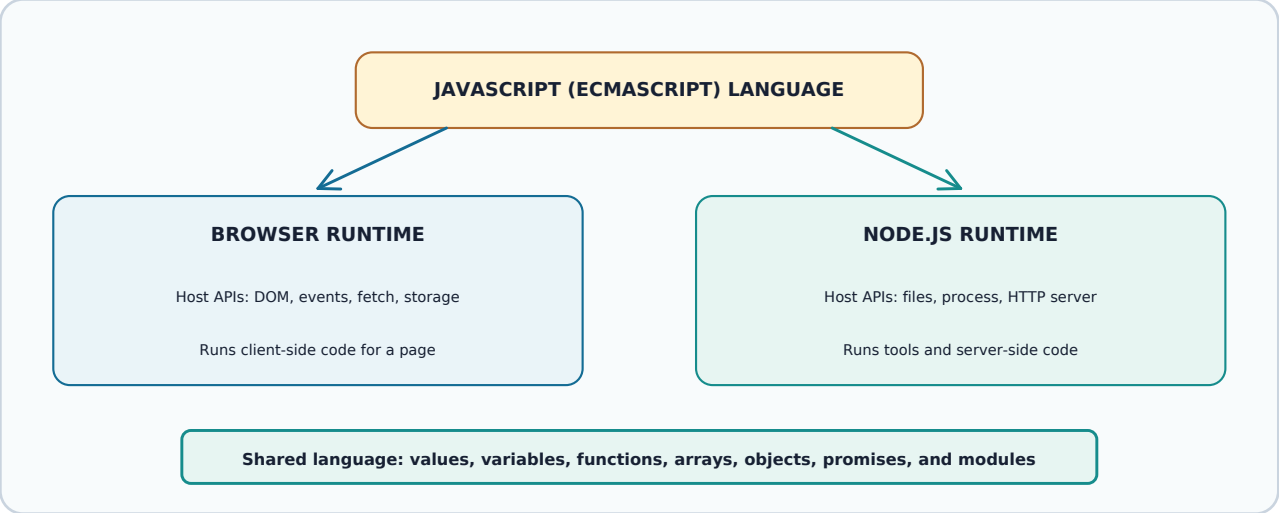


Figure 0.4. JavaScript runs in browsers and Node.js, while each environment supplies different host APIs.

HTML and Cascading Style Sheets (CSS) remain essential, but they have different jobs. HTML describes content and relationships. CSS describes presentation and layout. JavaScript expresses algorithms, behavior, state changes, and application logic. Calling all three “programming languages” hides distinctions that become important during debugging.

7.6 Verify JavaScript in the browser

Every supported browser already includes a JavaScript engine; there is no separate browser-JavaScript installer. To verify it without creating a file:

1. Open `https://example.com/` in the course browser.

2. Open developer tools and select **Console**. If Console is hidden, select the `>>` overflow menu and choose **Console**.
3. Click beside the `>` input prompt, type `1 + 2` yourself, and press Enter. The next line should be `3`.
4. Type `typeof document` and press Enter. The result should be `"object"` because the browser supplies a document for the current page.

The Console executes code immediately in the context of the open page. Never paste an unknown command into it. Some browsers display a self-XSS warning to protect users from being tricked into pasting malicious code. For this exercise, type the two short expressions manually and do not bypass the warning.

7.7 Verify JavaScript in Node.js

From any terminal in which `node --version` succeeds, run:

```
node -e "console.log(1 + 2)"
node -e "console.log(typeof document)"
```

The first command should print `3`. The second should print `undefined`. That second result is expected: Node.js executed JavaScript correctly, but it does not create a browser document. The `-e` option asks Node.js to evaluate the short program that follows it. Later chapters will place programs in `.js` and `.mjs` files, import modules, and test them rather than writing substantial programs on the command line.

7.8 How JavaScript develops across the course

The sequence deliberately separates concerns before combining them:

1. Chapter 1 explains where JavaScript fits in a web request and rendered page.
2. Chapters 2 and 3 establish semantic HTML and professional CSS so that scripts enhance a sound document rather than replacing one.
3. Chapter 4 teaches JavaScript from first principles: values, variables, types, control flow, functions, scope, arrays, objects, errors, debugging, and tests.
4. Chapter 5 applies the language to the DOM, events, and forms in the browser.
5. Chapter 6 explains asynchronous JavaScript, promises, `async`/`await`, `fetch`, JavaScript Object Notation (JSON), and Web APIs.
6. Chapter 7 adds ECMAScript modules, npm tooling, and then TypeScript. JavaScript comes first so that students do not learn a language, a type system, and the DOM simultaneously.
7. Chapter 8 uses Node.js to teach server-side JavaScript and HTTP services. Later projects connect browser code, server code, and persistent data while preserving their separate responsibilities.

This means client-side and server-side programming are taught separately first, then integrated in full-stack applications. The language is shared; the security boundaries, available APIs, lifetime, and responsibilities are not.

8. Create or obtain your remote repository before developing

The safest course workflow begins with a **private remote repository** assigned to the student. A repository is a project folder together with the history of changes recorded by Git. A **Git hosting service** is a website that stores Git repositories on an Internet-connected server and controls who may read or change them. GitHub, GitLab, and Bitbucket are examples. This course uses **GitHub**, at `https://github.com/`. Git is the version-control program on your computer; GitHub is the online service. They cooperate, but they are not the same product.

The word **local** means “on this computer.” The word **remote** means “stored at another location and reached through a network.” After cloning, the same project therefore exists in two places: a local project folder that you edit and a remote repository on GitHub that receives commits when you push. Saving a file does not update Git history, committing does not upload it, and pushing does not automatically include uncommitted changes.

Figure 0.5 shows the full relationship. The remote repository and local project folder are two related locations, not one magical shared folder.

Figure 0.5. A repository-first workflow begins with a private remote repository, creates one local project folder by cloning, and deliberately returns committed work with push.

8.1 Preferred course method: create from the instructor's GitHub template

This subsection describes one specific route: the instructor prepares a GitHub **template repository**, and each student creates a separate private repository from it. A template is a starting project whose folders and files can be copied through a button on GitHub. It is not a document template in Microsoft Word, and students do not edit the instructor's original repository.

Information that must come from the instructor

The Fall 2026 class uses the following private GitHub template repository. Open the Fall 2026 coursework template while signed in to the authorized GitHub account.

```
https://github.com/Mahyar2020/web-programming-coursework-2026_Fall
```

The instructor must also publish that exact clickable address in the course's **learning management system (LMS)**, such as Moodle, Canvas, or Blackboard. Students should use the LMS copy rather than an address received from an unverified account. In this address, `Mahyar2020` is the owner and `web-programming-coursework-2026_Fall` is the repository name.

Do not confuse it with the repository containing this book. The course-book source repository is used to produce the book and instructor resources; it is not the repository in which students complete laboratory work.

If the LMS contains no assignment link, or the link opens a `404` page, stop and ask the instructor for the semester's GitHub template or invitation. Do not guess the address, create an unrelated empty repository, use a repository found through a web search, or substitute **Fork** or **Download ZIP**

unless the instructor chooses one of those workflows. A private repository can also appear as `404` when the signed-in account has not been granted access.

Step 1 - Create or sign in to the correct GitHub account

Open `https://github.com/` in a browser. Choose **Sign up** if you do not have an account, or **Sign in** if you do. Use the account approved by the university. If the university created a managed account, follow its sign-in instructions rather than creating a second account. Confirm the signed-in username by selecting the profile picture in the upper-right corner; GitHub displays the username in that menu.

The browser is being used here to manage the online account. The terminal and the `git` command will be used later to create the local copy.

Step 2 - Protect the account with multi-factor authentication

Multi-factor authentication (MFA) requires more than one kind of evidence when signing in. GitHub calls its implementation **two-factor authentication (2FA)**. A password is one factor; a changing code from an authenticator application, a passkey, or a physical security key supplies another. This limits the damage if somebody learns the password.

GitHub currently recommends a time-based one-time password (TOTP) authenticator application rather than a text message. Prepare an authenticator application on a phone or computer, then follow these steps in GitHub:

1. Select the profile picture in the upper-right corner and choose **Settings**.
2. In the left sidebar under **Access**, choose **Password and authentication**.
3. Find **Two-factor authentication** and choose **Enable two-factor authentication**.
4. Use the authenticator application to scan the displayed Quick Response (QR) code. If the device cannot scan it, GitHub can display a setup key for manual entry.
5. Enter the six-digit code produced by the authenticator application. The code changes periodically, so use the current one.
6. Download the recovery codes when GitHub displays them. Store them in a secure password manager or another protected location separate from the laptop.
7. Confirm that the recovery codes have been saved. Add a passkey or security key as a backup method when one is available.

Recovery codes can restore access if the authenticator device is lost. Never put the QR image, setup key, password, authenticator code, or recovery codes in the course repository, a screenshot submitted for grading, email, or a shared chat. The detailed and update-sensitive procedure is maintained in GitHub's official two-factor authentication instructions.

Step 3 - Open and verify the instructor's exact link

First accept the private-repository invitation sent by GitHub. Open the notification while signed in to the GitHub account registered for this course, confirm that the invitation names `Mahyar2020` and `web-programming-coursework-2026_Fall`, and choose **Accept invitation**. If no invitation is visible, check the GitHub notification inbox and the email address registered with GitHub, then contact the instructor. Do not ask another student to forward an invitation because repository invitations are account-specific.

Return to the LMS and select the template link posted for this semester. Do not retype it when the clickable link is available. The browser address must be:

```
https://github.com/Mahyar2020/web-programming-coursework-2026_Fall
```

The page should display course files and a **Use this template** button above the file list. If the button is absent, one of three things is likely true: the link is not a template, the wrong account is signed in, or the account lacks access. Record the page address and ask the instructor which condition applies. Do not choose a different button merely to continue.

Step 4 - Create your own remote repository from the template

Select **Use this template**, then **Create a new repository**. On the creation page:

1. Set **Owner** to the personal account or university organization specified by the instructor.
2. Name the repository `web-programming-coursework-2026_Fall-STUDENT_NUMBER`, replacing `STUDENT_NUMBER` with your university student number. Do not include spaces.
3. Enter an optional description such as `Coursework for Web Programming`.
4. Choose **Private** unless the university explicitly requires another visibility. Private means only authorized accounts can read the repository; it does not mean security practices are unnecessary.
5. Leave the **Include all branches** checkbox **unchecked**. Unchecked means off: GitHub will copy only the default `main` branch. Select it only if the instructor explicitly announces that additional template branches are needed.
6. Do not add another README, `.gitignore`, or license. Those files already come from the template.
7. Choose **Create repository from template** and wait for the new page to load.

Confirm that the page now shows your chosen owner and repository name at the top. This new page is your remote repository. Creating it did not yet place a project folder on the computer.

Step 5 - Grant the instructor access to your private repository

The new repository belongs to the student, so access to the template does not automatically give the instructor access to the student's copy. On the new repository page, choose **Settings**, then **Collaborators** or **Collaborators and teams**. Choose **Add people**, enter `Mahyar2020`, verify the account name, and send the invitation. The instructor must accept it before private coursework can be reviewed.

Return to the repository's **Code** page after sending the invitation. Never add an unfamiliar account merely because its display name resembles the instructor's name.

Step 6 - Copy the HTTPS clone address

On your new repository page, choose the green **Code** button. Select **Local**, then **HTTPS**, and use the copy button beside the displayed address. It should resemble the following example, with your real owner and repository name:

```
https://github.com/YOUR_GITHUB_USERNAME/web-programming-coursework-2026_Fall-
YOUR_STUDENT_NUMBER.git
```

Secure Shell (SSH) is another valid connection method, but use it only after the course has taught SSH keys. Copying an address does not clone anything; it places text on the clipboard for the next command.

Step 7 - Choose the parent folder on the computer

In Windows File Explorer, create or open a parent folder such as `C:\Users\YourName\Documents\GitHub`. The parent folder will contain the project, but it is not the project itself. Click the File Explorer address bar, type `cmd`, and press Enter to open Command Prompt in that parent folder. The prompt should end with `Documents\GitHub>`.

On macOS or Linux, open a terminal and use `cd` to enter the chosen parent folder. Use `pwd` to print and verify its location.

Step 8 - Clone the remote repository to create the local project

At the terminal prompt, type `git clone`, add one space, paste the HTTPS address copied in Step 6, and press Enter. A representative command is:

```
git clone https://github.com/YOUR_GITHUB_USERNAME/web-programming-coursework-2026_Fall-
YOUR_STUDENT_NUMBER.git
```

Replace the entire example URL with the address copied from your repository. Do not type the placeholder words `YOUR_GITHUB_USERNAME` or `YOUR_STUDENT_NUMBER`. Git creates a new subfolder, downloads the project files and Git history, and normally names the remote connection `origin`. During the first private-repository operation, Git Credential Manager may open a browser and ask you to authorize Git. Complete that browser sign-in; do not put the GitHub password into the command itself.

Step 9 - Enter the project and prove that the two locations are connected

In Windows Command Prompt, use the actual folder name created by `git clone`:

```
cd /d "C:\Users\YourName\Documents\GitHub\web-programming-coursework-2026_Fall-
YOUR_STUDENT_NUMBER"
dir package.json
git status
git remote -v
```

On macOS or Linux, use `cd` with the corresponding path and replace `dir package.json` with `ls -l package.json`. Read the evidence rather than assuming success:

- `dir package.json` or `ls -l package.json` must display the project file;
- `git status` must name the current branch and report the working-tree state;
- `git remote -v` must show the HTTPS address of **your** remote repository next

to `origin` for fetch and push; and

- the owner and repository in that address must match the new GitHub page created

in Step 4, not merely the instructor's template.

If `git clone` reports `Repository not found`, first confirm the copied address and signed-in GitHub account. For a private repository, the message often means that authentication failed or that the account lacks permission; it does not necessarily mean that the repository has been deleted.

8.2 If the instructor supplies a clone URL instead of a template

Clone the supplied URL into the course-work parent folder, enter the new folder, and perform the same four verification commands. You can push only if the remote grants your account permission. Do not make experimental commits on a protected shared book branch. The instructor may instead create one remote repository per student or require a personal fork.

8.3 If you already have the repository on this computer

Do not clone a second copy merely because the terminal opened elsewhere. Find the existing outer project folder in File Explorer. For the hypothetical Windows location used in this orientation, the Command Prompt sequence is:

```
cd /d "C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall-12345678"
dir package.json
git status
```

If all three succeed, continue in that folder. Two clones with similar names make it easy to edit one copy and run another, so keep one clearly named working copy.

8.4 If the instructor provides a ZIP archive

1. Save the ZIP in the course-work parent folder and extract it completely.
2. Confirm that the extracted outer folder contains `package.json`; never work inside the compressed preview.
3. Run `git status` inside the extracted folder. If Git says it is not a repository, the ZIP contains files but no Git history.
4. Create the approved empty private remote repository in the browser. Do not add a README if the extracted project already has one.
5. In the extracted project root, follow the instructor's commands to initialize Git and connect that specific remote. Because a mistaken remote can publish work to the wrong place, the instructor should verify `git remote -v` before the first push.

8.5 Open the whole repository in VS Code

From the verified repository root, run:

```
code .
```

The dot means **this current folder**. In VS Code, the Explorer should show `labs`, `scripts`, `package.json`, and `.nvmrc`. The bottom terminal prompt should end with the project-folder name. If Explorer shows only one HTML file, use **File > Open Folder**, select the outer repository folder, and choose **Select Folder**. When VS Code asks whether you trust the authors, first verify that the repository came from the authorized course source.

8.6 Use the same save-check-commit-push cycle for developed work

At the end of each small, working change:

```
git status
git diff
git add path-to-the-file-you-changed
git diff --staged
git commit -m "Describe the completed change"
git push
```

Read `git status` and both diffs. Replace the example path and message; do not add every file blindly. The first push may ask the browser or a credential manager to authenticate. Use the hosting service's approved sign-in flow, not an account password embedded in a command. Refresh the remote repository page afterward and confirm that the latest commit is visible. That visible remote commit is the evidence that the push succeeded.

9. Run the reproducible readiness test

Do not begin with `npm test`. First prove where the terminal is. In Windows Command Prompt, a successful preflight looks like this:

```
cd
dir package.json
```

The first output must be the repository root. The second must display `package.json`. In PowerShell use `Get-Location` and `Get-Item package.json`; on macOS or Linux use `pwd` and `ls -l package.json`. Only then run these commands one at a time:

```
node --version
npm --version
git --version
npm test
```

`npm test` does not mean “test npm.” npm reads `package.json` and runs this project's named `test` script. In the student coursework repository, the script checks the required project files, the starter HTML documents, the Node.js major version, and an HTTP response from each Week 1-3 server. A

passing result demonstrates that the runtime can read the project and execute the supplied setup checks. It does not prove that every future program is correct.

Record the complete first failure, not only its final line. Common distinctions include:

- **command not found:** the tool is missing or not available through `PATH`;
- **wrong major version:** another installation or version manager is active;
- **file not found:** the terminal is probably in the wrong directory;
- **permission denied:** the current account cannot read or execute the target;
- **test assertion failed:** the tools ran, but project content does not meet a

checked requirement.

When the check finishes successfully, the last lines include messages such as `Local Markdown link verification passed`, `Week 1 verification passed`, and `Foundation verification passed`. Version numbers may differ in their minor and patch parts, but the Node.js line must begin with `v24.` for this edition.

10. Start and inspect the local server

10.1 What the book means by two terminals

There are not two different terminal products to install. A **server terminal** is simply an ordinary Command Prompt, PowerShell, macOS Terminal, or VS Code terminal tab in which a server is currently running. A **command terminal** is a second ordinary window or tab that remains free for `git status`, `npm test`, and navigation commands.

You need two because a foreground server keeps running and keeps control of the first terminal. When it is active, the normal prompt does not return. That is not a freeze. It is evidence that the process is waiting for browser requests. Figure 0.6 shows how to recognize the two roles.

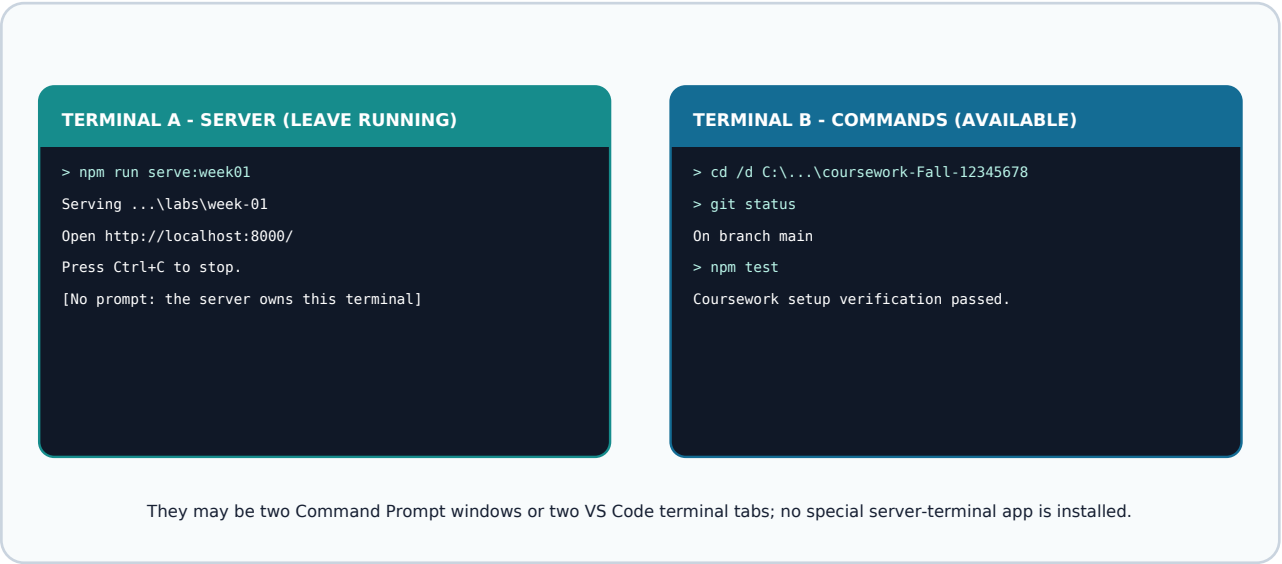


Figure 0.6. The server and command terminals are ordinary terminals with different current jobs.

To create two terminals in VS Code:

1. Open the repository with `code .`.
2. Choose **Terminal > New Terminal**. A panel opens at the bottom. Check that its prompt path ends with the repository folder name.

3 In the terminal panel, select the plus sign (**+**) to create a second tab. Rename the tabs **SERVER** and **COMMANDS** if your VS Code version offers **Rename** in the tab's context menu.

4 Select **SERVER** before running the server command. Leave **COMMANDS** unused until you need another command.

In Windows Command Prompt you can instead open two separate windows. Run the same `cd /d "..."` command in both windows because every terminal has its own current directory.

10.2 Start the server and read its output

In the terminal designated **SERVER**, prove that `package.json` is present and then start the server with the Week 1 page from the student coursework repository:

```
dir package.json
npm run serve:week01
```

At this point you are checking whether Node.js, the local server, and the browser cooperate; you are not yet expected to complete the Campus Hub document. The starter includes a visible setup message as well as comments marking the work for Session 1, so a successful server does not produce an unexplained white page.

On macOS or Linux, replace `dir package.json` with `ls -l package.json`. The expected final three server lines are:

```
Serving ...\labs\week-01
Open http://localhost:8000/
Press Ctrl+C to stop.
```

The beginning of the first path depends on your computer. The important ending is `labs\week-01` on Windows or `labs/week-01` on macOS and Linux. Leave this terminal running. Figure 0.7 pairs the expected terminal state with the expected browser result.

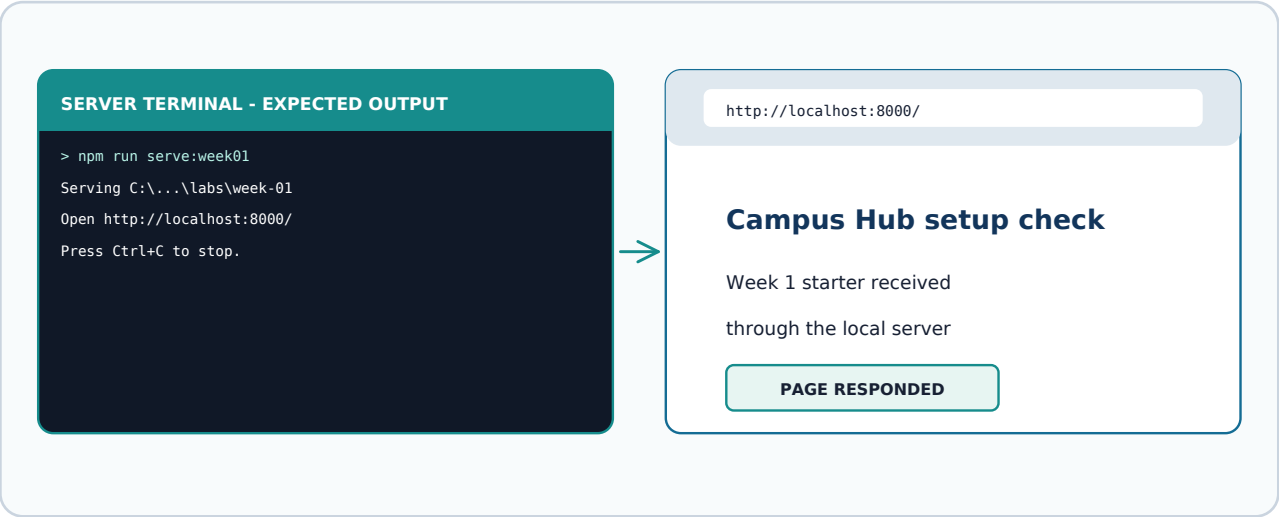


Figure 0.7. Terminal output and a browser page together prove that localhost is responding.

10.3 Open the address correctly

Click the browser's address bar, type exactly `http://localhost:8000/`, and press Enter. Do not include Markdown brackets, parentheses, quotation marks, or a final period. In particular, this is book-link notation, not something to type: `[http://localhost:8000/](http://localhost:8000/)`. The browser receives only the plain address.

`localhost` is a special host name for the current computer. `8000` is the port on which this course server is listening. This step does not publish anything to the Internet. The page should show the

heading **Campus Hub setup check** and a message stating that the browser received the Week 1 starter through the local server. In Session 1, you will replace that setup content with the complete Campus Hub structure.

If the address loads but the page is white, first confirm that the terminal's `Serving ...` line ends with `labs\week-01`, then open `labs/week-01/index.html` in VS Code and confirm that it contains the setup-check heading. Save the file and reload the browser. This is different from **This site can't be reached**, which means the browser could not connect to a listening server.

10.4 Find Network in developer tools

The **Network** panel belongs to the browser's developer tools, not to VS Code and not to Windows network settings.

1. Keep the Campus Hub tab selected.
2. Open developer tools with `F12` or `Ctrl+Shift+I` on Windows/Linux, or `Command+Option+I` on macOS.
3. A panel normally opens beside or below the page. Across its top are tabs such as **Elements**, **Console**, **Sources**, and **Network**. Select **Network**.
4. If **Network** is hidden because the panel is narrow, select the double-chevron `>>` overflow button and choose **Network**. You can also open the developer tools command menu and search for **Show Network**.
5. Network records activity from this point forward. If its table is empty, reload with `Ctrl+R`, `F5`, or the browser reload button while developer tools remains open.
6. Select the row named `localhost` or `/` whose **Type** is `document`. Do not select an image or icon row.
7. In the details pane, select **Headers**. Under **General**, find **Request URL**, **Request Method**, and **Status Code**. Under **Response Headers**, find **Content-Type**.

Figure 0.8 is a stable, representative screen map. Browser colors and exact tab positions may change, but the evidence names remain recognizable.

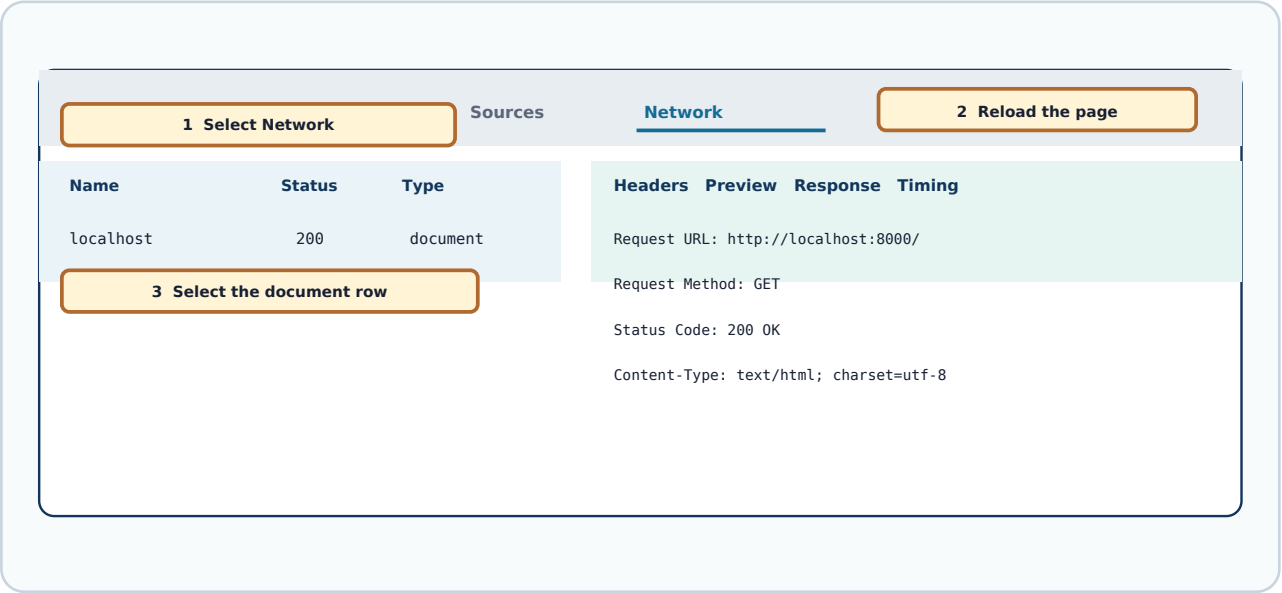


Figure 0.8. Network records requests; select the document row to inspect HTTP evidence.

Record these expected values:

Table 0.4 - Expected evidence for the Week 1 localhost document request

Evidence	Expected value
Request URL	`http://localhost:8000/`
Request method	`GET`
Status code	`200 OK`
Resource type	`document`
Response `Content-Type`	`text/html; charset=utf-8`

If the status is `404`, the server is reachable but the requested path does not map to a file. That is different from **This site can't be reached**, which means the browser could not establish the connection.

10.5 Stop the server deliberately

Return to the SERVER terminal, click inside it, and press `Ctrl+C` once. If Windows asks `Terminate batch job (Y/N)?`, type `Y` and press Enter. The ordinary prompt returns. Reload the browser: it should now report that the site cannot be reached. This controlled failure proves that the page depended on the server process rather than on a previously opened local file.

10.6 Optional troubleshooting: npm cannot find `package.json`

Skip this subsection when the server starts and the terminal prints the `Serving`, `Open`, and `Press Ctrl+C` lines. It describes a different failure that a learner may encounter later; it does not explain a white page.

`ENOENT` is an operating-system error name meaning that a required file or folder was not found. For example, if a learner runs npm from the Windows profile folder instead of the repository root, npm may print a line like this:

```
npm error path C:\Users\Student\package.json
```

This line is illustrative, not expected output from the orientation. If it does appear with your own account path, npm is functioning but the terminal is outside the repository. Reinstalling npm would not move the terminal. Figure 0.9 shows the diagnostic sequence for that hypothetical case.

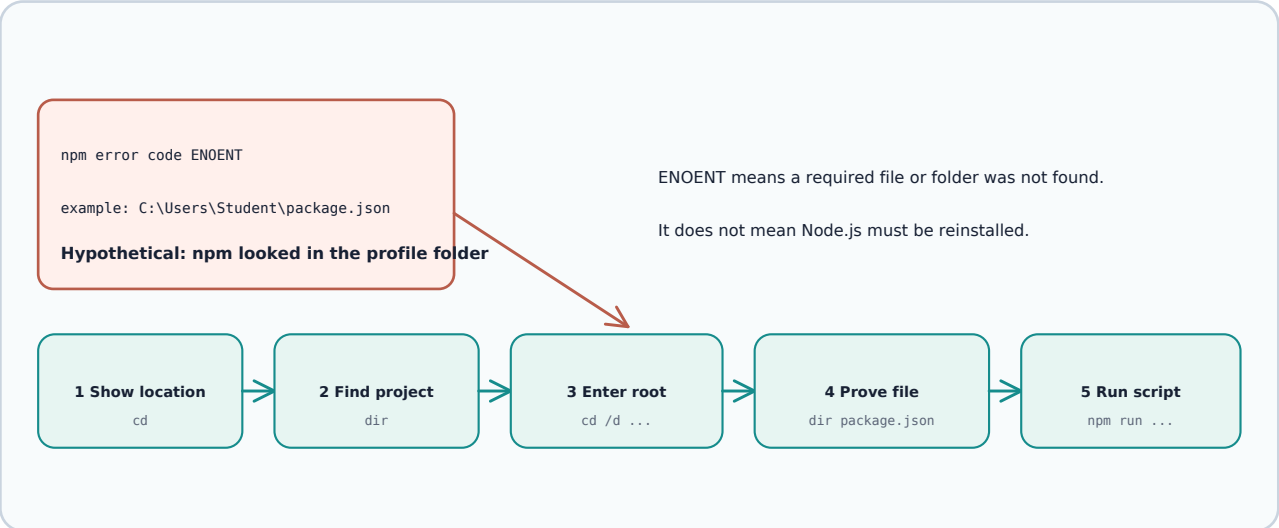


Figure 0.9. A hypothetical ENOENT path can reveal where npm looked for package.json.

For the example path used in this chapter, the repair would be:

```
cd /d "C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall-12345678"
```

```
dir package.json
npm run serve:week01
```

Do not open the browser until the terminal prints `Open http://localhost:8000/`. If `dir package.json` fails, confirm the folder in File Explorer instead of guessing another npm command.

If port 8000 is already in use, first check for a previous course server. On Windows PowerShell, use `Get-NetTCPConnection -LocalPort 8000`. On macOS, use `lsof -i :8000`. On Linux, use `ss -ltnp | grep 8000`. Do not terminate an unknown institutional process. Ask for help with the recorded process information.

11. Prepare accessibility inspection tools

No paid accessibility product is required. Prepare the following:

1. Learn the browser's Elements, Accessibility, and Network inspection areas.
2. Confirm that Tab and Shift+Tab move focus and that the operating system is not intercepting those shortcuts.
3. Practice browser zoom at 200 percent and restore it afterward.
4. Identify the built-in screen reader available on your platform: Narrator on Windows, VoiceOver on macOS, or an institution-supported Linux screen reader such as Orca. Screen-reader operation requires its own instruction; turning it on for one minute is not a complete accessibility test.
5. Bookmark the W3C Nu HTML Checker and the W3C accessibility evaluation overview.

Do not upload private coursework or personal data to an online checker unless the course and data policy permit it. Browser inspection, keyboard testing, validation, automated accessibility checks, assistive technology, and representative-user feedback provide different evidence. No single installation proves accessibility.

12. Create and push a personal proof-of-work commit

Use a student repository or branch designated by the instructor; do not commit setup notes directly to a protected shared book branch. Create a plain text file named `setup-evidence.txt` containing only non-secret evidence such as tool major versions, test result, operating system, and the date. Then run:

```
git status
git add setup-evidence.txt
git diff --staged
git commit -m "Record development environment verification"
git log -1 --oneline
git push
```

Read the staged diff before committing. `git add` selects content for the next commit; it does not upload a file. `git commit` records a local snapshot; it does not automatically push to a remote service. `git push` sends commits that are not yet on the configured remote branch. After pushing, refresh the repository page in the browser. Confirm that the newest commit message and time are visible. If Git reports that no upstream branch exists, do not guess a destination; verify `git remote -v` and use the exact first-push command supplied by the instructor.

Repeat this small cycle throughout the course: edit a coherent change, save it, run the relevant check, review `git diff`, stage only the intended files, commit with a meaningful message, and push. “Push everything” should mean “push every reviewed commit needed for the project,” not “upload caches, passwords, generated dependencies, or unrelated files.” `.gitignore` defines files that should remain outside version history.

13. Troubleshoot from evidence

Table 0.5 maps common symptoms to productive checks. Reinstalling every tool is rarely the best first response.

Table 0.5 - Setup symptoms and productive first checks

Symptom	First checks
`code` is not recognized	Restart the terminal; verify VS Code's command-line launcher is on `PATH`; use Open Folder temporarily
`node --version` is older than v24	Check which executable the shell finds; activate the intended version manager or reinstall the supported major
`npm` is blocked by PowerShell policy	Confirm the official Node installation; use the appropriate npm command shim or ask institutional support before changing execution policy
`npm test` cannot find `package.json`	Run the current-directory command and move to the repository root
Browser cannot reach localhost	Confirm the server terminal is still running, the URL and port match, and no firewall prompt was denied
Page does not change after saving	Save the file, verify the server root, inspect the requested URL, and reload with the Network panel open
Git refuses to commit	Read the message; configure author name/email, stage the intended file, and confirm the repository and branch
Git shows many unexpected line changes	Confirm `.editorconfig`, LF endings, and the repository-local `core.autocrlf` setting before committing
A managed computer rejects installation	Record the product, version, installer source, and error; use the university software center or request administrator support

When asking for help, provide the command as text, the complete error as text, the current directory, operating system, relevant version output, and what changed immediately before the failure. A screenshot can supplement that evidence but should not replace searchable error text. Remove usernames, private repository addresses, tokens, and personal information when they are not needed.

14. Readiness record

Complete Table 0.6. “Installed” is not adequate evidence.

Table 0.6 - Student workstation readiness record

Check	Evidence to record
Operating system and architecture	
Browser and successful DevTools opening	
VS Code version and repository folder opened	
Git version and configured author identity	
Node.js v24 output	
npm version output	
Browser Console: `1 + 2` and `typeof document` results	
Node.js: arithmetic and `typeof document` results	
`npm test` result	
Localhost URL, method, status, and `Content-Type`	
Server stopped with `Ctrl+C` and failure observed	
Latest personal Git commit	
Latest commit visible in the remote repository after push	
Accessibility tools identified	

The workstation is ready when every row has observed evidence or an approved accommodation. Keep this record for the first laboratory rather than repeating installation during teaching time.

Orientation exercises

Exercise 1 - Name the responsible tool

For each task, name the primary tool: changing `index.html`, running an npm script, interpreting HTML pixels, inspecting an HTTP status, preserving a local snapshot, and executing server-side JavaScript.

Exercise 2 - Diagnose the directory

`node --version` succeeds, but `npm test` reports that it cannot find `package.json`. State the most likely cause and give a non-destructive diagnostic sequence for Windows and for macOS/Linux.

Exercise 3 - Explain the evidence

A student opens `index.html` with a `file:` URL and says that the local web server works. Explain why that observation is insufficient and design the smallest test that demonstrates an HTTP exchange with the course server.

Exercise 4 - Protect secrets

A terminal error contains a remote URL with a personal access token. Explain how the student should obtain help without publishing the token and what immediate security action may be necessary.

Exercise 5 - Assign the terminal roles

A student starts the server and then tries to type `git status` in the same terminal, but no normal prompt is visible. Explain what has happened, how to keep the server running, and where to run `git status`.

Exercise 6 - Find an empty Network panel

Developer tools is open, but the Network table is empty. Give the exact sequence that should produce and inspect the document request. State the expected method, status, resource type, and `Content-Type`.

Exercise 7 - Separate connection and path failures

Compare these two browser results: **This site can't be reached** and an HTTP `404 Not Found` response. What does each observation prove, and which evidence would you inspect next?

Exercise 8 - One language, two environments

The expression `typeof document` returns `"object"` in the browser Console and `undefined` in Node.js. Explain why both results can be correct JavaScript. Then classify each facility as language-level, browser-provided, or Node.js-provided: an array, `document.querySelector`, `fetch`, `process.argv`, and a function.

Orientation worked answers

Answer 1

The code editor changes `index.html`. npm reads `package.json` and runs the named project script. The browser parses and renders HTML. Browser developer tools expose the HTTP status. Git records the local snapshot. Node.js executes JavaScript outside the browser. Several tools may participate in one task, but these are their primary responsibilities.

Answer 2

The shell is probably outside the repository root. First show the current directory with `Get-Location` on PowerShell or `pwd` on macOS/Linux. List items with `Get-ChildItem` or `ls` and look for `package.json`. Move with `Set-Location` or `cd` into the repository folder, list again, and rerun `npm test`. These checks do not change files. Reinstalling Node.js would not correct the current directory.

For the hypothetical Windows path used in the chapter, the complete non-destructive recovery begins with `cd`, then `dir`, then `cd /d "C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall-12345678"`, and finally `dir package.json`. A learner must substitute the actual repository path. Only after the last command displays the file should the student rerun npm. If an ENOENT message exists, its path is evidence of the folder npm searched; when no such message exists, this diagnosis does not apply.

Answer 3

A `file:` URL lets the browser read a local file without an HTTP server. Start `npm run serve:week01`, visit `http://localhost:8000/`, open the Network panel, reload, and select the document request. A recorded HTTP method, status, and response `Content-Type`, together with the running server process, demonstrate the exchange. Stop the server and observe that the URL no longer responds to confirm the process boundary.

Answer 4

The student should revoke or rotate the exposed token immediately through the hosting service, because redacting a later screenshot does not make an already disclosed credential safe. The help request should replace the secret with a marker such as `[REDACTED TOKEN]` while preserving the error type, command shape, host, and non-sensitive context. The token must not be committed, pasted into a chat, uploaded to an issue, or placed in setup evidence.

Answer 5

The server is a foreground process and currently owns that terminal. The absent prompt is expected while it listens for requests. Leave the first terminal open and create a second ordinary terminal window or VS Code terminal tab. Enter the repository root in the second terminal, prove that `package.json` is present, and run `git status` there. “Server terminal” and “command terminal” describe their current jobs; they are not different applications. Pressing `Ctrl+C` in the server terminal would stop the server, so that is appropriate only when the student intends to end the local session.

Answer 6

Keep developer tools open, select **Network** (using the >> overflow menu if necessary), and reload the page. Select the `localhost` or `/` row whose Type is `document`, open **Headers**, and read the General and Response Headers sections. For the Week 1 starter, the expected evidence is method `GET`, status `200 OK`, type `document`, and `Content-Type: text/html; charset=utf-8`. Opening Network after the page loaded may show an empty table because recording began too late; reloading creates a new observable request.

Answer 7

This site can't be reached means the browser could not complete a connection to a responding process at the given host and port. Check whether the SERVER terminal is still running, whether it printed the same port, and whether the URL was typed as a plain address. A `404 Not Found` response proves that an HTTP server did respond, but the requested path did not identify an available file. In that case inspect the Network request URL and the server's configured root and path. The distinction prevents a student from treating every browser failure as the same problem.

Answer 8

JavaScript defines the `typeof` operator, but the execution environment decides which global objects exist. A browser creates `document` to represent the loaded page, so `typeof document` is `"object"`. Node.js has no browser document by default, so reading its type safely produces `undefined`. Neither result implies that the language is different or broken.

Arrays and functions are language-level facilities. `document.querySelector` is provided by the browser's Document Object Model. `process.argv` is provided by Node.js. `fetch` began as a Web API and is now also supplied by current Node.js; its availability therefore belongs to the host

environment rather than to the core language. This last example shows why developers check both the language standard and the runtime documentation instead of assuming that every familiar name is built into JavaScript itself.

Orientation retrieval quiz

1. Why is opening an installer not proof that a tool is ready?
2. What is the difference between Node.js and npm?
3. What does `code .` mean?
4. Why does the current working directory matter?
5. What Node.js major version does this edition require?
6. Does `git commit` automatically upload work?
7. What evidence distinguishes a `file:` load from an HTTP load?
8. Why should browser extensions be kept minimal during laboratories?
9. What should never appear in `setup-evidence.txt`?
10. What is the productive first response to an unfamiliar error?
11. What makes an ordinary terminal a “server terminal”?
12. Why might Network be empty even when a page is visible?
13. What does the path in an npm ENOENT message tell you?
14. What is the difference between save, commit, and push?
15. What programming language is primary in this course?
16. Why can browser JavaScript use `document` while Node.js normally cannot?
17. When are client-side and server-side programming combined in the course?

Quiz answers

1. Readiness requires executable evidence: version output and successful use in the course workflow.
2. Node.js is the runtime that executes JavaScript outside the browser; npm is a package manager and project-script runner distributed with Node.js.
3. It asks VS Code to open the current directory as a project folder.
4. Relative paths and project discovery begin there; a correct command in the wrong directory can read or change the wrong target.
5. Version 24.x, as constrained by `package.json` and recorded in `.nvmrc`.
6. No. It creates a local snapshot. A separate push operation communicates with a remote repository.
7. An HTTP URL plus a Network-panel document exchange with a method, status, and HTTP response fields.
8. Extensions can inject content, change requests, add panels, or introduce variables that obscure the behavior being studied.
9. Passwords, access tokens, private keys, recovery codes, or unnecessary private student and repository information.

10 Preserve the exact error, command, current directory, versions, and recent change; identify which layer failed before changing one factor.

11 A foreground server process is currently running in it and has not returned the normal shell prompt.

12 Developer tools may have begun recording after the request completed; reload while Network is open.

13 It identifies the location in which npm tried to find the missing file, such as `package.json`.

14 Save writes the working file, commit records selected content in local Git history, and push transfers local commits to the configured remote.

15 Modern JavaScript, standardized as ECMAScript; TypeScript is introduced only after students understand JavaScript and modules.

16 `document` is supplied by a browser's DOM environment, not by the core JavaScript language. Node.js does not create a browser DOM by default.

17 They are taught separately first, so their APIs and responsibilities are clear, and are then integrated in later full-stack application projects.

Orientation summary

- A workstation is a system of cooperating tools with distinct responsibilities.
- Installation is complete only when version and workflow evidence both succeed.
- The course baseline is a current browser, VS Code, Git, Node.js 24 LTS, npm, browser developer tools, and the authorized repository.
- Modern JavaScript is the primary programming language; browser and Node.js execute the same core language with different host APIs.
- The repository root is the reference directory for course commands.
- `npm test`, a localhost request, Network evidence, a Git commit, and a verified remote push form the minimum reproducible readiness proof.
- Developer tools and automated checks provide evidence; they do not replace reasoning or user evaluation.
- Credentials and private data never belong in source, screenshots, logs shared publicly, or setup records.

Orientation references

1 Node.js, Node.js releases.

2 Node.js, Download Node.js.

3 Microsoft, Get started with Visual Studio Code.

4 Git, Install Git.

5 Chacon and Straub, Pro Git - First-Time Git Setup.

6 Chrome for Developers, Open Chrome DevTools.

7 Chrome for Developers, Inspect network activity.

8 GitHub Docs, Creating a repository from a template.

9 GitHub Docs, Cloning a repository.

10 GitHub Docs, Pushing commits to a remote repository.

11 GitHub Docs, About repositories.

12 GitHub Docs, Configuring two-factor authentication.

13 W3C, Nu HTML Checker.

14 W3C Web Accessibility Initiative, Evaluating Web Accessibility.

What changes often?

Installer screens, package versions, operating-system permission prompts, and browser panel layouts change frequently. The responsibility of each tool and the evidence-led workflow are more stable. Technical review date: **22 September 2026**.

Session 1 - How the Web Works

Essential question: *What actually happens between entering a Uniform Resource Locator (URL) and seeing a page, and how can a developer observe each step?*

Session map

Table 1.1 gives instructors and students a shared view of the session before the technical discussion begins.

Table 1.1 - Session scope, preparation, and evidence

Item	Details
Audience	Students with programming experience but no assumed web knowledge
Preparation	Complete the workstation orientation and bring its readiness record
Contact time	100-minute concept workshop plus a 110-minute laboratory
Independent work	Approximately 2-3 hours
Main artifact	A small Campus Hub page served locally and recorded in Git
Assessment	Network observation, repository history, exercises, and quiz

Learning outcomes

After this session, you should be able to:

1. Distinguish the Internet, the World Wide Web, a browser, a server, a website, a web page, and a resource.
2. Decompose a Hypertext Transfer Protocol Secure (HTTPS) URL into scheme, host, port, path, query, and fragment.
3. Read the structure and notation of Internet Protocol version 4 (IPv4) and Internet Protocol version 6 (IPv6) addresses, including prefix lengths.
4. Trace a simplified navigation through name resolution, connection setup, a Hypertext Transfer Protocol (HTTP) request, an HTTP response, and browser processing.
5. Read the method, target, status, and selected headers of an HTTP exchange.
6. Explain why `localhost`, a local file, and a public deployment are different.
7. Create a minimal valid HyperText Markup Language (HTML) document and serve it through HTTP.
8. Use browser developer tools to collect evidence instead of guessing.
9. Create a Git repository and make small, meaningful commits.

1. Begin with evidence

This session assumes that you have completed **Prepare Your Web-Programming Workstation**, the orientation immediately before this chapter. That orientation does more than list software: it explains what each tool is for, gives operating-system-specific installation steps, verifies the versions, starts a local server, opens browser developer tools, runs the automated checks, and records a first Git commit. If any of those steps is unfamiliar or fails, return to the orientation before continuing. The chapters deliberately reuse that same editor-terminal-browser-test-Git loop so that learning the subject is not confused with repeatedly changing tools.

Open a familiar website. Before reading further, write a short prediction:

- Is the page already stored on your computer?
- How does the browser find the correct machine?
- Does one page require one request?
- Which computer executes the HTML?
- Can the server see the `#fragment` portion of a URL?

Do not correct the prediction yet. At the end of the session, annotate it with what changed. Learning starts when a mental model becomes testable.

Keep two columns in your notes: **observation** and **interpretation**. “The Network panel shows status 404” is an observation. “The server has no matching route for this path” is an interpretation that the evidence may support. This habit prevents a plausible explanation from being mistaken for a measured fact.

2. Internet is not the Web

The **Internet** is a network of networks that uses a family of protocols to move data between machines. The **World Wide Web** is one application built on that network. Email, voice calls, online games, and the Web can all use the Internet, but they are not all the Web.

This distinction matters because a web developer works with several layers at once. A failure in the network layer is different from a failure in HTTP, and both are different from an error in an HTML document. When every failure is called “the website is down,” diagnosis becomes guesswork. Professional practice begins by naming the layer in which the evidence points. Figure 1.1 separates the Internet infrastructure from the Web application built on top of it.

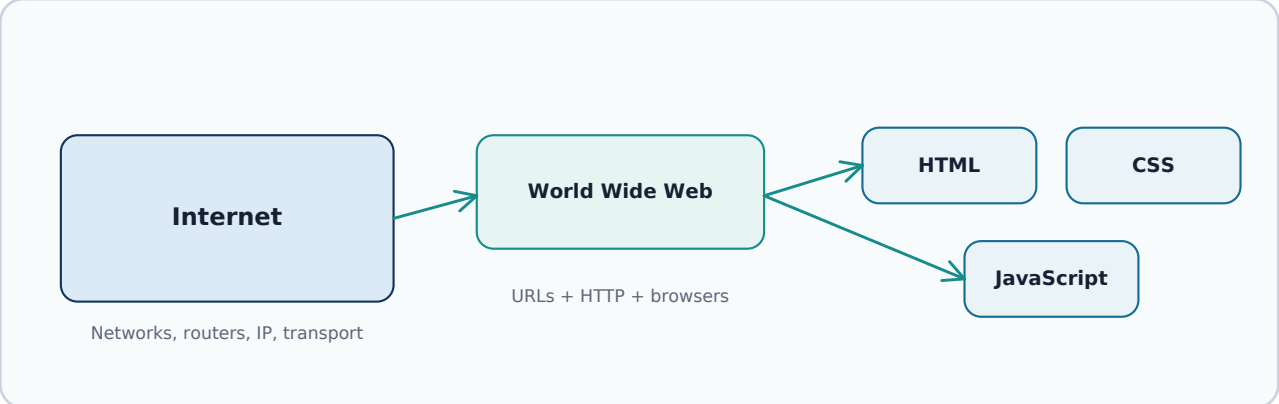


Figure 1.1. The Web is one application built on Internet infrastructure.

Figure 1.2 offers a more concrete view of the same idea. The laptop, Wi-Fi router, intermediate network devices, naming service, security step, and data-center server are not one giant machine. They are separate parts that cooperate. The dotted return path represents response data travelling back to the browser; the browser must still interpret that data before a page appears.

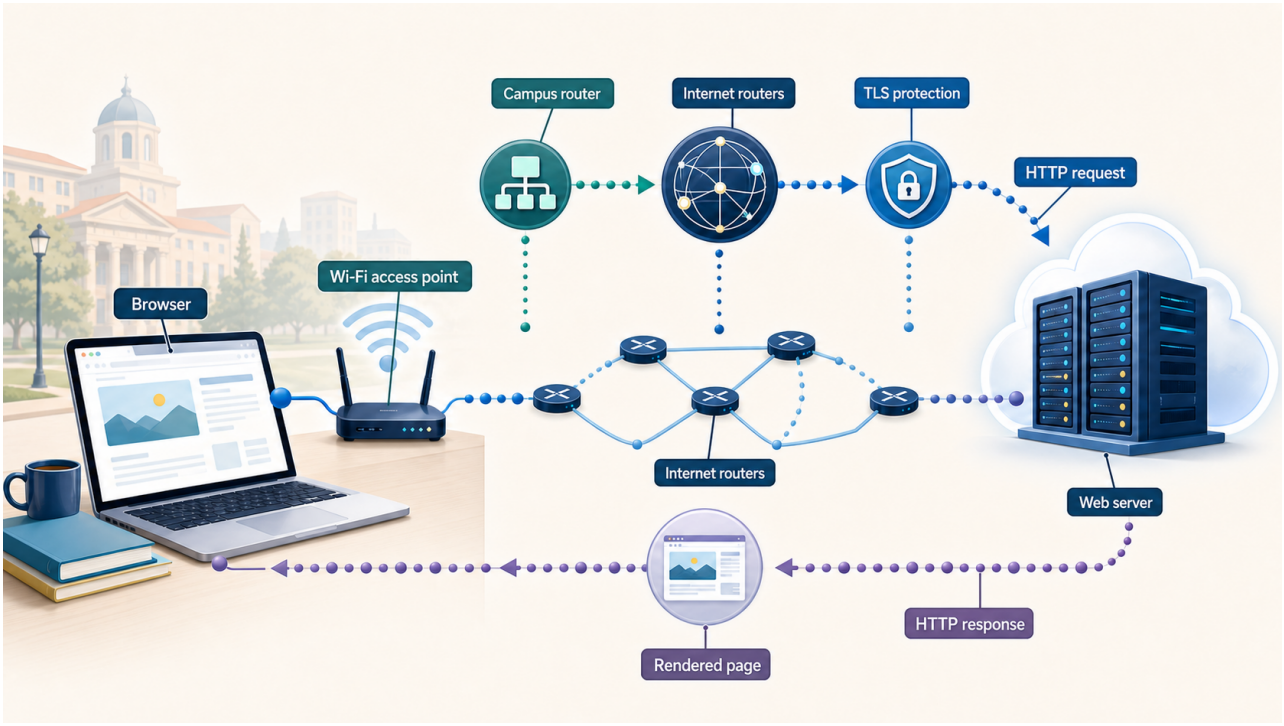


Figure 1.2. A browser request crosses local and interconnected networks to reach a web service; the response returns through the network and is rendered by the browser.

2.1 What is a network?

A **network** is a group of computing devices that can exchange data. Your home Wi-Fi network is one example. A university network is another. The Internet connects an enormous number of independently operated networks through common protocols. No single organization owns every cable, router, address, or service.

A device joins a network through a **network interface**, such as a Wi-Fi radio or an Ethernet adapter. A direct connection between interfaces is a **link**. Within a local network, a switch commonly forwards frames between wired devices, and a wireless access point connects radio devices to that local infrastructure. A **router** connects different networks and decides where an Internet Protocol packet should be forwarded next. These devices cooperate, but they do not all do the same job.

Data is normally divided into manageable units often called **packets**. Routers forward packets toward their destinations. Different packets belonging to the same exchange may encounter different delays, and networks can lose, duplicate, or reorder them. Higher layers compensate according to the service they promise.

Two everyday measurements are easy to confuse. **Bandwidth** describes how much data a path can carry per unit of time; **latency** describes how long data takes to travel and receive a response. A high-bandwidth connection can still feel slow when latency is high, and a low-bandwidth link can transfer a tiny message quickly while struggling with a large video.

An **Internet Protocol (IP) address** identifies a network interface for Internet Protocol delivery. Internet Protocol version 4 (IPv4) addresses are commonly written as four decimal numbers, such as `192.0.2.10`; Internet Protocol version 6 (IPv6) addresses use a larger hexadecimal notation, such as `2001:db8::10`. These examples come from ranges reserved for documentation. An IP address is not the same thing as a domain name, a URL, a person, or necessarily a single physical computer. Cloud services, proxies, load balancers, and shared hosting make one-to-one assumptions unreliable.

Many home and campus networks use private IPv4 addresses internally and share a public address through **Network Address Translation (NAT)** at a router. NAT is common, but it is not the definition of routing and it is not required for every network design. IPv6 provides a much larger address space; its deployment and privacy mechanisms are more nuanced than the claim that it "removes routers."

2.2 Clients and servers are roles

A **client** initiates a request for a service. A **server** listens for requests and produces responses. These are roles, not permanent categories of hardware. A laptop can run a browser as a client and a development server at the same time.

A **host** is a machine or virtual environment participating in a network; a **process** is a running program on that host. One host can run a browser process, a development-server process, a database process, and many unrelated programs at the same time. Ports help the operating system deliver arriving transport data to the intended process.

The word *server* is used in two related senses:

- a server **program**, such as a process listening for Hypertext Transfer

Protocol (HTTP) requests; and

- a server **machine**, meaning a computer that runs one or more server programs.

Keep the two meanings separate. "The server returned `404`" describes program behavior. "The server lost power" describes a machine or infrastructure failure.

A **browser** is a client application that understands web standards. A **web page** is normally a HyperText Markup Language (HTML) document. A **website** is a related collection of pages and other resources. A **resource** is anything identified on the Web: HTML, Cascading Style Sheets (CSS), an image, a font, JavaScript Object Notation (JSON) data, a generated report, or another result.

Client and server do not mean "weak computer" and "powerful computer," nor are they permanent identities. A server handling the browser's request may become a client when it requests data from another service. Likewise, the same laptop can run both sides during development. The role is defined by the exchange being described.

Figure 1.3 makes the change of role visible. The application server is a server in relation to the browser because it receives the browser's HTTP request and sends an HTTP response. When that same application process asks a database service for data, it initiates a new exchange and therefore acts as a client. The database server answers that second exchange. The labels describe each relationship; they do not assign one permanent identity to the machines.

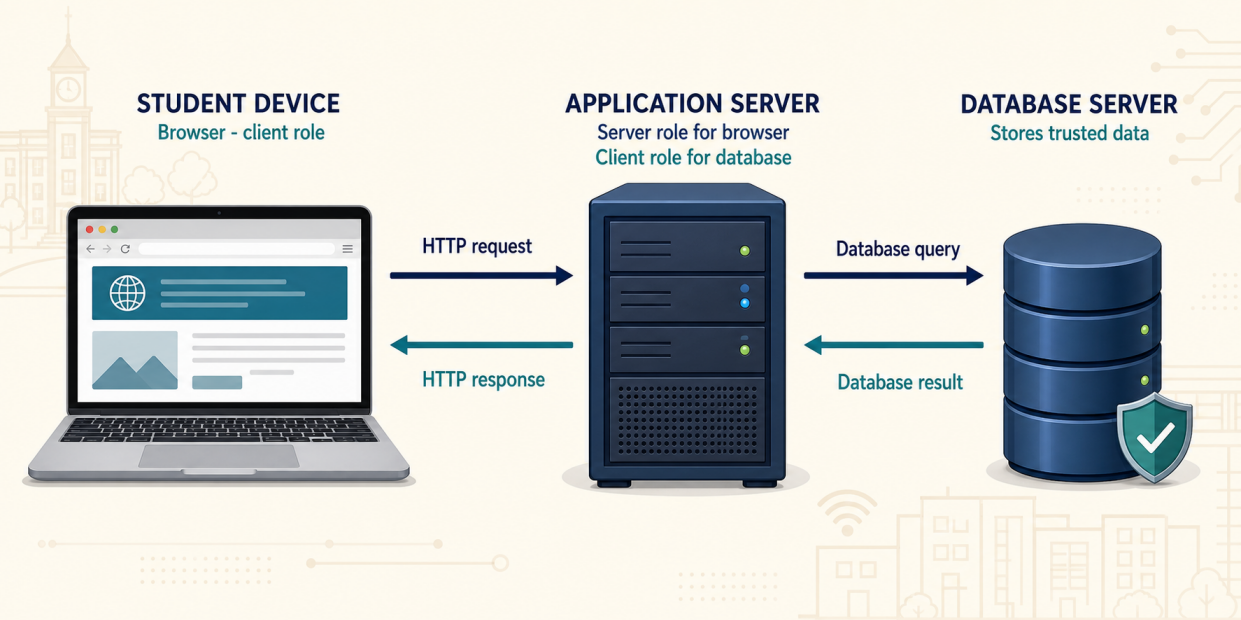


Figure 1.3. Client and server are roles within an exchange: an application server answers the browser, then acts as a client when it requests data from a database server.

2.3 HTML, CSS, and JavaScript have different jobs

Someone completely new to the Web often hears these three names together and assumes they are interchangeable programming languages. They are not.

HTML, or **HyperText Markup Language**, describes the structure and meaning of a document. “Hypertext” means that documents can link to other resources. “Markup” means that text is annotated with elements such as headings, paragraphs, links, images, lists, and forms. HTML is not primarily a language for writing algorithms. It describes what the parts of a document are.

CSS, or **Cascading Style Sheets**, describes presentation: typography, color, spacing, layout, responsive behavior, and other aspects of rendering. The word “cascading” refers to the rule system that combines declarations from different sources and resolves conflicts. CSS is much more than decoration; it controls layout across screens, print, different writing systems, user preferences, and accessibility needs.

JavaScript is a programming language. In a browser it can respond to events, calculate values, request data, update the document, and use browser-provided interfaces. JavaScript can also run outside browsers, including on servers through environments such as Node.js.

Despite the similar names, JavaScript and Java are different programming languages with different type systems, execution environments, and common uses. Modern JavaScript follows a language specification named **ECMAScript**. A JavaScript engine implements that language, while the surrounding runtime adds facilities that are not part of the language itself. A browser supplies the DOM, events, and Web APIs. Node.js supplies facilities for files, processes, network servers, and command-line programs. That is why a function or array works in both environments while `document` normally exists only in a browser.

This course uses modern JavaScript as its primary programming language on both sides of the HTTP boundary. It teaches the sides separately before integrating them. Students first learn language fundamentals, then browser programming with the DOM and events, asynchronous requests, and modules. They later apply the same language through Node.js to server routes and data access. Sharing a language does not erase the boundary: browser code runs on a user's device and cannot be trusted to enforce server rules; server code runs under the application's control and

must validate every request. TypeScript is introduced only after JavaScript and modules are understood.

Think of one event page. HTML says, “This is the main heading; this is the event date; this is a registration button.” CSS says, “Use this type scale, keep the text readable, and arrange the event cards according to the available width.” JavaScript may say, “When the student activates the filter, request the matching events and update the results without replacing the whole page.” The three technologies address different questions about the same interface.

The browser does not execute HTML as an algorithm. It parses HTML into the **Document Object Model (DOM)**, an in-memory tree representing the live document. CSS rules match parts of that tree and contribute presentation. A browser's JavaScript engine executes JavaScript, which can read or change the DOM through browser interfaces. Later chapters examine these mechanisms separately; for now, the important boundary is meaning, presentation, and behavior.

Table 1.2 distinguishes the three technologies by responsibility rather than by the file extensions in which students happen to encounter them.

Table 1.2 - Responsibilities of the core browser technologies

Technology	Primary responsibility	Simple example
HTML	Structure and meaning	“This text is the page's main heading.”
CSS	Presentation and layout	“Place these cards in a responsive grid.”
JavaScript	Behavior and state	“When the user submits, validate and update the page.”

Figure 1.4 shows the same page in three stages. Read it from left to right: a plain but meaningful document, the same document with professional presentation, and the same presented document after an interaction changes its state.

Figure 1.4. HTML supplies document meaning and structure; CSS controls presentation; JavaScript can respond to events and update state.

This separation is a design principle, not an absolute physical boundary. A JavaScript program can create HTML elements, and modern CSS can react to many conditions. Nevertheless, asking whether a concern is structural, presentational, or behavioral usually leads to clearer code.

The separation also improves resilience. If a stylesheet fails to load, a sound HTML document can still expose its headings, links, and form controls, although it will look plain. If optional JavaScript fails, ordinary navigation and a server-backed form can often remain useful when the page was designed with **progressive enhancement**. This does not mean that every application can work without JavaScript; it means that essential tasks should depend on no more technology than they genuinely require.

2.4 Website, web application, and API

A **website** commonly emphasizes documents and navigation. A **web application** usually emphasizes user interaction and changing state. The boundary is not a standardized technical line: a university portal may contain articles, search, forms, dashboards, and real-time notifications in one system.

These terms describe an emphasis, not three incompatible technologies. A news site can include interactive search. A web application can contain long-form documentation. Both can use the same URLs, HTTP semantics, HTML, CSS, JavaScript, authentication system, and server infrastructure.

An **API**, or **Application Programming Interface**, is a defined way for software components to communicate. A web API often accepts HTTP requests and returns JSON rather than a complete HTML document. A browser interface and an API can be two views of the same underlying application.

An API is a contract, not a database made public and not necessarily a screen a person visits. The contract defines operations, inputs, outputs, errors, and rules. For Campus Hub, public event pages could emphasize reading and navigation; an organizer dashboard could emphasize editing and changing state; and an API could provide event data to that dashboard, a mobile client, or another university service. The server may obtain the information from a database, but the API stands between clients and that internal storage.

Figure 1.5 uses three panels to compare those emphases. The left panel represents linked documents, the center represents an interface with interactive state, and the right represents structured exchanges between software clients and a service. The shared infrastructure beneath them emphasizes that one production system can provide all three.

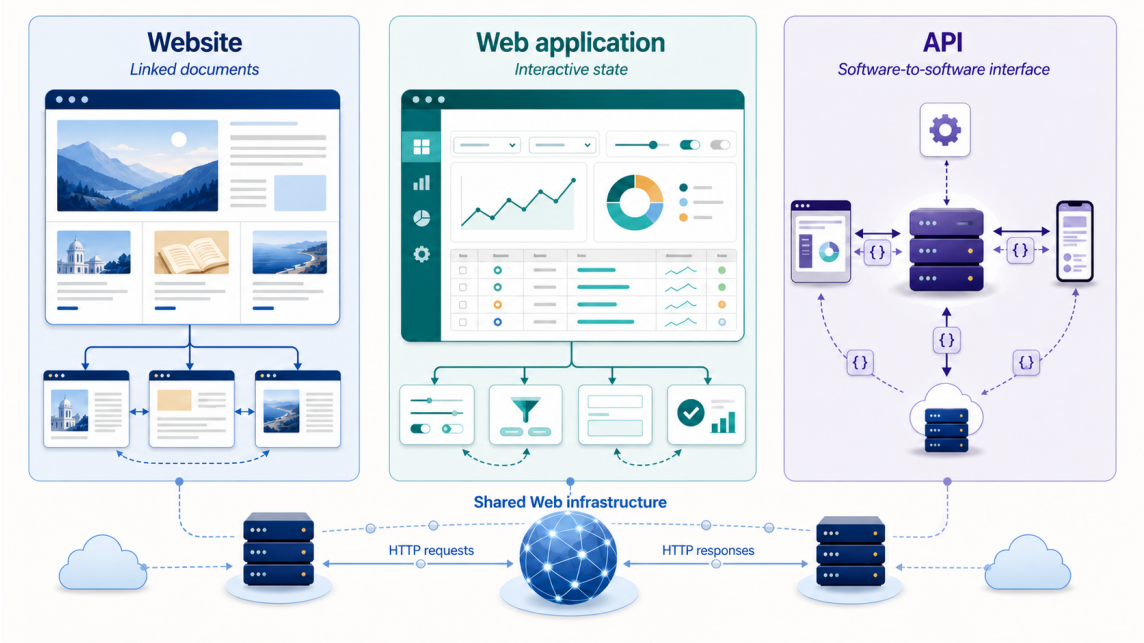


Figure 1.5. A content-oriented website, an interactive web application, and an API emphasize different interfaces while sharing Web infrastructure.

Whether the response is HTML or JSON does not decide who may access it. APIs still require validation, authorization, rate control, error handling, and careful privacy boundaries. Later chapters develop those responsibilities after students can first observe the underlying HTTP exchanges.

Checkpoint 1 - classify the roles

When your browser requests `http://localhost:8000/index.html`, which statements are true?

1. Your computer is both client host and server host.
2. The browser and server are the same program.
3. The exchange can use HTTP even though no packet leaves your computer.
4. `localhost` means “the instructor's server.”

Answer: 1 and 3. Client and server describe software roles. `localhost` normally resolves to the loopback interface of the machine on which the browser runs.

3. Read a URL as structured data

A Uniform Resource Locator (URL) is the address-like identifier a browser uses to refer to a resource and determine how to access it. It is not an informal label or a sentence. It is serialized data with a defined grammar, which is why characters such as `:`, `/`, `?`, `&`, and `#` have precise roles in particular positions. Reading a URL by components is the first step toward explaining navigation, security boundaries, routing, caching, and form submission without guesswork.

Consider the following URL.

```
https://events.campus.example:8443/clubs/robotics?semester=fall&limit=20#schedule
\___/   \_____________________/ \__/\_____________/ \___________________/ \______/
scheme            host        port       path               query          fragment
```

Figure 1.6 turns the same URL into labelled components so that each punctuation mark and substring can be discussed independently.

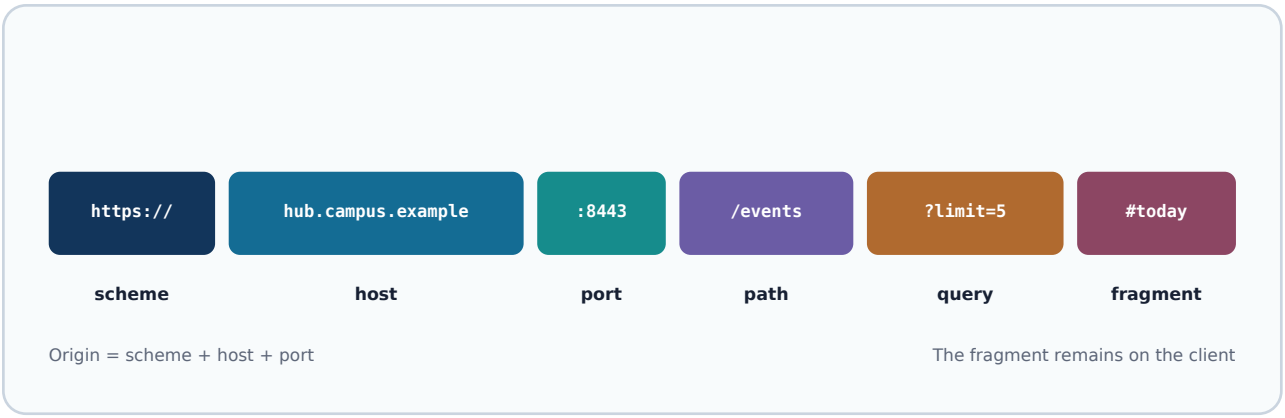


Figure 1.6. A URL is a structured identifier whose components have distinct roles.

Table 1.3 explains what each labelled URL component contributes.

Table 1.3 - Components of a URL and their purposes

Component	Purpose
Scheme	Identifies how the URL should be handled; `https` uses HTTP with transport security.
Host	Identifies the network host. A domain name can be resolved to an IP address.
Port	Selects a service endpoint on the host. It is often omitted when the default applies.
Path	Identifies a resource within the server's naming space; the resource does not have to be a physical file.
Query	Supplies additional parameters. Its meaning belongs to the application.
Fragment	Identifies a secondary location or state within the retrieved resource. It is handled by the client and is not sent in the HTTP request.

For HTTPS, the conventional default port is 443; for HTTP it is 80. A URL can omit the port without omitting the concept: the scheme supplies the default.

A note about the colon character

The colon has no single universal meaning. Its meaning comes from the notation in which it appears. In `https://`, the colon ends the URL scheme. In `campus.example:8443`, it separates the host from an explicitly written port. In the IPv6 address `2001:db8::10`, colons separate groups of hexadecimal digits, and the double colon compresses one run of zero groups. In ordinary prose, a colon may simply introduce an explanation or list. Code formatting in this book marks URL and address examples so that punctuation belonging to the technical value is easier to distinguish from punctuation in the surrounding sentence.

3.1 A path is a name, not necessarily a disk location

Early websites often mapped `/notes/week1.html` directly to a file named `week1.html`. That mapping remains possible, but modern servers frequently use a **router**. A path such as `/students/42` may instruct an application to retrieve student 42 from a database and generate a response. The client cannot determine from the URL alone whether a file, database query, cache entry, function, or another service produced the representation.

3.2 Query data and fragments

A query begins with `?`. Multiple name-value pairs are commonly separated with `&`, although an application's interpretation depends on its contract. Query parameters are appropriate for concerns such as filtering, sorting, search, and pagination when those choices belong in a shareable URL.

Queries are not private. They may appear in browser history, server logs, analytics systems, copied links, and intermediary records. Never put passwords, session identifiers, or other secrets in a URL.

A fragment begins with `#`. The browser retains it rather than including it in the HTTP request target. In a traditional document it can identify an element whose `id` matches the fragment. Client-side applications can assign other local meanings to fragments. A fragment is still not a secure storage location: browser history, screenshots, extensions, and client-side scripts may expose it.

3.3 Origin and same-origin reasoning

Suppose the browser has loaded the Campus Hub events page from the following URL.

> **This is a fictional address, not a website to open.** *The ending* `.example` *is reserved for names used in teaching and documentation. Therefore,* `hub.campus.example` *intentionally has no public Campus Hub server behind it. Some PDF viewers and browsers automatically make printed URL-shaped text look clickable. If you follow this example address, a message such as* `DNS_PROBE_FINISHED_NXDOMAIN` *is expected: it means the Domain Name System (DNS) could not find that fictional name. When a later activity expects you to open a working page, the book explicitly gives a* `localhost` *address or a verified public link.*

```
https://hub.campus.example/events
```

The complete URL identifies a resource, but the browser also needs a stable way to answer a security question: *Which other documents and services belong to the same source as this page?* The browser answers that question by calculating the page's **origin**.

For an ordinary HTTP or HTTPS URL, an origin contains exactly three values:

1. the **scheme**, such as `http` or `https`;
2. the **hostname**, such as `hub.campus.example`; and
3. the **effective port**, such as `443` for HTTPS.

The word *effective* matters. A URL does not have to show a port explicitly. HTTPS uses port 443 by default, while HTTP uses port 80 by default. Consequently, `https://hub.campus.example` and `https://hub.campus.example:443` have the same origin. The first URL omits the port from its spelling, but its effective port is still 443.

For the Campus Hub page, the origin can therefore be written as follows.

```
https://hub.campus.example
```

Notice what disappeared. The path `/events` is not part of the origin. Neither is a query nor a fragment. Figure 1.6 labels all of those URL components; when you make an origin comparison, use only the scheme, hostname, and effective port. Paths still distinguish resources, but they do not create different origins.

A mechanical way to compare two origins

Do not decide by asking whether two URLs “look related.” Apply the same four steps every time:

1. Resolve a relative reference, if necessary, so that both values are complete URLs.
2. Compare the schemes. If they differ, the origins differ.
3. Compare the hostnames. If they differ, the origins differ.
4. Compare the effective ports, inserting the scheme's default when a port is omitted. The URLs are same-origin only if all three comparisons match.

Apply those steps with `https://hub.campus.example/events` as the page URL.

- `https://hub.campus.example/clubs?active=true` is **same-origin**. Only the path and query differ.
- `https://hub.campus.example:443/help` is **same-origin**. The written port 443 is the default HTTPS port, so both effective ports are 443.
- `http://hub.campus.example/events` is **cross-origin**. The scheme is `http` instead of `https`; its default port is also 80 instead of 443.
- `https://api.campus.example/events` is **cross-origin**. `api.campus.example` and `hub.campus.example` are different hostnames. Sharing the ending `campus.example` does not make them the same origin.
- `https://hub.campus.example:8443/events` is **cross-origin**. The hostname matches, but port 8443 does not match the effective port 443.

This rule often surprises students during local development. A page at `http://localhost:8000` and an application programming interface (API) at `http://localhost:3000` run on the same computer, but they are cross-origin because their ports differ. Similarly, `http://localhost:8000` and `http://127.0.0.1:8000` are cross-origin because `localhost` and `127.0.0.1` are different host values. “Same machine” and “same origin” answer different questions.

Why the browser needs this boundary

Imagine that a student is signed in to a private Campus Hub page and then opens an unrelated, malicious page in another tab. If any script could freely read data from any origin that the browser could reach, the malicious page could try to read the student's private timetable, grades, or intranet pages. The **same-origin policy** limits how a document or script from one origin can interact with resources from another origin. It is one of the browser's central isolation mechanisms.

The policy is not a rule that says “no cross-origin traffic is ever allowed.” Browsers commonly allow a user to follow a link to another origin, allow a form to submit to another origin, and allow a page to display some cross-origin resources such as images. The important restrictions depend on the kind of interaction. In particular, JavaScript normally cannot read a cross-origin `fetch()` response unless the receiving server deliberately permits that access.

Cross-Origin Resource Sharing (CORS) is the HTTP mechanism through which a server can grant selected cross-origin access. CORS does not turn two origins into one origin, and browser code cannot grant itself permission. The server states its policy in response headers, and the browser enforces the result. Chapter 6 develops that request-and-response process in detail.

Also keep **origin** separate from the looser everyday word *website*. One website may deliberately use several origins, such as `www.campus.example` for pages and `api.campus.example` for data. Those names may belong to the same organization, yet the browser still treats them as different origins. Later security chapters will also distinguish *same-origin* from *same-site*; the two terms are not interchangeable.

Check the rule before moving on

A page is loaded from `http://localhost:8000/week-01/`. Decide whether each target is same-origin with the page.

- `/images/campus.svg` is same-origin because a relative URL inherits the page's scheme, hostname, and port.
- `http://localhost:8000/api/events` is same-origin because all three origin

values match.

- `http://localhost:3000/api/events` is cross-origin because the port differs.
- `https://localhost:8000/api/events` is cross-origin because the scheme

differs, even though the hostname and written port match.

- `http://127.0.0.1:8000/api/events` is cross-origin because the host differs,

even though both names may reach the same computer.

The habit to retain is precise: compare scheme, hostname, and effective port. Do not compare the path, do not judge by organizational ownership, and do not substitute “same computer” for “same origin.” Later chapters will connect this model to cookies, browser storage, iframes, CORS, and content isolation.

3.4 Encoding and normalization

URLs are serialized according to defined parsing and percent-encoding rules. A space may appear as `%20`, and form-encoded query data often represents a space with `+`. Visually similar or differently encoded URLs are not safe to compare by casual inspection. Applications should use a standards-compliant URL parser rather than split URL strings manually.

The left-to-right appearance of a hostname can mislead beginners. In `accounts.university.example`, the registered-domain boundary depends on naming policy, but the DNS hierarchy becomes more general toward the right. Never decide whether a site is trustworthy merely because a familiar word appears somewhere in the hostname.

Predict before running

Given this base URL:

```
https://hub.campus.example/events/today
```

predict the targets of these references:

```
../clubs
/help
?format=json
#registration
```

Answers:

- `../clubs` -> `https://hub.campus.example/clubs`
- `/help` -> `https://hub.campus.example/help`
- `?format=json` -> `https://hub.campus.example/events/today?format=json`
- `#registration` -> `https://hub.campus.example/events/today#registration`

Relative URL resolution follows defined rules; it is not plain string concatenation. A relative reference depends on a base URL. In an HTML document, that base is normally the document URL unless a `base` element explicitly changes it. The same text, such as `../clubs`, can therefore identify different targets when it appears in documents at different locations.

4. From navigation to pixels

The following model is intentionally simplified, but each step is observable or refinable later in the book.

Two abbreviations appear in the sequence. The Domain Name System (DNS) supplies naming information, and Transport Layer Security (TLS) protects a secure connection. The sections

immediately after the figure explain where these protocols fit and what they do not do. Figure 1.7 traces the whole page-navigation sequence before those individual responsibilities are examined.

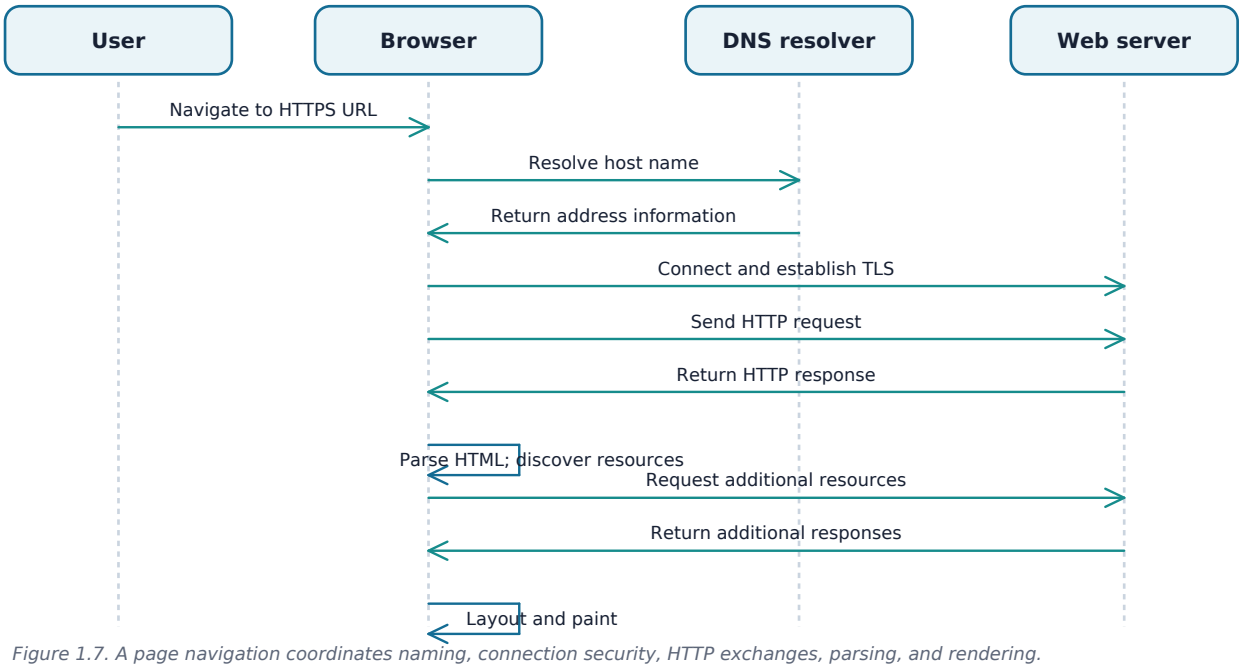


Figure 1.7. A page navigation coordinates naming, connection security, HTTP exchanges, parsing, and rendering.

Figure 1.7 is a sequence diagram: it emphasizes **when** each participant acts and which participant sends each message. It deliberately hides most of the physical network so that the causal order remains readable. The browser does not normally connect to a server by one uninterrupted cable. Its data crosses a succession of local and remote network links.

Figure 1.8 presents the same navigation from a spatial point of view. A laptop may send a Wi-Fi frame to an access point; the campus network carries the data through switches and a router; independently operated transit networks forward IP packets; and a data-center network finally delivers them to the server. Response data returns through interconnected networks, although it is not guaranteed to follow exactly the same route in reverse. Routers forward packets toward addresses; they do not interpret the page's headings, CSS rules, or JavaScript behavior.

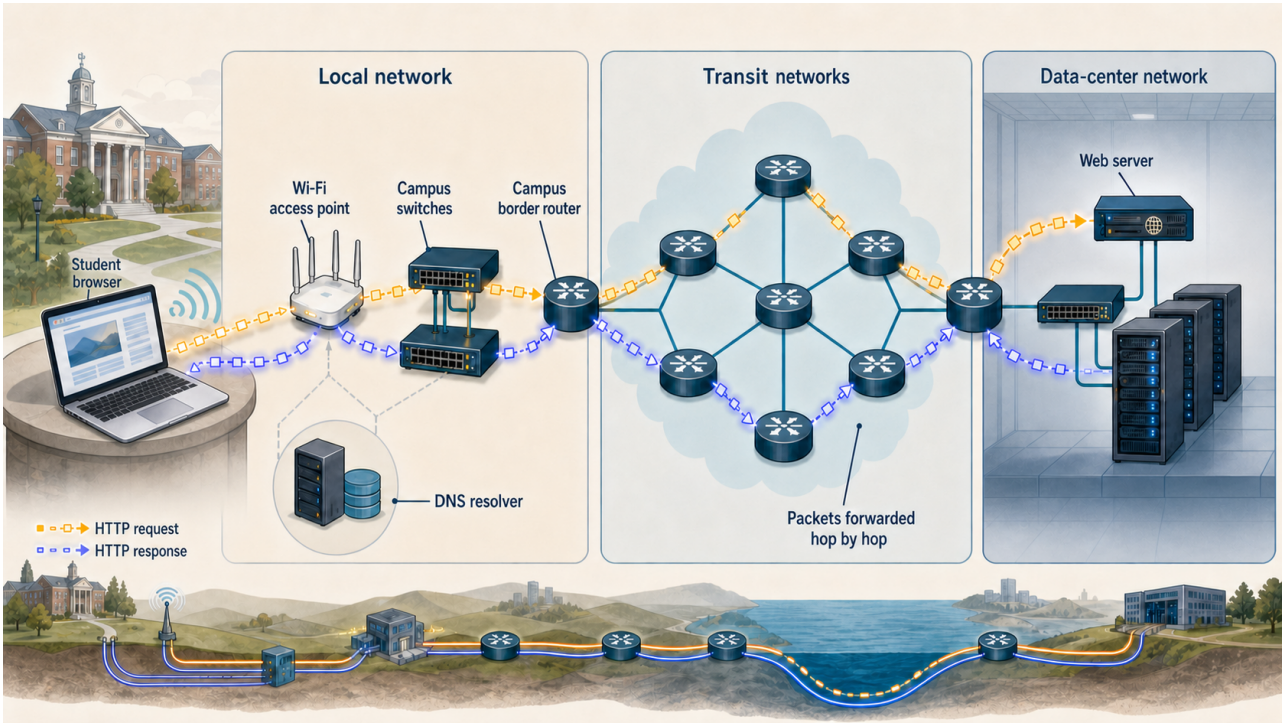


Figure 1.8. A request crosses a local network, independently operated transit networks, and a data-center network; the response returns across interconnected routes.

The DNS lookup shown near the beginning is a related exchange, not another cable inside the page request. It helps the client obtain destination information. Once the browser has suitable cached or newly resolved address information, it can establish the connection used for HTTPS. Together, Figures 8 and 9 provide two complementary views: one explains the order of work, and the other explains where the data travels.

4.0 The protocol stack: cooperation without confusion

The browser does not send one special object called “a website packet.” A page request is completed by several protocols working together. The word **stack** is helpful because each layer provides a service to the layer above it and relies on a service below it. The layers are not a menu from which the browser chooses only one item.

For one secure page request, several layers are active during the same overall exchange. The browser may create an HTTP request. Transport Layer Security (TLS) protects that HTTP data. Transmission Control Protocol (TCP) may carry the protected bytes reliably. Internet Protocol (IP) addresses and routes packets. Finally, a link technology such as Wi-Fi or Ethernet moves frames across each local network connection. A router removes one link-layer frame and constructs a new one for the next link, while the IP packet continues toward its destination.

Some entries in a stack are alternatives because different protocol versions solve the same layer's job differently. Hypertext Transfer Protocol version 2 (HTTP/2) commonly uses TLS over TCP. Hypertext Transfer Protocol version 3 (HTTP/3) uses QUIC, a secure transport protocol, over the User Datagram Protocol (UDP). QUIC includes the security handshake needed by HTTP/3, so an HTTP/3 stack does not place a separate TLS box below QUIC in the same way as HTTP/2 over TCP. Both examples still use IP and a link technology.

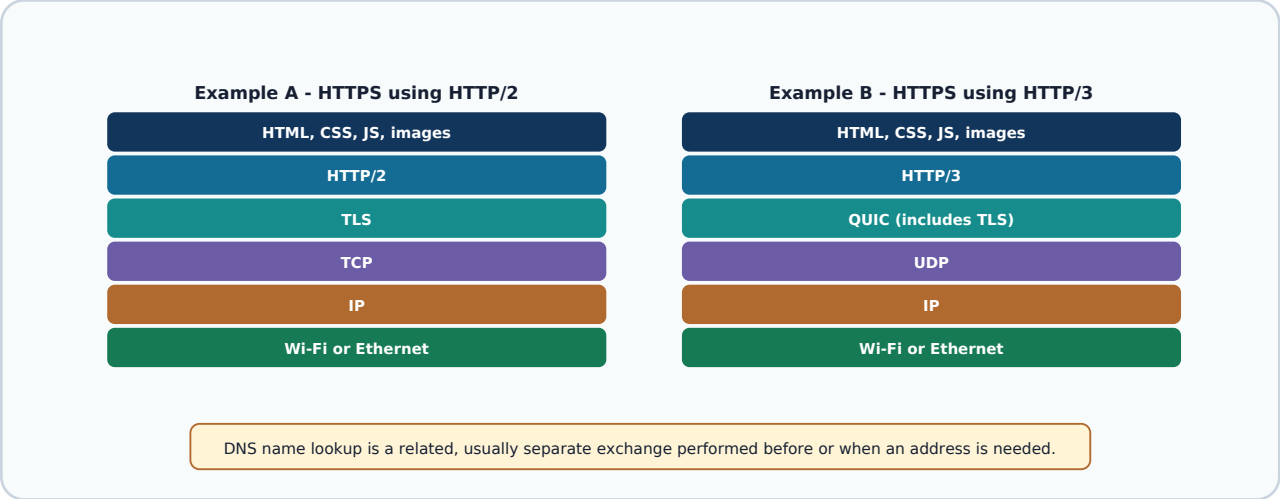


Figure 1.9. Two common HTTPS stacks. Read each column from the web content down to the network link.

Read each column in Figure 1.9 from top to bottom. A column represents one possible combination used for a page exchange; it does not mean that only one box operates. The boxes in one column cooperate. The two columns are alternatives selected by protocol support and negotiation.

At a useful introductory level, the responsibilities are:

- **Domain Name System (DNS)** lookup obtains typed naming information, often an

address. It is usually a separate exchange performed before the browser can open the page connection. DNS is not carried inside that page's HTTP request.

- **Internet Protocol (IP)** moves packets between network interfaces across

interconnected networks.

- **Transmission Control Protocol (TCP)** provides a reliable, ordered byte

stream for protocols such as HTTP/1.1 and HTTP/2.

- **QUIC** provides secure, multiplexed transport over the User Datagram Protocol

(UDP) and is used by HTTP/3.

- **Transport Layer Security (TLS)** protects data with authentication,

confidentiality, and integrity. In the HTTP/2 example it sits between HTTP and TCP; HTTP/3 obtains the corresponding protection through QUIC.

- **Hypertext Transfer Protocol (HTTP)** defines the meaning of requests and

responses about resources.

- **Page content** includes HyperText Markup Language (HTML), Cascading Style

Sheets (CSS), JavaScript, images, and JavaScript Object Notation (JSON). These resources are interpreted by applications at the endpoints.

This model is deliberately simplified. Real systems can reuse connections, encrypt DNS traffic, pass through proxies, or choose different link technologies at different points along the route. The essential idea remains stable: several protocols can be active together because they perform different jobs, while TCP and QUIC are examples of alternatives for the transport job. The application meaning of `GET`, `404`, and `Content-Type` remains useful even when the lower transport details differ.

Figure 1.10 turns those abstract layers into a journey across a network. At the sender, the page resource becomes the content of an HTTP request. For the TCP-based example, TLS protects the HTTP data, TCP supplies an ordered byte stream, IP supplies the packet addressing used by routers, and the first local link carries the packet inside a Wi-Fi frame. This wrapping is often called **encapsulation**: each layer adds the control information needed for its own job.

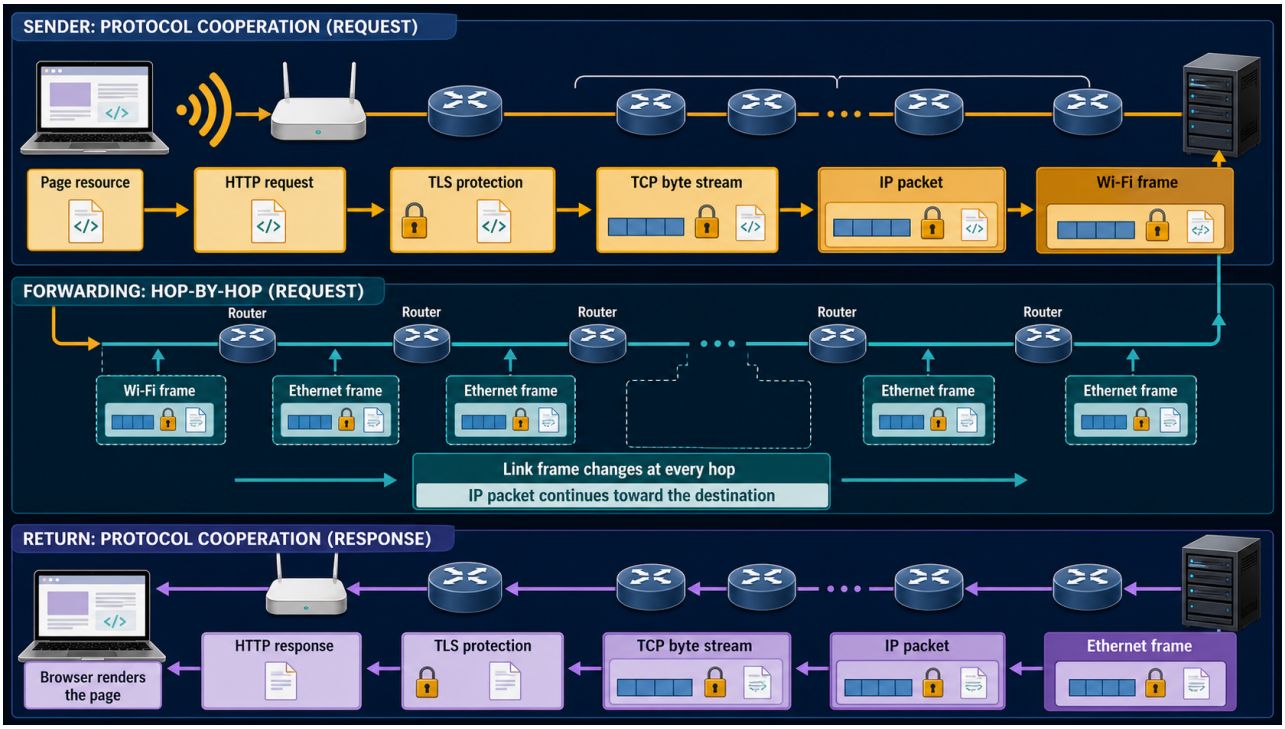


Figure 1.10. An HTTPS exchange combines application, security, transport, network, and link responsibilities while routers forward packets hop by hop.

At a router, the incoming link frame has reached the end of its local journey. The router removes that frame, examines the IP information needed for forwarding, and places the packet in a new frame suitable for the next link, such as Ethernet. The exact bytes and headers are more detailed than Figure 1.10 can show, but the boundary is essential: routers forward the IP packet hop by hop; the browser and server endpoints interpret HTTP and page content. The return response uses the same cooperating responsibilities in the opposite application direction.

4.1 Name resolution

Humans prefer domain names; network routing ultimately needs address information. The browser and operating system consult caches and a DNS resolver. Resolution is not necessarily a single journey to one central directory. DNS is distributed and hierarchical, and answers can be cached for a specified time.

Important correction: DNS does not download the page. It helps the client find where to send a request.

What DNS actually is

The **Domain Name System (DNS)** is a distributed, hierarchical database and a query protocol. It does not store one untyped answer beside each name. Instead, it stores typed entries called **resource records**. A client asks for a name and a record type. The type tells DNS what kind of information the client wants. Name-to-address lookup is the most familiar use, but the same system also stores information about mail delivery, authoritative servers, aliases, and other services.

A resource record is easier to understand when its fields are visible. Consider this teaching example.

```
hub.campus.example.  300  IN  A  192.0.2.40
```

Read the record from left to right.

- `hub.campus.example.` is the **owner name**, meaning the DNS name to which the

record belongs. The final dot explicitly represents the root of the DNS tree, so the name is complete rather than relative to a local suffix.

- `300` is the **time to live (TTL)** in seconds. A cache may normally reuse this

record for 300 seconds before asking for fresh information.

- `IN` is the **Internet class**. DNS was designed to support more than one data

class, but `IN` is the class students will encounter for ordinary Internet use.

- `A` is the **record type**. It says how to interpret the final field.
- `192.0.2.40` is the record's **data**. For an `A` record, the data is one IPv4

address. The documentation address used here is not a public Campus Hub server.

Reading `192.0.2.40` as an IPv4 address

Internet Protocol version 4 (IPv4) addresses contain exactly 32 binary digits, or **bits**. For human-readable display, those 32 bits are divided into four groups of eight bits. An eight-bit group is called an **octet**. Each octet is converted to an ordinary decimal number, and dots separate the four written numbers:

```
192 . 0 . 2 . 40
 |    |   |    |
octet octet octet octet
```

Eight bits can represent 256 distinct values, beginning with 0 and ending with

1 Therefore, every dotted-decimal IPv4 octet must be in the inclusive range

`0`-`255`. Values such as `192.0.2.40` and `10.24.0.255` are syntactically valid, whereas `192.0.2.256`, `192.0.-1.40`, and `192.0.2` are not valid complete IPv4 addresses. The four octets in this example have the following binary forms:

```
192       0         2         40
11000000  00000000  00000010  00101000
```

The dots belong to the written notation; they do not divide a packet into four separate transmissions. Nor do the four numbers correspond to the four levels in `hub.campus.example`. A domain name and an IP address use different structures for different purposes.

An IPv4 address identifies an IP endpoint or network interface for routing. It does not identify an application process by itself; a transport-layer port, such as TCP port 443 for conventional HTTPS, helps select the service on that host. The address alone also does not reveal where the network-prefix portion ends and the host portion begins. That boundary is supplied by a prefix length. For example, `192.0.2.0/24` says that the first 24 bits form the network prefix and the remaining eight bits vary within that prefix.

The block `192.0.2.0/24` is reserved for documentation and examples. Consequently, `192.0.2.40` is suitable for this book but should not be presented as the actual public address of Campus Hub. In a real DNS lookup, an `A` record can return one or several operational IPv4 addresses selected by the domain operator.

Reading `2001:db8::40` as an IPv6 address

Internet Protocol version 6 (IPv6) was designed with a vastly larger address space and improvements to Internet-layer operation. An IPv6 address contains exactly 128 bits: four times as many bits as an IPv4 address. This does not mean that an IPv6 address is written as four IPv4 addresses. IPv6 has its own notation and operating rules.

The size difference is enormous. IPv4 has `2^32`, or about 4.3 billion, possible bit patterns. IPv6 has `2^128`, or approximately 3.4 x 10^38, possible bit patterns. Addressing policies reserve or structure parts of these spaces, so neither number is simply a count of devices that can be connected. The comparison does explain why IPv6 can assign generously sized network prefixes without depending on widespread address sharing merely to conserve addresses.

Writing 128 individual zeroes and ones would be error-prone, so IPv6 uses **hexadecimal**, a base-16 number system. Hexadecimal has sixteen digit values: `0` through `9`, followed by `a` through `f` for decimal values 10 through 15. Each hexadecimal digit represents four bits. Four hexadecimal digits therefore represent 16 bits.

The full written form divides the 128 bits into eight 16-bit groups. Colons separate the groups:

```
2001 : 0db8 : 0000 : 0000 : 0000 : 0000 : 0000 : 0040
 |       |      |      |      |      |      |      |
16 bits 16 bits 16 bits 16 bits 16 bits 16 bits 16 bits 16 bits
```

Each group is in the inclusive hexadecimal range `0000-ffff`. Letter case has no numerical significance, so `0DB8` and `0db8` mean the same value; lowercase is the conventional form used in this book. The colons are separators in the written address. They are not transmitted as characters in an IPv6 packet.

For example, hexadecimal `2001` represents these sixteen bits. Reading four bits at a time makes the conversion visible:

```
hexadecimal      2      0      0      1
binary        0010   0000   0000   0001
```

IPv6 notation provides two shortening rules. Apply them carefully:

1. **Omit leading zeroes within a group.** For example, `0db8` becomes `db8`, `0040` becomes `40`, and `0000` becomes `0`. Do not remove a zero from the right-hand side of a nonzero group: `0040` becomes `40`, not `4`.
2. **Replace one consecutive run of all-zero groups with `::`.** The double colon may appear only once in an address, because two compressed runs would make it impossible to know how many zero groups belong in each position.

Applying both rules turns the full address above into:

```
2001:db8::40
```

This also gives useful syntax checks. `2001:db8::40`, `::1`, and `fe80::a:b:c:d` have valid IPv6 structure. `2001::db8::40` is invalid because it contains two double-colon compressions. `2001:db8:12345::40` is invalid because one group contains five hexadecimal digits. `2001:db8::g` is invalid because `g` is not a hexadecimal digit. A syntactically valid address may still be reserved, unassigned, unreachable, or inappropriate for a particular interface.

To expand a compressed address, count the visible 16-bit groups and insert enough `0000` groups at `::` to produce exactly eight groups. In `2001:db8::40`, three groups are visible (`2001`, `db8`, and `40`), so the double colon represents five all-zero groups. The following forms consequently describe the same 128-bit address:

```
2001:0db8:0000:0000:0000:0000:0000:0040
2001:db8:0:0:0:0:0:40
```

```
2001:db8::40
```

An address can use leading-zero omission without using `::`. Conversely, `::` can represent several zero groups or just one zero group. The address `::1`, for example, expands to seven zero groups followed by `0001` and identifies the IPv6 **loopback** interface: the current computer. The address `::` is the **unspecified address**. It means that no ordinary source address has yet been selected; it is not a public destination to place in a link.

Although compressing a single zero group is syntactically possible, the standard recommended text form does not use `::` for only one group. When several zero runs are equally long, the recommended form compresses the first run. These canonicalization rules help logs, configuration files, and comparison tools use one predictable spelling even though several spellings can represent the same address.

As with IPv4, a prefix length identifies the network portion. The notation `2001:db8:1234:5678::/64` means that the first 64 bits are the network prefix and the remaining 64 bits form the interface-identifier portion within that subnet. A `/64` is the standard size for most ordinary IPv6 local-area-network subnets, although other prefix lengths have defined uses. The slash is not part of the address itself; it belongs to the address-prefix notation.

Several address scopes are useful to recognize:

- **Global unicast addresses** are intended to be routable across IPv6 networks,

subject to routing and security policy. The broad currently allocated global unicast space begins within `2000::/3`.

- **Link-local addresses** use `fe80::/10` and operate only on the local network

link. Routers do not forward them to other links. An operating system may show a zone identifier such as `fe80::12ab%7` to specify which local interface is meant; `%7` is interface notation, not part of the 128-bit address.

- **Loopback** is `::1`, which refers to the current host.
- **Multicast addresses** begin with `ff00::/8` and identify a group of receivers.

IPv6 has no broadcast address; multicast and other mechanisms serve the cases for which IPv4 commonly used broadcast.

The prefix `2001:db8::/32` is reserved for documentation. Therefore, `2001:db8::40` is appropriate in this chapter but should not be described as a real Campus Hub server. A production host may have multiple IPv6 addresses, including a stable address and one or more temporary privacy addresses. Address assignment may be coordinated through router advertisements, Stateless Address Autoconfiguration (SLAAC), Dynamic Host Configuration Protocol for IPv6 (DHCPv6), manual configuration, or a combination chosen by the network operator.

IPv6 changes addressing; it does not automatically provide application security. A globally routable address still requires firewall policy, secure service configuration, Transport Layer Security (TLS) for HTTPS, authentication, and software maintenance. IPv6 also does not eliminate routers: routers still forward IPv6 packets between networks. Network Address Translation is not a normal requirement for conserving IPv6 addresses, but removing translation is not the same as removing security controls.

DNS stores IPv6 address information in an `AAAA` record. A name can have both `A` and `AAAA` records, allowing an IPv4/IPv6 **dual-stack** client to choose a working path. Modern connection algorithms commonly try available IPv6 and IPv4 possibilities without forcing the user to choose a protocol version manually. Receiving an IPv6 address from DNS still does not prove that the server is reachable or that HTTPS will succeed.

An IPv6 literal needs square brackets when it appears as the host in a URL. The brackets distinguish the address's internal colons from the colon that introduces an optional port:

```
https://[2001:db8::40]/status
https://[2001:db8::40]:8443/status
```

In the second URL, `2001:db8::40` is the host and `8443` is the port. The brackets are URL syntax; they are not part of the 128-bit IPv6 address. In production, domain names are normally preferable in public HTTPS URLs because certificates, service migration, and human recognition are organized around stable names.

Table 1.4 compares the record types that students are most likely to meet during introductory Web development and deployment.

Table 1.4 - Common DNS resource record types

Type and name	Value stored	What the record is commonly used for
`A` - Address	One 32-bit IPv4 address, written as four decimal octets such as `192.0.2.40`	Gives IPv4 address information for a name. One name can have several `A` records.
`AAAA` - IPv6 Address	One 128-bit IPv6 address, written in hexadecimal groups such as `2001:db8::40`	Gives IPv6 address information for a name. The four letters distinguish the newer address record from `A`; they are the record's official type name.
`CNAME` - Canonical Name	Another domain name, not an IP address	Declares that the owner name is an alias. A resolver follows the target name and asks for the needed record type there.
`MX` - Mail Exchange	A preference number and the domain name of a mail server	Directs email delivery for a domain. Lower preference numbers are tried before higher ones, subject to the mail protocol's rules.
`NS` - Name Server	The domain name of an authoritative name server	Delegates or identifies DNS authority for a zone. The server name must itself be resolved to an address before it can be contacted.
`TXT` - Text	One or more character strings	Carries text interpreted by a specific convention, including some email-security and domain-verification mechanisms. DNS itself does not assign one universal meaning to all TXT text.

The following examples use names and addresses reserved for teaching.

```
hub.campus.example.     300   IN  AAAA   2001:db8::40
www.campus.example.     300   IN  CNAME  hub.campus.example.
campus.example.         3600  IN  MX     10 mail.campus.example.
campus.example.         86400 IN  NS     ns1.campus.example.
campus.example.         3600  IN  TXT    "course-verification=sample-value"
```

The `AAAA` example provides the IPv6 address explained immediately before Table 1.4. Its record data stores the complete 128-bit value; the compressed spelling does not reduce the address itself. DNS transports address data in its protocol representation rather than as a browser copying the displayed punctuation.

The `CNAME` example does not say that `www.campus.example` has the literal address `hub.campus.example`. It says that the first name is an alias and the resolver should continue with the canonical target. The `MX` example assigns mail preference 10 to `mail.campus.example`; it does not provide that server's address. The `NS` example names an authoritative server, which also requires address records. The `TXT` example is merely a string until an application-level convention defines how to interpret it.

A DNS answer can contain several records, and different clients can request different types for the same owner name. Receiving an `A` answer therefore does not imply that no `AAAA`, `MX`, `TXT`, or other records exist.

Stub resolver, recursive resolver, and authoritative server

Most applications do not walk the DNS hierarchy themselves. They ask a local **stub resolver**, usually provided by the operating system. It contacts a **recursive resolver** operated by a network provider, institution, public DNS service, or local organization.

If the recursive resolver has no usable cached answer, it follows referrals. At a simplified level it may ask:

1. a root name server where to find information for the top-level domain;
2. a top-level-domain server where to find the domain's authoritative servers;
3. an authoritative server for the requested record.

The resolver returns the result to the client and may cache it. Real resolution can involve aliases, multiple addresses, DNSSEC validation, search suffixes, local policy, split-horizon DNS, or encrypted resolver transports. The simplified sequence is a foundation, not a complete implementation trace. Figure 1.11 combines the lookup path with the dotted-decimal structure of the returned IPv4 address. Read its left panel as a conversation: the browser asks its local stub resolver, the stub asks a recursive resolver, and the recursive resolver consults the DNS hierarchy only when it lacks a usable cached answer. Read the right panel as a numeric explanation of the `A` record data, not as additional DNS hierarchy.

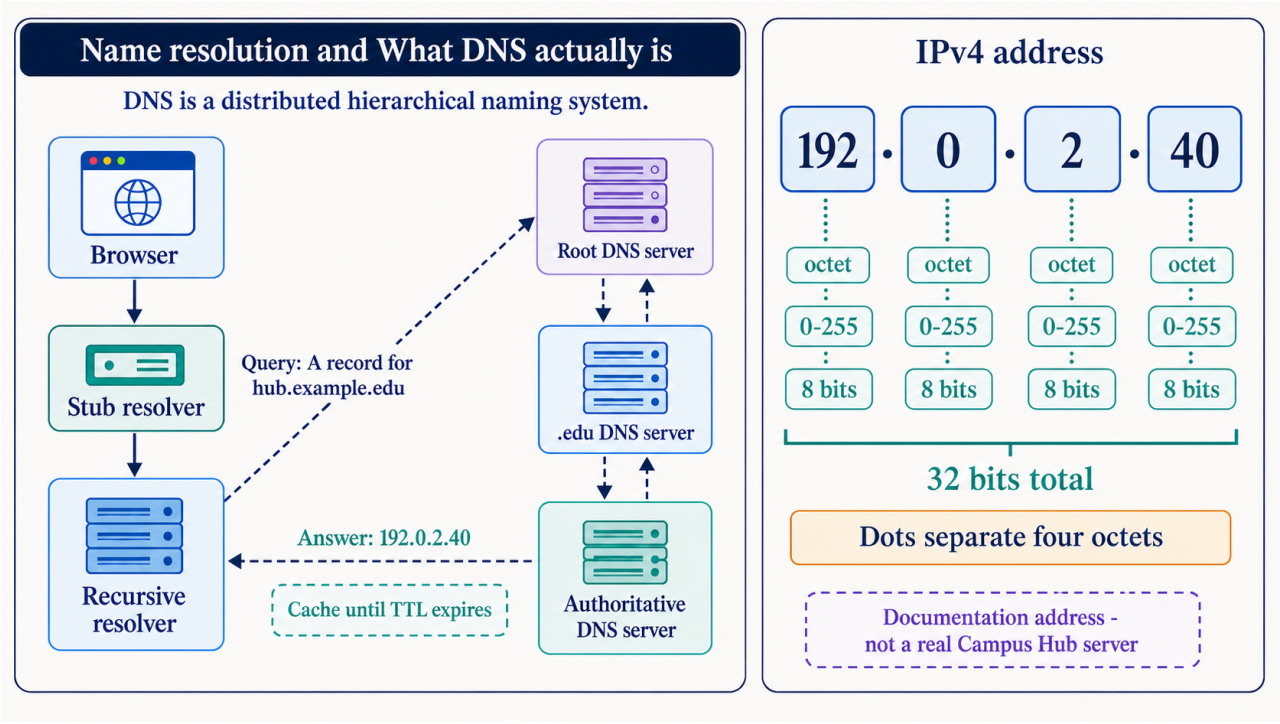


Figure 1.11. DNS resolution follows a hierarchy to obtain a typed answer; an IPv4 address contains four eight-bit octets written in dotted-decimal notation.

Caching and TTL

A DNS response can include a **time to live** (TTL), which tells caches how long the record may be reused. Caching improves latency and reduces load, but it also means a record change is not visible everywhere at once. “DNS propagation” is often a loose description of old cached answers expiring at different times.

Negative answers can also be cached. If a name did not exist when queried, a resolver may retain that result for a period. Repeatedly refreshing the browser does not necessarily bypass browser, operating-system, or resolver caches.

DNS does not guarantee application availability

A successful DNS answer proves only that the resolver obtained relevant naming data. The returned service can still be unreachable, refuse a connection, fail a TLS handshake, return `503`, or produce incorrect content. Conversely, a DNS failure prevents normal name-based access even if a server process is healthy.

4.2 Connection and TLS

The client needs a transport path to the server. Different HTTP versions use different transport details, which later chapters survey. For an HTTPS URL, the client and server also establish a protected channel and the client validates the server's certificate for the requested identity.

With TCP-based HTTP, connection establishment creates an ordered, reliable byte stream between endpoint addresses and ports. “Reliable” has a specific meaning: the transport detects loss and retransmits data. It does not promise that a server will answer quickly, remain online, or process an application request correctly.

TLS establishes cryptographic parameters and authenticates the server's identity, normally through a certificate chain anchored in a trust store. After a successful handshake, protected application data has three central properties:

- **confidentiality**: observers should not be able to read protected content;
- **integrity**: undetected modification should not be possible; and
- **authentication**: the client verifies the server identity it intended to

contact, subject to certificate and validation rules.

HTTPS protects data in transit and authenticates the server identity represented by the certificate. It does **not** prove that the application's claims are true, that its code has no vulnerabilities, or that the organization deserves trust.

Encryption also does not make endpoints invisible. Network observers can still learn some metadata, such as addresses, timing, and traffic volume. Privacy is a system property, not a padlock icon.

4.3 HTTP exchange

HTTP is an application-level, stateless protocol.

“Stateless” means that the protocol does not require the server to remember prior requests in order to interpret the current one. Applications still create state using databases, cookies, sessions, caches, and other mechanisms. Application state is built on top of HTTP rather than supplied automatically by HTTP. Figure 12 separates the request travelling from client to server from the response travelling back to the client.

Figure 1.12. HTTP separates client request intent from server response outcome.

A simplified request might be:

```
GET /events?semester=fall HTTP/1.1
Host: hub.campus.example
Accept: text/html
User-Agent: ExampleBrowser/1.0
```

A simplified response might be:

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 4182
Cache-Control: max-age=60

<!doctype html>
<html lang="en">
...
```

The message format differs in HTTP/2 and HTTP/3, but the semantic ideas-method, target, fields, status, and content-remain useful.

The method expresses request intent. `GET` asks to retrieve a representation. The status code summarizes the result. Table 1.5 provides a first reading of the five status-code families.

Table 1.5 - HTTP status-code families

Range	Meaning	Examples
1xx	Informational	`103 Early Hints`
2xx	Successful	`200 OK`, `201 Created`, `204 No Content`
3xx	Redirection	`301 Moved Permanently`, `304 Not Modified`
4xx	The request cannot be fulfilled as submitted	`400 Bad Request`, `403 Forbidden`, `404 Not Found`
5xx	The server failed while handling an apparently valid request	`500 Internal Server Error`, `503 Service Unavailable`

Do not translate every 4xx response as “the student made a mistake” or every 5xx response as “the server computer is broken.” Status codes describe protocol-level outcomes.

Methods and semantics

Frequently encountered methods are compared in Table 1.6.

Table 1.6 - Common HTTP request methods and their intended semantics

Method	Intended use	Important property
`GET`	Retrieve a current representation.	Safe and idempotent by definition.
`HEAD`	Retrieve the fields a `GET` would return, without response content.	Useful for metadata checks.
`POST`	Ask the target resource to process enclosed content.	Often creates or triggers something; not generally idempotent.
`PUT`	Create or replace the state of the target resource.	Idempotent.
`PATCH`	Apply a partial modification.	Not guaranteed to be idempotent.
`DELETE`	Remove the association represented by the target.	Idempotent in intended effect.

A **safe** method is intended to be read-only from the user's perspective. An **idempotent** method has the same intended effect when the same request is made once or several times. Idempotent does not mean “returns the identical response” or “has no logging side effects.” These properties shape retry behavior, caching, link traversal, and API design.

Fields and content negotiation

HTTP fields carry metadata. `Content-Type` describes the media type of content; `Content-Length` describes its size when present; `Accept` expresses formats a client can handle; `Cache-Control` carries caching directives; and `Location` can identify another resource after redirection or creation.

Never infer a response's format solely from its filename. A URL ending in `.html` can return JSON, and a URL with no extension can return HTML. The response's `Content-Type` is the protocol-level declaration. Incorrect media types can cause rendering failures and security problems.

Representations and resources

A resource is the conceptual target identified by a URL. A **representation** is a particular transferable form of its current or intended state. The same resource might have HTML and JSON representations, or localized forms selected through negotiation. This distinction explains why “URL equals file” is too weak for modern web architecture.

Redirects, caching, and conditional requests

A redirect is a response, not an invisible jump. The browser receives a 3xx status plus information such as a `Location` field and may issue a new request. The Network panel therefore often shows both the original and follow-up exchange.

Caching allows a stored response to satisfy later requests under defined rules. A client can make a conditional request using a validator such as an entity tag (`ETag`). A `304 Not Modified` response tells the client to reuse its stored representation; it does not mean that the server failed to respond.

4.4 Browser processing

Receiving HTML is not the end. The browser parses it into an internal document model, discovers referenced resources, applies styles, calculates geometry, and paints pixels. Scripts may later respond to events and change the document. One visible page commonly results from many HTTP exchanges. Figure 1.13 summarizes the browser pipeline from received bytes to visible pixels.

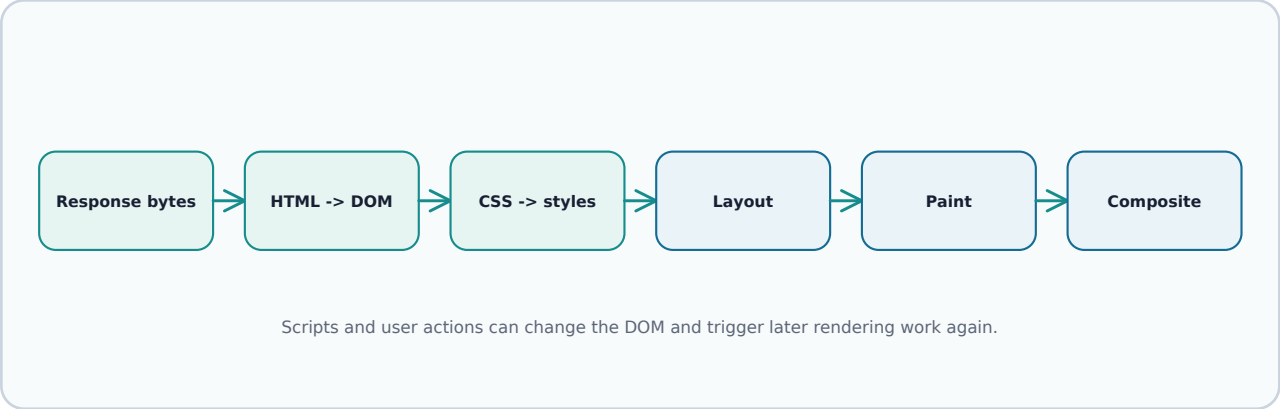


Figure 1.13. Browsers transform bytes and resources into pixels and interaction.

A useful rendering model is:

1. HTML bytes are decoded into characters and parsed into the **Document Object Model** (DOM), a tree of nodes.
2. CSS is parsed, selectors are matched, and computed styles are determined. A conceptual **CSS object model** participates in this work.
3. The browser determines which boxes need to be generated for rendering.
4. **Layout** calculates sizes and positions.
5. **Paint** records how backgrounds, borders, text, and images should be drawn.
6. **Compositing** combines layers for display.

This is a teaching model, not a promise that every browser executes six strictly sequential phases. Engines optimize, parallelize, cache, and revisit work. A script that changes layout-related properties can trigger additional style, layout, paint, or compositing work.

Progressive rendering and resource discovery

Browsers can begin parsing before the entire response arrives. The HTML parser discovers stylesheets, scripts, images, fonts, and other resources. Some resources can block later processing; others load asynchronously. Preload scanning, cache state, priority decisions, and script attributes affect the observed order.

This is why page performance cannot be reduced to “make the HTML file small.” Latency, request count, critical resources, server response time, cache policy, image dimensions, font strategy, JavaScript execution, and rendering cost all contribute.

5. Local file, local server, public server

These addresses describe different environments:

```
file:///C:/course/week-01/index.html
http://localhost:8000/index.html
https://student.campus.example/index.html
```

- A `file:` URL lets the browser read a local file directly. There is no HTTP request to a web server.
- `localhost` names the current machine. A server program must be listening on the selected port.
- A public HTTPS URL requires public name resolution, reachable hosting, and TLS.

`localhost` usually resolves to a loopback address such as `127.0.0.1` or `::1`. Traffic sent to loopback remains on the current host. The port still matters: `http://localhost:8000/` and `http://localhost:3000/` can reach different server processes.

The IPv4 address `127.0.0.1` and the IPv6 address `::1` both identify loopback, but they belong to different Internet Protocol versions. By contrast, a server configuration that says it is listening on `0.0.0.0` means "all available local IPv4 interfaces." `0.0.0.0` is a binding instruction, not a destination students should type into a public link. Binding beyond loopback can expose a development server to other devices on the local network, so it must be a deliberate choice.

Moving from a local server to a public service changes more than the hostname. Public deployment normally requires a reachable host, firewall policy, DNS records, a valid TLS certificate, and often a reverse proxy that accepts incoming requests and forwards them to the application process. Production logs, monitoring, updates, backups, and access controls also become operational responsibilities. A page that works at `localhost` has passed an important test, but it has not yet demonstrated that it is safely deployable.

Figure 1.14 compares these environments without hiding their boundaries. In local development, the browser and development server can communicate through loopback on the same computer; the request is not travelling across the public Internet. In the public example, the DNS lookup is a separate preliminary exchange that returns address information. The later HTTPS request does not pass through DNS: it crosses interconnected routers to an HTTPS gateway, which forwards it to the application server. The database remains behind the application server rather than accepting a direct public browser connection. The response returns along the application path toward the browser.

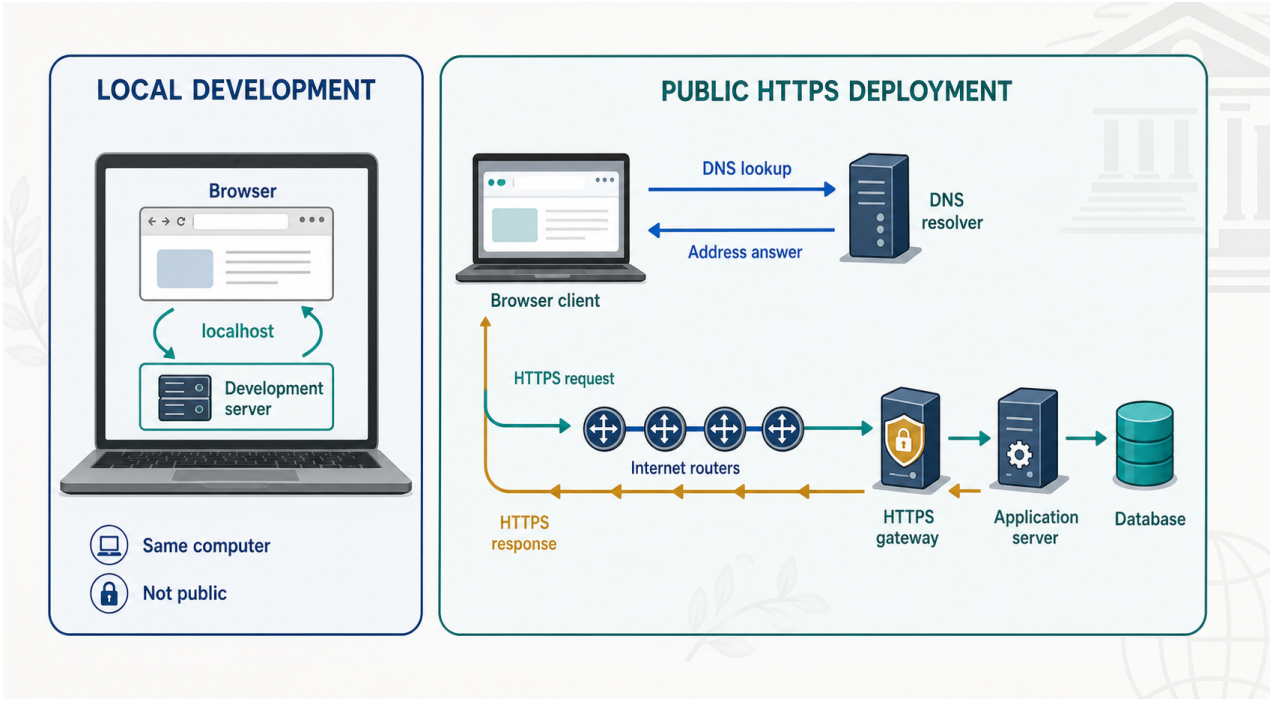


Figure 1.14. Local development keeps the browser and development server on one computer, whereas public deployment combines a separate DNS lookup with an HTTPS path through Internet routers, a gateway, an application server, and protected data storage.

A **development server** is designed to support editing and debugging. It may reload automatically, show verbose errors, and omit production hardening. A **production server** is configured for controlled deployment, security, reliability, performance, and observability. Never expose a development server to untrusted networks merely because it works locally.

Always use a local HTTP server for course work unless a lab explicitly studies `file:` behavior. Browser security rules and URL resolution can differ between local files and HTTP resources.

6. Developer tools are measuring instruments

Open the browser's developer tools and find the **Network** panel. Enable “Preserve log” if you expect a navigation or redirect to clear the list, then reload the page.

For one document request, record:

- complete request URL;
- request method;
- status code;
- remote address, if shown;
- request `Accept` value;
- response `Content-Type`;
- response `Cache-Control`, if present;
- transferred size and timing;
- whether more requests followed and what initiated them.

Evidence has limits. The panel may show a provisional header, a cached response, an extension-generated request, or a browser-specific timing breakdown. Record what the tool reports and distinguish observation from inference.

Developer tools contain several related instruments. The **Elements** panel shows the live Document Object Model (DOM) and matched styles. The **Console** reports script messages and lets a developer evaluate small expressions in the current page context. The **Network** panel records resource exchanges. The **Sources** or **Debugger** panel shows loaded files and supports breakpoints. The **Application** or **Storage** panel reveals client-side storage, cookies, caches, and service workers. The **Performance** panel records timed browser work. Names vary slightly between browsers, but the questions remain stable: what was requested, what was constructed, what code ran, what state exists, and where was time spent?

No panel is an all-seeing truth machine. A Network recording begins only when its capture is active; the Elements panel shows the live tree rather than necessarily the original source; and Console output can be filtered or produced by extensions. Good debugging records the tool, time, scope, and conditions of an observation.

A disciplined debugging loop

Use the same loop throughout the course:

1. **State the symptom precisely.** “The page is broken” is not precise. “The document returned `200`, but `logo.svg` returned `404`” is actionable.
2. **Form one hypothesis.** For example, the relative image URL resolves to the wrong path.
3. **Choose an observation that could disprove it.** Inspect the requested URL and initiator.
4. **Change one relevant factor.** Correct the path rather than changing unrelated browser settings.
5. **Repeat and record.** A fix without an explanation is fragile knowledge.

Figure 1.15 arranges these actions as a loop because the evidence from one test becomes the observation that begins the next cycle.

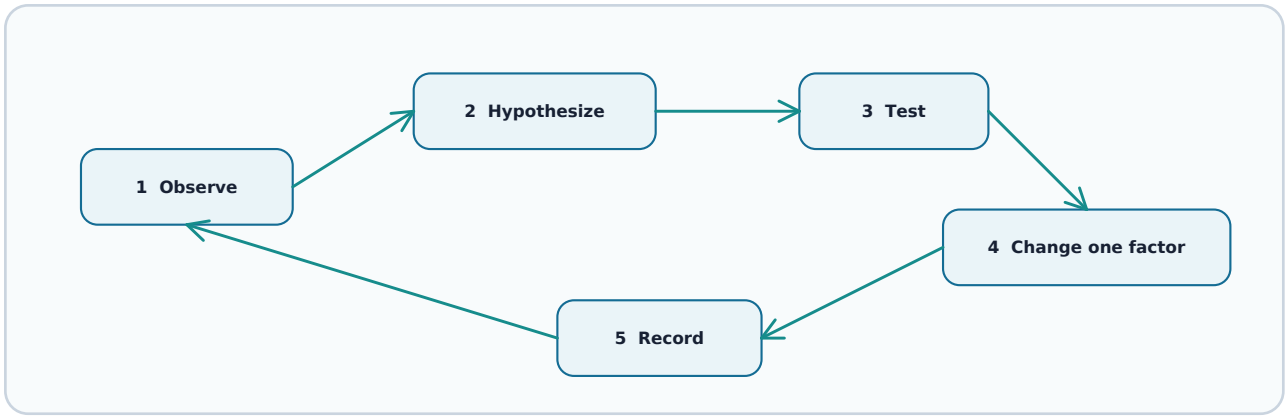


Figure 1.15. Evidence-driven debugging is a repeatable scientific loop.

7. The smallest useful HTML document

HTML describes document structure and meaning. It is the language in which most web pages are authored and the format a browser initially expects when an HTTP response declares `Content-Type: text/html`. A browser parses HTML; it does not “run” HTML as though the document were a sequence of programming instructions. During parsing, elements such as `link`, `img`, and `script` can identify other resources that the browser may fetch and use.

7.1 Elements, tags, content, and attributes

Consider:

```
<a href="/events">View events</a>
```

The whole construct is an **element**. `<a ...>` is the start tag, `</a>` is the end tag, and `View events` is the element's text content. `href` is an **attribute** whose value supplies the link destination. Informally, developers often say “tag” when they mean “element,” but the distinction is useful when reading specifications and debugging a document tree.

Some elements, such as `meta` and `img`, are **void elements** and do not have end tags or child content. Writing `<img />` is permitted syntax, but the slash does not turn HTML into XML and does not close arbitrary non-void HTML elements.

Elements can be nested to express relationships:

```
<article>
  <h2>Robotics laboratory open evening</h2>
  <p>Meet the team and explore current projects.</p>
</article>
```

The `h2` and `p` elements are children of `article`. Correct nesting matters because the parser constructs a tree, not a visual stack of opening and closing lines. Figure 1.16 makes those parent-child relationships explicit.

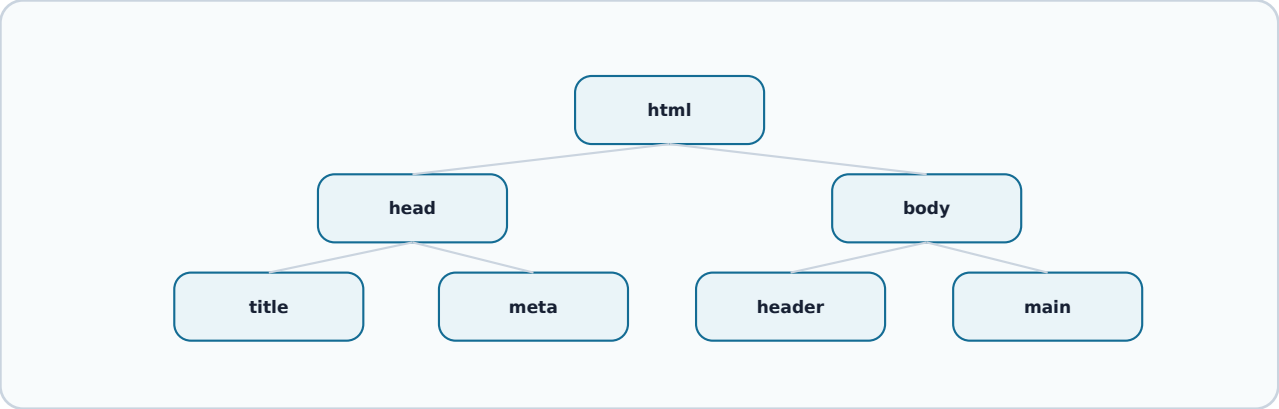


Figure 1.16. Nested HTML elements become a document tree.

7.2 HTML is semantic

Semantic HTML communicates purpose. A heading is a heading because it introduces a section, not because it happens to be large and bold. A button represents an action, not merely a rectangle with text. Meaning benefits keyboard users, assistive technology, search systems, reader modes, maintainers, and automated tools.

Choose elements for their meaning before styling them. Do not use a `div` with a click handler when a native `button` already provides activation behavior, keyboard support, and platform semantics.

7.3 Document skeleton

Session 2 studies semantics and accessibility in depth. For now, understand this complete minimum:

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Campus Hub</title>
  </head>
  <body>
    <header>
      <h1>Campus Hub</h1>
    </header>
    <main>
      <p>Find student events and communities.</p>
    </main>
  </body>
</html>
```

The code block is the complete input. Figure 1.17 shows the browser output from the larger Week 1 solution that uses the same structural ideas. Its plain appearance is intentional: HTML supplies structure and meaning, while Chapter 3 adds Cascading Style Sheets (CSS) presentation.

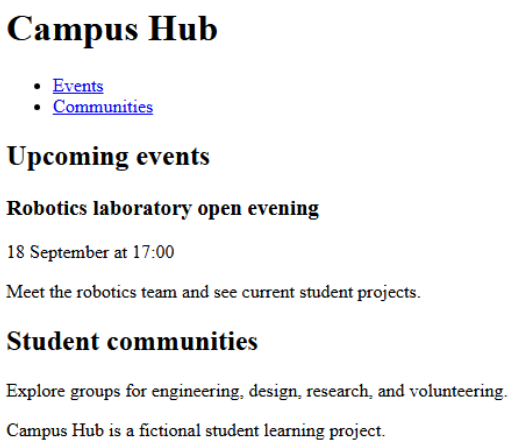


Figure 1.17. The browser's default rendering of the complete Week 1 HTML example makes the document hierarchy visible before CSS is added.

- `<!doctype html>` selects the modern HTML parsing mode.
- `lang` identifies the document language for software including assistive

technology.

- UTF-8 supports the text students need and should be declared early.
- The viewport declaration makes the layout viewport useful on mobile devices.
- `title` labels the browser tab, history entry, and bookmark.
- `h1` identifies the main heading; it is not a command to make text large.

The `head` contains document metadata and references to supporting resources. It is not the visible page header. The `body` contains the document content presented to the user. A visible site header belongs inside `body`, usually in a `header` element.

7.4 Parsing is error-tolerant, not error-free

Browsers recover from many HTML mistakes according to standardized parsing rules. That resilience is why malformed pages may appear to work. It is not permission to ignore correctness. Recovery can create a DOM different from the source a student imagined, producing inconsistent styling, scripting, and accessibility.

Use a validator and inspect the browser's Elements panel. The source file records what you wrote; the DOM records what the browser constructed.

7.5 HTML, DOM, and View Source are related but distinct

- The **HTML source** is the serialized text delivered or stored.
- The **HTML parser** interprets that text according to the HTML parsing model.
- The **DOM** is the in-memory object tree exposed to browser features and

JavaScript.

- Developer tools may show a DOM that scripts have already modified.

This distinction becomes essential when a value appears in the rendered page but not in View Source, or when invalid nesting is rearranged by the parser.

8. Git records decisions

Git is a distributed version-control system. It records snapshots and their relationships so that a team can inspect, compare, share, and recover work. Figure 1.18 distinguishes the working tree, staging area, local history, and remote repository.

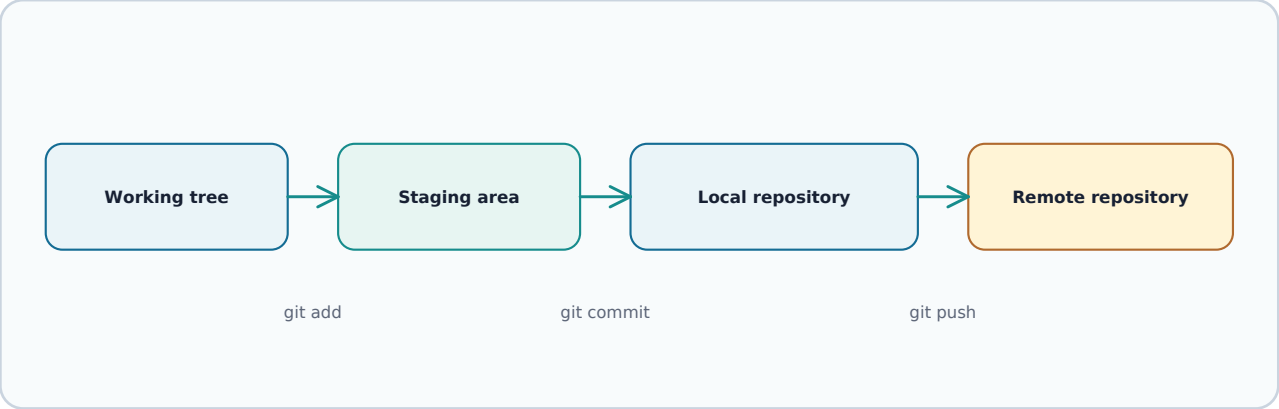


Figure 1.18. Git moves selected changes from the working tree into durable history.

Three ideas are enough for the first laboratory:

1. The **working tree** contains the files you are editing.
2. The **staging area** selects the changes intended for the next commit.
3. A **commit** records a snapshot plus metadata and a parent relationship.

```
git init
git status
git add index.html
git commit -m "Create the Campus Hub document"
```

Run `git status` before and after each operation. A commit message should state the meaningful change, not the keystrokes. “Add event navigation” is more useful than “update file.” Never commit passwords, tokens, private keys, or personal student data.

Repository, commit, branch, and remote

A **repository** stores the project's history and metadata. A **commit** is an identified snapshot with author information, time, message, and parent relationship. A **branch** is a movable name pointing into that history. A **remote** is a named reference to another repository location used for collaboration; GitHub is one possible hosting service, not Git itself.

Git works locally. You can initialize, stage, commit, inspect history, create branches, and restore earlier content without a network connection. Commands such as `push` and `fetch` exchange objects and references with another repository. Creating a local commit does not automatically upload, deploy, publish, or back up the project. Those are separate operations with separate destinations and access rules. This distinction prevents the common mistake of assuming that code is safe on a remote host merely because `git commit` succeeded.

Commit quality

A useful commit is coherent: it represents one explainable change and leaves the project in a state another developer can inspect. Commit early enough to preserve reasoning, but not after every keystroke. Before committing:

1. run the relevant checks;
2. inspect `git diff` and `git diff --staged`;

3 remove accidental files and secrets; and

4 write a message that completes the sentence “This commit will …”.

Version control is not a substitute for backups, access control, or secret management. Once a secret has been committed and shared, deleting it in a later commit does not remove it from existing history. Revoke or rotate the credential.

Guided laboratory - Observe and publish locally

Goal and acceptance criteria

In the personal repository created from the Fall 2026 coursework template, start from `labs/week-01/index.html`, create a small Campus Hub document, and serve it over HTTP. The template repository itself is read-only course material; commit and push work only to the private repository that you created from it.

Your result must:

- include the required document metadata;
- contain one `header`, one labeled `nav`, one `main`, at least two sections, and

one `footer`;

- have one clear `h1` and a logical heading order;
- contain working relative links to section fragments;
- use no inline CSS, JavaScript, framework, or copied template;
- be usable at 200% browser zoom;
- pass the supplied Week 1 check;
- have at least three meaningful commits;
- include a completed observation record.

Safety and privacy

Use fictional event names and contact details. Network panels can reveal query parameters, cookies, authorization data, and internal hostnames. Do not include secrets in screenshots or submissions.

Step 1 - Establish the baseline

Open a terminal at the root of your personal coursework repository. Prove that you are in the correct folder, then inspect the existing Git working tree:

```
Get-Item package.json
git status
```

Open `labs/week-01/index.html` in VS Code. For comparison, you may open that file directly in the browser and note the `file:` URL. In developer tools, look for an HTTP document request. You should not find one for the direct local-file load.

Complete the first TODOs, then commit:

```
git add labs/week-01/index.html
git commit -m "Create the Campus Hub document"
```

Checkpoint A: `git log --oneline` shows one commit, and the tab has a useful title.

Step 2 - Add navigable content

Add a header and navigation, then two main sections with IDs such as `events` and `communities`. Each navigation link should use a relative fragment reference, such as `href="#events"`.

Add a footer with a short statement that this is a student learning project. Commit again:

```
git add labs/week-01/index.html
git commit -m "Add Campus Hub navigation and sections"
```

Checkpoint B: selecting each link changes the URL fragment and moves focus or the viewport to the intended section. No server request is needed merely to change a fragment within the loaded document.

Step 3 - Serve through HTTP

From the root of your personal coursework repository, run the supplied dependency-free server at port 8000:

```
npm run serve:week01
```

The npm script serves `labs/week-01`. The server binds only to the local loopback address and does not download a package.

Navigate to `http://localhost:8000/`. Keep the server terminal open.

Checkpoint C: the Network panel shows a document request using HTTP. Record the method, status, request URL, and response `Content-Type`.

Step 3A - Observe name resolution

`localhost` normally resolves without contacting the public DNS hierarchy because the operating system treats it as a loopback name. Confirm the result from a PowerShell terminal:

```
Resolve-DnsName localhost
```

On systems without `Resolve-DnsName`, use `nslookup localhost`. The command may show `127.0.0.1`, the IPv6 loopback address `::1`, or both. These addresses mean “this computer”; they do not identify another machine on the Internet.

Now inspect a public teaching-safe name:

```
Resolve-DnsName example.com
```

Record the returned record type and address. A result obtained from a cache is still valid evidence of resolution; DNS caching is part of the system, not a shortcut around it. Do not assume that the displayed address is permanent. A domain can legitimately resolve to different addresses over time or from different networks.

Checkpoint D: explain, in your own words, why successful DNS resolution does not prove that an HTTP server is running at the returned address.

Step 4 - Cause and diagnose a failure

Navigate to `http://localhost:8000/missing.html`.

Before looking at the network entry, predict the status. Then record the actual status and response body. Explain why seeing an error page does not imply that no HTTP response arrived.

Return to the working page, correct any issues, and commit:

```
git add labs/week-01/index.html
git commit -m "Complete the Week 1 HTTP observation"
```

Step 5 - Verify

In your personal coursework repository, run:

```
npm test
```

The automated check confirms the supplied project structure, Node.js version, starter documents, and local HTTP servers. It does not grade the completed Week 1 HTML. Compare your document with every acceptance criterion above and with the worked solution in Appendix A, but only after you have completed and committed your own attempt.

Observation record

Use Table 1.7 during the laboratory to separate what the tools displayed from what you concluded from that evidence.

Table 1.7 - Laboratory observation record

Observation	Your evidence
Direct-file URL	
Local-server URL	
Request method	
Successful status	
Missing-resource status	
Response `Content-Type`	
One request header	
One response header	
`localhost` address information	
Public DNS record type and value	
Number of requests on initial load	
Observation vs inference	“I observed …; I infer … because …”

Cleanup

Stop the local server with `Ctrl+C`. Confirm with `git status` that all intended work is committed and no secret or unrelated file is staged.

Exercises

Attempt all exercises before opening the answers.

Foundation

Exercise 1 - URL anatomy

For this URL, identify every component and state which component is not sent in the HTTP request:

```
https://library.campus.example:9443/search/books?q=distributed+systems#results
```

Exercise 2 - Order the events

Put these simplified events in a valid causal order:

- browser parses HTML;
- DNS resolver returns address information;
- user enters a URL;
- server sends an HTTP response;
- browser sends an HTTP request;
- browser discovers an image URL.

Exercise 3 - Read the response

Interpret this status line and the two fields. What can and cannot be concluded?

```
HTTP/1.1 304 Not Modified
Cache-Control: max-age=120
ETag: "events-v7"
```

Application

Exercise 4 - Diagnose three observations

For each case, give the most useful next inspection:

1. `file:///.../index.html` displays, but no document request appears in Network.
2. `http://localhost:8000/` reports that the connection was refused.
3. The document returns `200`, but one image returns `404`.

Exercise 5 - Draw the boundary

A student says, “The fragment is private because HTTPS encrypts it before sending it to the server.” Correct the statement precisely.

Challenge

Exercise 6 - One page, multiple requests

Suppose `index.html` references one stylesheet, two images, and one script. With empty caches and no inlining, what is the minimum expected number of HTTP resource requests for a normal load? Why might the actual number differ?

Exercise 7 - Git evidence

Explain why the following history is weak evidence of an engineering process, and propose a better three-commit history:

```
a13f99c final
```

Exercise 8 - Follow a DNS answer

A recursive resolver already has these cached records:

```
www.department.example.  CNAME  web-host.example.net.  TTL 240
web-host.example.net.     A      192.0.2.40             TTL 45
```

What must the resolver return so that an IPv4 client can attempt a connection? Which cached fact can become stale first, and what is the earliest stated time at which that can happen? Explain why the answer does not prove that a website is healthy.

Exercise 9 - Source HTML and DOM

Consider this invalid source:

```
<p>First paragraph
<p>Second paragraph
```

A browser's Elements panel shows two separate `p` elements, even though neither closing tag appears in the source. Has the browser changed the source file? What has happened, and which view should be used to inspect each representation?

Exercise 10 - Method properties

Classify each operation as safe or unsafe and as idempotent or non-idempotent. Justify the classification from the intended semantics, not from a particular implementation.

1. `GET /courses/42`
2. `PUT /students/7/preferences` with a complete replacement document
3. `POST /registrations` to create a new registration
4. `DELETE /registrations/91`

Exercise 11 - Compare origins

For the document at `https://portal.campus.example/index.html`, state whether each URL has the same origin. Explain every answer using the scheme-host-port tuple.

1. `https://portal.campus.example/help`
2. `http://portal.campus.example/help`
3. `https://api.campus.example/help`
4. `https://portal.campus.example:443/help`
5. `https://portal.campus.example:8443/help`

Exercise 12 - Diagnose by layer

A browser reports `DNS_PROBE_FINISHED_NXDOMAIN` for one URL. A second URL resolves but reports `ERR_CONNECTION_REFUSED`. A third establishes HTTPS and receives `HTTP 503 Service Unavailable`. Identify the failing or responding layer in each case and name one productive next check. Explain why treating all three as “the Internet is down” would obstruct diagnosis.

Exercise 13 - Expand, compress, and interpret IPv6

Answer all parts and explain each transformation.

1. Expand `2001:db8:12::40` to eight groups of four hexadecimal digits.
2. Write `2001:0db8:0000:0000:0000:00ab:0000:0001` in a shorter valid form.
3. Explain why `2001::db8::1` is invalid.
4. In `https://[2001:db8::40]:8443/status`, identify the host, port, and path.
5. What does `/64` mean in `2001:db8:12:34::/64`, and does it mean that the address is using Transmission Control Protocol (TCP) port 64?

Worked answers

Answer 1

- Scheme: `https`
- Host: `library.campus.example`
- Port: `9443`
- Path: `/search/books`
- Query: `q=distributed+systems`
- Fragment: `results`

The fragment is not transmitted as part of the HTTP request target. The browser uses it locally after obtaining the resource. Query data is transmitted and must not be treated as secret.

Answer 2

One valid order is:

1. user enters a URL;
2. DNS resolver returns address information;
3. browser sends an HTTP request;
4. server sends an HTTP response;
5. browser parses HTML;
6. browser discovers an image URL.

The browser can then issue another request for the image. Real implementations can perform work concurrently, reuse connections and cached DNS answers, or begin processing a streamed response before all bytes arrive. The exercise expresses causality, not a complete implementation schedule.

Answer 3

`304 Not Modified` means the client made a conditional request and the server says the stored representation can be reused; the response does not contain a new representation body for that target. `Cache-Control: max-age=120` expresses cache freshness information, and the entity tag is a validator associated with a representation.

You can conclude neither that “nothing happened” nor that the resource is globally unchanged forever. You also cannot infer page correctness from the HTTP status.

Answer 4

1. Nothing is necessarily wrong. A `file:` load does not make an HTTP document request. Inspect the address bar, then serve the directory locally and compare.
2. Check that a server process is running and listening on port 8000; then check the terminal for startup errors and confirm the URL/port.
3. Select the failed image request and inspect its requested URL and initiator; then compare the HTML reference, path resolution, filename case, and server files.

Answer 5

The fragment is not sent in the HTTP request at all, so its absence at the server is not caused by HTTPS encryption. The browser retains and interprets it. However, fragments can still appear in browser history, screenshots, copied links, client logs, or client-side code; “not sent as the request target” does not mean “safe for secrets.”

Answer 6

The minimum is five: one HTML document, one stylesheet, two images, and one script. The actual number may differ because of redirects, CSS-referenced assets, fonts, source maps, favicons, script-initiated requests, service workers, browser extensions, preloads, errors, caching, or resources embedded directly as data.

Answer 7

A single “final” commit conceals intermediate decisions, makes review difficult, and offers few useful recovery points. A better history could be:

```
8ac501e Complete HTTP observation notes
340de18 Add Campus Hub navigation and sections
bd102aa Create the Campus Hub document
```

The exact hashes differ. The improvement is a sequence of coherent, reviewable changes with messages that communicate intent.

Answer 8

The resolver must follow the alias and provide the address associated with its canonical target: `www.department.example` is an alias for `web-host.example.net`, whose IPv4 address is `192.0.2.40`. A complete response can include the CNAME and the A record so the client can understand both the alias relationship and the usable address.

The A record can expire first because its remaining TTL is 45 seconds, compared with 240 seconds for the CNAME. After 45 seconds the resolver must no longer rely on that cached address without refreshing it; the alias relationship can remain cached longer. TTL is a cache lifetime, not a promise that a server will remain reachable for that entire period. DNS establishes naming information only. The host could be offline, a firewall could reject the connection, TLS could fail, or the HTTP application could return an error.

Answer 9

The browser has not edited the source file. Its HTML parser applied the HTML parsing rules, which implicitly close the first paragraph when the second `<p>` start tag is encountered. The resulting in-memory Document Object Model therefore contains two paragraph nodes. “View Source” or the original editor shows the bytes that were delivered or saved; the Elements panel shows the live DOM after parsing and after any JavaScript changes. The distinction matters because valid browser recovery does not make invalid source good engineering practice. Validate and correct the source even when the displayed result looks acceptable.

Answer 10

1. `GET /courses/42` is safe and idempotent. Its defined purpose is retrieval, not a requested state change, and repeating the same retrieval has the same intended effect. Logging and access counters may change internally without changing the method's semantics.
2. `PUT` with a complete replacement is unsafe but idempotent. It intentionally changes server state, yet sending the identical replacement once or several times has the same intended final state.
3. `POST` to create a registration is unsafe and generally non-idempotent. Repeating it may create multiple registrations unless the application adds an idempotency mechanism.
4. `DELETE` is unsafe but idempotent. The first successful request removes the target; repetitions may return different statuses, but the intended final state-no such registration-remains the same.

“Idempotent” does not mean that every response is byte-for-byte identical. It describes the intended server-side effect of repeating the request.

Answer 11

The reference origin is `(https, portal.campus.example, 443)` because HTTPS uses port 443 when no port is written.

1. Same origin: all three components match.
2. Different origin: the scheme is `http`, whose default port is also 80 rather than 443.
3. Different origin: `api.campus.example` is a different host, even though it may belong to the same organization.
4. Same origin: explicitly writing the default HTTPS port does not change the effective port.
5. Different origin: port 8443 differs from port 443.

Paths do not participate in the origin tuple. Two resources can have different paths and still share an origin.

Answer 12

`DNS_PROBE_FINISHED_NXDOMAIN` indicates that name resolution reported that the domain name does not exist. Check the spelling, the configured name, and then the resolver's answer. An IP connection has not yet been attempted.

`ERR_CONNECTION_REFUSED` means address information was available and the client reached the connection stage, but no service accepted the connection at that address and port (or a device actively rejected it). Check whether the server is running and listening on the expected interface and port.

`503 Service Unavailable` is an HTTP response. DNS, connection establishment, and HTTPS succeeded far enough for an HTTP server or intermediary to answer. Inspect response fields, server logs, health checks, capacity, and upstream dependencies.

Calling every condition “the Internet is down” discards the strongest evidence: each message locates progress at a different layer. Layer-aware diagnosis narrows the search and avoids changing unrelated systems.

Answer 13

1. The expanded address is `2001:0db8:0012:0000:0000:0000:0000:0040`. Four groups are visible before expansion (`2001`, `db8`, `12`, and `40`), so `::` must supply four all-zero groups. Leading zeroes are then restored within every group.
2. One short form is `2001:db8::ab:0:1`. Leading zeroes are removed from each group, and the longest run of consecutive all-zero groups is replaced once by `::`. The remaining isolated zero group stays as `0`.
3. The address contains `::` twice. Each double colon would claim an unknown number of omitted zero groups, so the expansion to exactly eight groups would be ambiguous.
4. The square brackets delimit the IPv6 host `2001:db8::40`. The port is `8443`, written after the closing bracket, and the path is `/status`. The brackets and port are URL syntax, not bits in the IPv6 address.
5. `/64` is an IPv6 prefix length: the first 64 address bits identify the network prefix for routing and subnet membership. It has no relationship to a transport-layer port. A port is written separately in a URL or socket endpoint.

Debugging clinic

A student starts the server in `C:\course` but stores the file at `C:\course\week-01\index.html`. They navigate to `http://localhost:8000/` and see a directory listing instead of Campus Hub.

Diagnosis

The server root is `C:\course`, so `/` names that directory. The HTML document is at `/week-01/index.html`. Either navigate to that path or stop the server and restart it from `C:\course\week-01`. Renaming the network, clearing browser data, or editing DNS would not address the observed mapping.

Retrieval quiz

Write answers without looking back. Then use the explanations to correct your mental model.

1. In one sentence, distinguish the Internet from the Web.
2. Which URL component is normally interpreted only by the client: query or fragment?
3. What does DNS contribute to a normal navigation?
4. Does HTTPS guarantee that a website is honest and bug-free?
5. What does an HTTP method communicate?
6. A missing page returns visible HTML with status `404`. Did the server respond?
7. Why can one visible page cause many HTTP requests?
8. What is the difference between the Git working tree and staging area?
9. Name one observation that distinguishes an HTTP load from a `file:` load.
10. Why should a developer record both the observation and the inference?
11. What is the difference between HTML and the DOM?
12. Why can a DNS record have a TTL?
13. Which three values define a web origin?
14. What does it mean to say that HTTP is stateless?
15. Why does a `200 OK` status not prove that a page is correct?
16. How many groups appear in a fully expanded IPv6 address, and what does `::` mean in a compressed form?

Quiz answers and misconception notes

1. The Internet is the underlying network of networks; the Web is an application using that network to identify and exchange linked resources. They are related, not synonyms.
2. Fragment. A query is part of the request target; a fragment is retained by the client.
3. It resolves a hostname to address information needed to contact a service. DNS does not fetch or render the page.
4. No. HTTPS protects the connection and authenticates the certified server identity; it does not certify application truthfulness or correctness.
5. The request's intended semantics, such as retrieving a representation with `GET`.
6. Yes. The error document is response content and `404` is the response status.
7. Parsing the document reveals references to stylesheets, scripts, images, fonts, and data that can require separate retrieval.
8. The working tree contains current file content; the staging area selects the content intended for the next commit.
9. An HTTP load has an `http:` or `https:` URL and a document exchange with method, status, and HTTP fields in Network; a direct `file:` load does not.
10. Tools show evidence, while explanations add assumptions. Separating them makes reasoning reviewable and prevents a plausible story from being mistaken for a measurement.

11 HTML is the markup language and source syntax used to describe a document; the DOM is the browser's live, in-memory tree produced by parsing that markup. The DOM may also reflect parser repairs and later JavaScript changes, so it is not necessarily a character-for-character copy of the source.

12 A TTL tells resolvers how long they may reuse a cached DNS record before they must obtain fresher information. Caching reduces delay and DNS traffic, while expiration permits authoritative data to change. A TTL is not a guarantee that the addressed service remains available.

13 Scheme, host, and effective port define an origin. The path, query, and fragment do not. This precise tuple is important because browsers use origins as a major security boundary.

14 HTTP does not inherently remember a conversation between independent requests. Applications build continuity explicitly with mechanisms such as cookies, authorization credentials, or tokens; “stateless” does not mean the server cannot store data.

15 `200 OK` reports that the server successfully handled the HTTP request under its semantics. The body can still contain incorrect data, inaccessible HTML, a broken user interface, or even a human-readable error mistakenly served with status 200. Protocol success and application correctness are different claims.

16 A fully expanded IPv6 address has eight groups, each representing 16 bits with four hexadecimal digits. One `::` replaces enough consecutive all-zero groups to bring the expanded address to eight groups. It can appear at most once because two unknown-length omissions would be ambiguous.

Self-check scale

- **14-16:** ready for Session 2;
- **10-13:** revisit IP addressing, URL, DNS, and HTTP, then explain the sequence aloud;
- **0-9:** repeat the Network lab with a peer or teaching assistant before moving on.

Session summary

- The Web is an application on the Internet.
- Clients initiate requests; servers produce responses. One machine can play both roles.
- URLs are structured identifiers, not unstructured strings.
- An origin combines a URL's scheme, hostname, and effective port; browsers use that combination as a security boundary.
- IPv4 uses four decimal octets for a 32-bit address. IPv6 uses eight hexadecimal groups for a 128-bit address and permits carefully defined compression.
- DNS resolution, connection setup, HTTP exchange, and browser processing are distinct responsibilities.
- HTTP communicates request intent and response outcomes through standardized semantics.
- HTTPS protects data in transit but does not make an application trustworthy.

- One page commonly depends on many resources and therefore many requests.
- `file:`, `localhost`, and public HTTPS environments are not interchangeable.
- Developer tools provide evidence; Git records meaningful snapshots and intent.

Glossary

Table 1.8 consolidates the chapter's working vocabulary for revision and classroom retrieval practice.

Table 1.8 - Session 1 glossary

Term	Working definition
API	An application programming interface: a defined way for software components to request operations or exchange data. A web API commonly exposes resources through HTTP.
Browser	A web client that retrieves, interprets, and presents web resources.
Client	Software that initiates a request for a service.
CSS	Cascading Style Sheets, the language used to describe the presentation and layout of structured documents.
DNS	A distributed naming system used, among other purposes, to obtain address information for domain names.
DOM	The Document Object Model: the live tree of nodes that a browser constructs for a document and exposes to scripts.
Domain name	A hierarchical human-readable name, such as the fictional `www.campus.example`, whose DNS records can describe addresses and other services.
Fragment	A client-side secondary reference after # in a URL.
HTML	HyperText Markup Language, the language that gives a web document its structure and meaning.
HTTP	A stateless application-level protocol for exchanging messages about resources.
HTTPS	HTTP communicated over an authenticated, encrypted transport channel.
Internet	The global network of interconnected networks using the Internet protocol family.
IP address	A numerical identifier used by the Internet Protocol to address an interface and route packets, such as `192.0.2.40` or `2001:db8::40`.
IPv4	Internet Protocol version 4, which uses 32-bit addresses commonly written as four decimal octets.
IPv6	Internet Protocol version 6, which uses 128-bit addresses commonly written as eight hexadecimal groups with defined shortening rules.
JavaScript	A programming language used in browsers and many server environments to express behavior and application logic.
Localhost	A name for the current host's loopback interface.
Origin	The scheme, host, and effective port considered together; browsers use this tuple as an important security boundary.
Packet	A bounded unit of data carried through a packet-switched network, with control information needed for delivery.
Port	A transport-layer number used to direct traffic to a particular service on a host.
Recursive resolver	A DNS service that performs or coordinates lookups for a client and caches answers according to their TTLs.
Representation	A particular transferable form of a resource, such as an HTML document or a JSON description.
Resource	Something identified by a URL and acted upon through web protocols.
Router	A network device or software function that forwards packets between networks according to routing information.
Server	Software that listens for requests and provides responses.
TLS	Transport Layer Security, the protocol that provides authentication, confidentiality, and integrity for HTTPS connections.
TTL	Time to live; in DNS, the period for which a record may be reused from a cache before it must be refreshed.

Term	Working definition
URL	A structured identifier used to locate and access a resource.
Web	A linked information and application system built on Internet technologies.

Primary references and further reading

1 WHATWG, HTML Living Standard - Web developer edition.

2 WHATWG, URL Living Standard.

3 WHATWG, Fetch Standard. The CORS sections define how HTTP responses may be shared across origins.

4 MDN, Same-origin policy. This guide gives practical examples of same-origin and cross-origin browser interactions.

5 IETF, RFC 9110: HTTP Semantics.

6 IETF, RFC 1034: Domain Names-Concepts and Facilities.

7 IETF, RFC 3596: DNS Extensions to Support IP Version 6.

8 IETF, RFC 8200: Internet Protocol, Version 6 (IPv6) Specification.

9 IETF, RFC 4291: IP Version 6 Addressing Architecture.

10 IETF, RFC 5952: A Recommendation for IPv6 Address Text Representation.

11 IETF, RFC 9846: The Transport Layer Security (TLS) Protocol Version 1.3. This July 2026 specification obsoletes RFC 8446.

12 IETF, RFC 9000: QUIC-A UDP-Based Multiplexed and Secure Transport.

13 IETF, RFC 9114: HTTP/3.

14 W3C, CSS Snapshot 2026.

15 MDN, How the web works.

16 Chacon and Straub, Pro Git, 2nd edition.

17 IANA, Example Domains. This explains why reserved example names are suitable for documentation but should not be treated as dependable web services.

18 IETF, RFC 2606: Reserved Top Level DNS Names. This standard reserves `.example` for documentation and illustrative examples.

What changes often?

The conceptual roles and HTTP semantics in this chapter are stable. Browser developer-tool labels, exact timing panels, installation screens, and command output change more often. The technical review date for this chapter is **29 September 2026**.

Session reflection

- Which prediction from the beginning of the session changed, and what evidence

changed it?

- Which distinction-DNS versus HTTP, client versus server, or `file:` versus

HTTP-would you now be most confident explaining to another student?

- What remains uncertain, and which observation could reduce that uncertainty?

Learning journal notes

Use the space below for your learning journal.

Session 2 - Semantic HTML and Accessible Forms

Essential question: How can one HyperText Markup Language (HTML) document remain understandable to people, browsers, search engines, and assistive technology before any Cascading Style Sheets (CSS) or JavaScript is added?

Session map

Table 2.1 summarizes the preparation, teaching time, expected artifact, and evidence for this session.

Table 2.1 - Session scope, preparation, and evidence

Item	Plan
Audience	Students who completed Session 1; no prior accessibility experience is assumed
Preparation	Complete the workstation orientation; this session repeats the edit-save-serve-reload workflow before introducing HTML concepts
Contact time	Approximately 3 hours lecture/workshop plus a 2-hour laboratory
Independent work	Approximately 3 hours
Main artifact	An accessible event-submission page for Campus Hub
Assessment	Validator evidence, keyboard test, semantic rationale, exercises, and quiz

Learning outcomes

After this session, you should be able to:

1. locate the Campus Hub HTML source, edit and save it in a code editor, serve the correct directory, and verify the rendered output in a browser;
2. distinguish an element, tag, attribute, content, void element, Boolean attribute, and character reference in conforming HTML syntax;
3. explain why HTML communicates meaning rather than merely arranging boxes;
4. establish document language, direction, title, encoding, description, and viewport metadata;
5. choose headings and landmark elements from the document's information structure;
6. write useful text, links, images, lists, tables, dates, and media alternatives;
7. construct a form whose controls have names, labels, descriptions, groups, and appropriate native types;
8. predict which controls and values a form will submit with `get` or `post`, including the encoding required for a file upload;
9. distinguish native browser constraint validation from complete application validation;
10. design labels, instructions, error summaries, and field-level errors that support recovery;
11. describe how source HTML contributes to the DOM and accessibility tree;
12. operate and inspect a page using only a keyboard; and
13. combine conformance, automated, keyboard, zoom, assistive-technology, and user evidence when evaluating accessibility.

Before the concepts: find, edit, and view Campus Hub

HTML does not require a special visual design program. An HTML document is a plain-text file whose name normally ends in `.html`. You write the source in a text editor, save the file, and ask a browser to load it. The browser reads the markup and produces the visible page. The editor and browser therefore show two views of the same work: the editor shows instructions; the browser shows the result of interpreting those instructions.

Find the correct Campus Hub file

Your personal repository, created from the Fall 2026 coursework template, contains one student working file for each laboratory. In Session 2, edit `labs/week-02/index.html`. It already contains the outer document and the heading **Submit a campus event**, but its comments mark work that has not yet been completed. The complete reference answer appears in Appendix A of this book; it is intentionally absent from the private student repository.

The repeated filename `index.html` is intentional. Web servers commonly treat `index.html` as the default document for a directory. The folders distinguish the files. A tab that says only `index.html` is not enough evidence; read its folder path in the Explorer panel or editor breadcrumb.

The PDF you are reading is not the website source. Editing the PDF cannot change the Campus Hub page. The source to edit in this session is:

```
web-programming-coursework-2026_Fall-YOUR_STUDENT_NUMBER
  labs
    week-02
      index.html       <- edit this file
```

Use a plain-text code editor

Any plain-text editor can write HTML, including Notepad, but this course uses Visual Studio Code (VS Code) because it shows folders, line numbers, syntax coloring, an integrated terminal, and useful error indicators. Do not use a word processor such as Microsoft Word: a word processor can add formatting data that is not HTML source.

The tools have distinct jobs:

- **VS Code Explorer** shows the repository's folders and files.
- **VS Code editor** is where you type and save HTML source.
- **Terminal** runs the local server and other development commands.
- **Browser** requests the saved file and renders the page.
- **Browser developer tools** reveal the request, parsed Document Object Model

(DOM), accessibility information, and other evidence.

Open the project and the correct file step by step:

1. Start VS Code.
2. Choose **File > Open Folder**.
3. Select your outer `web-programming-coursework-2026_Fall-...` folder, the one containing `package.json`, `labs`, and `scripts`.
4. In the Explorer panel, expand `labs`, then `week-02`.
5. Select `index.html`. Confirm that the breadcrumb or path includes `labs > week-02 > index.html`.
6. If the editor shows a preview rather than editable text, right-click the file, choose **Open With**, and select **Text Editor**.

Make one visible change and save it

Find this heading in the starter:

```
<h1>Submit a campus event</h1>
```

On the next line, add a paragraph:

```
<p>This page is being edited in Visual Studio Code.</p>
```

The opening tag `<p>` begins a paragraph, the sentence is its text content, and the closing tag `</p>` ends it. Indent the paragraph to the same level as the nearby heading because both are children of `main`. Indentation helps people read the source; the tags establish the actual structure.

Press `Ctrl+S` on Windows or Linux, or `Command+S` on macOS. The small unsaved indicator on the editor tab should disappear. Saving writes the change to disk; it does not automatically reload every browser.

Start the local server and open the browser output

Open **Terminal > New Terminal** in VS Code. The prompt must be at the repository root. On Windows Command Prompt, verify the location with:

```
dir package.json
npm run serve:week02
```

On PowerShell, `Get-Item package.json` provides the same proof. On macOS or Linux, use `ls -l package.json` before the npm command. A successful server prints lines with this meaning:

```
Serving ...\labs\week-02
Open http://localhost:8000/
Press Ctrl+C to stop.
```

Leave that terminal running. It is now the server terminal and will not display a new prompt until the server stops. Open a browser, put the following plain address in the address bar, and press Enter:

```
http://localhost:8000/
```

Do not type the address into the terminal. In the browser, the starter should show **Submit a campus event** and the paragraph you added. The page looks plain because Session 2 intentionally uses no Cascading Style Sheets (CSS). Plain presentation is not a failure; it lets you inspect the HTML structure before styling is introduced.

Each time you change the source, repeat this short cycle:

1. edit `labs/week-02/index.html` in VS Code;
2. save the file;
3. return to the browser; and
4. reload with `Ctrl+R`, `F5`, or the browser's reload button.

Figure 2.1 shows the complete path. The arrows represent steps used together, not alternative programs: VS Code saves the file, the Node.js server reads it when the browser sends an HTTP request, and the browser parses and renders the response.

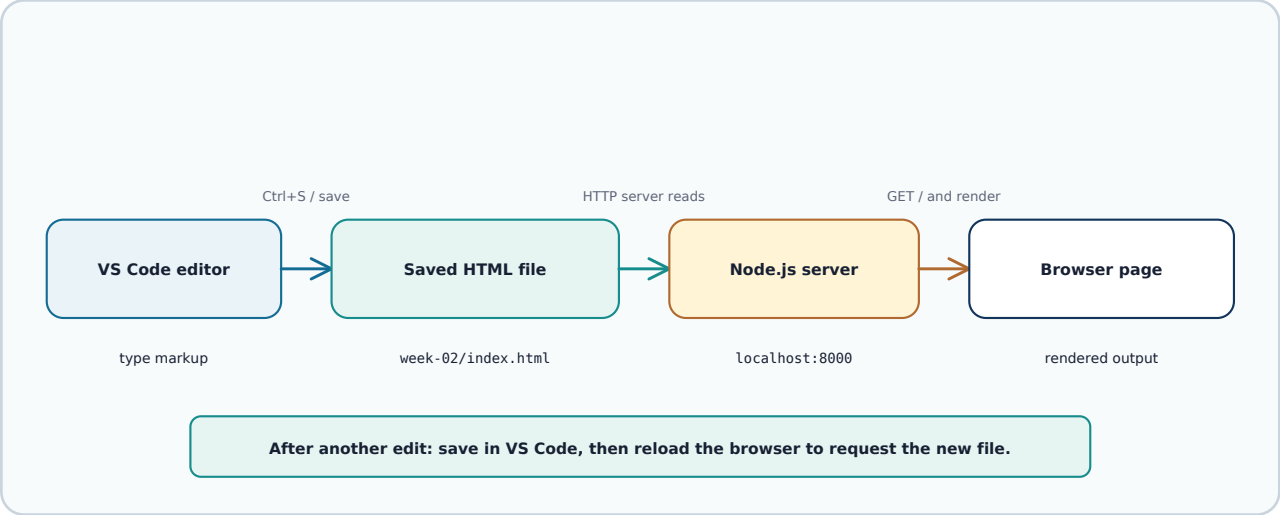


Figure 2.1. Editing, saving, serving, requesting, and rendering are distinct steps in the Campus Hub feedback loop.

The supplied server does not provide automatic live reload. If the old page remains visible, first confirm that the editor tab is saved, then reload the browser. Also confirm that the server's `Serving` line ends in `labs\week-02`. If you edit a different copy of `index.html`, reloading this address cannot display that other file.

A real browser screenshot later in this session shows the completed Week 2 solution. Your starter will contain fewer controls until you build them during the laboratory, so do not expect it to match that completed page immediately.

Read the evidence when the result differs

A browser page and an error message are different kinds of evidence:

- If **Submit a campus event** appears, the browser reached the correct starter.
- If the page loads with status `200 OK` but your new paragraph is absent, the

likely causes are an unsaved editor tab, a missing reload, or editing a different `index.html`.

- If the browser shows an almost empty page, open developer tools, inspect the

`body`, and read the server's `Serving` path. You may be serving an intentionally incomplete starter from another week.

- If the browser says **This site can't be reached**, no server is responding at

that address and port. Return to the server terminal.

- If the browser receives `404 Not Found`, the server responded, but the requested

path did not identify an available file.

The browser's **Elements** or **Inspector** panel shows the parsed DOM, not an editable replacement for the saved source file. Temporary edits made there are useful experiments, but they disappear on reload. Make durable corrections in the VS Code file and save them.

1. Begin with the user task

Imagine an event form that looks polished. Each field is represented by a pale rectangle. Placeholder text says “Name,” “Date,” and “Email.” Clicking the final rectangle submits the form. A mouse user with good vision may complete it. A keyboard user cannot see where focus has moved. A screen-reader user hears “edit text, blank” three times. When a value is rejected, the border becomes red but no message explains why.

The page has a visual arrangement, but it does not yet have a reliable interface. Accessibility is not an optional decoration for a completed design. It is the quality of the relationships, names, states, and interactions that the system offers to different users and technologies.

Before reading further, write a short prediction:

- Which information would remain if all CSS disappeared?
- Which operations can be completed without a mouse?
- What name would assistive technology announce for each control?
- How would a person know why submission failed?

These questions turn accessibility from a vague virtue into observable engineering behavior.

2. HTML is a language of meaning

HTML elements describe what content is and how pieces relate. A heading introduces a section. A list groups related items. A button represents an action. A link navigates to a resource. A form control accepts input. Browsers attach behavior and accessibility semantics to these elements.

It is technically possible to make a `div` look like a button. Appearance does not give it keyboard behavior, an accessible role, a disabled state, or activation semantics. Recreating all of that correctly is unnecessary when the native `button` element already expresses the concept.

```
<button type="submit">Publish event</button>
```

The principle is simple: use the element whose meaning matches the task. Use generic containers such as `div` and `span` when no more specific element applies. Figure 2.2 shows how semantic landmark elements identify regions by purpose.

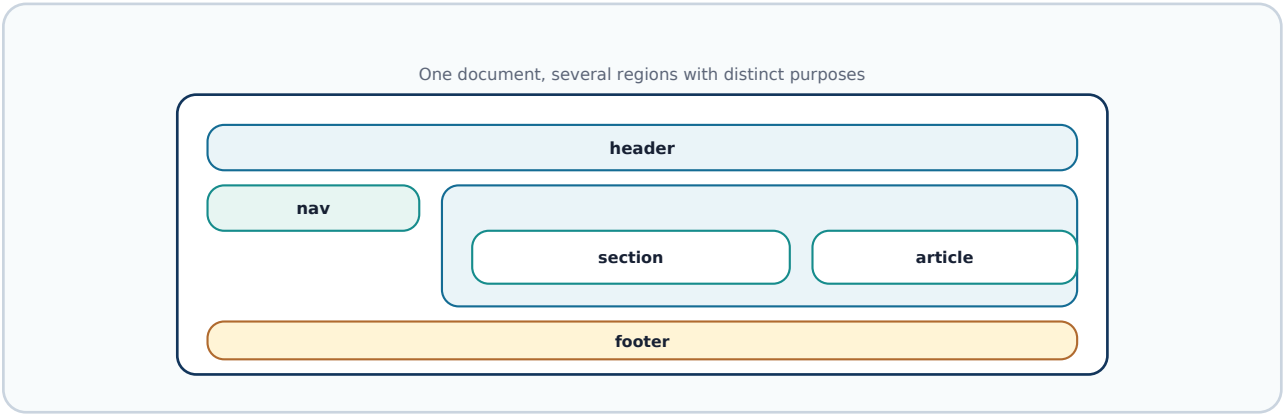


Figure 2.2. Semantic landmarks divide a page into named, navigable regions.

The same semantic document should not become a separate “accessible version.” As Figure 2.3 illustrates, its structure can support visual presentation, keyboard operation, assistive technology, and different viewport sizes from one shared source.

Figure 2.3. One semantic document can support visual browsing, keyboard operation, assistive technology, and different viewport sizes.

2.1 Read the syntax before choosing the element

HTML is written as text, but its punctuation has grammatical meaning. Consider one complete anchor element:

```
<a class="event-link" href="/events/robotics">Robotics event</a>
```

The opening tag begins with `<a` and ends with `>`. The closing tag is `</a>`. Together with the text between them, they form the **element**. `class` and `href` are **attributes** in the opening tag. Each attribute has a name and a quoted value. “Robotics event” is the element's text content. A tag is markup at an element boundary; it is not a synonym for the complete element.

Figure 2.4 labels these parts and shows their contribution to the parsed element. The browser does not display the tags as prose. It parses them into a Document Object Model (DOM) node with properties, content, and relationships.

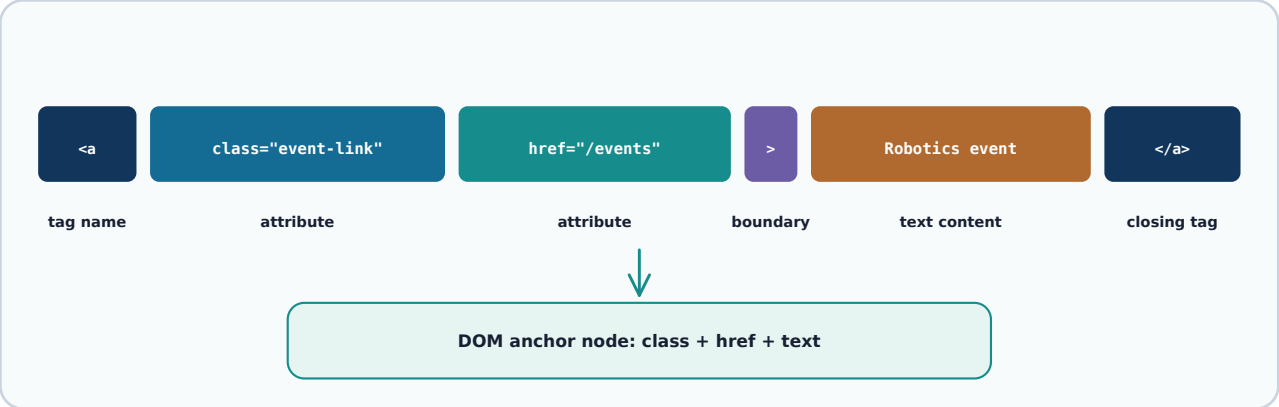


Figure 2.4. An HTML element combines tags, attributes, content, and a parsed DOM representation.

Most elements have an opening tag, content, and a closing tag. Elements must be properly nested: close the most recently opened child before closing its parent.

```
<!-- Conforming nesting -->
<p>Read the <strong>important guidance</strong> first.</p>

<!-- Overlapping elements: do not write this -->
<p>Read the <strong>important guidance</p></strong>
```

Browsers apply error-recovery rules to malformed HTML, but recovery is not proof that the source expresses the intended tree. A different error or context can produce a different DOM. Write conforming source, validate it, and inspect the constructed DOM when the result is surprising.

Void elements cannot contain child content and do not have closing tags. Common examples are `img`, `input`, `meta`, `link`, `br`, `hr`, and `source`. Write `<img src="map.png" alt="Campus map">`, not `<img>Campus map</img>`. A trailing slash in `<img ... />` is tolerated in HTML syntax but does not close the element in the XML sense and is not required.

An attribute normally has a quoted value. Consistent double quotes prevent spaces or punctuation from being misread as new attributes. A **Boolean attribute** is true when present and false when absent. Therefore, `required="false"` still makes a control required because the attribute is present. Write `required` to turn it on and omit the attribute to turn it off.

Whitespace usually collapses in ordinary text, so several spaces do not create a layout system. The `pre` element preserves relevant preformatted whitespace. When `<`, `>`, or `&` must appear as text rather than markup, use a character reference such as `<`, `>`, or `&`. Comments use `<!-- explanation -->`; comments are delivered to the client and must never contain passwords, tokens, private notes, or server-only information.

Global attributes can occur on many elements, but they have different jobs:

- `id` is a document-wide unique identifier used by fragments, labels, scripts,

and accessibility relationships;

- `class` assigns one or more reusable classifications for CSS and scripts; it

supplies no user-facing semantics by itself;

- `data-*` stores application-specific data for scripts; it is not a substitute

for visible content or a standard semantic attribute;

- `lang`, `dir`, `hidden`, `title`, and `tabindex` affect language, direction,

exposure, advisory information, or focus behavior and must be used deliberately.

HTML also defines **content models**: rules about which children and descendants are permitted. A `ul`, for example, represents a list whose direct children are `li` elements. A `p` cannot contain another paragraph. A form must not contain a nested form. Memorizing every rule is unnecessary; learn to consult the current element reference and use the W3C Nu HTML Checker as one source of conformance evidence. A validator can find a duplicate `id` or forbidden child, but it cannot decide whether a heading or alternative text communicates the correct meaning.

2.2 Document-wide meaning begins in `html` and `head`

Before the first visible heading, the document already communicates important information. The `doctype` selects the modern HTML parsing mode. The `lang` attribute identifies the document's human language so pronunciation, spelling, translation, and other language-sensitive tools can behave appropriately. The `title` names the page in browser tabs, history, bookmarks, and assistive navigation. Character encoding tells the browser how bytes represent text.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="description" content="Submit a student event to Campus Hub.">
    <title>Submit an event - Campus Hub</title>
  </head>
</html>
```

The viewport declaration asks mobile browsers to treat the layout viewport as the device width. It does not make a page responsive by itself; the HTML and CSS must still adapt. Do not disable zoom with values such as `user-scalable=no`. People use zoom for many reasons, including low vision, fatigue, small screens, and temporary environmental conditions.

The description metadata gives search, sharing, and indexing systems a concise summary they may choose to use. It is not guaranteed display text and does not replace a visible introduction. The `title` and visible `h1` have related but different audiences: the title identifies the page in browser and navigation contexts, while the `h1` identifies the main subject inside the document. Both should be concise and specific, and they need not be character-for-character identical.

Use a valid BCP 47 language tag, such as `en`, `tr`, or `fa`, and mark a passage whose language changes with `lang` on the nearest containing element. Writing direction is a separate property. Use `dir="rtl"` for a right-to-left passage when its base direction requires it, and prefer `dir="auto"` for user-generated text whose direction is not known in advance. Language and direction metadata are functional information, not search engine decoration.

When an inline user-supplied name may have an unknown writing direction, `bdi` (Bidirectional Isolate) prevents that text's direction from unexpectedly reordering surrounding punctuation:

```
<p>Organizer: <bdi>مریم</bdi></p>
```

2.3 Structure follows the document, not its font size

A heading level expresses nesting. An `h1` names the document's main subject. An `h2` begins a major subsection of that subject; an `h3` is subordinate to the preceding `h2`. Choosing `h4` because its default font happens to be small confuses presentation with structure. CSS controls appearance later.

Consider this outline:

```
Campus Hub                          h1
  Upcoming events                   h2
    Robotics laboratory evening     h3
    Sustainable design workshop     h3
  Student communities               h2
```

The structure remains meaningful in a text-only view, a browser reader mode, a search index, and assistive navigation. Do not choose levels merely to avoid a large default heading.

2.4 Landmarks divide the page into navigable regions

Common structural elements are summarized in Table 2.2.

Table 2.2 - Semantic elements for common document regions

Element	Appropriate use
`header`	Introductory material for a page or section
`nav`	A major group of navigation links
`main`	The page's unique primary content; normally one visible instance
`section`	A thematic grouping that usually has a heading
`article`	A self-contained composition that could stand independently
`aside`	Content indirectly related to the surrounding material
`footer`	Closing information for a page or section

An element's name does not prescribe its screen position. A `nav` can appear at the bottom; an `aside` need not be visually placed at the side. Semantics describe purpose.

When a page contains more than one navigation region, give each a distinct name:

```
<nav aria-label="Primary navigation">...</nav>
<nav aria-label="Event categories">...</nav>
```

A skip link is an ordinary link to the primary content, normally placed as the first useful focusable item in `body`. It lets keyboard users bypass repeated navigation:

```
<a href="#main-content">Skip to main content</a>
<main id="main-content">...</main>
```

Mark the link for the current page or step with `aria-current="page"` or another appropriate token. Do not remove its `href`; current-page information supplements the link's native navigation semantics.

Use Accessible Rich Internet Applications (ARIA) features to fill a genuine semantic gap or supply an accessible name. Do not add redundant roles to native elements. `<button role="button">` says nothing that the element does not already say.

An ARIA role is a promise about behavior. Adding `role="button"` to a `div` does not make it focusable, activate it from the keyboard, submit a form, or supply a disabled state. If native HTML can express the control, native HTML is normally the more reliable starting point. ARIA is powerful when a real semantic gap remains, but incorrect ARIA can override accurate native information.

3. Text content, links, and media

3.1 Text elements communicate rhetorical structure

Use `p` for a paragraph, `em` for stress emphasis that changes the intended reading, and `strong` for strong importance or urgency. Use `blockquote` for an extended quotation and `q` for a short inline quotation. The `cite` element names a creative work, not the person who said a quotation. Use `code`, `kbd`, and `samp` to distinguish source code, user input, and program output. Use `pre` when line breaks and spaces are genuinely part of preformatted content, often with a `code` child for a block of source code.

Several less frequent elements prevent generic markup from erasing meaning:

- `figure` groups self-contained content with an optional `figcaption`;
- `address` contains contact information for the nearest article or page, not an

arbitrary postal address mentioned in prose;

- `mark` identifies text highlighted because it is relevant in the current

context, such as a search match;

- `small` represents side comments such as legal or attribution text, not every

sentence that should look visually small;

- `sub` and `sup` represent subscripts and superscripts when that position carries

meaning, such as H₂O or x²;

- `ins` and `del` represent inserted and deleted content; and
- `details` with `summary` provides a native disclosure when hiding the content

does not prevent the user from discovering essential information.

`div` and `span` are neutral containers for grouping when no specific semantic element fits. A meaningful `class` name such as `class="event-card"` helps authors maintain code, but it does not create an article, button, heading, or landmark in the accessibility tree.

Do not select these elements merely for their default appearance. For example, `strong` is not a generic instruction to make arbitrary words bold, and `br` is not a spacing tool. Meaning belongs in HTML; later CSS controls presentation.

3.2 Lists express grouping

Use `ul` when order is not essential, `ol` when sequence or ranking matters, and `dl` for name-value groups. A row of navigation links is still conceptually a list even if CSS later displays it horizontally.

3.3 Link text should make sense in context

“Click here” tells the reader how to operate a mouse but not where the link goes. Prefer text that names the destination or action:

```
<a href="/events/robotics-open-evening">Robotics open-evening details</a>
```

Do not use a link for an action that changes the current interface without navigating. That is a button. Do not use a button merely to navigate. Correct elements produce predictable keyboard, history, context-menu, and assistive technology behavior.

If a link downloads a file, name the file type and, when useful, its approximate size. If it opens a new browsing context, warn the reader before activation. Avoid placing a URL itself in prose unless the literal address is what the reader needs; descriptive link text is usually easier to understand and revisit.

3.4 Images require a purpose decision

The `alt` attribute supplies a text alternative for an image. Its content depends on the image's purpose in this context, not only on what the pixels contain.

- An informative image needs concise equivalent information.
- A decorative image normally uses `alt=""` so it is ignored by screen readers.
- A linked image needs text that names the link's destination or action.
- A complex chart needs a short alternative and a nearby detailed explanation or

data table.

Do not begin every alternative with “image of.” The software already identifies the element as an image. Do not copy a caption into `alt` if that makes users hear the same sentence twice.

```
<figure>
  <img src="capacity.svg" width="640" height="360"
       alt="Registration rises from 20 to 48 students over four weeks">
  <figcaption>Workshop registration by week.</figcaption>
</figure>
```

The example is suitable only if that short trend is the important information. If exact values matter, provide the underlying values as text or a table.

When the intrinsic image dimensions are known, `width` and `height` attributes help the browser reserve the correct aspect-ratio space before the file arrives. CSS can still scale the image responsively in Chapter 3. For an icon-only link or button, the image alternative must identify the operation or destination, not the icon's geometric appearance. Inline Scalable Vector Graphics (SVG) and images of text require the same purpose analysis; using a different file format does not remove the need for an accessible name or equivalent information.

3.5 Dates, abbreviations, and data can have machine-readable values

The `time` element lets visible wording and a machine-readable date or time coexist. Its `datetime` value uses a standardized form that software can parse.

```
<p>The workshop begins <time datetime="2026-10-14T18:30:00+03:00">14 October at 18:30</
time>.</p>
```

The visible text is written for the reader; the attribute includes the year and Coordinated Universal Time (UTC) offset needed to identify the instant. A local event whose time zone is already unambiguous in context may use a local date or time value, but software must not invent a time zone that the content did not specify.

Use `abbr` when expanding an abbreviation genuinely helps, and expand uncommon abbreviations in ordinary prose at first use. Do not put essential teaching in a `title` tooltip: it is not consistently available to keyboard, touch, or assistive-technology users.

3.6 Audio and video require equivalent access

Controls alone do not make recorded media accessible. Prerecorded video with meaningful speech needs synchronized captions. Audio-only material needs a transcript. Important visual information that is not conveyed in the soundtrack may require audio description or an equivalent text account. Do not rely on automatic captions without reviewing names, technical vocabulary, punctuation, speaker changes, and timing.

The `track` element can associate caption files with `video`, but authoring the media element and producing an accurate alternative are separate responsibilities. Also provide a fallback link when a browser cannot play the chosen format.

3.7 Tables are for relationships in data

A data table represents relationships between headers and cells. It is not a page-layout mechanism.

```
<table>
  <caption>Remaining places by workshop</caption>
  <thead>
    <tr><th scope="col">Workshop</th><th scope="col">Places</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row">Robotics safety</th><td>12</td></tr>
    <tr><th scope="row">Accessible design</th><td>8</td></tr>
  </tbody>
</table>
```

The caption names the table. Header cells and `scope` expose row and column relationships. Visual alignment alone cannot communicate those relationships to software.

For a simple table, `scope="col"` and `scope="row"` are normally sufficient. A multi-level table may use `scope="colgroup"` or `scope="rowgroup"` when its structure genuinely contains those groups. When relationships cannot be expressed clearly with scope, give each header a unique `id` and list the applicable header IDs in a data cell's `headers` attribute. Complex tables impose cognitive and testing costs, so first ask whether the data can be divided into several simpler tables. Do not use an empty header cell merely to create visual spacing, and do not rely on color or position alone to identify a total or category.

4. Forms are conversations with constraints

A form gathers values for an operation. Each successful control contributes a name-value pair when the form is submitted. The `name` attribute supplies the name used in that submission; `id` identifies an element within the document. They often share a value but serve different purposes.

```
<form action="/events" method="post">
```

```
  <label for="event-title">Event title</label>
  <input id="event-title" name="title" type="text" required>

  <button type="submit">Submit event</button>
</form>
```

The label's `for` value matches the control's `id`. Activating the label focuses the input. Assistive technology can announce “Event title, edit text, required.” The control's `name="title"` determines the submitted field name. The form's `action` identifies the submission destination, while `method` selects the HTTP method used by the browser's form-submission algorithm. Figure 2.5 follows these relationships from the visible label to the submitted value and then to server-side validation.

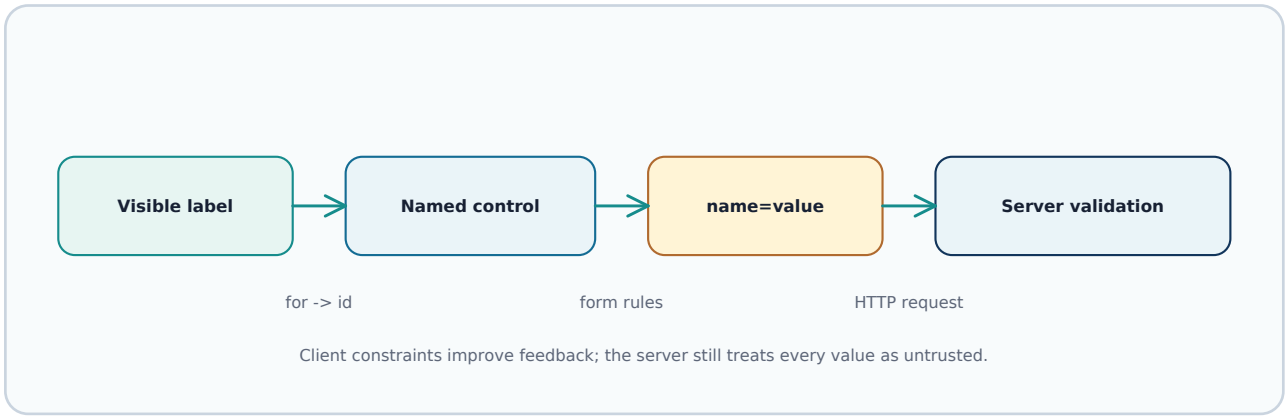


Figure 2.5. Labels identify controls; names carry values across the request boundary.

Figure 2.6 shows the browser output of the complete Week 2 form. It deliberately uses native browser presentation so students can see what correct HTML provides before custom CSS is introduced.

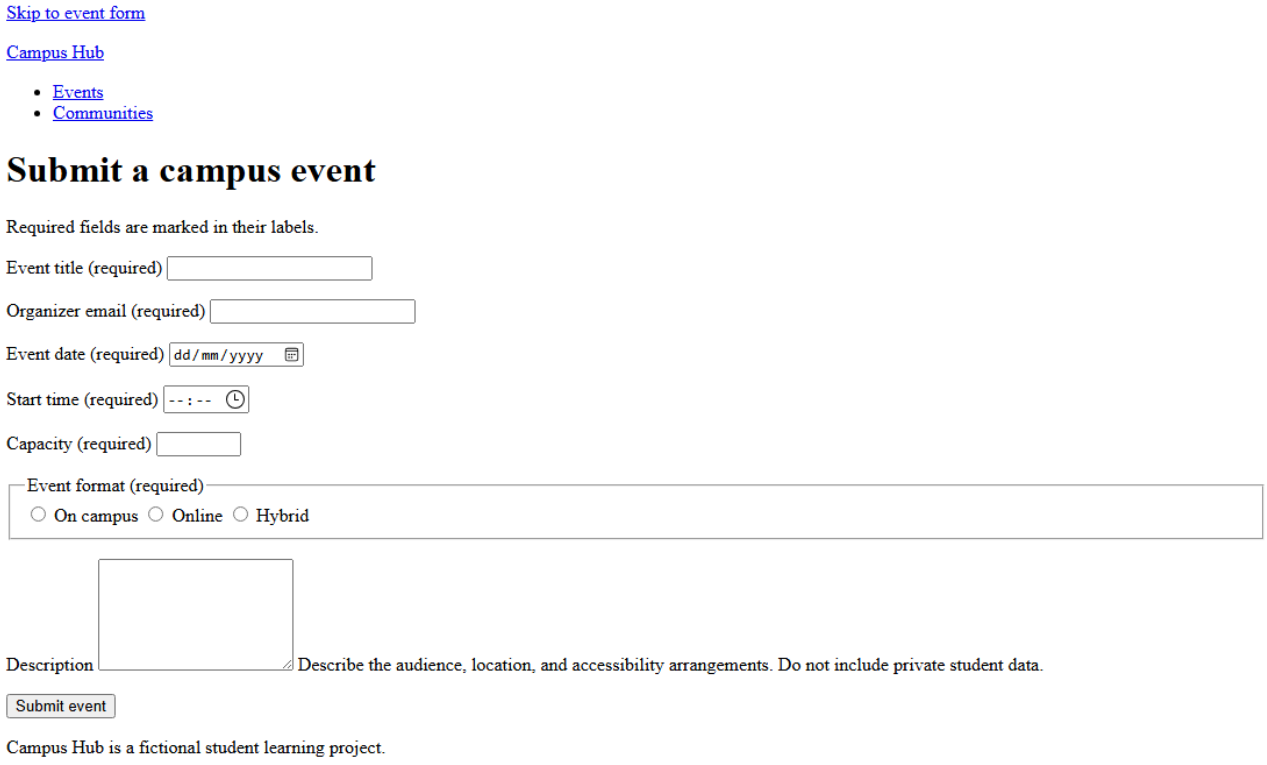


Figure 2.6. The browser output of the complete semantic form example shows visible labels, native controls, grouped choices, help text, and a submit button.

4.1 Choose the submission method from the operation

A `get` form represents retrieval. Its successful name-value pairs normally become a query in the resulting URL, which makes a search easy to bookmark and share. A `post` form sends form data in the request content and is suitable for an operation that creates or changes server state. `post` does not encrypt data and does not make an operation safe: use HTTPS, validate on the server, authorize the operation, and avoid collecting information the service does not need.

The method is not merely a syntax preference. Refreshing, bookmarking, caching, history, logs, and retries behave differently. A search form can use `get`; an event-creation form should use `post`. Later server chapters add Cross-Site Request Forgery protection and show how the application handles the submitted data.

4.2 Labels and descriptions remain available

A placeholder is an example or hint, not a replacement for a label. It disappears as the user types, is often displayed with weak contrast, and does not consistently provide an accessible name. Keep a visible label. If format guidance matters, associate permanent help text with the control:

```
<label for="event-date">Event date</label>
<input id="event-date" name="date" type="date"
       aria-describedby="event-date-help" required>
<p id="event-date-help">Choose a date within the current semester.</p>
```

The connected `label` provides the **accessible name**: the short identity of the control. `aria-describedby` contributes an **accessible description**: additional instructions, format guidance, or context. A description does not replace a name. Keep important instructions visible so people who do not use assistive technology receive the same information.

4.3 Select the narrowest appropriate native control

Input types communicate intent and can provide mobile keyboards and built-in checks. Examples include `email`, `url`, `date`, `time`, `number`, `search`, and `password`. Use `textarea` for multi-line input, `select` for a constrained list, checkboxes for independent choices, and radio buttons for one choice from a set.

Choose the type from the value's meaning, not from the characters it happens to contain. A postal code, telephone number, student number, credit-card number, or one-time code is an identifier, not a quantity on which arithmetic is performed. It normally uses a text-like input with suitable `autocomplete` and `inputmode` rather than `type="number"`. The `inputmode` attribute is a keyboard hint; it does not validate the value. Table 2.3 summarizes common decisions.

Table 2.3 - Choosing native form controls and supporting attributes

User task	Native starting point	Important considerations
Short general text	`input type="text"`	Add a visible label, useful `autocomplete`, and length constraints only when the task defines them.
Email address	`input type="email"`	The browser checks an email-shaped value, not whether the mailbox exists or belongs to the user.
Password	`input type="password"`	Use the correct `autocomplete` token; allow password managers and paste; never treat masking as encryption.
Search	`input type="search"`	Normally belongs in a GET form because it retrieves or filters information.
Quantity	`input type="number"`	Use for arithmetic quantities with meaningful `min`, `max`, and possibly `step`; not for digit-only identifiers.
Telephone or numeric identifier	`input type="tel"` or text with `inputmode="numeric"`	Preserve leading zeroes and formatting; validate according to the real domain rather than assuming one national pattern.
Date or time	`input type="date"` or `input type="time"`	Native presentation varies; state time zone and allowed range when they matter.
Several lines of text	`textarea`	Its initial value is text between the tags; provide rows as a useful starting size, not a content limit.
One option from a short visible set	Radio buttons in `fieldset`	A shared `name` forms the choice; the `legend` names the group.
Independent yes/no options	Checkboxes	Each choice needs a label and an explicit value; unchecked boxes are omitted from submission.
One option from a longer set	`select` with `option`	Use `optgroup` for meaningful groups; make any prompt option non-ambiguous.
Suggested values with free entry	`input` with `list` and `datalist`	Suggestions are not a closed constraint; server validation must accept or reject the actual value.
File selection	`input type="file"`	The POST form normally needs `enctype="multipart/form-data"`; `accept` is only a picker hint.

Group related choices with `fieldset` and `legend`:

```
<fieldset>
  <legend>Attendance mode</legend>
  <label><input type="radio" name="mode" value="campus" required> On campus</label>
  <label><input type="radio" name="mode" value="online"> Online</label>
</fieldset>
```

The shared `name` makes the radio buttons one submitted choice. The legend names the group. A collection of visually adjacent controls without this relationship is more difficult to interpret.

Button type also matters. Inside a form, a `button` defaults to submission. Write `type="submit"` for the submitter and `type="button"` for a button that performs only a client-side action. Avoid reset buttons in ordinary forms because an accidental activation can erase substantial work.

Do not nest one form inside another; HTML does not support nested forms. A control can be explicitly associated with a form through the `form` attribute, but keeping controls visibly and structurally near their form is usually clearer. Pressing Enter may submit some forms through the browser's implicit-submission behavior, so every form must remain coherent without depending only on a pointer click.

4.4 Predict which controls will be submitted

The browser does not submit every element inside `form`. Table 2.4 summarizes the rules students most often need when debugging missing or surprising data.

Table 2.4 - Common successful-control submission rules

Control condition	Submitted?	Reason or consequence
Enabled control with a `name`	Usually yes	Its current value is associated with that name.
Control without a `name`	No	An `id` can label or target the element but does not create a submitted field name.
Disabled control	No	Disabled controls are not successful controls and are normally not operable.
Read-only text control	Yes	The value cannot be edited through the control, but it can still be submitted and must not be trusted.
Unchecked checkbox or radio button	No	No name-value pair is contributed for that control.
Checked checkbox or radio button	Yes	Its explicit `value` is submitted; without one, a browser-defined default such as `on` can surprise the server.
Submit button that initiated submission	Yes	Its name and value can identify which submit action was chosen. Other submit buttons are omitted.
File input with no selected file	Browser-dependent empty file entry under multipart rules	The server must handle the no-file case explicitly rather than infer a usable upload.

Submission rules explain transport, not trust. Hidden and read-only values can be modified by a client. The server must derive security-sensitive facts, such as the current user or an authorized price, from trusted server-side state.

4.5 Choose the encoding, especially for files

Form submission must represent names and values as bytes. With GET, form data is encoded into the URL query. With an ordinary POST form, the default encoding is `application/x-www-form-urlencoded`, which represents fields in a query-like name-value form inside the request body. Spaces and reserved characters require percent-encoding; the server framework later decodes them.

A file upload normally requires POST and `multipart/form-data`:

```
<form action="/event-materials" method="post" enctype="multipart/form-data">
  <label for="agenda">Agenda document</label>
  <input id="agenda" name="agenda" type="file" accept=".pdf">
  <button type="submit">Upload agenda</button>
</form>
```

Multipart encoding separates fields into parts so file bytes and ordinary text can travel in one request. `accept=".pdf"` helps a file picker present likely choices; it is not a security boundary. The server must enforce allowed type, size, content, authorization, storage name, malware policy, and retention. Do not trust a client-supplied filename, extension, media type, or local path. The `text/plain` encoding exists but is not a robust general application encoding.

4.6 Autocomplete, input purpose, and data minimization

The `autocomplete` attribute helps browsers and assistive technologies identify the purpose of common personal-data fields. Tokens such as `name`, `email`, `organization`, and `current-password` are more informative than arbitrary field names. Autocomplete can reduce effort and errors, especially for people with motor or cognitive disabilities. Do not disable it reflexively.

Choose fields from the task, not from curiosity. Explain why information is required, avoid collecting secrets in examples, and do not place confidential values in GET URLs. `readonly` and `disabled` are interface states, not privacy or authorization mechanisms.

Attributes such as `inputmode`, `autocapitalize`, and `spellcheck` can reduce entry effort, but they are hints rather than validation. Disable spell checking for a value when sending it to a spelling service would be inappropriate, such as a password or one-time code. Do not disable paste in password or confirmation fields; password managers and assistive workflows depend on it.

4.7 Constraint validation is useful but incomplete

Attributes such as `required`, `min`, `max`, `minlength`, `maxlength`, and `pattern` express constraints to the browser. They improve the experience and reduce avoidable requests. They are not a security boundary. A client can disable JavaScript, alter markup, or send a Hypertext Transfer Protocol (HTTP) request directly. The server must validate every untrusted value again.

Validation should answer three questions:

1. What is wrong?
2. Where is the problem?
3. How can the user correct it?

Color alone cannot answer them. Provide specific text, preserve valid values, associate messages with controls, and move focus or an error summary appropriately after a failed submission.

Mark required and optional status in words, preferably before the user begins the form, and include the status in or programmatically with each relevant label. An asterisk without an explanation is ambiguous. Native `required` already exposes the required state; adding `aria-required="true"` to the same native control is normally redundant.

4.8 Design error recovery before adding JavaScript

A server-rendered response can return the form with an error summary and field-level messages without any JavaScript. The page title should announce the failed state, the summary should appear before the form, and each message should identify the problem and correction. Preserve valid values unless security makes that unsafe.

```
<title>2 errors - Submit an event - Campus Hub</title>

<div role="alert" tabindex="-1">
  <h2>There are 2 errors</h2>
  <ul>
    <li><a href="#event-title">Enter an event title.</a></li>
    <li><a href="#capacity">Enter a capacity from 1 to 300.</a></li>
  </ul>
</div>

<label for="capacity">Capacity (required)</label>
<input id="capacity" name="capacity" type="number"
       min="1" max="300" required aria-invalid="true"
       aria-describedby="capacity-error">
<p id="capacity-error">Enter a whole number from 1 to 300.</p>
```

`aria-invalid="true"` belongs on a control only while its current value is known to be invalid. The description connects the specific correction. `tabindex="-1"` allows a script or server-navigation enhancement to move focus to the summary without adding it to the ordinary Tab sequence. A static server response is already announced as a new page; `role="alert"` becomes especially relevant when errors are inserted dynamically, which later JavaScript chapters treat carefully.

Predict before running

Given this control:

```
<input id="capacity" type="number" min="1" max="300" required>
```

Will a value be submitted under the name `capacity`? No. The control has an `id` but no `name`, so it does not contribute that name-value pair. The browser can still validate and display it. This distinction is a frequent source of missing server data.

5. Three related representations

The browser maintains several related models:

1. Source HTML is the text received or read from a file.
2. The DOM is the parsed, live document tree.
3. The accessibility tree is a platform-oriented representation containing relevant roles, names, states, values, and relationships for assistive tools.

Figure 2.7 locates the accessibility tree between the live Document Object Model (DOM) and the assistive technology that consumes the platform representation.

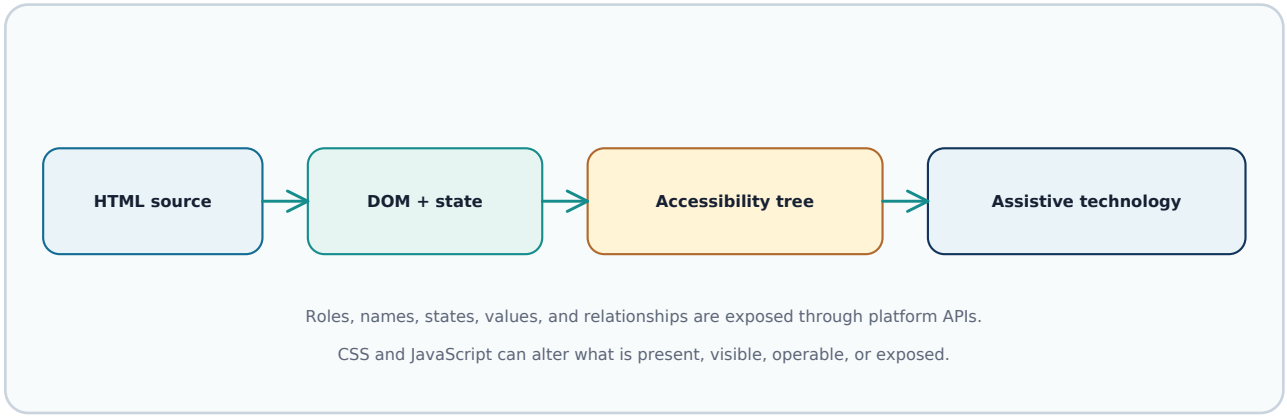


Figure 2.7. HTML and browser state contribute to a platform accessibility representation.

These representations are not identical. CSS can hide content. The HTML parser can repair omitted structure. JavaScript can alter the DOM. Some DOM nodes do not need accessibility exposure, while one compound widget may create several accessible relationships.

The HTML `hidden` attribute, CSS `display: none`, and the `hidden` state of a closed disclosure normally remove descendant content from both visual rendering and the accessibility tree. Visually hidden content intended only for assistive technology requires a carefully tested CSS pattern introduced in Chapter 3; do not use `hidden` for that purpose. Conversely, visually placing content off-screen without understanding focus can create a focus target that the keyboard reaches but a sighted user cannot see.

The accessibility tree is not “a screen reader version of the DOM.” It is a semantic interface used through operating-system accessibility application programming interfaces (APIs). Different browsers and platforms can expose details differently, but good native HTML provides a strong interoperable basis.

5.1 Accessible name, description, role, and state

Interactive controls need a name that identifies their purpose. A button's text usually provides its name. An input normally obtains its name from a connected `label`. An image can obtain a name from `alt`. `aria-label` can supply a name when no visible text can do so, but visible text is usually preferable because all users receive the same wording. A description supplies supporting information; it should not repeat the name word for word. Role communicates what kind of thing the object is, while state communicates information such as checked, expanded, selected, invalid, or disabled.

For speech-input users, the words they can see should occur in the accessible name. If the visible label says “Organizer email” while an `aria-label` silently renames the control “Contact address,” a voice command based on the visible words may fail. Prefer the connected visible label and use `aria-describedby` for extra help rather than replacing good visible text.

Do not place important visible text in CSS-generated content and assume it will be exposed consistently. Put meaningful content in HTML.

5.2 Keyboard operation is an engineering test

Use the Tab key to move through interactive elements. Use Shift+Tab to reverse direction, Enter to activate links and many buttons, Space to activate buttons and toggle checkboxes, and arrow keys within controls that define arrow-key behavior.

A useful keyboard inspection asks:

- Is every operation reachable?
- Is the focus indicator visible against its surroundings?
- Does focus order follow the reading and task order?
- Can a user escape components such as dialogs and menus?
- Does focus move only when there is a clear reason?

Positive `tabindex` values create a separate focus order that becomes difficult to maintain. Prefer logical source order and native elements. Use `tabindex="-1"` when a normally non-focusable element must receive programmatic focus, such as an error summary, not to insert it into ordinary Tab navigation.

6. Accessibility is broader than conformance

The Web Content Accessibility Guidelines (WCAG) 2.2 organize accessibility under four principles. Content should be **perceivable**, **operable**, **understandable**, and **robust**. The initials form the useful memory aid POUR. Table 2.5 connects each principle to questions a student can ask of Campus Hub.

Table 2.5 - WCAG principles as engineering questions

Principle	Engineering question
Perceivable	Can necessary information be obtained through more than one relevant sensory pathway, including text alternatives and captions?
Operable	Can people complete the task with keyboard, pointer, touch, zoom, and other supported input without traps or obscured focus?
Understandable	Are labels, instructions, navigation, errors, and changes predictable enough to act upon?
Robust	Do names, roles, states, values, and relationships remain available across supported browsers and assistive technologies?

WCAG success criteria are assigned conformance levels A, AA, or AAA. Level AA is a common organizational target, but conformance is evaluated against defined criteria for complete pages and processes; it is not a general score produced by one scanning tool. A product can meet a formal criterion and still impose a poor experience, while an isolated automated warning may require human interpretation.

Standards provide testable requirements, but an automated checker cannot decide whether alternative text communicates the correct purpose, whether instructions are understandable, or whether a workflow places an unreasonable cognitive burden on a user. Combine:

- valid semantic HTML;
- automated checks;
- keyboard testing;
- browser accessibility inspection;
- zoom and reflow testing;
- testing with assistive technology when possible; and
- feedback from disabled people and representative users.

Passing an automated scan is evidence, not proof of accessibility.

Figure 2.8 shows these evidence sources operating in parallel. Automated checks find some programmatically detectable failures. Keyboard and zoom testing reveal interaction and reflow problems. Accessibility-tree inspection and a screen reader expose names, roles, states, and reading behavior. Representative users show whether the real task is understandable and achievable. The conclusion is based on the combined evidence; no station can replace all the others.

Figure 2.8. Accessibility judgments combine automated, keyboard, zoom and reflow, accessibility-tree, assistive-technology, and user-task evidence; no single test proves accessibility.

Guided laboratory - Build an accessible event form

Goal and acceptance criteria

Complete the Week 2 starter as a no-CSS, no-JavaScript event-submission page. The page must remain understandable through structure alone.

Your result must contain:

- a useful title, language, encoding, description, and viewport declaration;
- a skip link whose target is the `main` element;
- one `h1` followed by logically nested headings;
- labeled primary navigation;
- a form using `post` with a meaningful `action`;
- visible labels for title, organizer email, date, start time, capacity, format,

and description;

- native input types and reasonable constraints;
- appropriate autocomplete metadata for the organizer email;
- an `aria-current="page"` indication on the relevant navigation link;
- a `fieldset` and `legend` for the event format;
- persistent help text associated with the description;
- a submit button with an explicit type;
- no positive `tabindex`, ARIA role that duplicates a native role, inline event

handler, secret, or real personal data; and

- at least three coherent Git commits.

Step 1 - Read before editing

From the repository root, open `labs/week-02/index.html` in the VS Code text editor. Verify the full folder path rather than trusting an `index.html` tab name. In the SERVER terminal, run `dir package.json` on Windows or `ls -l package.json` on macOS/Linux, followed by `npm run serve:week02`. Leave that process running, open `http://localhost:8000/`, and confirm that the baseline page displays **Submit a campus event**. Keep a second terminal available for Git and tests.

Now identify every TODO in the source. Sketch the intended heading outline and form data names on paper. Decide which information is instruction, which is a label, and which is a constraint. Save each meaningful edit and reload the browser; the local server displays saved files, not unsaved editor contents.

Step 2 - Establish landmarks and navigation

Add a skip link as the first useful link in `body`, a page header, labeled navigation, `main id="main-content"`, and a footer. Activate the skip link with a keyboard and confirm that its target exists.

Step 3 - Build the form from native controls

Use labels and native input types. Choose names that a server could interpret, such as `title`, `organizerEmail`, `date`, `startTime`, `capacity`, `format`, and `description`. Do not use placeholder text as the only instruction.

Step 4 - Define the submission contract

Confirm that the form uses `post` because it creates an event. Write the expected successful field names in order, and predict which controls disappear from the submission when they are disabled or unchecked. Explain why no field is trusted merely because it was read-only or hidden in the interface.

Step 5 - Express constraints

Require all fields except the description. Set capacity to a reasonable range. Set a useful maximum length for free text. Associate description help text with `aria-describedby`.

Step 6 - Test without a mouse

Reload at the top of the page, set the mouse aside, and complete the form. Record the focus order. Trigger at least two native validation messages. At 200% zoom, confirm that no information is lost.

Step 7 - Validate the source and inspect the tree

Submit the source to the W3C Nu HTML Checker by direct input, or use an approved local equivalent when the document is private. Correct every source error. Then inspect the heading structure, landmarks, labels, and the capacity control's name, role, value, description, and required state in browser accessibility tools. Record one thing a validator can establish and one meaning it cannot judge.

Step 8 - Verify, explain, and commit

Keep the server terminal running and open a second terminal with **Terminal > New Terminal**. This second terminal is the commands terminal: it displays a prompt and accepts new commands while the first terminal remains occupied by the server. From the repository root, run:

```
npm test
```

The final output should report `Coursework setup verification passed.` This check proves that the supplied structure and server still operate; it does not grade every semantic decision in the completed form. If it fails, read the first reported check, correct the source or project location, save, and run the command again. Then compare the page with every acceptance criterion and write a five-sentence rationale explaining one native semantic choice, one keyboard observation, one limitation of browser validation, one privacy decision, and one improvement you would test with users. Review the staged diff and create the final laboratory commit only after the browser, accessibility inspection, validator, and automated check agree with the intended source.

Laboratory observation record

Table 2.6 is an evidence record, not a checklist to mark without observation. Students should write what they saw and then explain what that observation means.

Table 2.6 - Laboratory accessibility observation record

Check	Evidence and interpretation
Heading outline	
Skip-link target	
First five focus stops	
Accessible name of capacity control	
Method and action rationale	
Expected successful field names	
Two native validation results	
Data excluded for privacy	
Automated-check limitation	
Nu HTML Checker result and limitation	
Accessibility-tree name, role, state, and description	

Exercises

Foundation

Exercise 1 - Element choice

Choose the best native element for each item and explain why: a navigation destination, an operation that opens a filter panel, a self-contained news item, a set of event categories, and the page's unique content region.

Exercise 2 - Label and name

Explain the independent purposes of `label[for]`, `id`, and `name` in a form. What breaks if an input has only `id="email"`?

Exercise 3 - Alternative text

An event card contains a decorative confetti pattern, a speaker portrait next to the visible name “Dr. Ada Kaya,” and a chart of registration by faculty. Propose an alternative-text strategy for each.

Application

Exercise 4 - Repair the outline

Repair this structure without selecting headings by desired size:

```
<h3>Campus Hub</h3>
<h1>Events</h1>
<h4>Robotics evening</h4>
<h2>Communities</h2>
```

Exercise 5 - Diagnose the inaccessible control

Why is this not an adequate button, and what is the smallest correct replacement?

```
<div class="button" onclick="submitEvent()">Submit</div>
```

Exercise 6 - Table relationships

Design semantic table markup for three rooms and their capacity, building, and availability. Identify the caption, column headers, and row headers.

Challenge

Exercise 7 - Validation boundary

The browser rejects an empty title, but an attacker successfully submits an empty title to the API. Explain how both observations can be true and specify validation responsibilities at the browser, server boundary, and database.

Exercise 8 - Accessibility-tree reasoning

A visually hidden checkbox remains in the DOM but is removed from the accessibility tree with `display: none`. A styled `div` is visible and clickable but has no role or keyboard behavior. Explain what different users and tools may perceive, and propose native-HTML repairs.

Exercise 9 - Predict the submitted data

Consider this form. Assume the user enters `Ada`, leaves the checkbox unchecked, and activates the Save button. Write the name-value pairs the browser will submit and explain why the other controls do or do not contribute data.

```
<form action="/profiles" method="post">
  <input name="displayName" value="Ada">
  <input name="theme" value="dark" disabled>
  <input name="studentId" value="CE-104" readonly>
  <input type="checkbox" name="newsletter" value="yes">
  <button type="submit" name="intent" value="save">Save</button>
</form>
```

Exercise 10 - Choose a testing strategy

A scanner reports no accessibility errors for a course-registration form. State why that result is insufficient. Design a compact test plan that gathers at least four different kinds of evidence, and identify one question each method can and cannot answer reliably.

Exercise 11 - Repair the syntax and semantics

Identify every independent problem in this fragment, write a conforming repair, and explain what the browser cannot infer from the original source.

```
<p class=event note>Register <strong>before Friday</p></strong>
<input id="email" type="email" required="false" placeholder="Email">
<p id="help">Use your university address & include the department.</p>
```

Exercise 12 - Choose controls and encoding

Design the HTML form contract for an event search, a six-digit one-time code, a seat quantity from 1 to 20, and an optional PDF agenda upload. Choose the method, control types, keyboard hints, names, constraints, and encoding. Explain why one form should not necessarily contain all four operations.

Exercise 13 - Design error recovery

A server rejects `capacity=500` and returns the form. Specify the page title, error summary, link target, field attributes, field-level message, value-preservation decision, and focus plan. Explain what changes if JavaScript inserts the error into the existing page instead of returning a new page.

Exercise 14 - Model a complex table

A timetable has two column groups, Monday and Tuesday, each with Morning and Afternoon columns. Every row begins with a room header. Design the header relationships using `caption`, `thead`, `tbody`, `colgroup`, and appropriate `scope` values. Then state when you would replace the table with two simpler tables or use explicit `id` and `headers` associations.

Worked answers

Answer 1

Use `a` with an `href` for the navigation destination because it participates in history, link menus, copying, and ordinary link keyboard behavior. Use `button` for opening a filter panel because it performs an action in the current interface. Use `article` for the self-contained news item, `ul` with `li` for categories when order is unimportant, and `main` for the unique primary content. CSS can give any of these the required appearance without erasing their meaning.

Answer 2

`id` uniquely identifies the element in the document. `label for="email"` uses that identifier to associate visible label text with the control. `name` supplies the key used when the form constructs submitted data. With only `id="email"`, a connected label can work, but the control does not submit a field named `email`. The three attributes may share a value, yet they are not interchangeable.

Answer 3

The decorative confetti should use an empty alternative, `alt=""`, because it adds no information. The portrait can also use `alt=""` when the adjacent visible name already provides all relevant identity; repeating the name would be noisy. If the portrait communicates additional relevant information, include only that information. The chart needs a concise summary of its important conclusion plus nearby access to exact values, normally a table or textual description. “Chart” alone is not an equivalent.

Answer 4

A defensible repair is one `h1` for Campus Hub, `h2` for Events and Communities, and `h3` for Robotics evening under Events. The original levels suggested that Events was the document title and Campus Hub was deeply subordinate. Heading appearance should be styled later with CSS.

```
<h1>Campus Hub</h1>
<h2>Events</h2>
<h3>Robotics evening</h3>
<h2>Communities</h2>
```

Answer 5

The `div` has no native button role, focusability, keyboard activation, disabled state, or form semantics. An `onclick` handler supplies only one implementation path. The smallest repair is `<button type="button">Submit</button>` with the behavior attached separately. If it submits a form, use `type="submit"` and allow the form's submit event and native behavior to coordinate the operation.

Answer 6

Use a caption such as “Teaching-room availability.” Put Room, Capacity, Building, and Availability in `th scope="col"` cells inside `thead`. In each body row, put the room name in `th scope="row"` and the other values in `td` cells. This exposes the same row-column relationships that visual alignment communicates to a sighted reader. A layout made of generic elements would require software to guess.

Answer 7

Browser validation is executed by a client under the user's control. An attacker can modify the document, disable checks, or call the API directly. The browser should provide immediate, accessible feedback for ordinary mistakes. The server must treat every request as untrusted, normalize and validate the value, and return a consistent error. The database should enforce durable invariants with constraints such as `NOT NULL` and, when appropriate, a check on trimmed length. No single layer replaces the others: they serve user experience, trust-boundary protection, and persistent integrity respectively.

Answer 8

`display: none` removes the checkbox from visual layout and normally from the accessibility tree, so neither sighted users nor screen-reader users can operate it even though a DOM node exists. The clickable `div` may be visible to a mouse user, but it has no native control semantics and is not ordinarily in the Tab order. Use a real checkbox with a connected visible label for a binary choice and a real button for an action. If custom visuals are required, style the native control or its label while preserving focus, name, state, and keyboard behavior.

Answer 9

The submission contains `displayName=Ada`, `studentId=CE-104`, and `intent=save`. The display-name control is enabled and named, so its current value is successful. `readonly` prevents ordinary editing but does not prevent the student identifier from being submitted. The disabled theme control is omitted. The unchecked checkbox is also omitted; a checkbox contributes its configured value only when checked. The activated submit button contributes its own `name=value` pair, which lets the server distinguish Save from another possible submission action. These browser rules describe data construction, not trust: the server must still authorize and validate every received field.

Answer 10

An error-free scan means only that the chosen tool did not detect a rule it knows how to test in the page state it inspected. It does not prove that labels are meaningful, keyboard order follows the task, reflow remains usable, screen-reader feedback is understandable, or the overall registration journey can be completed. A defensible plan combines: an automated standards check for detectable markup errors; keyboard-only completion while observing focus order and visibility; zoom and narrow-viewport reflow inspection; accessibility-tree and screen-reader inspection of names, roles, states, errors, and status messages; and, where possible, representative-user task testing. Automation can find a missing label association but cannot reliably judge whether the wording explains the field. Keyboard testing can expose unreachable actions but cannot establish spoken output. Tree and screen-reader inspection can expose an incorrect name or state but cannot by itself prove that the workflow suits real users. User testing reveals task barriers in context but does not replace systematic conformance checks. The methods answer different questions, so their evidence must be read together.

Answer 11

The unquoted class value stops at `event`; `note` is parsed as a separate Boolean attribute rather than a second class token. The `p` and `strong` elements overlap instead of nesting. `required="false"` still turns the Boolean required state on. The input has no connected visible label and no `name`, so its value would not be submitted. A placeholder is not a durable label. Finally, the literal ampersand should be written as `&` in conforming text where it could begin a character reference. One repair is:

```
<p class="event note">Register <strong>before Friday</strong>.</p>

<label for="email">University email</label>
<input id="email" name="email" type="email" aria-describedby="help">
<p id="help">Use your university address &amp; include the department.</p>
```

Whether the email is required is a product decision. The repair omits `required` to match the literal false intention; if the field is required, add the bare attribute and say so in the label or form instructions. Error recovery can create a DOM, but the browser cannot infer the intended second class, business rule, visible label, submitted field name, or authorial nesting with certainty.

Answer 12

The event search is a retrieval operation and belongs in a GET form, commonly with `<input type="search" name="q">`. Its value becomes a bookmarkable query. The one-time code is an identifier rather than a quantity; use a text input such as `<input name="code" inputmode="numeric" autocomplete="one-time-code" pattern="[0-9]{6}" maxlength="6">`, while recognizing that the server must still validate the exact six-digit policy. The seat count is a quantity and can use `<input type="number" name="seats" min="1" max="20" step="1">`.

The agenda upload uses `<input type="file" name="agenda" accept=".pdf">` inside a POST form with `enctype="multipart/form-data"`. The server must verify the file's content, type, size, authorization, and storage policy. Search is safe, shareable retrieval, while an upload changes server state and carries file bytes. Putting both into one submission would blur method semantics and encoding, so separate forms are clearer. The one-time code likely belongs to a separate authentication workflow rather than an event search or upload.

Answer 13

Use a specific title such as “1 error - Submit an event - Campus Hub.” Place a summary before the form with a heading such as “There is 1 error” and a link to `#capacity`. Keep `id="capacity"`, set `aria-invalid="true"`, and associate `aria-describedby="capacity-error"`. The visible message should say what works, for example, “Enter a whole number from 1 to 300.” Preserving `500` can help the user understand and correct the mistake; preserve all other safe, valid fields as well. Passwords and other security-sensitive values may need different handling.

For a new server-rendered page, move focus to the summary through normal page navigation or a deliberate focus enhancement and ensure the title reports the failure. If JavaScript inserts the error without navigation, the change needs an appropriate live announcement, focus must be managed without creating a trap, and repeated submissions must not produce duplicate IDs or stale messages.

Answer 14

Give the table a caption that names the timetable. In `thead`, the first row has a corner cell followed by `<th scope="colgroup" colspan="2">Monday</th>` and the equivalent Tuesday header. The second header row contains Morning and Afternoon as `<th scope="col">` cells under each day. Use two `colgroup` elements, each spanning the relevant pair of day columns, after the room column. In `tbody`, each room name is `<th scope="row">`, followed by four data cells.

Test the resulting associations in more than one accessibility combination. Divide the table when users naturally compare one day at a time or when the combined grid becomes difficult to understand at narrow widths. If header levels, spans, or irregular cells cannot be represented unambiguously with scope, assign unique IDs to header cells and list all applicable IDs in each data cell's `headers` attribute. Visual position alone is not a sufficient relationship.

Debugging clinic

A student reports, “The Email text is immediately above the field, so the field is labeled.” Inspection shows `<p>Email</p><input type="email" name="email">`.

Diagnosis

Visual proximity is not a programmatic relationship. The input has no accessible name from a connected label. Replace the paragraph with `<label for="email">Email</label>` and add `id="email"` to the input, or wrap the input inside its label. Then inspect the accessibility tree and activate the label to confirm that it focuses the control. Do not add `aria-label` merely to avoid correcting the visible markup.

Retrieval quiz

1. What does semantic HTML communicate?
2. Why should heading level not be chosen from default font size?
3. When is `article` more appropriate than `section`?
4. What is the difference between a link and a button?
5. Why is placeholder text not an adequate label?
6. What does a form control's `name` do?
7. Why must the server repeat validation performed by the browser?
8. What information belongs in an accessible name?
9. Why is `alt=""` sometimes correct?
10. What does the accessibility tree contain?
11. Name two limitations of automated accessibility testing.
12. Why should positive `tabindex` values usually be avoided?
13. What is the practical difference between `disabled` and `readonly` controls during form submission?
14. When is GET normally preferable to POST for an HTML form?

15 Name the four WCAG principles and explain why satisfying one does not imply satisfying the other three.

16 What is the difference between a tag and an element?

17 Why does `required="false"` still make a native control required?

18 Why is `type="number"` usually wrong for a student number or postal code?

19 Which form encoding is normally required for a file upload?

20 Name four parts of an accessible server-rendered validation failure.

Quiz answers and misconception notes

1 It communicates the roles and relationships of content and controls. Semantics are not a promise about visual style.

2 Levels express document nesting. CSS, not an incorrect level, should change size and spacing.

3 Use `article` when the composition is independently meaningful or reusable, such as a news story or event card. A `section` is a thematic division of its surrounding document.

4 A link navigates to a resource; a button performs an action. Native behavior, not just appearance, follows that distinction.

5 A placeholder disappears, is not consistently an accessible name, and cannot remain as permanent instruction. Keep a visible connected label.

6 It supplies the key for that control's submitted value.

7 The client is outside the server's trust boundary and can be bypassed or modified. Server validation protects the operation.

8 A concise description of the control's purpose, normally derived from its visible label or text.

9 An empty alternative intentionally removes a decorative or redundant image from the accessibility experience.

10 Relevant roles, names, states, values, and relationships exposed through platform accessibility APIs.

11 Automated tools cannot reliably judge whether text alternatives convey the intended meaning or whether an entire workflow is understandable. Many other usability and assistive-technology interactions also require human testing.

12 They create a manually managed focus order that easily diverges from reading and source order. Prefer logical HTML order and native focusability.

13 A disabled control cannot ordinarily be operated and is omitted from submitted form data. A read-only text control cannot ordinarily be edited, but it remains focusable in common browsers and its named value is submitted. Neither makes a client-provided value trustworthy to the server.

14 Use GET when submission retrieves or filters information without requesting a state change and when a bookmarkable, shareable query is useful. Use POST when the request creates or changes server state, carries a larger body, or should not place ordinary field values in the URL. HTTPS is still required for confidentiality; POST alone is not encryption.

15 Perceivable, Operable, Understandable, and Robust. They address different failure modes: content may be perceivable but impossible to operate, operable but confusing, or understandable visually while exposing invalid semantics to assistive technology. Evidence is required across all applicable principles.

16 A tag is source markup at an element boundary, such as `<p>` or `</p>`. The element is the complete parsed object, including its tags when applicable, attributes, content, and relationships.

17 `required` is a Boolean attribute. Presence means true regardless of a string that looks like “false.” Omit it to turn the native required state off.

18 Those values are identifiers rather than arithmetic quantities. Numeric input can reject formatting or leading zeroes and implies quantity behavior. Use an appropriate text-like control with `inputmode` and domain validation.

19 `multipart/form-data` on a POST form. The picker and `accept` attribute do not remove the server's responsibility to validate the uploaded bytes.

20 A failure-specific page title, an error summary with links, field-level messages associated with their controls, and invalid-state/focus handling. Preserving safe valid values and explaining how to correct each error are also essential.

Self-check scale

- **17-20:** ready to style the document without sacrificing its structure.
- **12-16:** revisit syntax, forms, and the accessibility-tree model, then repeat the keyboard test.
- **0-11:** rebuild the laboratory form from native elements and explain each choice to a peer or teaching assistant.

Session summary

- The Session 2 working source is `labs/week-02/index.html`; VS Code edits it, the local Node.js server serves it, and the browser renders it.
- The basic feedback cycle is edit, save, reload, and inspect. An unsaved file or a different `index.html` cannot change the page currently being served.
- HTML describes structure, relationships, controls, and meaning.
- Conforming syntax distinguishes tags, elements, attributes, Boolean presence, void elements, nesting, and character references.
- The document language, title, character encoding, and viewport metadata provide meaning and operating assumptions for the whole page.
- Native elements provide tested semantics and behavior before customization.
- Headings form a hierarchy; landmarks divide the page into navigable regions.
- Links navigate and buttons act.
- Images need alternatives chosen from purpose, not a mechanical description.
- Data tables require captions and explicit header relationships.
- Simple tables use scope; genuinely complex tables may require row/column groups or explicit header IDs, and are often clearer when divided.
- Form labels, identifiers, and submitted names have different responsibilities.
- GET and POST express different request intentions, and successful-control rules determine which form fields become submitted data.

- Ordinary text fields and file bytes require different form encodings;

multipart upload hints never replace server validation.

- Browser validation improves experience but never establishes server trust.
- Accessible error recovery combines a failure-specific title, summary, linked

field messages, preserved safe values, state, and deliberate focus behavior.

- Accessible names identify controls; descriptions supply supporting instructions

or constraints without replacing concise labels.

- The source, DOM, and accessibility tree are related but distinct.
- Accessibility work considers content that is Perceivable, Operable,

Understandable, and Robust, and combines automated and human evidence.

Glossary

Table 2.7 collects the terms that instructors can reuse for retrieval questions and that students can revisit while explaining their laboratory evidence.

Table 2.7 - Session 2 glossary

Term	Working definition
Accessibility tree	A semantic representation of relevant roles, names, states, values, and relationships exposed to assistive technologies.
Accessible name	Text by which software identifies an interface element to a user.
Accessible description	Supporting information associated with an element, such as a format hint or constraint; it supplements rather than replaces the accessible name.
ARIA	Accessible Rich Internet Applications specifications for adding accessibility semantics when native HTML cannot express a needed pattern.
Boolean attribute	An attribute whose presence represents true and whose absence represents false; the string `"false"` does not turn it off.
Character reference	HTML syntax such as `&` or `<` used to represent a character that would otherwise be interpreted as markup.
Content model	The HTML rules that define which kinds of content an element may contain.
Constraint validation	Browser mechanisms that evaluate form constraints such as required values and ranges.
Element	A parsed HTML component with a defined meaning, attributes, content when permitted, and relationships in the document tree.
Form encoding	The representation used to serialize successful controls, such as `application/x-www-form-urlencoded` or `multipart/form-data`.
Focus	The current destination for keyboard input.
Landmark	A major navigable page region such as navigation or main content.
Native semantics	Meaning and behavior supplied by a standard HTML element.
Tag	Source markup that begins or ends an HTML element; a tag is not the complete element.
Semantic HTML	Element selection based on the meaning and relationship of content.
Successful control	A form control whose name and current value are included in submitted form data under the form rules.
Text alternative	Text that communicates the relevant purpose or information of non-text content.
Void element	An HTML element such as `img` or `input` that cannot contain child content and has no closing tag.
WCAG	Web Content Accessibility Guidelines, organized around the principles Perceivable, Operable, Understandable, and Robust.

Primary references and further reading

1 WHATWG, HTML Living Standard - Web developer edition.

2 W3C, Web Content Accessibility Guidelines 2.2.

3 W3C, WAI-ARIA Authoring Practices - Read Me First.

4 MDN, HTML: A good basis for accessibility.

5 MDN, Web forms.

6 MDN, Core modules: Semantic HTML.

7 Manuel Matuzovic, Web Accessibility Cookbook.

8 MDN, Basic HTML syntax.

9 W3C Web Accessibility Initiative, Forms Tutorial.

10 Jennifer Niederst Robbins, *Learning Web Design*, 6th edition, O'Reilly Media, 2025.

11 Adam Silver, *Form Design Patterns*, Smashing Media.

12 Heydon Pickering, *Inclusive Components*.

What changes often?

HTML's central semantics and WCAG 2.2 requirements are stable. Browser validation messages, accessibility-tree inspection panels, and assistive-technology details vary by platform and version. Technical review date: **22 September 2026**.

Session reflection

- Which native element prevented the most custom work in your form?
- Which keyboard observation changed your design?
- Which accessibility claim still requires evidence from a representative user?

Session 3 - Cascading Style Sheets, Layout, and Responsive Design

Essential question: How can Cascading Style Sheets (CSS) help one semantic document adapt to different screens, preferences, languages, and amounts of content without becoming fragile?

A browser can display the Chapter 2 document without any author-written CSS. Its headings are large, paragraphs form blocks, links are recognizable, and form controls use platform conventions. Those defaults are intentionally modest, but they reveal an important starting point: CSS does not create the document's meaning. It participates in how that meaningful document is presented.

This chapter begins at the first character in a stylesheet and continues through the cascade, typography, color, boxes, positioning, Flexbox, Grid, media queries, container queries, accessibility preferences, and browser debugging. The live session should emphasize the core path marked in Table 3.1; the additional sections make the chapter useful later as a reference instead of forcing a beginner to search for missing foundations in unrelated tutorials.

Session map

Table 3.1 summarizes the prerequisites, expected work, and evidence for this session.

Table 3.1 - Session scope, preparation, and evidence

Item	Plan
Prerequisites	Sessions 1-2; semantic Campus Hub HyperText Markup Language (HTML)
Contact time	Approximately 3 hours workshop plus a 2-hour laboratory
Independent work	Approximately 5 hours, including reference sections and exercises
Main artifact	A responsive, accessible Campus Hub shell
Assessment	Three viewport checks, keyboard evidence, overflow audit, exercises, and quiz

The recommended teaching route is CSS syntax and loading, selector matching, the cascade, the box model, normal flow, Flexbox, Grid, responsive constraints, and the guided laboratory. Typography, color, positioning, container queries, and the expanded debugging material can be assigned before or after the workshop and revisited in later chapters. They are not optional professional knowledge; the separation protects the pace of the first encounter.

Learning outcomes

After this session, you should be able to:

1. connect an external stylesheet and explain CSS rule, selector, declaration, property, value, at-rule, and error-recovery syntax;
2. use type, class, attribute, state, relationship, and pseudo-element selectors without coupling presentation to accidental document nesting;
3. explain how relevance, origin, importance, cascade layers, specificity, scoping proximity, and order of appearance choose a declared value;
4. distinguish inheritance from selector matching and use custom properties and CSS-wide keywords deliberately;
5. reason about display, the box model, intrinsic sizing, containing blocks, overflow, positioning, stacking, and logical axes;

6 choose units, colors, typography, backgrounds, borders, and form styles that remain readable and adaptable;

7 create robust one-dimensional layouts with Flexible Box Layout (Flexbox) and two-dimensional layouts with Grid;

8 design mobile-first media queries and component-level container queries from content constraints rather than named devices;

9 preserve focus visibility, contrast, target size, source order, zoom, reflow, forced colors, and reduced-motion preferences;

10 diagnose missing declarations, cascade conflicts, overflow, and layout failures with browser developer tools; and

11 organize CSS so that later changes remain local and predictable.

1. CSS is a language for presentation and constraints

CSS does not paint a fixed screenshot. It is a declarative language: an author describes desired relationships, and the browser determines the resulting presentation for the current conditions. The browser combines CSS rules with document structure, fonts, available space, user preferences, writing direction, zoom, and actual content.

A design that works only for one exact viewport and one exact paragraph is not responsive. It is a picture of a single test case. Professional CSS establishes constraints under which many valid cases can be laid out.

Before adding CSS, test the Chapter 2 page at narrow width, 200% zoom, with a long event title, and with keyboard focus. Record what already works because of normal HyperText Markup Language (HTML) flow. CSS should improve that behavior without destroying it.

The short code examples in this chapter build one cumulative page rather than a series of unrelated demonstrations. Figures 3.9 and 3.10 show the completed page at wide and narrow viewports, so every rule that affects this example can be connected to visible output.

1.1 Read a CSS rule as a complete sentence

Consider the first rule added to `styles.css`.

```
.event-card {
  border: 1px solid #cad4df;
  padding: 1rem;
}
```

The selector `.event-card` identifies which elements the rule can match. The curly braces contain a declaration block. Each declaration contains a property, a colon, a value, and normally a terminating semicolon. In `padding: 1rem;`, `padding` is the property and `1rem` is the value. The colon belongs to CSS syntax; it is not part of either name. Figure 3.1 labels these parts and shows the path from a matched element to proposed presentation values.

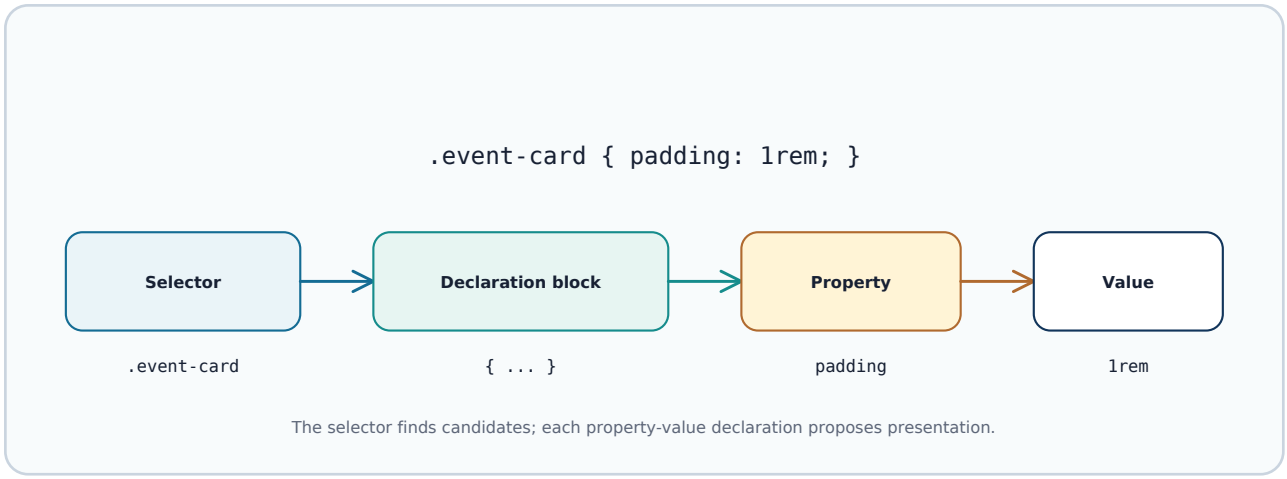


Figure 3.1. A CSS rule connects a selector to a declaration block containing property-and-value proposals.

A property describes the aspect being controlled. A value uses the grammar that the property accepts. `padding: teal` is not meaningful merely because `teal` is a valid CSS color: the `padding` property expects lengths, percentages, or related keywords. Browser developer tools usually cross out or omit a declaration whose value cannot be parsed for that property.

A semicolon separates declarations. The final semicolon is technically optional in a block, but keeping it makes later additions safer. CSS comments begin with `/*` and end with `*/`. The `//` notation used for comments in some programming languages is not a CSS comment.

CSS is designed to recover from many errors. If a browser does not understand one declaration, it normally ignores that declaration and continues with the rest of the rule. This resilience supports progressive enhancement, but it can also make a spelling error look silent. `colr: navy` does not stop the entire stylesheet; it simply does not set `color`.

1.2 Rules, at-rules, and shorthands are different structures

An **at-rule** begins with `@` and controls a larger condition or operation. Examples in this chapter include `@layer`, `@media`, `@container`, and `@supports`. Some at-rules end with a semicolon; others contain a block. Read the particular at-rule's grammar instead of assuming every `@` construct behaves the same way.

A shorthand property sets several related longhand properties together. For example, this declaration proposes width, style, and color for all four borders.

```
.event-card {
  border: 1px solid #cad4df;
}
```

Shorthands are concise, but they also reset omitted longhands to their initial values. Use them when that reset is intended. During debugging, the Computed panel can reveal the resulting longhand values.

1.3 Connect CSS to HTML deliberately

The preferred connection for this project is an external stylesheet referenced from the document `head`.

```
<link rel="stylesheet" href="styles.css">
```

`rel="stylesheet"` identifies the relationship, and `href="styles.css"` resolves relative to the HTML document's address. The browser requests the CSS resource, parses it, and applies applicable rules. The Network panel should show a successful request whose response has a CSS

media type. A `404 Not Found` response, a misspelled filename, or a path relative to the wrong document can leave the page unstyled even when the CSS itself is correct.

CSS can also appear in a `style` element or a `style` attribute. A `style` element can be useful for a self-contained demonstration. A style attribute is occasionally generated by applications, but widespread inline styling duplicates decisions, bypasses selectors, and makes maintenance difficult. The project uses an external stylesheet so one tested file can serve multiple documents and be cached independently.

The `@import` at-rule can load another stylesheet, but it may create additional serial discovery and makes dependency order less visible. Prefer HTML `link` elements for major stylesheets unless the project's architecture justifies an import.

1.4 The browser already has a stylesheet

Browsers apply a **user-agent stylesheet** before the author's stylesheet. It is why an unstyled heading appears bold and why a button resembles a platform control. Users may also provide their own preferences or styles. The cascade is designed to reconcile these sources; author CSS is not the only participant.

A reset should remove only defaults the design is prepared to replace. Erasing all margins, focus indicators, list markers, and control appearance can destroy useful affordances. This project uses a small baseline: predictable box sizing, responsive media, and a few intentional spacing decisions. It does not attempt to make every element visually blank.

2. Selectors match; the cascade decides

A selector answers one question: which elements does this rule target? It does not traverse the document and change HTML. The browser evaluates the selector against the Document Object Model (DOM), creates a set of matches, and considers the declarations for each matched element. Figure 3.2 separates selector matching from the later cascade decision.

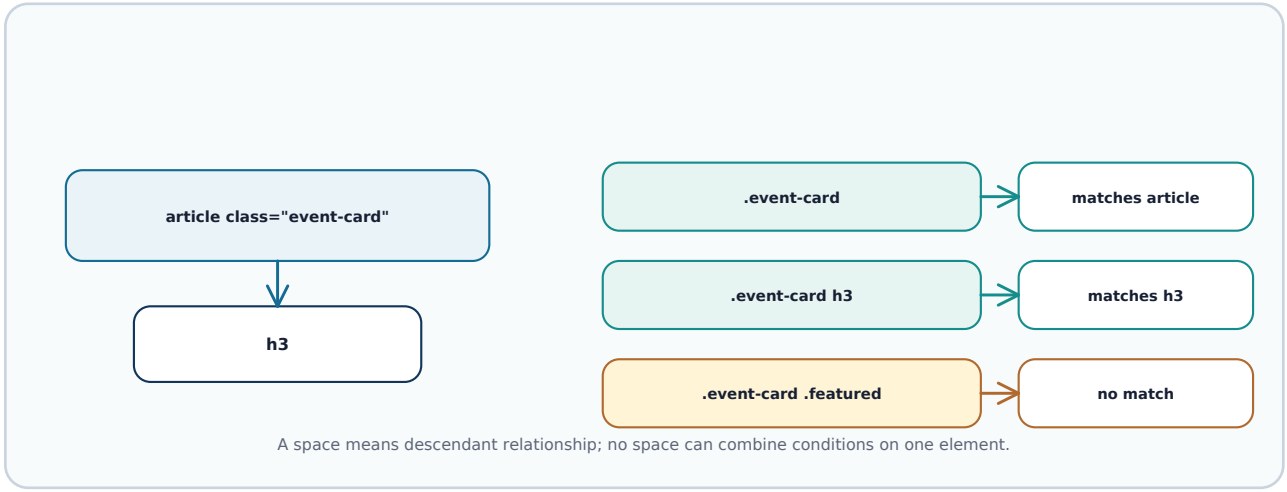


Figure 3.2. Selectors express different relationships and produce different sets of matched elements.

2.1 Build selector literacy before composing long selectors

Table 3.2 introduces the selector families that a beginning web programmer must be able to read. The examples assume the semantic Campus Hub document.

Table 3.2 - CSS selector families and the relationships they express

Selector family	Example	Meaning
Type	`article`	Every `article` element
Class	`.event-card`	Elements whose `class` list contains `event-card`
ID	`#events`	The element whose document ID is `events`
Attribute	`input[required]`	`input` elements with a `required` attribute
Attribute value	`input[type="email"]`	`input` elements whose `type` value is `email`
Pseudo-class	`a:hover`, `:focus-visible`	Elements in a state or structural condition
Pseudo-element	`.event-card::before`	A stylable generated box associated with a matched element
Descendant combinator	`.event-card h3`	Matching `h3` elements anywhere inside an event card
Child combinator	`.site-navigation > ul`	Matching `ul` elements that are direct children
Next-sibling combinator	`h2 + p`	A matching paragraph immediately after an `h2`
Subsequent-sibling combinator	`h2 ~ p`	Matching later sibling paragraphs after an `h2`
Selector list	`h1, h2, h3`	The union of several selectors; each item is evaluated separately

Whitespace around `>` or `+` improves readability, but the space between `.event-card` and `h3` is itself the descendant combinator. By contrast, `.event-card.featured` contains no space and matches one element that has both classes.

Classes are normally the most reusable styling hooks. IDs are valid selectors, but their high specificity and one-document identity make them better suited to fragment targets, labels, and scripting identities than routine component styling. Attribute selectors are valuable when presentation genuinely follows a semantic attribute or state.

Pseudo-classes such as `:hover`, `:focus-visible`, `:checked`, `:disabled`, `:valid`, and `:invalid` describe conditions. Hover must never be the only way to reveal essential information because touch, keyboard, and assistive input may not produce it. Pseudo-elements such as `::before` and `::after` can add decorative boxes, but generated content is not a reliable home for essential text.

Modern functional pseudo-classes make selectors more expressive. `:where()` always contributes zero specificity, which is useful for low-priority defaults. `:is()`, `:not()`, and `:has()` take the specificity of their most specific selector argument rather than contributing an extra pseudo-class unit of their own. Use `:has()` for relationships that CSS could not previously express, but do not hide an unclear document model behind a difficult selector.

2.2 The cascade resolves competing declarations

When more than one declaration proposes a value for the same property on the same element, the cascade resolves the conflict. Figure 3.3 turns that decision into an ordered path rather than the misleading rule “the last declaration wins.”

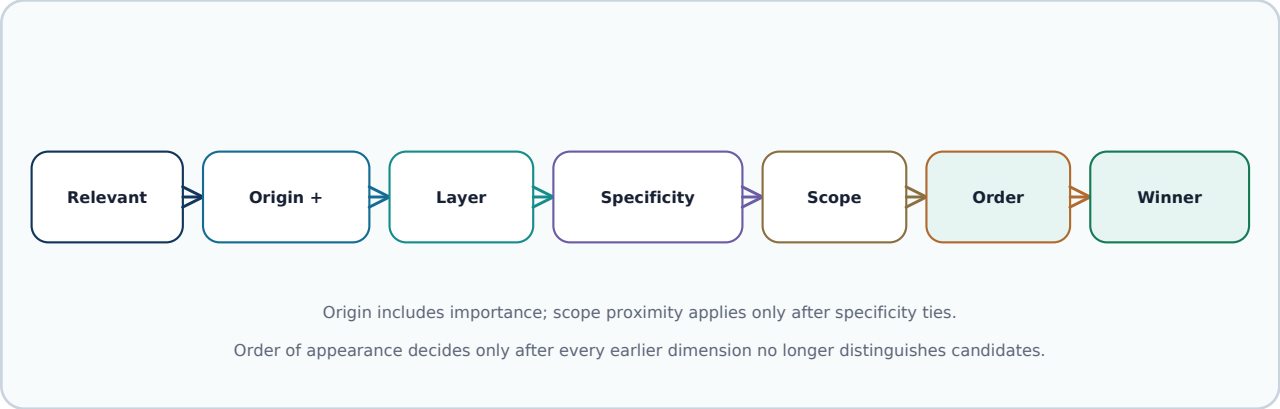


Figure 3.3. The cascade resolves relevant declarations through an ordered decision process.

At a practical introductory level, ask these questions in order:

1. Is the declaration relevant to this element, document, condition, and medium?
2. Which origin and importance level has precedence?
3. Within that origin and importance level, which cascade layer has precedence?
4. Which selector has greater specificity?
5. If the rules use `@scope` and specificity ties, which scope root is nearer?
6. If the earlier dimensions tie, which declaration comes later in the style order?

The full cascade also gives running transitions and animations special positions. Table 3.3 records the parts that most often surprise authors. It is a reasoning guide, not an invitation to override user needs.

Table 3.3 - Practical cascade precedence and important qualifications

Situation	What to remember
Normal author declarations	Unlayered author rules outrank normal author rules inside layers; among normal layers, later declared layers outrank earlier layers.
Important declarations	`!important` reverses layer precedence; important declarations in earlier layers protect foundational constraints.
User important declarations	Important user preferences outrank important author declarations so users can override author presentation.
Keyframe animations	Animated values outrank normal declarations but not important declarations.
Running transitions	A property currently transitioning has the highest cascade precedence for its duration.
Inline style	A style attribute participates as author styling with special precedence in specificity comparisons; avoid treating it as a routine architecture.
Source order	Later wins only after relevance, origin, importance, layer, specificity, and scoping proximity fail to distinguish candidates.

Do not memorize the table as a license to use `!important`. First inspect which declaration won and why. The browser Styles panel normally shows losing declarations crossed out and identifies their source files and line numbers.

2.3 Specificity is a comparison, not a score to maximize

Specificity is compared as three categories: ID selectors; class, attribute, and pseudo-class selectors; and type and pseudo-element selectors. Compare the ID category first. If it ties, compare the class-like category; if that ties, compare the type-like category. Do not interpret `1-0-0` and `0-12-0` as decimal values: one ID category still wins the first comparison.

```
/* Specificity 0-1-0: reusable component hook */
.event-card__title { color: var(--color-heading); }

/* Specificity 1-2-3: coupled to one location and difficult to override */
#main-content section.events article.event-card h3 { color: navy; }
```

Prefer shallow class selectors for component styling. Use source order and explicit cascade layers intentionally. Avoid `!important` as a routine escape; it changes the conflict level and tends to create an escalating override contest. When third-party CSS must be controlled, place it in a deliberately low-priority layer rather than fighting every selector.

2.4 Inheritance is not selector matching

Properties such as `color` and `font-family` normally inherit. Properties such as `margin`, `border`, and `width` normally do not. A directly targeted declaration on a child wins over an inherited value even when the ancestor's selector has greater specificity, because specificity is not compared between direct and inherited declarations.

```
body {
  color: #172033;
  font-family: system-ui, sans-serif;
}
```

Descendant text normally inherits these values. A card does not inherit the body's margin because margin is not inherited.

CSS-wide keywords make the decision explicit:

- `inherit` takes the parent's computed value even for a property that does not

normally inherit;

- `initial` uses the property's specification-defined initial value, which is not

necessarily the browser's element default;

- `unset` behaves like `inherit` for inherited properties and `initial` for the

others;

- `revert` rolls back the current origin's contribution; and
- `revert-layer` rolls back the current cascade layer's contribution.

These keywords are precise tools. A global `all: unset` can remove display and interaction affordances that a component still needs.

2.5 Custom properties carry decisions through the cascade

Custom properties can represent repeated decisions:

```
:root {
  --color-ink: #172033;
  --color-accent: #146c94;
  --space-2: 0.5rem;
  --space-4: 1rem;
  --radius: 0.5rem;
}

.event-card {
```

```
    color: var(--color-ink);
    border-radius: var(--radius);
  }
```

Custom property names begin with two hyphens, are case-sensitive, participate in the cascade, and inherit by default. The `var()` function substitutes their value where a property's grammar permits it. A fallback such as `var(--card-surface, white)` is used when the referenced custom property is missing or invalid at substitution time; it is not a compatibility fallback for an arbitrary unsupported value.

A token name should express purpose when possible. `--color-accent` survives a palette change better than `--blue`. Tokens centralize decisions, but they do not automatically create a coherent design system; document their purpose and test every combination in which they are used.

3. Boxes turn declared values into geometry

Most visible layout questions eventually become box questions: which box is this, how large is it allowed to become, which containing block supplies its reference, and does it remain in normal flow?

3.1 `display` establishes outer and inner behavior

An element's `display` value controls how its principal box participates in its parent's layout and, for layout containers, how its children are arranged.

- `display: block` normally creates a block-level box that takes the available

inline space and begins on a new line.

- `display: inline` participates in line layout; ordinary inline boxes do not use

`width` and `height` like block boxes.

- `display: inline-block` participates inline while its inside behaves as a block

container.

- `display: flex` and `display: grid` create block-level layout containers for

their children.

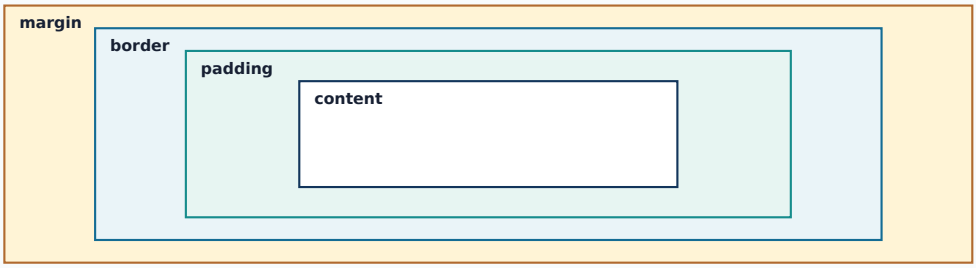
- `display: none` removes the element's generated boxes and normally removes it

from the accessibility tree. It is not merely visual transparency.

HTML semantics and CSS display are related only through browser defaults. A `nav` element can be laid out with Grid, and a `span` can be displayed as a block, without changing what those elements mean.

3.2 Content, padding, border, and margin form the box model

The standard box model consists of content, padding, border, and margin. Figure 3.4 nests the four regions so their boundaries are easier to distinguish.



border-box includes content, padding, and border; margin remains outside.

Figure 3.4. Padding, border, and margin surround an element's content box.

With the default `box-sizing: content-box`, a declared `width` or `inline-size` applies to the content box. Padding and borders are added outside it. A common project baseline is:

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

Now a declared size includes content, padding, and border. Margin remains outside. This does not make every box fit: a fixed `20rem` border box still cannot fit a `17rem` container, and an unbreakable string can still impose a larger minimum.

Padding belongs inside the border and inherits the element's background. Margin is transparent space outside the border. Borders contribute size and can convey boundaries, but a difference that exists only in a faint border may not be perceivable. `outline` differs from `border`: an outline does not occupy box-model space, which makes it useful for focus indicators that should not shift layout.

3.3 Normal flow is a resilient default

Block content ordinarily follows document order and expands in the block direction to contain its content. Inline content participates in line layout. Normal flow handles unknown text length, font substitution, translation, and many zoom cases well. Removing an element from flow means its size no longer reserves ordinary space for neighbors.

An absolutely positioned card with fixed coordinates may overlap when text grows. A normally flowing card contributes its size to surrounding layout. Begin with flow, then introduce a different layout model because a real relationship demands it.

3.4 Sizing begins with intrinsic content

An **intrinsic size** comes from the content itself. A word has an unbroken minimum contribution; an image has intrinsic dimensions; a paragraph has a preferred width at which its lines would not wrap. CSS sizing combines these contributions with available space and author constraints.

`min-inline-size`, `max-inline-size`, `min-block-size`, and `max-block-size` express boundaries without forcing one exact size. The following page shell can shrink with its container but stops growing at `72rem`.

```
.page-shell {
  inline-size: min(100% - 2rem, 72rem);
  margin-inline: auto;
}
```

`min-content`, `max-content`, and `fit-content()` expose useful intrinsic sizing ideas, but they are not synonyms for small, large, and responsive. Test them with real content. Flex and Grid items also

have an automatic minimum size that can surprise authors. `min-inline-size: 0` allows a particular item to shrink below its min-content contribution when wrapping or clipping behavior makes that safe.

Replaced elements such as images and video bring intrinsic dimensions and aspect ratios from outside ordinary text layout. Width and height attributes in HTML can reserve the correct ratio before a resource loads. CSS can then constrain the rendered box. The `aspect-ratio` property is useful when the ratio is part of the design, while `object-fit: cover` or `contain` controls how replaced content fits inside a deliberately sized box. Cropping with `cover` must not remove information that users need.

3.5 Logical properties follow the writing system

`left`, `right`, `top`, and `bottom` are physical directions. CSS also defines logical directions relative to the writing mode. The **inline axis** follows a line of text; the **block axis** follows the direction in which lines and blocks accumulate. In English the inline axis is normally horizontal and begins on the left. In Arabic or Persian it normally begins on the right. Some writing systems can use vertical flow.

Figure 3.5 shows why `margin-inline`, `padding-block`, `inline-size`, and `inset-inline-start` are more adaptable than assumptions about left, right, width, and top.

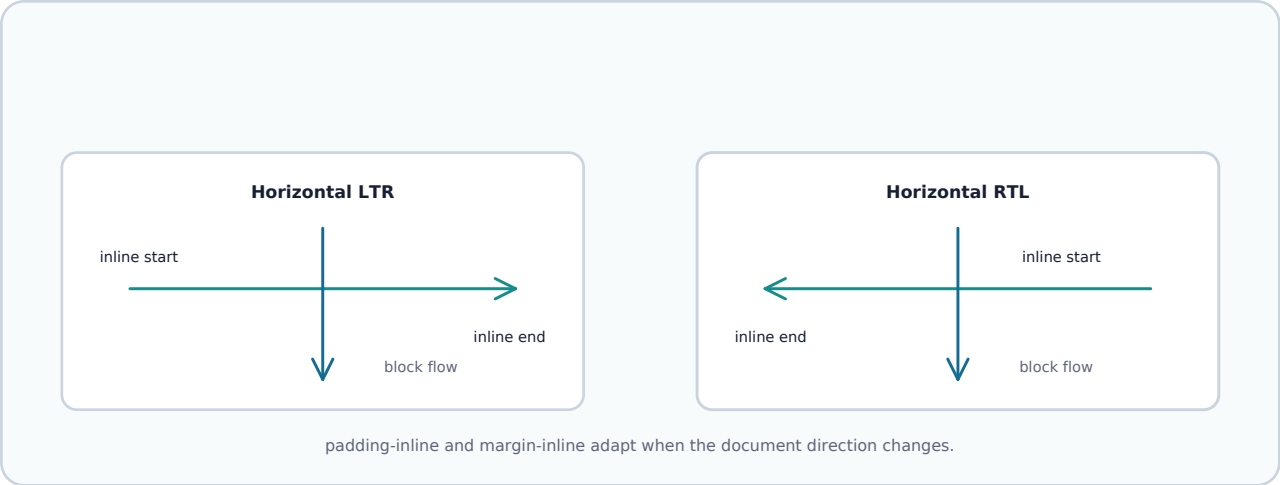


Figure 3.5. Logical inline and block axes follow writing mode and direction instead of assuming physical sides.

```
.event-card {
  padding-block: 1.5rem;
  padding-inline: 1rem;
  border-inline-start: 0.25rem solid var(--color-accent);
}
```

This border follows the inline start edge when direction changes. Logical properties do not translate content or correct an incorrect HTML `dir` value; they allow layout to respect direction information already present.

3.6 Margin collapse and overflow provide evidence

Adjacent block-axis margins in normal flow can collapse under defined conditions, so the resulting separation may equal the larger margin instead of their sum. Padding, a border, a new formatting context, or a layout container changes those conditions. If spacing appears to disappear, inspect the boxes instead of adding larger and larger margins.

Overflow often reveals an incorrect constraint:

- a fixed size wider than the available inline size;

- unbreakable content such as a long Uniform Resource Locator (URL);
- a Flex or Grid item whose automatic minimum prevents shrinking;
- an image without a responsive maximum;
- a child sized to `100vw` inside a page that also has padding or a scrollbar; or
- a transformed or positioned object extending beyond the scrollable area.

Choose overflow behavior locally. `overflow: auto` can create a scroll container where two-dimensional content genuinely needs it, such as a wide data table. `overflow: clip` or `hidden` may be correct for deliberate visual effects, but clipping must not make focused or essential content unreachable. Do not hide a page-wide symptom with `overflow-x: hidden`; find the box that exceeds its container and correct the cause.

3.7 Positioned layout changes reference and flow

Positioning is useful when an object must be anchored to a box or viewport, but each mode has a different reference:

- `static` is the normal positioned state; inset properties do not reposition it;
- `relative` keeps the box's normal-flow space and offsets its visual position;

it can also establish the containing block for positioned descendants;

- `absolute` removes the box from normal flow and positions it relative to an

appropriate containing block;

- `fixed` is generally positioned relative to the viewport; and
- `sticky` behaves in flow until a scroll threshold is reached within its scroll

container.

Sticky and fixed headers can obscure headings, skip-link targets, or keyboard focus. Test them at high zoom and with `scroll-margin-block-start` where anchored content needs clearance.

`z-index` participates in **stacking contexts**; it is not one global number line for the whole page. Positioning, opacity, transforms, isolation, and other properties can create separate contexts. If `z-index: 9999` appears ineffective, inspect the ancestor stacking contexts rather than choosing a still larger number. Transforms also change geometry visually without reserving new normal-flow space, so they are not a substitute for layout.

4. Values express type, scale, color, and visual hierarchy

CSS values are typed. A property may accept a keyword, number, length, percentage, color, image, position, function, or a combination defined by its grammar. Browsers move values through several stages: a declaration supplies a specified value; the cascade and inheritance contribute a computed value; layout resolves a used value; rendering may finally round it to device pixels. This is why the Computed panel can show something different from the text originally written in the stylesheet.

4.1 Units communicate relationships

Table 3.4 compares units and functions by the relationship they express rather than treating them as interchangeable ways to write a number.

Table 3.4 - CSS units and responsive sizing functions

Unit or function	Relationship and useful role
`px`	A CSS reference pixel, not necessarily one physical screen pixel; useful for fine borders and precise thresholds
`rem`	A root-em length: `1rem` equals the computed `font-size` of the root `html` element; useful when many elements should share one scale
`em`	A local font-relative length: for most properties, `1em` equals the element's own computed font size; in a `font-size` declaration, it is resolved from the inherited parent size
`%`	Relative to a property-specific reference; inspect the property's definition instead of assuming it always means parent width
`vw`, `vh`	One percent of the viewport width or height; useful carefully because browser UI and extreme aspect ratios change the result
`svh`, `lvh`, `dvh`	Small, large, and dynamic viewport heights; useful when mobile browser interfaces change the visible viewport
`ch`	The advance measure of the `0` glyph; a practical approximation for readable line-length constraints
`fr`	A fraction of remaining space in Grid after non-flexible track sizing is considered
`min()`, `max()`	Choose a boundary from comparable values at used-value time
`clamp(min, preferred, max)`	Bound a preferred fluid value between a minimum and maximum

What `rem` means

The name `rem` is short for **root em**. In an HTML document, the root element is `html`. A length written in `rem` therefore asks the browser to begin with the computed `font-size` of `html` and multiply it by the number before the unit. If the root size is represented by R, the calculation is:

```
x rem = x × R
```

Many browsers, with their ordinary settings unchanged, compute the initial root font size as `16px`. That is a useful classroom example, not a promise made by CSS. A user can select a larger default text size, a browser or operating system can use another setting, and an author can change the root size. Always reason from the **computed root font size**, not from a memorized assumption that `1rem` must equal 16 pixels.

With a computed root size of `16px`, the arithmetic is direct:

```
0.5rem = 0.5 × 16px = 8px
1rem   = 1   × 16px = 16px
1.5rem = 1.5 × 16px = 24px
2rem   = 2   × 16px = 32px
```

If the root size becomes `20px`, the same declarations resolve to `10px`, `20px`, `30px`, and `40px`. The stylesheet has not changed; the common reference has changed. Figure 3.6 follows that one root reference into several declarations and contrasts it with the local chain used by `em`.

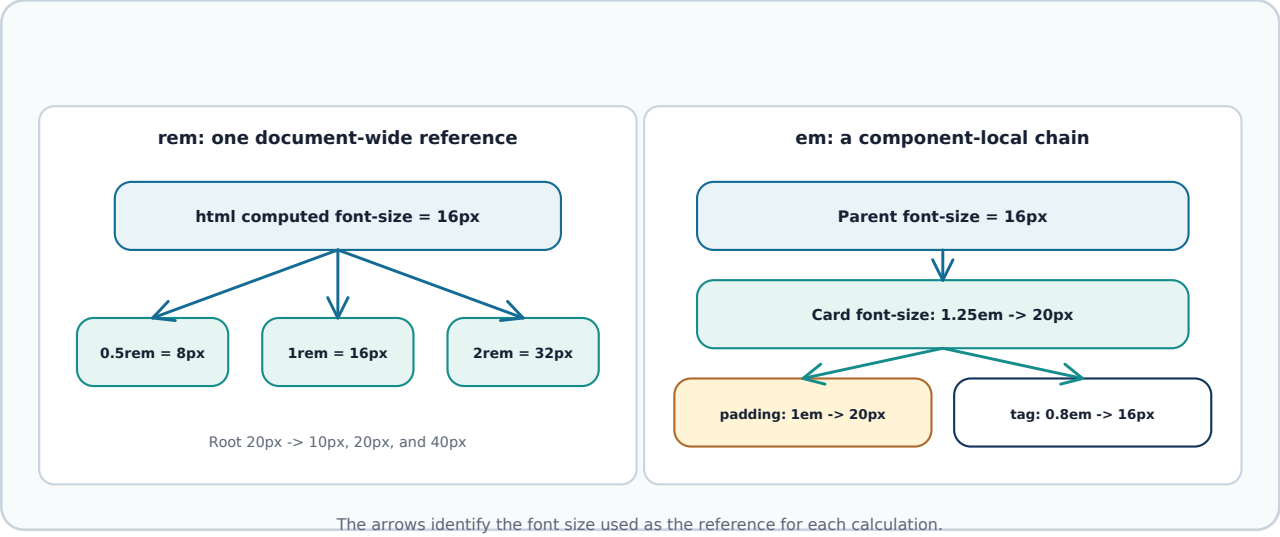


Figure 3.6. Every rem length uses one root reference, while em can follow a component's local font-size chain.

The most important relationship in Figure 3.6 is not the particular number `16px`. It is that every `rem` length points back to the same root. This makes a spacing and type scale easier to reason about across nested components.

Read a `rem` declaration as a calculation

Consider a small part of the Campus Hub stylesheet.

```
body {
  font-size: 1rem;
}

.event-card {
  padding: 1rem;
  gap: 0.75rem;
}

.event-card__title {
  font-size: 1.5rem;
}
```

If the computed root font size is `16px`, the body text is `16px`, the card padding is `16px`, the gap is `12px`, and the title is `24px`. Nesting the title inside the card does not change those calculations because every value still uses the root. In developer tools, the rule may remain visible as `1.5rem` in the Styles panel while the Computed panel reports the resolved size, such as `24px`. The authored value communicates the relationship; the computed value supplies evidence of what the browser is using now.

Do not mentally replace every `rem` with pixels while designing. Write the relationship first, then use the current root size when a concrete measurement is needed for debugging. The conversion changes if the root changes.

`rem` and `em` answer different questions

Both units are font-relative, but they do not normally consult the same element. `rem` asks, “How large is this compared with the document's root text size?” `em` usually asks, “How large is this compared with this component's text?”

One detail deserves careful attention. In a `font-size` declaration, an `em` value is resolved from the font size inherited from the parent because the element's own new font size has not yet been established. For most other properties, such as `padding`, `gap`, or `border-radius`, `em` is resolved from the element's computed font size.

```
.event-card {
  font-size: 1.25em;
  padding: 1em;
}

.event-card__tag {
  font-size: 0.8em;
}
```

Suppose the card's parent has a computed font size of `16px`. The card's `font-size: 1.25em` becomes `20px`. Its `padding: 1em` then becomes `20px` because padding uses the card's computed font size. The tag inherits `20px` from the card, so `0.8em` becomes `16px`. This local scaling can be exactly what a self-contained badge, icon, or button needs. It can also surprise you when nested components multiply sizes repeatedly. Using `rem` avoids that accidental compounding when the intended reference is the document-wide scale.

A practical starting policy is:

- use `rem` for the main type scale, page and component spacing, and dimensions

that should follow the user's root text setting consistently;

- use `em` when a detail should deliberately scale with the text of its own

component, such as an icon, badge padding, or a button's internal spacing;

- use `px` where a CSS reference pixel expresses the intended relationship well,

such as a thin border; and

- choose percentages, viewport units, container units, `ch`, Grid fractions, or

sizing functions when their different reference is the relationship you need.

This is a design policy, not a rule that makes one unit universally superior. For example, browser zoom enlarges content written in pixels too. The benefit of `rem` is the explicit connection to the root font setting; accessibility still requires rendered testing.

Preserve the user's starting point

You may encounter the following shortcut in existing stylesheets.

```
html {
  font-size: 62.5%;
}
```

When the browser's starting size happens to be `16px`, `62.5%` produces `10px`, which makes decimal conversion to pixels look convenient. The result is not guaranteed to be `10px`, however, because the user's starting size may differ. It also makes the document's normal text smaller unless later rules enlarge it. This course does not change the root merely to make mental arithmetic resemble decimal notation. Leave the user's root setting intact unless the design has a well-tested reason to change it, and never treat `62.5%` as required boilerplate.

Relative units reduce some rigid relationships, but they cannot prevent clipped text, fixed-height controls, or layouts that fail under enlargement. Verify the rendered page at 200% text enlargement or the applicable browser test, at browser zoom, and at narrow reflow widths. The goal is not the presence of `rem`; the goal is content and functionality that remain available.

Checkpoint - calculate before opening developer tools

The root computed font size is `18px`. A card has `padding: 1.25rem`; its title has `font-size: 1.5rem`. The card also has `font-size: 1.1em` inside a parent whose computed font size is `20px`, and a local badge uses `padding-inline: 0.5em`. What values should you predict?

Answer: the card padding is `22.5px` because `1.25 × 18 = 22.5`; the title is `27px` because `1.5 × 18 = 27`. The card's own font size is `22px` because `1.1 × 20 = 22`. If the badge inherits that `22px` font size and does not set a different one, each inline padding is `11px` because `0.5 × 22 = 11`. The common mistake is to calculate every value from `18px`: only the `rem` values use the root; these `em` values follow the local font-size chain.

Physical units such as `cm`, `mm`, `in`, and `pt` are anchored to CSS reference units on screens and do not promise an accurate physical ruler. They are most appropriate when print output has a real physical design requirement.

Avoid setting ordinary paragraph text in viewport units alone; extreme viewport dimensions can make it unreadably small or large and can resist effective text scaling. A bounded fluid size is safer:

```
h1 {
  font-size: clamp(2rem, 1.4rem + 3vw, 4rem);
}
```

This rule contributes to the heading visible in Figures 3.10 and 3.11. The minimum and maximum protect extremes; the middle term allows gradual change. Always test zoom and user font settings rather than assuming a formula is accessible.

4.2 Color notation is separate from color usability

CSS accepts named colors, hexadecimal notation, `rgb()`, `hsl()`, and newer color functions. The Campus Hub palette uses hexadecimal values because they are compact and familiar, but the notation does not determine quality. `#146c94` and an equivalent `rgb()` value describe the same sRGB color.

Alpha controls transparency. Semi-transparent foregrounds blend with whatever is behind them, so their final contrast depends on the complete stack. The `currentColor` keyword reuses an element's computed `color`, which can keep icons and borders coordinated. System color keywords can cooperate with forced-color modes where author colors may be replaced.

Table 3.5 connects visual decisions with their accessibility implications.

Table 3.5 - Color, contrast, and non-color evidence

Decision	Required reasoning
Ordinary text contrast	Web Content Accessibility Guidelines (WCAG) 2.2 Level AA normally requires at least 4.5:1 against the actual background.
Large text contrast	Qualifying large text normally requires at least 3:1; do not enlarge text merely to obtain the lower threshold.
User-interface and graphical boundaries	Meaningful visual information normally needs at least 3:1 contrast against adjacent colors under the relevant criterion.
Error, status, or selection	Color may reinforce the state but cannot be its only cue; include text, shape, iconography, or another perceivable difference.
Focus indicator	It must remain visible against the component and surrounding surface; test actual colors and states rather than trusting the palette name.

Contrast ratios are measurements, not aesthetic judgments. A compliant pair can still be tiring at small sizes, and a strong-looking pair may fail when opacity or a background image changes. Measure the rendered combination and test high contrast or forced-color settings.

4.3 Typography controls reading, not merely decoration

A font stack provides ordered alternatives:

```
body {
  font-family: system-ui, -apple-system, "Segoe UI", sans-serif;
```

```
  font-size: 1rem;
  line-height: 1.6;
}
```

The browser chooses the first available face capable of representing the text. Generic families such as `sans-serif` provide a final category rather than a specific file. Font availability, glyph coverage, weight synthesis, and metrics can change line breaks and component height, so test more than one platform and include the languages the course supports.

Use unitless `line-height` for ordinary text when descendants should scale the ratio with their own font sizes. Very tight line height can cause lines to collide; excessive line height separates related text. A useful body-text starting point is often between 1.4 and 1.7, followed by testing with the actual typeface, language, width, and user overrides.

Readable paragraphs also need a controlled measure. Approximately 45-75 characters per line is a useful starting range, not a conformance rule. The project uses `max-inline-size: 68ch` and tests actual content. Avoid justified alignment in narrow digital columns when it creates disruptive gaps. Preserve link underlines unless another strong, non-color affordance makes links evident.

Web fonts can strengthen identity, but they add network requests and may cause late metric changes. Use only needed weights and styles, choose appropriate fallbacks, and evaluate performance and layout movement. The `font-display` descriptor belongs inside `@font-face`; its choice trades immediate fallback text against the timing of the downloaded face.

4.4 Backgrounds, borders, and shadows must support content

`background-color` paints beneath content and padding. Background images can be positioned, sized, and repeated, while CSS gradients are generated images. They are presentation: essential content and alternative text still belong in HTML.

```
.event-card {
  background: white;
  border: 1px solid var(--color-border);
  border-radius: var(--radius);
  box-shadow: 0 0.25rem 1rem rgb(18 53 91 / 8%);
}
```

The completed rule appears in the cards shown in Figures 3.9 and 3.10. The border supplies a dependable boundary; the subtle shadow adds depth but is not the only separation. Large blurred shadows and full-page background images can increase paint cost and reduce text contrast.

4.5 Form controls retain platform behavior

Form controls do not all inherit typography consistently, and their internal rendering varies across operating systems and browsers. Begin with a small, testable baseline rather than removing every native feature.

```
button,
input,
select,
textarea {
  font: inherit;
}

button,
select {
  min-block-size: 2.75rem;
  padding-block: 0.6rem;
  padding-inline: 0.9rem;
}
```

The filter controls in Figures 3.9 and 3.10 use this rule. The minimum block size creates a comfortable target under the tested font settings, but a CSS declaration alone does not prove

WCAG target-size conformance: measure the rendered target and its spacing. Preserve recognizable states, keyboard operation, labels, and native semantics. `appearance: none` transfers more responsibility to the author and should be used only when every required affordance is rebuilt and tested.

5. Choose a layout model from the relationship

Normal flow, Flexbox, and Grid are cooperating tools, not successive generations where the newest replaces everything before it. Table 3.6 begins with the content relationship and then selects the smallest model that expresses it.

Table 3.6 - Choosing a CSS layout model

Relationship	Likely starting model	Reason
Headings and paragraphs in reading order	Normal flow	Content naturally stacks and expands without explicit tracks.
Navigation, toolbar, button group, or aligned row/column	Flexbox	One primary axis controls distribution and alignment.
Card gallery, page regions, or timetable	Grid	Rows and columns or repeated tracks must coordinate.
Image with surrounding prose	Float, used narrowly	Float remains appropriate for content wrapping around an item, not for whole-page layout.
Genuine tabular data	Semantic HTML table	Data relationships require table semantics; changing CSS display does not create them.

Figure 3.7 contrasts Flexbox's primary-axis coordination with Grid's row-and-column coordination.

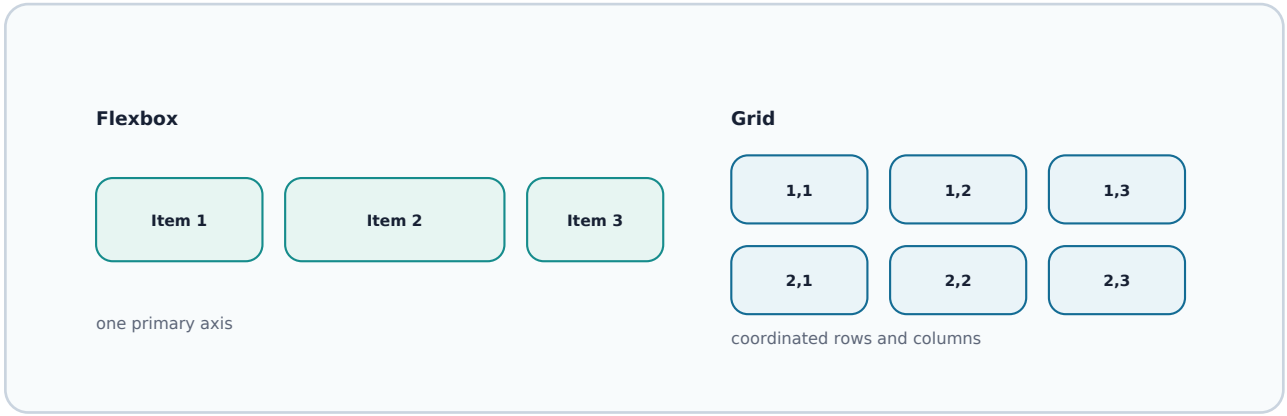


Figure 3.7. Flexbox coordinates one primary axis; Grid coordinates rows and columns.

5.1 Flexbox distributes items along a main axis

Setting `display: flex` creates a flex container. Its direct children become flex items. The **main axis** follows `flex-direction`; the **cross axis** is perpendicular. In the default horizontal writing mode, `row` follows the inline axis, but “row” should not be memorized as “left to right” because direction and writing mode affect it.

```
.site-navigation ul {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  gap: 0.5rem 1rem;
}
```

Wrapping is a resilience decision. Without it, translated labels, user font settings, or zoom can force horizontal overflow. `justify-content` distributes free space along the main axis. `align-items` aligns items on the cross axis. These properties align boxes; they do not replace padding within a control.

Each flex item has grow, shrink, and basis components. The shorthand `flex: 1 1 16rem` means the item may grow, may shrink, and begins flexing from a `16rem` basis. Avoid using `flex: 1` without understanding its expanded values. A long word or intrinsically wide child may still resist shrinking because of the automatic minimum; inspect the item and use wrapping or `min-inline-size: 0` when that is the correct content policy.

Auto margins can absorb available main-axis space for a specific relationship. They are often clearer than adding an empty spacer element. `gap` belongs to the container and creates consistent separation without special first/last-child margin rules.

5.2 Grid coordinates tracks in two dimensions

Setting `display: grid` creates a grid container. Its direct children become grid items. Rows and columns are **tracks**; the boundaries are **grid lines**; and the space between tracks is the gap.

```
.event-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 17rem), 1fr));
  gap: 1rem;
}
```

The inner `min(100%, 17rem)` permits a track to shrink when the container itself is narrower than `17rem`. `minmax()` sets the track's lower and upper bounds. `auto-fit` creates as many repeated tracks as fit and collapses empty ones. `1fr` shares remaining track space after fixed and intrinsic contributions have been considered. This rule produces the one- and three-column arrangements in Figures 3.9 and 3.10 without naming phones, tablets, or laptops.

The explicitly declared tracks form the **explicit grid**. Auto-placement can create rows or columns in the **implicit grid** when more items exist. Use `grid-auto-rows` or `grid-auto-columns` when those tracks need a deliberate policy. Items can be placed by line numbers or named areas, but auto-placement is often more resilient for collections whose item count changes.

`subgrid` allows a nested grid to use track sizing from its parent along a chosen axis. It is useful when internal labels or actions across several cards must align, but it should solve a real shared-track relationship rather than compensate for unnecessary nesting.

5.3 Alignment is not ordering

Flexbox and Grid can change visual placement with `order`, explicit grid lines, or named areas. The DOM order still governs important sequential experiences such as reading and keyboard focus. Do not use visual reordering to repair incorrect HTML order. Choose a coherent source order first, then place it responsibly and test the linearized page.

Alignment also has a vocabulary. `justify-*` and `align-*` act on different axes, and the exact effect depends on the layout method. When a property seems to do nothing, ask whether free space exists on that axis and whether the property applies to the container or the item.

6. Responsive design treats context as an input

Responsive design is not a list of device sizes. A page must work in resized windows, split-screen views, embedded panels, browser zoom, different font settings, portrait and landscape orientations, and devices that did not exist when the stylesheet was written. Begin with flexible content and narrow constraints, then enhance when a real relationship has room.

6.1 Viewport behavior begins in HTML

Chapter 2 supplied this metadata in the document `head`:

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

It asks a mobile browser to make the layout viewport correspond to the device's CSS-pixel width at the initial scale. It does not make a fixed layout responsive. Never disable user scaling with `user-scalable=no` or a restrictive maximum scale; zoom is an essential user capability.

6.2 Media queries respond to the environment

Start with a useful single-column layout. Add a media query when the content has enough space to support an enhancement:

```
.page-shell {
  width: min(100% - 2rem, 72rem);
  margin-inline: auto;
}

@media (min-width: 48rem) {
  .dashboard {
    grid-template-columns: minmax(0, 2fr) minmax(16rem, 1fr);
  }
}
```

The breakpoint is justified by the layout's content, not by a specific device model. New devices and split-screen windows need not match a familiar label. Figure 3.8 compares the same card content under narrow and wide constraints.

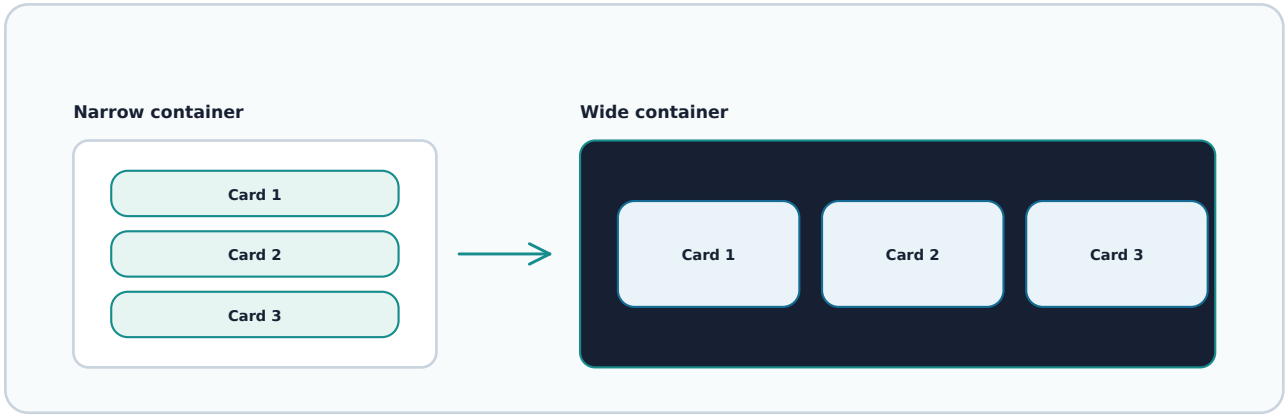


Figure 3.8. Responsive constraints reflow the same content without changing its meaning.

Figure 3.9 provides a more concrete view of that principle across a desktop, tablet, and phone. The content remains recognizably the same; only its layout adapts to the available space.

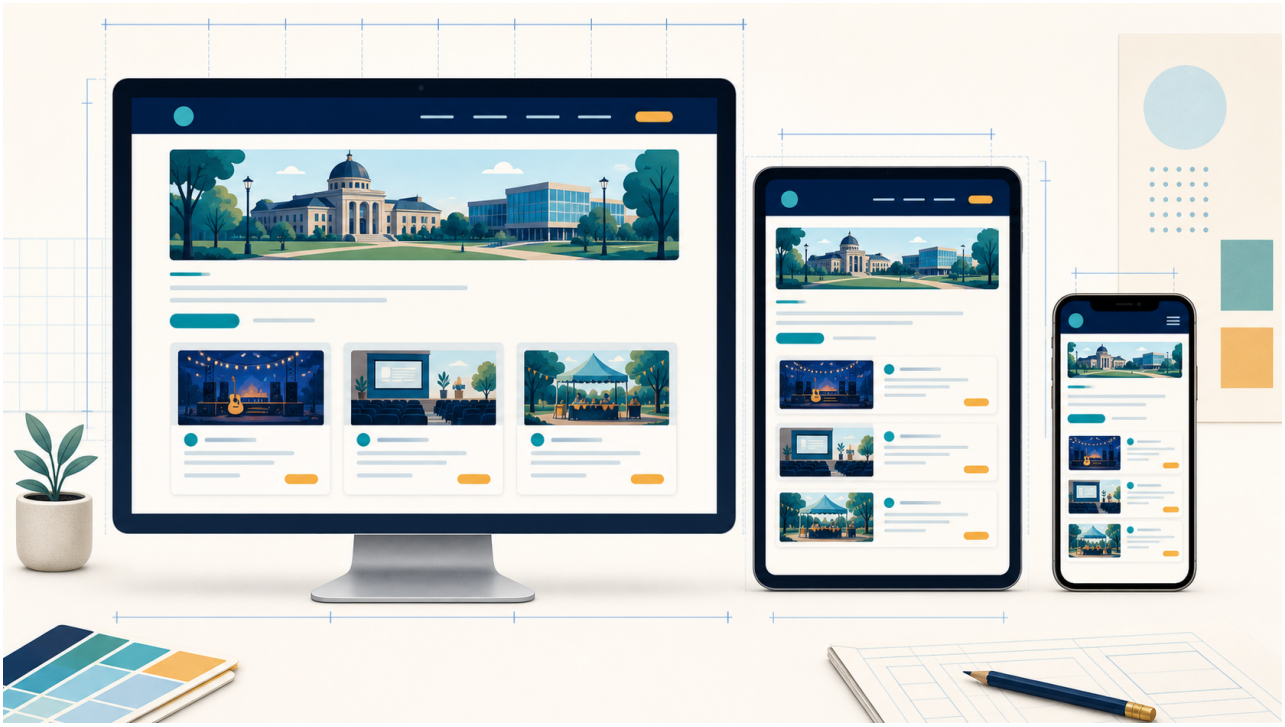


Figure 3.9. Responsive design preserves content and reading order while the layout adapts to the available viewport.

The complete example produces the wide browser output in Figure 3.10. When the viewport narrows, the same source produces Figure 3.11 without a separate mobile document or duplicated content.

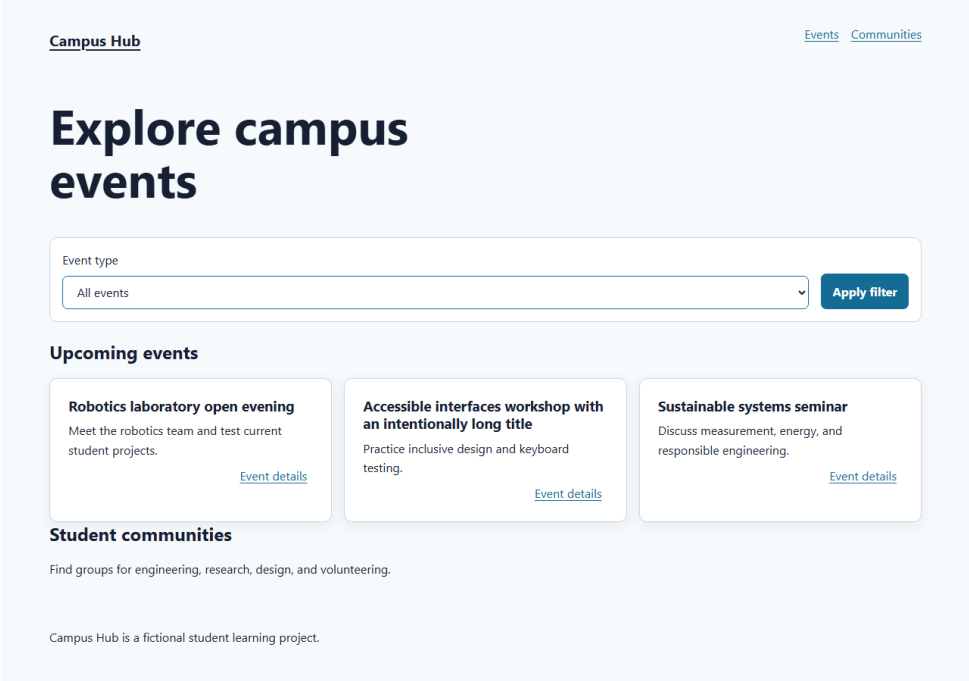


Figure 3.10. At a wide viewport, the completed CSS example presents the filter and event cards with space for enhanced component layout.

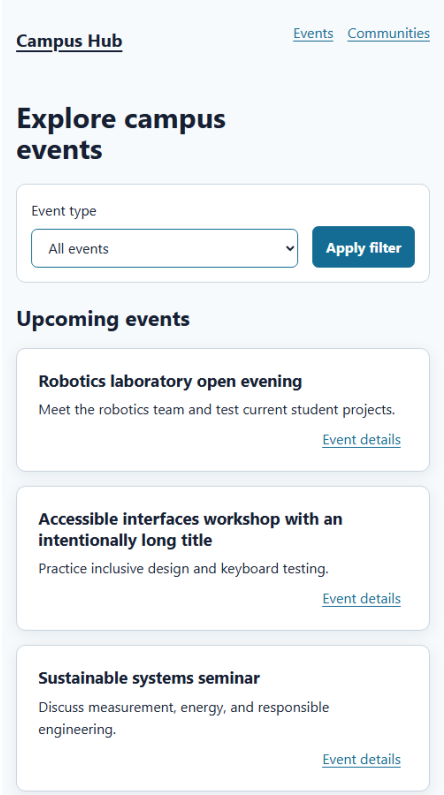


Figure 3.11. At a narrow viewport, the same HTML and CSS reflow the filter and event cards into one readable column.

Use range syntax or the established `min-width`/`max-width` syntax consistently. Width queries describe the viewport, not the screen hardware. Media features can also describe orientation, hover capability, pointer precision, color scheme, contrast preference, forced colors, and motion preference. Avoid inferring a person or device class from one feature; a laptop may have both touch and a mouse.

6.3 Container queries make components context-aware

A media query knows about the viewport. A reusable component often needs to know about the space its parent gives it. The same event card may appear wide in a main region and narrow in a sidebar at the same viewport width.

```
.event-card {
  container-name: event-card;
  container-type: inline-size;
}

@container event-card (min-width: 20rem) {
  .event-card__content {
    grid-template-columns: 1fr auto;
    align-items: end;
  }
}
```

`container-type: inline-size` establishes size containment needed for inline-size queries. The named query applies to descendants of the container, not the container box itself. Figure 3.12 shows two instances of one component responding differently because their containers provide different inline sizes.

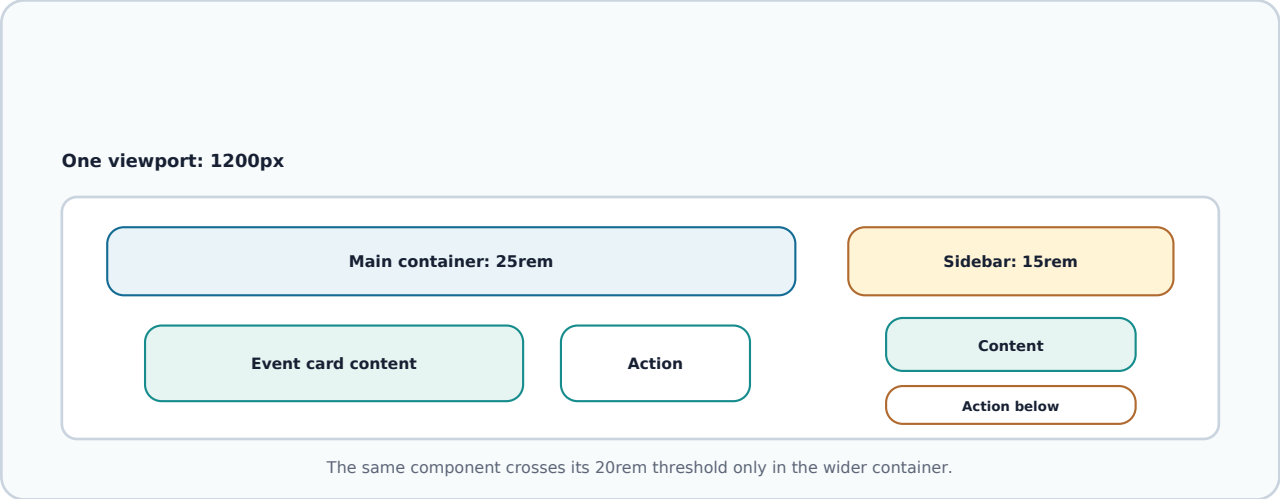


Figure 3.12. One component can adapt to the inline size of its own container at a single viewport width.

Container queries complement media queries. Use media queries for page-level environment changes and user preferences; use container queries when a component's own allocated space should govern its internal arrangement. Container query units such as `cqi` are relative to a query container, but ordinary `rem`, percentages, and intrinsic sizing often remain easier to reason about.

6.4 Images and embedded media

Prevent ordinary images from exceeding their containers:

```
img,
svg,
video {
  max-inline-size: 100%;
  block-size: auto;
}
```

Reserve aspect ratio or dimensions when possible so late-loading media does not cause avoidable layout movement.

CSS constrains how an image's box is presented; HTML chooses which image resource best matches content and loading conditions. The `srcset` and `sizes` attributes support resolution switching, while the `picture` element supports art direction when crops or compositions genuinely change. Do not use CSS background images for content that needs alternative text or document meaning.

6.5 User preferences are responsive inputs

Respect reduced-motion preferences for non-essential animation:

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    scroll-behavior: auto;
    animation-duration: 0.01ms;
    animation-iteration-count: 1;
  }
}
```

This example is a broad project policy and should be adapted if an animation communicates essential state. Also consider color-scheme preferences and forced colors. Do not make interaction depend on motion, hover, or color alone.

`prefers-color-scheme` can select a light or dark palette, but every resulting token combination still needs contrast testing. The `color-scheme` property tells the browser which schemes a component supports so built-in controls and scrollbars can cooperate. In `forced-colors: active`, user-agent

colors may replace author colors. Preserve borders, focus, and state with system color keywords or other forced-color-aware techniques instead of fighting the mode.

Do not use `prefers-reduced-motion` to remove essential feedback. Replace non-essential movement with an immediate state change and ensure that progress, errors, and completion remain perceivable.

7. Accessible presentation survives real user conditions

Accessibility is not a theme added after layout. CSS can make semantic content unreadable, unreachable, misleading, or motion-sensitive, so every presentation decision must be tested against the ways people actually perceive and operate it.

7.1 Interaction states must remain perceivable

Links and controls need distinguishable hover, focus, active, disabled, valid, and invalid states where relevant. Never remove the focus outline without a replacement that is at least as visible across every surface.

```
:focus-visible {
  outline: 3px solid #f0a202;
  outline-offset: 3px;
}
```

Use `:focus-visible` to emphasize keyboard-style focus while allowing the browser to decide when that indicator is appropriate. Test it rather than assuming every browser interaction follows the same heuristic.

Focus must also remain unobscured. Sticky headers, cookie panels, and fixed toolbars can cover the focused control even when its outline is technically rendered. Tab through the page at narrow width and high zoom. For a robust focus indicator across varying surfaces, a two-color indicator or separated outline can maintain contrast where one color cannot.

Disabled controls should look unavailable without becoming unreadable. Validation states must not rely on red and green alone. Hover styling must be accompanied by focus behavior where the same information or action matters to keyboard users.

7.2 Target size is measured after layout

WCAG 2.2 Level AA generally requires pointer targets to be at least 24 by 24 CSS pixels or to satisfy one of the criterion's spacing or contextual exceptions. That is a minimum conformance threshold, not necessarily a comfortable design target. The Campus Hub controls aim for a `2.75rem` minimum block size under the tested root settings and retain space between adjacent targets.

Inline links inside sentences have a specific exception, and browser-provided controls can have another. Do not quote “24 pixels” without the criterion's conditions, and do not assume declared CSS dimensions equal the final clickable area. Inspect the rendered box and test touch or imprecise pointer use.

7.3 Zoom, text spacing, and reflow are layout tests

At 400% browser zoom, a page initially 1280 CSS pixels wide commonly presents a layout viewport close to 320 CSS pixels wide. WCAG reflow expects most content to remain readable without two-dimensional scrolling at the specified equivalent dimensions, with exceptions for content such as data tables that genuinely needs two dimensions. Test both narrow width and zoom because they expose related but not identical failures.

Users may override line height, paragraph spacing, letter spacing, and word spacing. Avoid fixed heights and clipped text. A card that works only with its original copy, font, and spacing is not a robust card.

7.4 Hiding and visual order can change access

`display: none` and the HTML `hidden` attribute normally remove content from both visual layout and the accessibility tree. `visibility: hidden` preserves layout space while hiding the element and its descendants. `opacity: 0` can leave an invisible control present and focusable. Off-screen positioning can also create unexpected scrolling. Choose a hiding mechanism from the intended state rather than copying a “hide element” recipe.

CSS-generated content, background images, and visual reordering do not repair missing semantic order. Test the page without CSS, with keyboard focus, and in a linearized reading sequence.

7.5 Motion, contrast, and forced colors require combined evidence

Measure text and non-text contrast for every state, including hover, focus, disabled, and invalid conditions. Verify meaning without color. Emulate reduced motion and forced colors, then use a real operating-system mode where possible. Automated checks can find some contrast and target problems, but keyboard, zoom/reflow, preference, and human reading tests remain necessary.

Predict before running

What happens when this card is placed in a 280px-wide container?

```
.card { width: 20rem; padding: 2rem; }
```

With a typical 16px root size, 20rem is 320px before considering the containing space. Under `border-box`, padding is included in that 320px; under `content-box`, it increases the outer box further. Either way, a fixed width can overflow the 280px container. A resilient alternative is `width: min(100%, 20rem)`.

8. CSS architecture should make change inexpensive

CSS is global by default: a selector can match anywhere in the document, and a custom property can flow through many descendants. Maintainability comes from controlling that power, not from pretending the cascade does not exist.

A maintainable small project can use this explicit layer order:

```
@layer reset, base, layout, components, utilities;
```

1. `reset` contains the small interoperability baseline the project intentionally owns;
2. `base` contains tokens and low-specificity element defaults;
3. `layout` contains page shells and reusable spatial relationships;
4. `components` contains cohesive interface objects such as event cards; and
5. `utilities` contains narrow, documented single-purpose exceptions.

Normal declarations in a later layer outrank normal declarations in an earlier layer without requiring higher specificity. Declare the order before loading layer contents. Remember that unlayered normal author rules outrank layered normal author rules, so an accidental unlayered file can bypass the architecture.

Names should communicate purpose and ownership. `.event-card` is more durable than `.blue-box-left`. Avoid encoding one page's entire nesting into every selector. A component may

use a naming convention such as `.event-card__title`, but the convention is a team tool rather than CSS syntax.

Native CSS nesting can group related selectors, but deep nesting recreates the same coupling as long descendant selectors. Keep the resulting selector clear and understand how functional pseudo-classes affect specificity. Comments should explain a non-obvious decision, compatibility boundary, or constraint, not translate obvious syntax into English.

Use `@supports` for progressive enhancement when a feature needs a guarded fallback:

```
.dashboard {
  display: block;
}

@supports (display: grid) {
  .dashboard {
    display: grid;
  }
}
```

Modern Flexbox and Grid do not normally require this particular fallback in the course's supported browsers; the example demonstrates the decision pattern. Check current compatibility and audience requirements before shipping a newer feature. A declaration that an older browser ignores can be a safe enhancement only when the earlier presentation remains usable.

9. Debug CSS by tracing evidence, not by guessing values

When a page looks wrong, separate four possible questions:

1. Did the stylesheet load?
2. Did the selector match the intended element?
3. Did the declaration win the cascade and compute to a valid value?
4. Did layout have the space and content assumptions the author expected?

Figure 3.13 maps those questions to browser evidence.

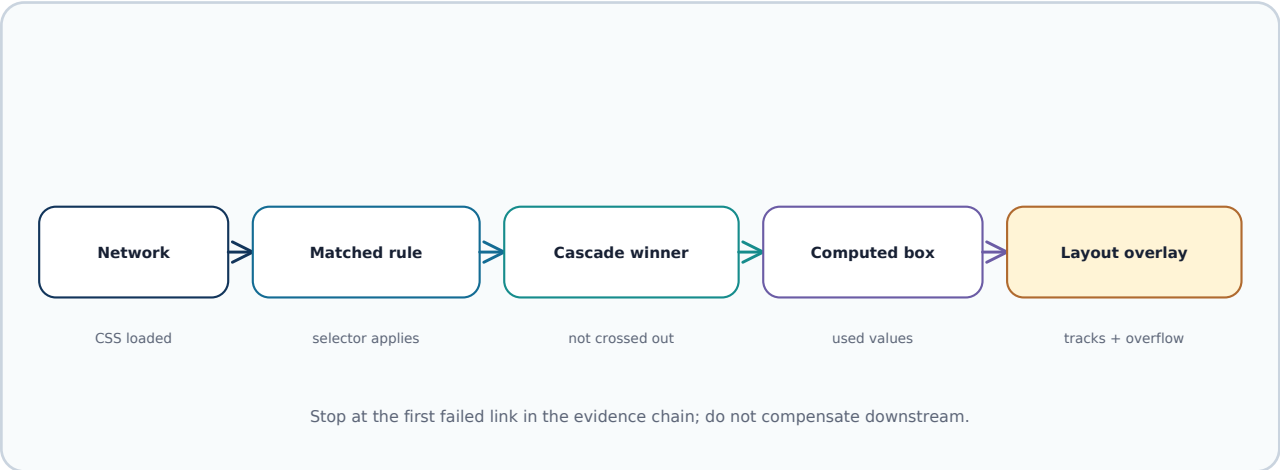


Figure 3.13. CSS debugging traces evidence from resource loading through matching, cascade, computed values, and layout.

9.1 Open the correct browser panels

1. Open the Campus Hub page in the browser.
2. Press `F12`, or press `Ctrl+Shift+I` on Windows and Linux. On macOS, press `Command+Option+I`. Browser and keyboard settings can change these shortcuts; the browser menu normally contains **Developer tools** when a shortcut is not available.

3. Select the **Network** panel and reload. Filter for `CSS` or find `styles.css`. A successful status and CSS response establish that the file was requested. If no row appears, inspect the HTML `link`; if the status is `404`, inspect the resolved request address and filename.
4. Select the **Elements** or **Inspector** panel and activate the element picker. Click the visible card or control whose presentation is unexpected.
5. In **Styles** or **Rules**, examine matched selectors. A crossed-out property lost the cascade or was overridden by a shorthand. A warning icon or missing value can indicate invalid syntax.
6. In **Computed**, search for the property and expand it to trace the winning declaration. Inspect the box-model visualization for content, padding, border, and margin.
7. Open the Flexbox or Grid overlay in the **Layout** panel. Turn on line numbers, track sizes, gaps, and item overlays where the browser supports them.
8. Use responsive-design mode to set an exact CSS-pixel viewport. Test zoom in the ordinary page as well; responsive emulation and browser zoom are related but not interchangeable evidence.

Edits made inside developer tools are temporary experiments. Once an experiment explains the failure, reproduce the minimal correction in `styles.css`, save it, reload, and confirm the evidence again.

9.2 Trace horizontal overflow systematically

Start at the page's horizontal scrollbar and inspect outward evidence:

1. Record the viewport width and zoom level.
2. Temporarily outline candidate boxes in developer tools.
3. Find the first box whose right or inline-end edge extends beyond its containing block.
4. Inspect its declared size, intrinsic minimum, margins, transforms, positioned offsets, and unbreakable descendants.
5. Correct the cause locally, then test long text, another language direction, and every required viewport again.

Do not keep a global diagnostic outline in the production stylesheet. Do not declare success because the scrollbar disappeared after clipping; confirm that all content and focus indicators remain reachable.

Guided laboratory - Build the responsive Campus Hub shell

Before you begin

Work in your personal repository created from the Fall 2026 coursework template. Its root is the folder that contains `package.json`, `labs`, and `scripts`. In Visual Studio Code, open **Terminal > New Terminal** and run the command below in PowerShell to display the current location.

```
Get-Location
```

If the path is not your cloned `web-programming-coursework-2026_Fall-YOUR_STUDENT_NUMBER` folder, change to the repository using its actual path. Replace the placeholder with your own student number; do not copy this example literally.

```
cd "C:\path\to\web-programming-coursework-2026_Fall-YOUR_STUDENT_NUMBER"
```

Confirm that the project manifest is visible before running an npm command.

```
Get-Item package.json
```

Create a working branch and commit the untouched starting point only if your course workflow asks for that baseline. Otherwise, begin from the instructor's provided starter commit.

```
git switch -c chapter-03-responsive-campus-hub
```

Open two ordinary terminal tabs. The first becomes the **server terminal** only because it runs the long-lived server process. The second remains available for tests and Git commands. Their windows are not different kinds of software; their current jobs are different.

In the first terminal, run:

```
npm run serve:week03
```

Leave that process running. Open the address printed by the command, normally `http://localhost:8000/`, in the browser. If npm reports that it cannot find `package.json`, the terminal is in the wrong directory. If the browser cannot connect, confirm that the server terminal is still running and that the port in the browser matches the printed address.

Goal and acceptance criteria

Style the supplied semantic Campus Hub page without changing its meaning. Your result must:

- use an external stylesheet;
- define a small token set for color, spacing, radius, and readable width;
- apply `border-box` consistently;
- preserve logical source and focus order;
- use Grid for the event collection and Flexbox for navigation/actions;
- style the semantic filter form without removing its native keyboard behavior;
- demonstrate one component-level container query;
- work without horizontal scrolling at 320, 768, and 1440 CSS pixels;
- remain usable at 200% zoom;
- show a strong keyboard focus indicator;
- provide comfortably sized, separated pointer targets;
- wrap long titles and navigation labels;
- respect reduced motion;
- remain understandable with forced colors and without color cues;
- avoid `!important`, ID selectors, positive `tabindex`, and fixed page heights;
- pass the supplied repository check and every manual acceptance check; and
- contain at least three meaningful commits.

Step 1 - Inspect the unstyled document

Open `labs/week-03/index.html` and read the heading, landmark, event-card, and filter-form structure. In the browser, press `Tab` repeatedly and write down the focus order. Narrow the viewport to 320 CSS pixels in responsive-design mode. Do not change HTML merely to make a selector convenient.

Expected evidence: the content remains in a meaningful order, the form has visible labels, and the page uses browser defaults. It does not yet resemble Figures 3.9 and 3.10 because the starter stylesheet contains only its assignment comment.

Step 2 - Establish tokens and global constraints

Open `labs/week-03/styles.css`. Add the layer declaration, purpose-named custom properties, inherited body typography, the universal `border-box` baseline, responsive media sizing, a centered page shell, and a visible `:focus-visible` rule. Save the file and reload.

In developer tools, select one event card and confirm that the Computed box model reports `border-box`. Select a link with the keyboard and confirm that the focus outline is visible against its actual background. Commit the completed baseline.

```
git add labs/week-03
git commit -m "Establish CSS tokens and resilient baseline"
```

Step 3 - Build component layouts

Use a wrapping Flexbox list for navigation. Use responsive Grid tracks for event cards. Give cards internal spacing and a subtle boundary while preserving their article semantics. Style the filter with wrapping Flexbox and allow its controls to inherit typography. Do not remove native select or button appearance.

Turn on Flexbox and Grid overlays. Explain which axis Flexbox controls in the navigation and which tracks Grid creates in the event collection. Make the second commit after the page relationships work at narrow and wide sizes.

Step 4 - Add a content-driven enhancement

At a breakpoint justified by the content, allow the page header or dashboard to use additional columns. Record why that width supports the enhancement. Then make each event card an inline-size query container. At a threshold justified by the card's own content, align its details link beside the metadata while preserving the card's logical reading order.

Resize the page until one card crosses the container threshold. Record the card's container width before and after the rule activates. Figure 3.12 explains why the component responds to its container rather than a device name.

Step 5 - Conduct an overflow audit

Test 320, 768, and 1440 CSS pixel widths and 200% zoom. Insert a deliberately long event title. If horizontal scrolling appears, use the layout inspector to identify the exact box before changing CSS. Also paste one long unbroken URL into a card, switch the document direction temporarily to `rtl`, and check that logical spacing follows the new inline start and end.

Remove temporary test content only after recording the cause and correction. Make the third commit when the audited layout is stable.

Step 6 - Verify preferences and interaction

Tab through the page and capture evidence of visible, unobscured focus. Measure the rendered filter targets. Emulate reduced motion and forced colors in browser tools, then verify an actual operating-system high-contrast mode if available. Check that every state remains understandable without color alone.

In the second terminal, run the supplied coursework check from the repository root.

```
npm test
```

A passing command reports `Coursework setup verification passed.` It confirms that the repository structure, Node.js version, starter documents, and local servers remain usable; it does not grade every CSS decision. If it fails, read the reported file or environment problem before editing. Then evaluate every acceptance criterion manually and compare your committed attempt with the tested reference solution in Appendix A.

Finally, compare the completed page with the wide output in Figure 3.10 and the narrow output in Figure 3.11. Differences in exact font rendering are expected across operating systems; the relationships, states, and responsive behavior must agree.

Use Table 3.7 to record visible evidence at every required viewport and user preference rather than writing “works” without a reproducible observation.

Table 3.7 - Responsive-layout laboratory observation record

Check	Evidence and interpretation
Stylesheet request	
Matched selector and computed value	
320px overflow	
768px layout	
1440px readable line length	
200% zoom	
Long-title wrapping	
Right-to-left logical spacing	
Container-query threshold	
Focus visibility	
Target dimensions and spacing	
Reduced-motion behavior	
Forced-color behavior	

Exercises

Foundation

Exercise 1 - Cascade decision

Given `.card p { color: navy; }`, `.message { color: teal; }`, and an element `<p class="message">`, which declaration wins in the same layer and origin? Show the specificity comparison.

Exercise 2 - Box calculation

Under `content-box`, calculate the outer inline size of an element with `width: 240px`, 16px padding on each side, 2px borders, and 12px margins. Repeat for `border-box`.

Exercise 3 - Calculate and choose font-relative units

The root computed font size is `20px`. A card uses `padding: 1.25rem` and its heading uses `font-size: 1.6rem`. Calculate both values. Then choose and justify a unit or function for body line length, component padding, a badge whose padding must scale with its own text, full-width media, and a fluid but bounded title.

Application

Exercise 4 - Repair overflow

Explain why `width: 100vw; padding: 2rem` can create horizontal scrolling inside a page and propose a robust alternative.

Exercise 5 - Flexbox or Grid

Choose a layout system for a wrapping navigation bar, a two-dimensional timetable, a vertical button group, and a responsive card gallery. Explain each choice.

Exercise 6 - Focus regression

A reset contains `*:focus { outline: none; }`. Explain the failure and provide a repair that does not depend on color change alone.

Challenge

Exercise 7 - Source order

A designer asks you to display the event sidebar before the main event list on wide screens but after it on narrow screens. Explain how to choose a DOM order and CSS strategy without producing a confusing focus or reading sequence.

Exercise 8 - Responsive reasoning

Design a Grid declaration that produces as many cards as fit, prefers a 17rem minimum, never exceeds the container, and shares remaining space evenly. Explain every function and unit.

Exercise 9 - Diagnose an unstyled page

The HTML contains `<link rel="stylesheet" href="styles.css">`, but the Network panel shows `404 Not Found` for `http://localhost:8000/pages/styles.css`. The actual file is `http://localhost:8000/assets/styles.css`. Explain why no CSS rule can apply and give two correct repairs, assuming the HTML document is in `/pages/index.html`.

Exercise 10 - Match selectors deliberately

For `<article class="event-card featured"><h3>Robotics night</h3></article>`, state whether each selector matches an element and identify that element: `.event-card`, `.event-card.featured`, `.event-card .featured`, `article > h3`, and `article::before`.

Exercise 11 - Inheritance and CSS-wide keywords

A card inherits `color: navy` from `body`, but a component rule directly sets `.event-card { color: teal; }`. What color does ordinary card text inherit? What would `color: inherit`, `color: initial`, and `color: unset` mean on the card?

Exercise 12 - Positioning and stacking

A modal child has `z-index: 9999`, but it still appears below a transformed sidebar. Explain why increasing the number may not help and describe the developer tools evidence you would inspect before changing the architecture.

Exercise 13 - Media or container query

The same event card is 25rem wide in the main area and 15rem wide in a sidebar, both while the viewport is 1200px wide. Choose a media query or a container query for changing the card's internal layout. Provide a minimal rule and explain the containment requirement.

Exercise 14 - Accessible control styling

A design removes the select arrow with `appearance: none`, removes the focus outline, uses pale gray text, and gives the control a 16px-high click target. Identify the separate usability and accessibility risks. Propose a safer baseline without claiming that CSS alone proves conformance.

Exercise 15 - Developer-tools evidence plan

A student's `padding-inline: 1rem` appears crossed out. Write an evidence sequence that determines whether the file loaded, the selector matched, the declaration was valid, which declaration won, and what used padding the browser produced.

Exercise 16 - Architecture review

Review this selector and repair strategy:

```
#main-content section.events article.event-card h3.title {
  color: purple !important;
}
```

Explain the maintenance costs and propose a layered, low-specificity alternative that still allows a featured card to vary its title color.

Worked answers

Answer 1

`.card p` contains one class selector and one type selector. `.message` contains one class selector and no type selector. Their class category ties, and the type category breaks the tie, so `.card p` is more specific and its navy value wins if all earlier cascade dimensions are equal. Later source order would not defeat the higher specificity in this case.

Answer 2

With `content-box`, the outer size including margins is `240 + 32 + 4 + 24 = 300px`. Without margins, the border box is 276px. With `border-box`, the declared 240px already includes padding and borders, so the outer size including margins is `240 + 24 = 264px`. Margins never become part of the declared border-box width.

Answer 3

`1.25rem` resolves to `25px` because `1.25 × 20 = 25`; `1.6rem` resolves to `32px` because `1.6 × 20 = 32`. Use a `ch`-based maximum such as `max-inline-size: 68ch` for body line length and `rem` for component padding that should follow the user's root text size consistently. Use `em` for the badge's internal padding because that spacing is intentionally local to the badge's own computed text size. Use `max-inline-size: 100%` for media containment and `clamp()` combining `rem` and a fluid term for a bounded title. These are starting decisions that require testing with actual content, fonts, user text settings, zoom, and reflow. The common mistake is to call every relative unit interchangeable; each unit encodes a different reference relationship.

Answer 4

`100vw` describes the viewport width, not the remaining content width inside a parent. With `content-box`, padding is added outside that width, and a vertical scrollbar can also make viewport units differ from the document's available inline size. Prefer normal block sizing or `width: 100%` within the containing block, use `border-box`, and let the parent provide page padding.

Answer 5

Use Flexbox for wrapping navigation because its primary relationship is one row that may wrap. Use Grid for the timetable because rows and columns must align. Use Flexbox with a column direction for the button group. Use Grid with responsive tracks for the card gallery because both track creation and column alignment matter. These are reasoned defaults, not laws; content relationships decide.

Answer 6

The reset removes the primary visible indication of keyboard location. Users can reach controls but may not know which one will receive input. Remove the rule and style `:focus-visible` with a sufficiently thick outline and offset that contrasts with adjacent colors, for example a 3px amber outline plus spacing. Test on every surface; a single color may not contrast with all backgrounds.

Answer 7

Choose the DOM order that makes the most coherent reading and keyboard sequence, usually main content followed by complementary content. On wide screens, place the regions into named Grid areas without changing their logical sequence. A visual left-right arrangement need not dictate DOM order. Test linearized reading and Tab order at both widths. If the sidebar must logically precede the main list, use that order consistently rather than creating breakpoint-dependent meaning.

Answer 8

```
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(min(100%, 17rem), 1fr));
  gap: 1rem;
}
```

`repeat()` defines recurring tracks. `auto-fit` creates as many as fit and collapses unused tracks. `minmax()` provides minimum and maximum track sizes. The inner `min()` prevents the preferred 17rem minimum from overflowing a narrower container. `1fr` shares remaining track space evenly.

Answer 9

The `href` is resolved relative to `/pages/index.html`, so `styles.css` becomes `/pages/styles.css`. The server's `404` response means the browser did not receive the stylesheet at that address; correct CSS in another file cannot participate in matching or the cascade.

One repair is to use a root-relative address:

```
<link rel="stylesheet" href="/assets/styles.css">
```

Another is the correct document-relative address:

```
<link rel="stylesheet" href="../assets/styles.css">
```

Which form is preferable depends on the deployment's URL structure. The common misconception is to edit selector specificity while the resource itself never loaded. Reload with the Network panel open and require a successful CSS response before investigating rules.

Answer 10

- `.event-card` matches the `article` because its class list contains that token.
- `.event-card.featured` also matches the same `article`; no space means the one

element must have both classes.

- `.event-card .featured` does not match anything in the supplied fragment. The

space asks for a descendant with class `featured`, not the card itself.

- `article > h3` matches the `h3` because it is a direct child of the article.
- `article::before` selects a generated pseudo-element associated with the

article. It does not match an HTML child, and it generates no visible box unless applicable declarations, often including `content`, cause one to be rendered.

The important misconception is that punctuation is cosmetic. A space changes the relationship being selected.

Answer 11

The card's directly declared `teal` value wins for the card. Descendant text that has no direct color declaration normally inherits teal from the card. The body's selector specificity is irrelevant to that direct-versus-inherited distinction.

On `.event-card`, `color: inherit` explicitly takes the parent's computed color, so it would normally take navy. `color: initial` uses the specification's initial color value, commonly represented by the user-agent canvas text color rather than the author's body navy. Because `color` is normally inherited, `color: unset` behaves like `inherit` here and also takes the parent value. A common misconception is that `initial` means “the browser's normal style for this HTML element”; it means the property's defined initial value.

Answer 12

`z-index` compares boxes within stacking contexts. A transformed ancestor can create a context, so a child with `z-index: 9999` may still be trapped beneath a sibling context whose entire stack is above it. Larger numbers do not allow the child to escape its ancestor's context.

Inspect the modal and sidebar ancestors in the Elements panel. Identify which properties create stacking contexts-such as transforms, positioned elements with non-auto `z-index`, opacity, or

isolation-and inspect the browser's stacking or layers visualization if available. Then decide whether the modal belongs higher in the DOM or whether an unnecessary ancestor context should be removed. The misconception is that `z-index` is one global ranking shared by every element.

Answer 13

Use a container query because both cards share one viewport but receive different component space.

```
.event-card {
  container-type: inline-size;
}

@container (min-width: 20rem) {
  .event-card__content {
    grid-template-columns: 1fr auto;
  }
}
```

The card establishes an inline-size query container, and the rule styles its descendant when that container reaches `20rem`. A media query at the 1200px viewport would affect both cards and therefore cannot express the actual relationship. The queried container cannot ordinarily style itself through that same size query; the query applies to descendants.

Answer 14

Removing native appearance removes a familiar affordance and makes the author responsible for recreating the select indicator across states and platforms. Removing focus styling hides keyboard location. Pale text may fail text contrast, and a 16px-high target can be difficult to activate and may fail the WCAG 2.2 target-size criterion unless a stated exception or spacing condition applies.

A safer baseline keeps native appearance, uses `font: inherit`, supplies readable foreground and background colors, preserves or strengthens `:focus-visible`, and adds padding with a comfortable tested minimum block size. Measure the rendered target, spacing, text contrast, non-text boundaries, focus contrast, and forced colors. CSS declarations are implementation choices; conformance is established from the resulting behavior and measurements.

Answer 15

First, reload with the Network panel open and confirm that the intended stylesheet returned successfully. Second, select the element and find the rule in the Styles panel; absence suggests that the selector did not match or the conditional rule was inactive. Third, check for a warning or invalid value. Fourth, expand the crossed-out declaration or search `padding-inline` in the Computed panel to find the winning shorthand or longhand and its source. Finally, inspect the box-model diagram or computed `padding-inline-start` and `padding-inline-end` values to see the resolved result.

Make a temporary developer-tools change only to test the hypothesis, then fix the source file and reload. The common mistake is to add `!important` before learning whether the declaration was valid, relevant, or loaded.

Answer 16

The selector couples the title to an ID, several ancestors, element names, and two classes. Its high specificity makes reuse and later variation expensive, while `!important` moves the declaration into a different precedence level and invites more important overrides.

A low-specificity component layer can express the default and a modifier:

```
@layer components {
```

```
  .event-card__title {
    color: var(--event-title-color, var(--color-heading));
  }

  .event-card--featured {
    --event-title-color: var(--color-accent-strong);
  }
}
```

The title rule remains reusable, while the featured card changes a documented component decision through the cascade. Another class-based modifier could be valid; the important principle is to express ownership and variation without encoding accidental page nesting. The misconception is that a selector becomes safer when it names more ancestors. It actually becomes more coupled.

Debugging clinic

At 400% zoom, an event card overlaps the footer. The card has `height: 18rem` and its description has `overflow: hidden`.

Diagnosis

The fixed height assumes a particular text size and amount. Zoom increases text space requirements, but the card is forbidden to grow and the description is clipped. Remove the fixed height, allow normal block growth, and use `min-height` only if a minimum visual rhythm is needed. Retest with long content and zoom. Do not conceal the failure with additional clipping.

Retrieval quiz

1 What are the selector, declaration block, property, and value in a CSS rule?

2 Why can one invalid declaration fail without stopping the complete stylesheet?

3 How does an external stylesheet address resolve, and which browser panel proves that the file loaded?

4 What is the difference between `.card.featured` and `.card .featured`?

5 What does a selector do, and what does the cascade do?

6 When does source order decide a conflict?

7 Why is specificity not an ordinary decimal score?

8 How does inheritance differ from a selector directly matching a child?

9 What changes when `box-sizing` becomes `border-box`?

10 Why are normal flow and intrinsic sizing valuable?

11 What do the inline and block axes represent?

12 Why can a very large `z-index` still appear below another component?

13 When is Flexbox usually preferable to Grid?

14 What does `minmax(min(100%, 17rem), 1fr)` protect against?

15 When should a component use a container query instead of a media query?

16 Why is visual reordering risky?

17 Why is `overflow-x: hidden` a poor first repair?

18 What must a focus design provide beyond declaring `:focus-visible`?

19 How do zoom, reflow, and target-size tests contribute different evidence?

20 How should reduced-motion and forced-color preferences affect design?

Quiz answers and misconception notes

1 In `.card { padding: 1rem; }`, `.card` is the selector, the braces contain the declaration block, `padding` is the property, and `1rem` is the value. The colon separates property from value; the semicolon terminates the declaration.

2 CSS has error-recovery rules so an unsupported or malformed declaration can be ignored while later valid material remains usable. The misconception is that resilience makes errors harmless; a misspelled property can silently remove an essential visual state.

3 A relative `href` resolves from the HTML document's address. Reload with the Network panel open and verify the requested URL, response status, and CSS response. The Styles panel cannot prove that an absent resource loaded.

4 `.card.featured` matches one element with both classes. `.card .featured` matches a descendant with class `featured` inside an ancestor with class `card`. The space expresses a relationship.

5 A selector identifies matching elements; the cascade resolves competing property declarations for each matched element and property.

6 Source order decides only after relevance, origin and importance, layer, specificity, and scoping proximity no longer distinguish the declarations. “Last always wins” is therefore incomplete.

7 Specificity compares ordered categories-ID, class-like, and type-like-not one base-ten number. Many class selectors do not carry into the ID category.

8 A directly matched declaration participates in the cascade for the child. Inheritance supplies a value only through the parent relationship when the child does not otherwise establish one. High ancestor specificity does not defeat a directly targeted child declaration.

9 Declared width and height include content, padding, and border. Margin remains outside, and intrinsic minimums or fixed sizes can still cause overflow.

10 Normal flow reserves space and accommodates changing content; intrinsic sizing lets content contribute real constraints. Removing both too early replaces adaptable behavior with assumptions about text, fonts, and viewport size.

11 The inline axis follows text within a line, while the block axis follows the direction in which lines and blocks accumulate. Logical properties adapt those relationships to writing direction and mode.

12 `z-index` is compared inside stacking contexts. A child cannot escape an ancestor context merely by using a larger number, so the ancestor structure and context-creating properties must be inspected.

13 Flexbox is usually preferable when one main axis governs a row or column of items. Grid is preferable when rows and columns or repeated tracks must coordinate. Neither choice changes source semantics.

14 The inner `min()` lets a preferred `17rem` minimum yield to a container that is even narrower. `minmax()` then allows growth to `1fr`, and repeated tracks share remaining space instead of overflowing.

15 Use a container query when a component's allocated space-not the viewport- determines its internal layout. Establish an appropriate query container and remember that the size query styles descendants, not the queried container itself.

16 Visual order can diverge from DOM reading and keyboard order, producing a page that looks coherent but is sequentially confusing.

17 Global clipping hides the evidence and can make content or focus indicators unreachable. Identify the first oversized box and correct its local constraint.

18 The focus indicator must be visible, sufficiently contrasting, and not obscured on every relevant surface. It must also follow a meaningful focus order. A selector alone proves none of those rendered outcomes.

19 Zoom tests text enlargement and can activate responsive conditions; reflow tests whether content remains readable without unnecessary two-dimensional scrolling; target testing measures the final interactive area and spacing. Together they reveal different failures.

20 Reduce or replace non-essential motion while preserving understandable state. In forced colors, allow user-agent colors to replace author colors and retain boundaries, focus, and state through compatible cues rather than color alone.

Self-check scale

- **17-20:** ready to add behavior to a resilient interface.
- **13-16:** revisit the answers you could not explain without notes.
- **8-12:** repeat the cascade, sizing, and responsive investigations.
- **0-7:** rebuild the shell from normal flow, then add Flexbox and Grid one

decision at a time.

Session summary

- CSS supplies adaptable constraints rather than fixed screenshot coordinates.
- CSS syntax connects selectors with typed property values, while error recovery

allows unsupported declarations to fail locally.

- A stylesheet must load and a selector must match before the cascade can matter.
- The cascade considers origin, importance, layers, specificity, scope, and order.
- Low-specificity component rules are easier to maintain.
- Inheritance and custom properties carry decisions through the document tree.
- Display, intrinsic sizing, the box model, and containing blocks explain most

apparent size and spacing surprises.

- Normal flow should be preserved unless a real relationship requires otherwise.
- Logical properties follow writing direction and improve international resilience.
- `rem` lengths share the root element's computed font size, while `em` lengths

can express deliberate component-local scaling.

- Typography, color, controls, backgrounds, and borders must be tested as rendered

combinations rather than judged from declarations alone.

- Flexbox coordinates one primary axis; Grid coordinates rows and columns.
- Media queries respond to the environment; container queries respond to a component's allocated space.
- Logical DOM order must remain coherent regardless of visual placement.
- Focus, target size, zoom, reflow, long content, direction, reduced motion, and forced colors are required test inputs.
- Browser developer tools connect a visible failure to loading, matching, cascade, computed values, boxes, and layout.
- Overflow is diagnostic evidence, not something to hide globally.

Glossary

Table 3.8 consolidates the vocabulary needed to explain cascade and layout decisions during review.

Table 3.8 - Session 3 glossary

Term	Working definition
Box model	Content, padding, border, and margin regions generated for an element.
At-rule	A CSS construct beginning with `@` that defines a condition, grouping, or operation, such as `@media` or `@layer`.
Cascade	The algorithm that resolves competing CSS declarations.
Cascade layer	An author-defined precedence group that organizes declarations independently of selector specificity.
Computed value	A property's value after cascade and much of value processing, before final layout use.
Containing block	The reference box used to resolve sizes or positions for another box.
Container query	A conditional rule based on a query container, commonly its inline size, rather than the viewport.
Custom property	A cascading author-defined property such as `--color-accent`, commonly used as a design token.
Declaration	A property-and-value proposal such as `padding: 1rem`.
`em`	A font-relative CSS length unit. In a `font-size` declaration it refers to the inherited parent size; for most other properties it refers to the element's computed font size.
Flexbox	A layout model for distributing and aligning items primarily along one axis.
Grid	A layout model that coordinates rows and columns.
Inheritance	Transfer of selected property values from an element to its descendants.
Intrinsic size	A size contribution derived from an element's content rather than only an author-declared external dimension.
Logical property	A property expressed relative to block and inline flow rather than fixed physical directions.
Media query	A conditional block based on characteristics such as available width or user preference.
Normal flow	The default system by which block and inline boxes participate in document layout.
Pseudo-class	A selector component representing a state or structural condition, such as `:focus-visible`.
Pseudo-element	A selector component representing a stylable generated or abstract part, such as `::before`.
`rem`	Root em, a CSS length unit equal to the computed font size of the document's root element.
Selector	A pattern that identifies elements to which a rule may apply.
Stacking context	A local three-dimensional painting context within which `z-index` values are compared.
Specificity	The selector comparison used within the relevant cascade level to resolve conflicts.

Primary references and further reading

1 World Wide Web Consortium (W3C), CSS Snapshot 2026. Use the snapshot to understand that modern CSS is a family of coordinated modules, not one numbered “CSS3” specification.

2 MDN, CSS fundamentals curriculum. Use its outcomes to review syntax, selectors, boxes, conflicts, values, forms, and debugging.

3 MDN, CSS text styling curriculum. Use it for deeper typography, links, lists, and web-font study.

4 MDN, CSS layout curriculum. Use it to extend normal flow, positioning, Flexbox, Grid, and responsive-design practice.

5 W3C, CSS Cascading and Inheritance Level 5. Use it when the introductory cascade model needs normative detail about layers and value processing.

6 W3C, Selectors Level 4. Use it to verify selector grammar and modern functional pseudo-classes.

7 W3C, CSS Flexible Box Layout. Use it to resolve precise flex sizing and alignment questions.

8 W3C, CSS Grid Layout Level 2. Use it for track sizing, placement, auto-placement, and subgrid behavior.

9 W3C, CSS Containment Level 3. Use it for normative container-query and containment behavior.

10 W3C Web Accessibility Initiative, Understanding WCAG 2.2. Use it to evaluate contrast, reflow, focus, target size, and preference-sensitive presentation as outcomes rather than isolated declarations.

11 Harvard University, CS50 Web Week 0: HTML and CSS. Compare its concise lecture-and-project progression with this chapter's slower novice reference path.

12 Stanford University, CS142 CSS lecture notes. Use the rule-anatomy, selector, color, box, positioning, display, and layout outline as a compact university-level review.

13 Jennifer Niederst Robbins, *Learning Web Design*, 6th edition (O'Reilly, 2025). Use Chapters 11-17 for an extended beginner-to-intermediate treatment of CSS, typography, color, boxes, responsive layout, and transitions.

14 Keith J. Grant, *CSS in Depth*, 2nd edition (Manning). Use its treatment of the cascade, relative units, flow, boxes, layout, and maintainable component systems for instructor enrichment.

15 W3C, CSS Values and Units Module Level 4. Use its definitions of root-relative and local font-relative length units to verify `rem` and `em` calculations.

16 W3C Web Accessibility Initiative, Understanding Resize Text. Use it to connect relative sizing decisions with the required rendered outcome: enlarged text without loss of content or functionality.

What changes often?

Cascade, flow, Flexbox, and Grid principles are stable. Selectors, container queries, nesting, color spaces, viewport units, preference media features, and browser developer tools continue to evolve. Compatibility decisions must use the project's tested browser policy and current support data rather than a remembered label such as “CSS3.” Technical review date: **21 October 2026**.

Session reflection

- Which CSS declaration removed the most special cases?
- Which overflow diagnosis changed your mental model?
- Which constraint would make this layout safer under translation or user zoom?

Instructor teaching guide

Suggested five-hour sequence

1 **Opening evidence, 20 minutes:** compare the unstyled Chapter 2 document with the completed page. Ask which meaning remains when CSS is disabled.

2 **Language and cascade, 50 minutes:** annotate Figure 3.1, live-edit selector matches, and trace one conflict through Figure 3.3 and developer tools.

3 **Boxes and values, 55 minutes:** measure one card under both box-sizing models, calculate `rem` and `em` from inspected computed font sizes, then demonstrate intrinsic overflow and logical axes.

4 **Break, 10 minutes.**

5 **Layout and responsive systems, 55 minutes:** build navigation with Flexbox, cards with Grid, and one container-query enhancement. Require students to predict each change before resizing.

6 **Accessible presentation, 30 minutes:** test focus, target size, 200% zoom, 320 CSS pixels, reduced motion, and forced colors.

7 **Laboratory, 120 minutes:** use the six checkpoints and Table 3.7 as formative evidence. Reserve the final 15 minutes for peer explanation and commit review.

Demonstrations that expose misconceptions

- Break the stylesheet path before discussing specificity. Students must use the Network panel and learn that an unloaded rule cannot lose the cascade.
- Compare `.card.featured` with `.card .featured` on Figure 3.2's document tree.
- Give a low-specificity rule a later layer and a high-specificity rule an earlier layer so students see that specificity is not the first cascade dimension.
- Put a long unbroken string in a Flex or Grid item and inspect its automatic minimum before adding `min-inline-size: 0`.
- Change the browser's default font size, inspect the computed root size, and ask students to predict which `rem` and `em` measurements will change before reloading the Campus Hub page.
- Place the same card in wide and narrow containers at one viewport to distinguish a container query from a media query.
- Temporarily remove focus styling and enable forced colors; ask students to describe evidence rather than say that the design “looks accessible.”

Common misconceptions to listen for

- “CSS3 is the current single version of CSS.” Modern CSS consists of modules with independent levels and maturity.
- “The last declaration wins.” It wins only after earlier cascade dimensions tie.
- “Inheritance means a selector matched every descendant.” Inheritance and direct matching are different value paths.
- “`border-box` prevents overflow.” It changes size accounting but cannot override every fixed or intrinsic minimum.
- “`1rem` always equals `16px`.” Sixteen pixels is a common starting value, but the unit follows the computed root font size, which can reflect user and author settings.
- “`rem` automatically makes a page accessible.” A relative unit can preserve a useful relationship, but only resizing, zoom, and reflow tests reveal clipping or loss of functionality.
- “Flexbox is old Grid” or “Grid replaces Flexbox.” They express different dimensional relationships and cooperate with normal flow.
- “Responsive means phone, tablet, desktop.” Content, containers, zoom, input, preferences, and window configuration are the real inputs.

- “A high `z-index` is globally on top.” Stacking contexts are local.
- “Passing an automated accessibility checker proves the CSS is accessible.” It supplies partial evidence only.

Accommodations and alternatives

Provide completed source files to students whose assistive workflow makes timed typing an irrelevant barrier; assess their explanation, diagnosis, and adaptation instead. Supply textual descriptions of every diagram through the surrounding prose, allow browser zoom and personal styles, and avoid grading pixel-identical rendering across platforms. A learner who cannot use a pointing device must be able to complete every required task by keyboard or an equivalent input method.

Formative rubric

- **Conceptual reasoning, 30%:** explains matching, cascade, boxes, and layout choices with correct boundaries.
- **Responsive resilience, 25%:** passes required widths, zoom, long-content, direction, and container tests without concealing overflow.
- **Accessible presentation, 20%:** preserves focus, contrast, target usability, source order, preferences, and perceivable states.
- **Engineering evidence, 15%:** uses Network, Styles, Computed, box, and layout tools and records reproducible observations.
- **Maintainability, 10%:** uses purposeful names, low specificity, documented layers, cohesive commits, and no forbidden repair shortcuts.

Session 4 - JavaScript from First Principles

JavaScript turns the static Campus Hub page into a program. This chapter begins before the first line of code. It assumes that you may never have installed a programming runtime, opened a project folder, used a terminal, or interpreted an error message. We will prepare the computer, run one deliberately small program in Node.js, run another in a browser, and learn how to tell source code from its output. Only then will we introduce values, names, decisions, repetition, functions, arrays, objects, errors, and tests.

Document Object Model (DOM) manipulation begins in Chapter 5. Keeping that boundary clear is useful: first we learn the JavaScript language, and then we use browser facilities to change a page. A reader who already programs should still complete the environment checks because later laboratories depend on the same folders, commands, and evidence.

Session map

Table 4.1 maps each learning phase to the evidence students and instructors can inspect before moving forward.

Table 4.1 - Session 4 learning sequence and evidence

Phase	Main question	Evidence
Orient	Where does JavaScript run?	Browser and Node.js produce distinguishable results
Represent	What kinds of values can a program hold?	Predicted types and evaluated expressions
Decide and repeat	How does execution choose a path or revisit work?	Traced branches and loop iterations
Abstract	How does a function turn inputs into a result?	Calls, returns, and scope traces
Model	How do arrays and objects represent Campus Hub data?	Filtered and summarized event records
Verify	How do we know the program is correct?	Passing tests plus inspected browser output

Prerequisites

You need ordinary familiarity with opening files and visiting a web address. Chapter 0 remains the complete orientation reference, but this chapter repeats the exact preparation needed for JavaScript so that no reader is blocked by an unexplained tool. No previous programming course is assumed. Readers with prior programming experience should still complete the JavaScript-specific checkpoints on equality, coercion, scope, and mutation.

Learning outcomes

After this chapter, you should be able to:

1. install or verify the editor, browser, Git, Node.js, and npm required by the course;
2. identify the project folder, current terminal directory, source file, command, and output;
3. create and run a first JavaScript file with Node.js and in a browser;
4. distinguish the ECMAScript language from browser and Node.js host facilities;
5. create and inspect primitive values, arrays, objects, and functions;
6. use `const` and `let` according to whether a binding must be reassigned;

7 trace expressions, statements, branches, loops, and function calls;

8 explain strict equality, truthiness, conversion, `null`, and `undefined`;

9 design functions with explicit inputs, return values, and useful errors;

10 transform arrays without accidentally changing source data;

11 explain the difference between rebinding a name and mutating an object;

12 diagnose syntax, reference, type, and assertion failures from evidence; and

13 write and run small automated tests with Node.js.

0. Prepare the computer and run your first JavaScript

A programming language is learned by making predictions and checking them against a running program. Reading alone cannot provide that feedback. Before we study JavaScript vocabulary, we therefore need a small, dependable path from a file you edit to output you can observe.

Figure 4.1 shows the complete beginner workflow. The boxes are sequential steps, not alternative technologies. Visual Studio Code edits the source file. Node.js executes it. The terminal shows the output. Git records the useful change after the program behaves as intended.

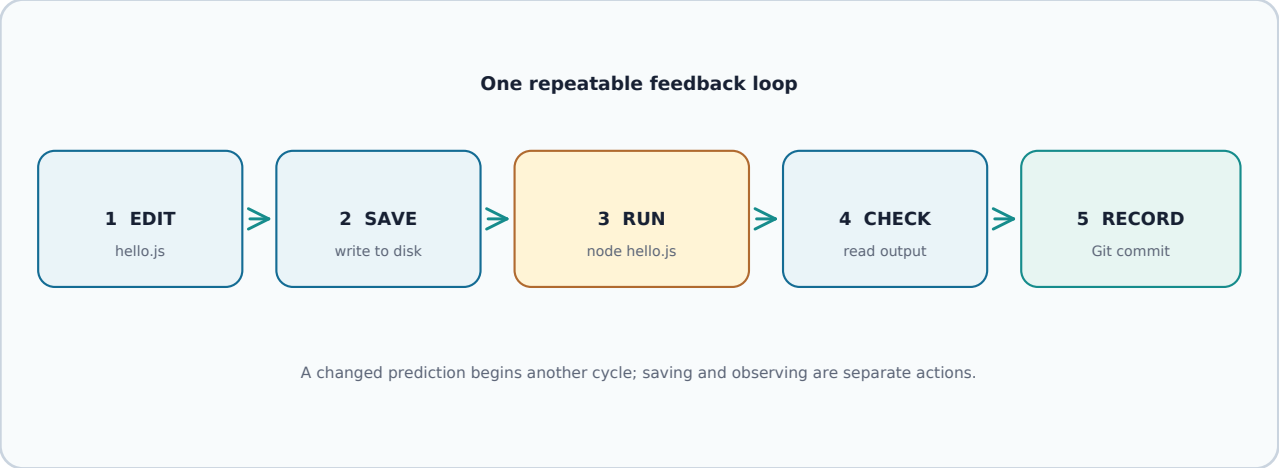


Figure 4.1. A first program moves through an observable edit-save-run-check-record cycle.

0.1 Understand the five tools before installing them

The tools have different responsibilities.

- **Visual Studio Code (VS Code)** is the text editor used to open the project folder and write source files. It does not become JavaScript merely because a file is open in it.
- **Node.js** is a JavaScript runtime: a program that can read and execute a JavaScript file outside a browser.
- **npm** is the package manager installed with Node.js. It reads project

instructions from `package.json` and runs named scripts such as `npm test`. Its official name is written lowercase and is not expanded as a current acronym. npm is not the JavaScript language and it is not the web server itself.

- A **web browser** runs client-side JavaScript and supplies browser Application

Programming Interfaces (APIs) such as the DOM, events, and network requests. We use an up-to-date Chromium- based browser and also test important work in Firefox.

- **Git** records versions of the project. GitHub stores a remote copy and helps

people collaborate, but Git and GitHub do not execute JavaScript.

Table 4.2 gives the command or visible evidence that proves each required tool is available. A successful installation is not established by remembering that an installer was clicked; it is established by current evidence.

Table 4.2 - JavaScript workstation tools and verification evidence

Tool	Purpose in this chapter	Verification evidence
VS Code	Edit the complete project folder and its files	The Explorer shows `package.json` and the `labs` folder
Node.js 24 Long-Term Support (LTS)	Execute JavaScript and course tools	`node --version` prints a value beginning with `v24.`
npm	Run commands defined by the project	`npm --version` prints a version number
Browser	Run client-side code and expose Developer Tools	The Console accepts `2 + 3` and displays `5`
Git	Record and inspect changes	`git --version` prints a version number

The course is tested with the latest available patch release in the Node.js 24 Long-Term Support (LTS) line. “LTS” means that the release line receives a longer period of maintenance than a short-lived current release. Use the newest 24.x patch offered by the official Node.js download page; do not search for an old patch number copied from a screenshot.

0.2 Install Node.js on Windows, step by step

If `node --version` already reports `v24.x.x`, continue to Section 0.4. Otherwise, use the following sequence.

1. Open a browser and visit the official download page at `https://nodejs.org/en/download`.
2. Select the latest **LTS** release in the 24.x line and the Windows Installer for your computer. Most current Windows computers use the x64 installer; an ARM-based Windows computer needs the ARM64 installer. Open **Settings > System > About > System type** if you do not know the architecture.
3. Open the downloaded `.msi` file. Windows may ask whether the installer is allowed to make changes. Confirm only when the publisher and download source are the official Node.js project.
4. Accept the license and keep the default installation features, including npm and the option that adds Node.js to `PATH`. `PATH` is the operating system's list of folders searched when you type a command such as `node`.
5. Finish the installer. Close every old Command Prompt, PowerShell, and VS Code terminal window. An already-open terminal may not know about the updated `PATH`.
6. Open a new terminal and run `node --version`, followed by `npm --version`.

On macOS, use the official `.pkg` installer for the current LTS release, finish the installer, open a new Terminal window, and run the same verification commands. On Linux, prefer the installation method approved for the course or institution; distribution repositories can provide a different major version. Whatever installation method is used, the evidence required by this book is the same: a new terminal must report Node.js 24 and a working npm command.

Do not use an online playground as a substitute for this setup. Playgrounds are useful for experiments, but they do not teach the local folders, module files, tests, server process, and Git workflow used throughout the course.

0.3 Install or verify Visual Studio Code

Visit the official download page at `https://code.visualstudio.com/Download`. On Windows, the **User Installer** is appropriate for most students because it installs for the current account without requiring a system-wide installation. Run the installer and keep the option that adds the `code` command to `PATH` when it is offered. Then close and reopen terminals.

Open VS Code. Choose **File > Open Folder**, select your personal coursework repository, and choose **Select Folder**. A folder opened this way is the workspace. In the Explorer at the left, confirm that `package.json`, `labs`, and `scripts` are visible at the top level. Opening only `events.js` is not equivalent: npm commands depend on the project root that contains `package.json`.

If VS Code displays **Workspace Trust**, trust the folder only if it is your own coursework clone or the authorized course repository. Restricted Mode is the appropriate place to inspect unfamiliar code before allowing it to run.

0.4 Open the correct terminal and prove its current directory

In VS Code, choose **Terminal > New Terminal**. The terminal panel normally opens below the editor. The text before the cursor is the **prompt**. For example, PowerShell may display the following prompt.

```
PS C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall>
```

Do not type the `PS`, the path, or the final `>` when copying a command. They are evidence showing which shell is running and which directory it currently uses. Type the following commands one line at a time, pressing Enter after each line.

```
dir package.json
node --version
npm --version
git --version
```

On macOS or Linux, `ls package.json` replaces `dir package.json`; the other commands are the same. Stop if the first command says that `package.json` cannot be found. The terminal is not at the repository root. Use **File > Open Folder** again, or change directory with `cd` followed by the exact repository path. A path containing spaces must be enclosed in quotation marks.

```
cd "C:\Users\Student\Documents\GitHub\web-programming-coursework-2026_Fall"
```

Figure 4.2 distinguishes three things beginners often merge together: the file saved on disk, the command typed into a shell, and the output produced by the runtime. Editing the file does nothing until the updated file is saved and run.

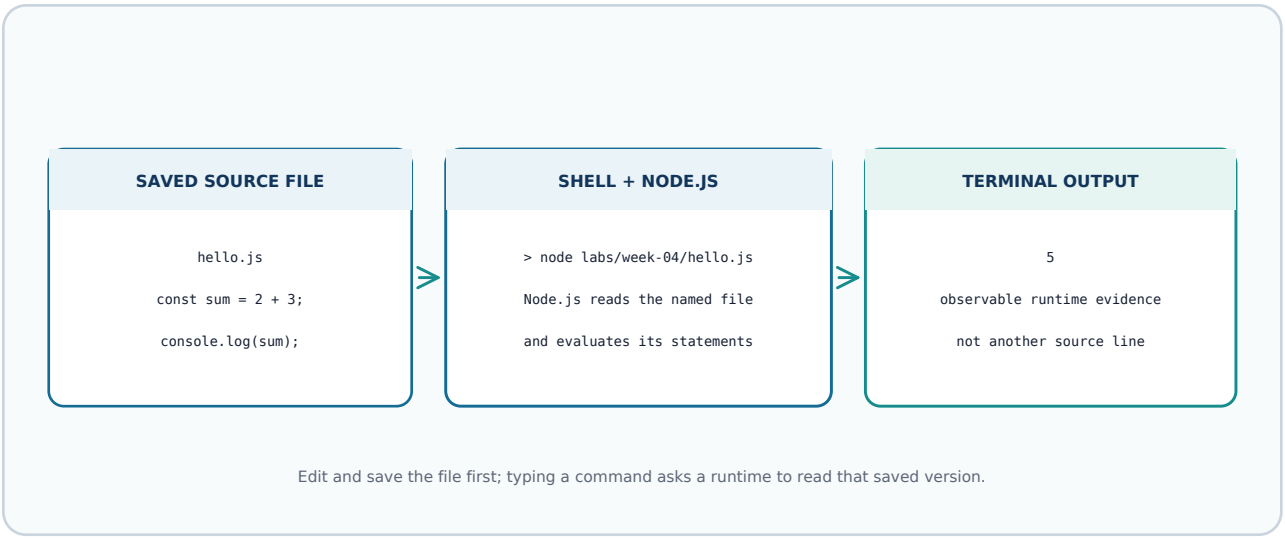


Figure 4.2. The saved source file, shell command, runtime process, and terminal output are related but distinct.

0.5 Create and run the first Node.js program

In the VS Code Explorer, expand `labs`, expand `week-04`, and open `hello.js`. If your instructor has asked you to create the file, right-click `week-04`, choose **New File**, type `hello.js`, and press Enter. The `.js` extension tells tools that the file contains JavaScript source code.

Enter the following program exactly, then save it with **File > Save** or Ctrl+S on Windows and Linux, or Command+S on macOS.

```
const courseName = "Web Programming";
const sum = 2 + 3;

console.log(`Hello from ${courseName}!`);
console.log(`Two plus three is ${sum}.`);
```

Before running it, read the punctuation. Table 4.3 names each important symbol and explains the job it performs in this first program.

Table 4.3 - Symbols in the first JavaScript program

Source text	Meaning in this program
`const`	Declare a name that will not be assigned a different value later
`courseName` and `sum`	Names chosen by the programmer; JavaScript is case-sensitive
`=`	Assign the value on the right to the name on the left; it does not mean equality here
`"Web Programming"`	A string value: a sequence of text characters
`2 + 3`	An expression that adds two number values and produces `5`
`` `...${...}...` ``	A template literal that inserts an evaluated expression into text
`console.log(...)`	Call the `log` function of the host's `console` object
`(` and `)`	Enclose the argument supplied to the function call
`.`	Select the named `log` property from `console`
`;`	Mark the intended end of a statement in this course's style

From the repository root, run the file with Node.js.

```
node labs/week-04/hello.js
```

Expected terminal output follows. Do not type these two output lines as commands.

```
Hello from Web Programming!
Two plus three is 5.
```

Change `2 + 3` to `2 * 3`, save, predict the new result, and run the same command again. If the output still says `5`, the file was probably not saved or a different file was executed. This small experiment establishes an essential cycle: edit, save, predict, run, observe, and explain.

0.6 Run JavaScript in a browser

Node.js and a browser both implement JavaScript, but they are different host environments. Open `hello-browser.html`. Near the end of its `<body>`, the following element asks the browser to obtain and execute another file.

```
<script src="hello-browser.js"></script>
```

The `src` attribute is a relative Uniform Resource Locator (URL). From `hello-browser.html`, it identifies the neighboring `hello-browser.js` file. The closing `</script>` tag is required even when the code lives in a separate file.

Why are these two files separate?

The two files answer different questions. `hello-browser.html` is the document the browser displays. It gives the page its structure and meaning: headings, paragraphs, links, and the place where the browser may load a program. It can be opened and read as a document even when JavaScript is disabled.

`hello-browser.js` is a JavaScript source file. It contains instructions for the JavaScript engine, such as calculating `2 + 3` and writing messages with `console.log()`. It is not a second page and it is not HTML. The browser does not execute it merely because both files are in the same folder; the `script` element creates the explicit relationship.

Keeping the files separate gives each one a clear responsibility. HTML describes the document, CSS describes presentation, and JavaScript describes computation and interaction. The same JavaScript file can serve several pages, the HTML can remain meaningful when the script fails, and each file can be tested and edited without hiding one language inside another. In a larger application this separation also makes the Network panel, source control history, and error messages easier to interpret.

Read the two files as a small contract:

File	Responsibility	What you can observe
`hello-browser.html`	Supplies the document structure and requests the script	The page heading and the `<script src="hello-browser.js">` element
`hello-browser.js`	Runs JavaScript instructions in the browser host	The two messages in Developer Tools > Console

The browser follows the path in Figure 4.3. It first receives the HTML document, discovers the script reference while parsing it, requests the JavaScript file, and only then executes the source. If the HTML request succeeds but the script request returns `404`, the page can still appear while the Console remains empty. That is why the two files must be debugged separately: inspect the HTML request and the script request in Network, then inspect execution messages in Console.

Start the local server from the repository root.

```
npm run serve:week04
```

Leave that terminal open. When it prints a line containing `http://localhost:8000/`, the server is waiting for browser requests; the absence of a new prompt is normal. Open a second terminal for later commands.

In the browser, visit `http://localhost:8000/hello-browser.html`. Open Developer Tools with F12 or Ctrl+Shift+I on Windows and Linux, or Command+Option+I on macOS. Select **Console**. The two messages from the JavaScript file should appear there.

Figure 4.3 shows the path. The browser first requests the HyperText Markup Language (HTML) document. After parsing the `script` element, it makes a second request for the JavaScript file. Only then can the JavaScript engine execute the source and write to the Console. These requests happen in sequence; the HTML file does not physically contain the external JavaScript file.

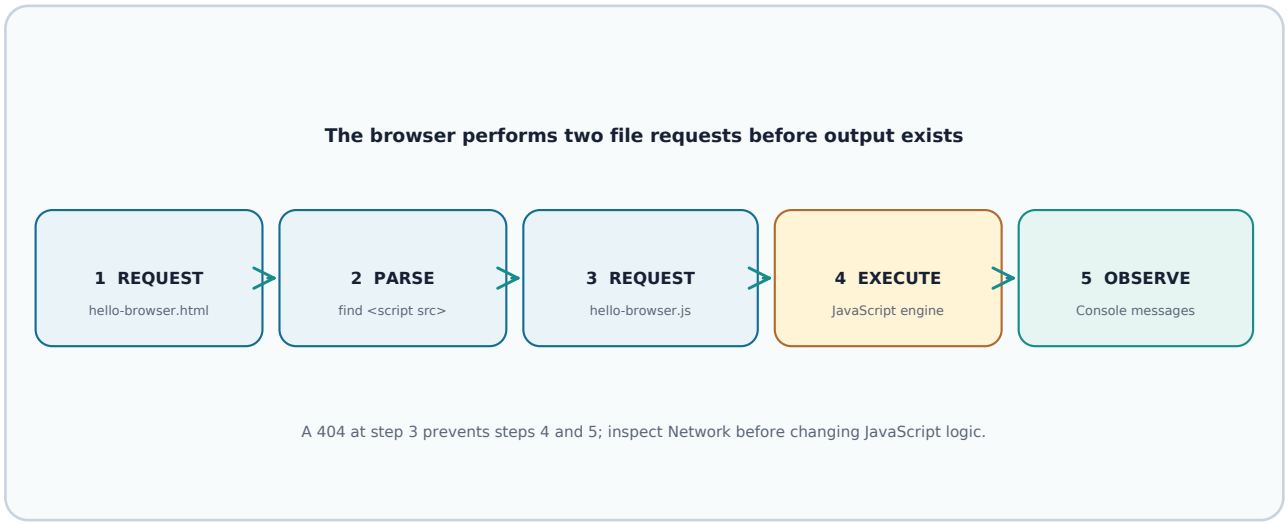


Figure 4.3. The browser requests HTML, discovers and requests JavaScript, executes it, and exposes Console evidence.

The Console is a developer evidence panel, not part of the visible web page. Changing page content uses the DOM and begins in Chapter 5. For now, logging is a controlled way to observe values without adding another new concept.

0.7 Read the first failures instead of starting over

An error message is a report about an attempted operation. Read it before changing anything. Table 4.4 maps common observations to the first useful check.

Table 4.4 - First-run symptoms, meanings, and next checks

Observation	Likely meaning	First check or repair
`'node' is not recognized` or `command not found`	The shell cannot find Node.js	Install Node.js, close all terminals, then open a new one and run `node --version`
`Cannot find module ...hello.js`	Node.js cannot find the named file at that path	Run `dir labs\week-04` on Windows or `ls labs/week-04`; compare spelling and current directory
npm reports `ENOENT` for `package.json`	npm searched the wrong directory	Run `dir package.json`; return to the repository root before the npm command
The server command prints no prompt afterward	The server is still running normally	Leave it open; use a second terminal for tests and Git
The browser says it cannot reach the site	No server is listening at that address and port	Read the server terminal; start the server and copy the printed URL exactly
The HTML appears but no Console message appears	The script was not loaded or did not execute	Save both files, reload, inspect Console, then inspect the Network request for `hello-browser.js`
Network shows `404` for the script	The requested URL does not match an available file	Compare `src`, capitalization, filename, and folder location
`SyntaxError` names a line	JavaScript could not parse the source	Inspect that line and the preceding line for a missing quote, bracket, parenthesis, or backtick
`EADDRINUSE` mentions port 8000	Another process already owns the port	Return to the earlier server terminal or stop that process with Ctrl+C before restarting

Do not respond to every failure by reinstalling software. Installation repairs a missing tool; it does not repair a misspelled filename, unsaved source, wrong directory, or syntax error. Diagnose the boundary named by the evidence.

---

1. A program is an ordered explanation for a machine

The Campus Hub page already contains information and presentation, but HTML and Cascading Style Sheets (CSS) do not calculate how many seats remain. A program supplies an ordered set of instructions and decisions. Given capacity `80` and registrations `62`, it can evaluate `80 - 62`, name the result, classify the event as open, and produce a summary.

```
const capacity = 80;
const registered = 62;
const remaining = capacity - registered;

console.log(remaining);
```

The expected console output is the number `18`. The code is source text; `18` is runtime evidence. A semicolon ends each declaration here. JavaScript can insert some omitted semicolons, but this course writes them consistently so statement boundaries remain visible to beginners.

Figure 4.4 separates the language from two environments that can execute it. Both environments understand subtraction, bindings, arrays, and functions. A browser additionally supplies `document`, while Node.js supplies objects such as `process`. Neither host object is part of the core language.

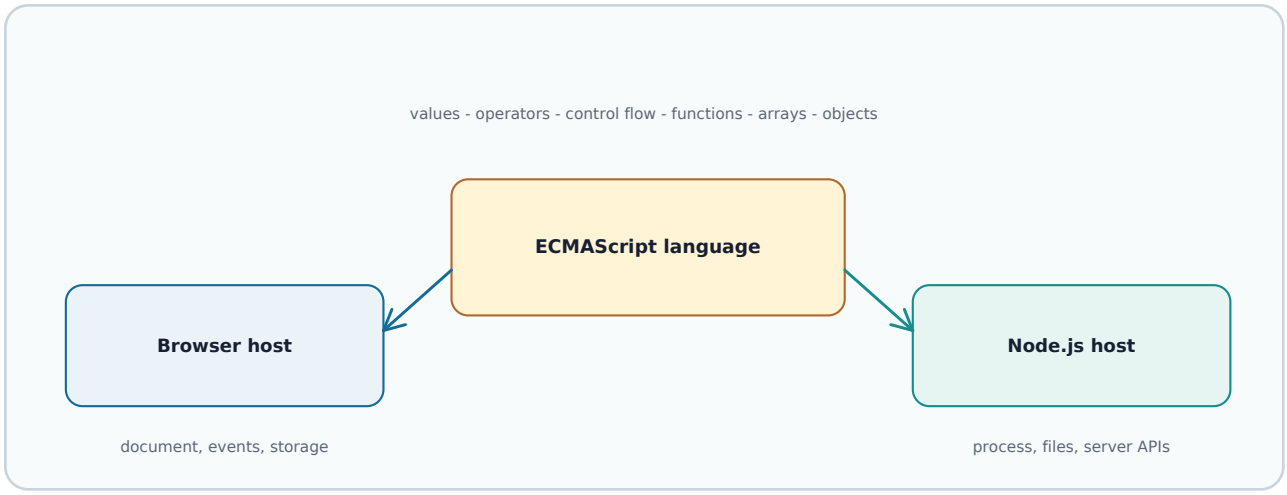


Figure 4.4. The ECMAScript language runs inside a host; browsers and Node.js add different environment APIs.

1.1 ECMAScript and JavaScript

ECMAScript is the standardized language specification. JavaScript is the name used for implementations of that language together with the surrounding platform in which code runs. The standard defines values, operators, statements, functions, objects, and built-in facilities. HTML, the DOM, browser storage, and Node.js file access come from hosts and are studied at the appropriate boundary.

1.2 Your first file, from an exact directory

The first program proved that one file can execute. We now move to the continuing Campus Hub example. In your personal coursework repository, open `labs/week-04/events.js` in Visual Studio Code. From a terminal whose current directory contains `package.json`, run:

```
npm run serve:week04
```

Keep that terminal open; it is the server terminal. Visit `http://localhost:8000/`, open Developer Tools, and select **Console**. The starter prints a table of three event objects. If the page does not load, first inspect the server terminal. If the page loads but the table is absent, inspect the Console and Network panels for a failed `events.js` request or a JavaScript error.

1.3 Execution order, comments, and case sensitivity

For the synchronous programs in this chapter, JavaScript normally evaluates statements from top to bottom. A later statement can use a binding created by an earlier declaration. Moving `console.log(sum)` above `const sum = 2 + 3` causes an error because the program tries to read the binding before initialization. Asynchronous work changes the timing model and is introduced in Chapter 6.

JavaScript is case-sensitive. `eventTitle`, `eventtitle`, and `EventTitle` are three different names. Choose one spelling and capitalization, then use it consistently. Course code uses lower camel case for variables and functions: the first word begins in lowercase and each later word begins with a capital, as in `seatsRemaining`.

Comments explain intent that the code itself cannot express clearly.

```
// One-line comment: registration cannot create a negative seat count.
const remaining = Math.max(0, capacity - registered);

/* A block comment can span
```

```
    more than one source line. */
```

The engine ignores comments, but a comment is not a place to preserve obsolete code. Git already records history. Prefer a meaningful name and a clear function over a comment that merely translates syntax into English.

2. Values have types; bindings have names

A **value** is data the program can work with. A **type** describes the operations and behavior associated with a value. JavaScript has seven primitive types: string, number, bigint, boolean, undefined, symbol, and null. Everything else is an object; arrays and functions are specialized objects.

JavaScript is dynamically typed. The value has a type at runtime; a variable declaration does not permanently acquire one type from its name. A `let` binding can therefore refer to a number and later to a string. That flexibility does not make the two values interchangeable. Reliable programs keep a stable meaning for each binding and validate data when it crosses a boundary.

A value written directly in source, such as `18`, `true`, or `"open"`, is a **literal**. A program can also obtain values by evaluating expressions, reading input, calling functions, or receiving network data. The same displayed text can hide different types: the number `18` can be added numerically, while the string `"18"` is text until deliberately converted.

Table 4.5 collects the value categories used in this chapter. The examples are values, while the final column explains the boundary that matters in programs.

Table 4.5 - JavaScript values introduced in this chapter

Type	Example	Purpose and important boundary
String	`"Robotics night"`	Text; strings are immutable sequences
Number	`18`, `2.5`, `NaN`, `Infinity`	Floating-point numeric values; integers share this type
BigInt	`9007199254740993n`	Integers beyond safe Number precision; cannot be mixed directly with Number arithmetic
Boolean	`true`, `false`	Logical outcomes used by conditions
Undefined	`undefined`	Commonly means no value has been supplied or produced
Null	`null`	An intentional absence chosen by the program or data contract
Symbol	`Symbol("id")`	A unique primitive key; important later, uncommon in beginner application data
Object	`{ title: "Robotics night" }`	A mutable collection of keyed properties

Use `typeof` as an observation tool, not as a complete theory of values.

```
typeof "Robotics night"; // "string"
typeof 18;               // "number"
typeof true;             // "boolean"
typeof undefined;        // "undefined"
typeof null;             // "object" - a long-standing historical result
typeof [];               // "object"
typeof function () {};   // "function"
```

Use `Array.isArray(value)` to distinguish arrays from other objects. Test for `null` directly with `value === null`; do not infer that `null` is an ordinary object from the historical `typeof` result.

2.1 Expressions produce values

An expression can be evaluated to produce a value. `capacity - registered`, `event.title`, and `seatsRemaining(event)` are expressions. A statement performs an action in the program's execution, such as declaring a binding or choosing an `if` branch. Figure 4.5 traces tokens through evaluation and assignment.

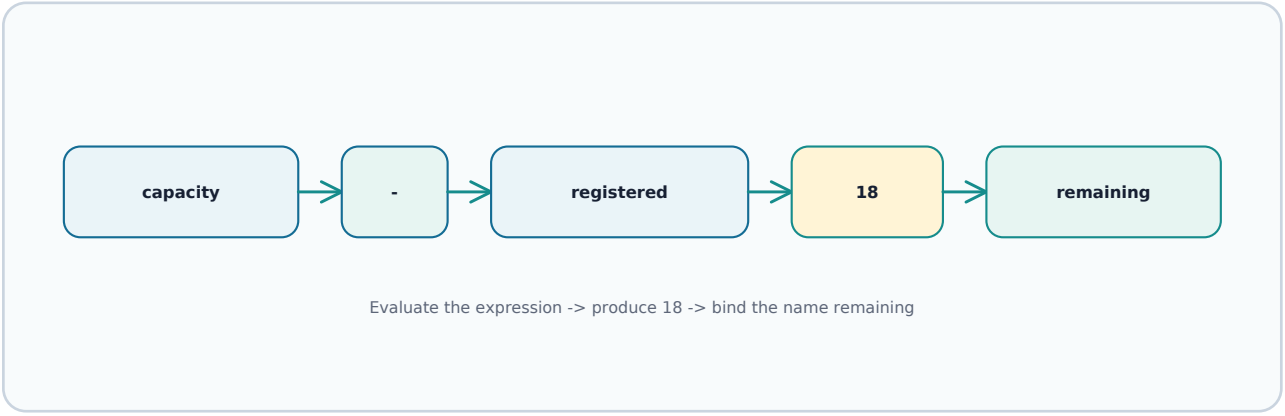


Figure 4.5. Evaluation produces a value, and a declaration connects that value to a binding name.

Evaluation works from smaller expressions toward a result. In `capacity - registered`, JavaScript first reads the two operand values and then applies subtraction. Parentheses make grouping explicit. Prefer `(registered / capacity) * 100` to relying on a reader to remember every precedence rule. Parentheses do not turn a value into a function call unless they follow a callable expression.

2.2 `const` and `let`

A declaration creates a **binding**, which connects a name to a value.

```
const eventTitle = "Robotics night";
let registered = 62;
registered = registered + 1;
```

A declaration contains a keyword, an identifier, an assignment operator, an initializer expression, and normally a semicolon. An **identifier** is a legal source name. It may contain letters, digits after the first character, underscores, and dollar signs, but it cannot contain spaces or be a reserved word such as `if`. Prefer names that express domain meaning; `remainingSeats` carries more information than `x`.

Use `const` by default. It prevents the name from being assigned a different value. Use `let` when reassignment is part of the algorithm, as with a counter. Do not use `var` in new course code; its function scope and hoisting rules obscure the block-scoped model students need first.

`const` does not freeze an object:

```
const event = { registered: 62 };
event.registered = 63; // mutation is allowed
// event = {};         // rebinding would fail
```

The binding remains connected to the same object; a property inside that object changed. Figure 4.8 later makes this distinction visible.

2.3 Strings and template literals

Single and double quotes create strings. Backticks create a template literal, which can interpolate an expression inside `${...}`.

```
const message = `${eventTitle}: ${remaining} seats remaining`;
```

Interpolation converts the embedded result to text. It is clearer than a long chain of `+` operations, where the same operator can mean numeric addition or string concatenation.

Strings are immutable: operations produce new strings rather than changing the characters inside an existing string. `message.length` reports the number of UTF-16 code units, and `message[0]` reads the first one because indexes begin at zero. Escape sequences represent characters that would otherwise end or disrupt a literal: `"Line one\nLine two"` contains a newline, and `"She said \"hello\""` contains quotation marks. Template literals can span lines, but preserve those line breaks in the resulting string.

---

3. Operators create relationships

Arithmetic operators include `+`, `-`, `*`, `/`, `%`, and `**`. Comparison operators such as `<`, `<=`, `>`, and `>=` produce booleans. Use strict equality `===` and strict inequality `!==` for ordinary application comparisons.

An operator receives one, two, or sometimes three operands. The unary `-` in `-capacity` has one operand; `capacity - registered` has two. Assignment `=` stores a value in a binding or property. Equality `===` asks a question and produces a boolean. Confusing these roles can change program state instead of testing it.

```
18 === 18;    // true
18 === "18";  // false
18 == "18";   // true because loose equality converts a value
```

Loose equality is governed by defined coercion rules, not randomness, but those rules add reasoning that most application comparisons do not need. This course uses strict equality unless a specific algorithm deliberately requires another sameness relation.

3.1 Conversion must be visible

Data from form controls and URL parameters commonly arrives as strings.

```
const enteredCapacity = "80";
const capacity = Number(enteredCapacity);

if (!Number.isFinite(capacity)) {
  throw new TypeError("Capacity must be a finite number.");
}
```

`Number()` makes the conversion explicit. `Number.isFinite()` rejects `NaN` and infinities. Remember that `NaN === NaN` is false; use `Number.isNaN()` when that specific test is required.

Conversion can succeed syntactically while producing an unusable number. `Number("")` produces `0`, while `Number("eighty")` produces `NaN`, whose name means “Not a Number.” Validate the original input when an empty value must be rejected, then validate the converted result. `parseInt()` and `parseFloat()` accept numeric prefixes such as the `12` in `"12 seats"`; use them only when that prefix behavior is part of the input contract, not as a general repair for bad data.

3.2 Truthiness and short-circuiting

Conditions convert their expression to a boolean. `false`, `0`, `-0`, `0n`, an empty string, `null`, `undefined`, and `NaN` are falsy. Objects and arrays are truthy, including an empty array.

`&&` stops at the first falsy operand; `||` stops at the first truthy operand; `??` uses the right operand only when the left is `null` or `undefined`.

```
const label = suppliedLabel ?? "Untitled event";
```

Use `??` when `0`, `false`, or an empty string may be valid data. Using `||` in that situation can replace a meaningful value accidentally.

---

4. Control flow chooses and repeats work

An `if` statement selects a branch. Only one branch in the following chain runs.

```
if (remaining === 0) {
  status = "full";
} else if (remaining <= 10) {
  status = "almost full";
} else {
  status = "open";
}
```

Figure 4.6 is a decision flow, not three independent tests. Once one condition succeeds, execution skips the remaining branches and continues after the chain.

The expression inside `if (...)` is the condition. The braces delimit a block. Indentation does not control execution, but it exposes the block structure to human readers and should match the braces. Trace a branch by writing the current input, the boolean result of each condition actually reached, the selected branch, and the value after the chain. This prevents guessing from the final line alone.

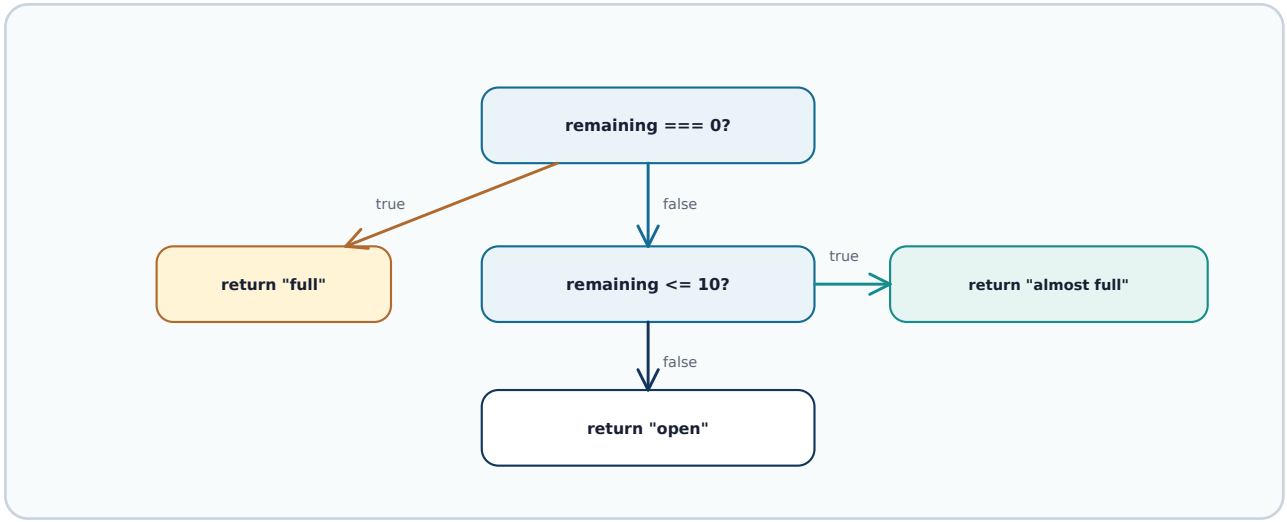


Figure 4.6. An if-else chain tests conditions in order and executes only the first matching branch.

4.1 Loops repeat with a changing state

Use `for...of` when you need each value in an iterable collection.

```
for (const event of events) {
  console.log(event.title);
```

```
}
```

On each iteration, `event` is bound to the next array value. The body runs once for that value. With three events, the body runs three times. The loop itself does not create a new array; it performs the statements in its body.

A counted loop exposes initialization, continuation condition, and update in one header.

```
for (let index = 0; index < events.length; index += 1) {
  console.log(index, events[index].title);
}
```

First `index` is initialized to zero. Before each iteration, `index < events.length` is tested. After the body, `index += 1` increases the index. Replacing `<` with `<=` would attempt one index beyond the last element because the final valid index is `length - 1`.

Use a counted `for` loop when the index itself matters, and a `while` loop when repetition is governed by a condition whose number of iterations is not known in advance. Every loop needs a progress argument: identify what changes and why the condition must eventually become false. `break` exits the nearest loop; `continue` skips to its next iteration. Avoid `for...in` for array values because it iterates enumerable property keys rather than expressing value iteration.

---

5. Functions give a calculation a name and boundary

A function receives inputs through parameters and can return an output.

```
function seatsRemaining(event) {
  return Math.max(0, event.capacity - event.registered);
}

const remaining = seatsRemaining(events[0]);
```

Defining a function does not run its body. The body runs when a call expression uses parentheses. For the call above, JavaScript evaluates `events[0]`, creates the local parameter binding `event`, executes the body, evaluates the return expression, ends the call, and places the returned number into the surrounding declaration.

`event` is a parameter in the declaration. `events[0]` is the argument supplied by this call. `return` ends the call and sends one value to the calling expression. If execution reaches the end without `return`, the result is `undefined`.

Figure 4.7 shows one call frame. The argument is connected to the parameter, the body evaluates using local bindings, and the result returns to the call site.

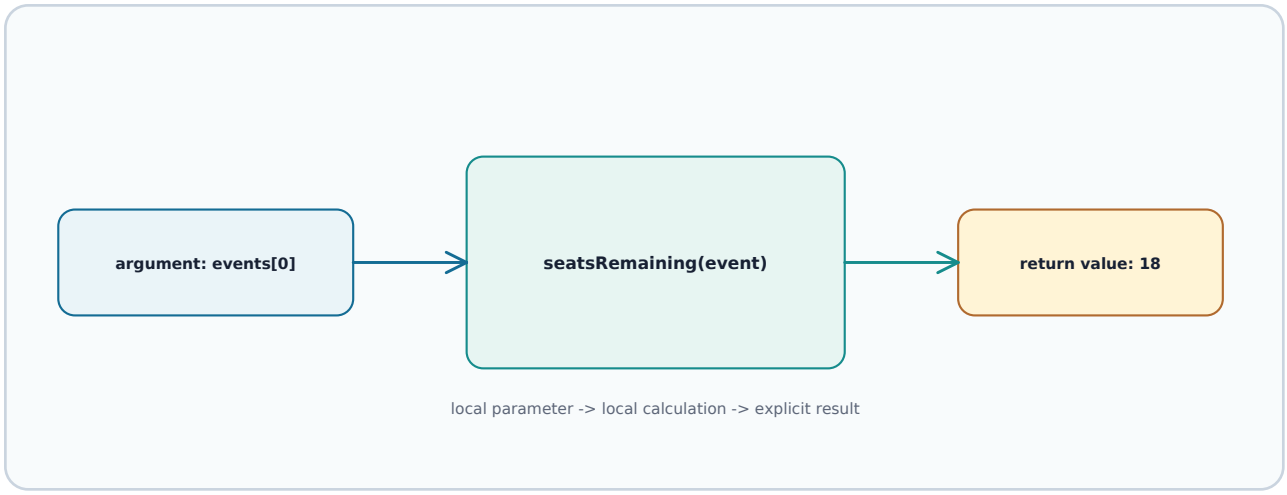


Figure 4.7. A function call connects arguments to local parameters and returns one result to the call site.

5.1 Declarations, expressions, and arrow functions

JavaScript provides several function forms.

```
function double(value) { return value * 2; }
const doubleAgain = function (value) { return value * 2; };
const doubleBriefly = (value) => value * 2;
```

They are not interchangeable in every advanced situation: `this`, construction, and hoisting differ. For this chapter, use function declarations for named domain operations and short arrow functions for callbacks such as array transformations.

A function should have a clear contract: accepted inputs, returned result, possible error, and observable side effects. A **pure function** returns the same result for the same inputs and does not change external state. Purity is not a requirement for every operation, but calculation functions such as `seatsRemaining()` are easier to reason about and test when they remain pure.

5.2 Scope answers where a name is available

JavaScript uses lexical scope: availability follows the nesting of source blocks and functions. A binding declared with `let` or `const` inside `{ ... }` is not available outside that block. A function can read bindings in surrounding scopes, but callers cannot read its local bindings.

```
const course = "Web Programming";

function describeEvent(event) {
  const remaining = seatsRemaining(event);
  return `${course}: ${event.title} has ${remaining} seats`;
}
```

Keep data local and pass needed inputs explicitly. Excessive global state makes functions depend on invisible context and makes tests interfere with one another.

6. Arrays model ordered collections

An array stores an ordered sequence. Indexes begin at zero.

```
const categories = ["technology", "arts", "community"];
categories[0];     // "technology"
categories.length; // 3
```

Reading beyond the existing indexes produces `undefined`; it does not wrap or throw automatically. Methods such as `push()` mutate an array. Spread syntax can create a shallow copy: `const copy = [...categories]`.

`length` is one greater than the highest ordinary sequential index in a dense array. The first item is index `0`, the second is `1`, and the last is `array.length - 1`. Arrays can technically mix value types, but application data is easier to validate when one array has one clear element shape, such as "event objects." Use `Array.isArray(value)` at an untrusted boundary because `typeof value` reports `"object"` for both arrays and ordinary objects.

6.1 Transformations express intent

`filter()` selects elements, `map()` transforms each element, `find()` returns the first matching element, and `some()` or `every()` answer boolean questions.

```
const featuredTitles = events
  .filter((event) => event.featured)
  .map((event) => event.title);
```

The first callback returns a boolean; the second returns the new value for each selected event. Both methods return new arrays. They do not deeply copy the objects inside those arrays.

Use `reduce()` when the operation is genuinely an accumulation and the accumulator remains easy to name. A loop is preferable when it communicates the state change more clearly to the intended reader.

---

7. Objects model named facts

An object literal groups properties that describe one entity.

```
const event = {
  title: "Robotics night",
  capacity: 80,
  registered: 62,
};

event.title;
event["capacity"];
```

Dot notation is concise when the property name is known in source code. Bracket notation accepts an expression and is necessary for dynamic keys or names that are not identifier syntax.

Reading a missing property normally produces `undefined`. Assigning a property that does not yet exist creates it. These rules are convenient, but a misspelled property such as `event.capcity` can therefore look like absent data rather than an immediate syntax error. Validate required object shapes at boundaries and use consistent names.

Destructuring extracts named properties, while object spread creates a shallow copy with selected replacements.

```
const { title, capacity } = event;
const updated = { ...event, registered: 63 };
```

7.1 Primitive copying and object identity

Primitive values are immutable. Object variables hold references to objects.

```
const first = { registered: 62 };
const second = first;
second.registered = 63;
console.log(first.registered); // 63
```

Both bindings refer to the same object. Figure 4.8 contrasts that shared identity with two separately copied number values. Spread creates a new outer object, but nested objects remain shared unless copied deliberately.

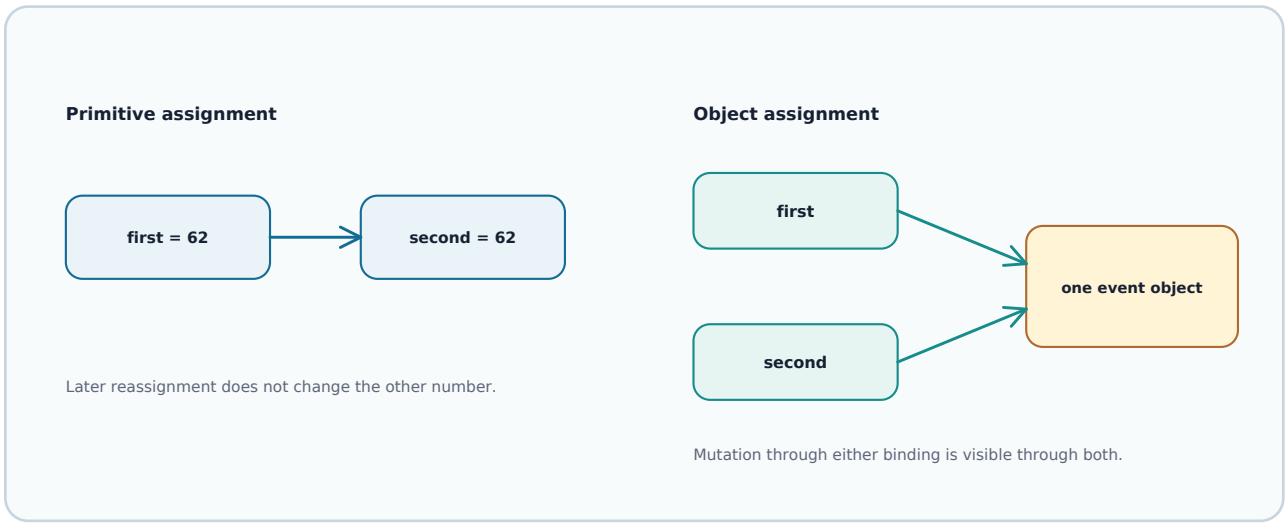


Figure 4.8. Primitive assignment copies a value, while object assignment can give two bindings the same object identity.

Optional chaining, as in `event.organizer?.email`, stops and produces `undefined` when the value before `?.` is `null` or `undefined`. It does not validate that an email is correct, and it should not hide a property that the program contract requires.

8. Errors are evidence, not verdicts

A syntax error means the source cannot be parsed as written. A reference error often means a name is unavailable. A type error means an operation is unsuitable for the received value. An assertion error means observed behavior disagreed with a test expectation.

A logical error is different: the program runs but produces the wrong behavior, such as reporting 18 occupied seats when the requirement asks for remaining seats. Logical errors may produce no exception, so predictions, examples, and tests matter. A stack trace is a sequence of active function calls reported with an error; the first line in external library code is not necessarily the source of the application defect.

Read an error from the first relevant frame of its stack, not only from the last line. Record the error name, message, file, line, operation being attempted, actual input, and expected contract. Figure 4.9 turns those observations into a repeatable diagnosis path.

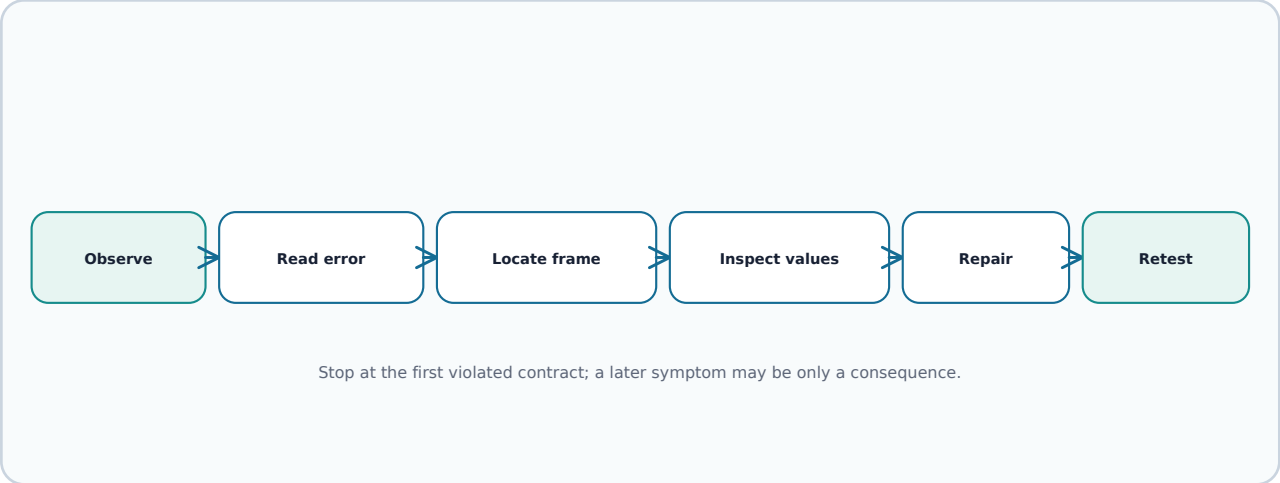


Figure 4.9. JavaScript debugging follows evidence from the observed failure to the violated contract and verified repair.

Throw an `Error` object when a function cannot honor its contract.

```
if (!Number.isFinite(event.capacity)) {
  throw new TypeError("Event capacity must be a finite number.");
}
```

Do not catch every error merely to suppress it. Catch an exception only where the program can add context, recover, translate it into an appropriate boundary response, or perform necessary cleanup. Otherwise allow it to remain visible during development.

---

9. Small tests make behavior repeatable

A test supplies an input, observes a result, and compares it with an expectation. Pure functions are especially testable because their return value depends on explicit arguments rather than hidden page or network state.

```
import test from "node:test";
import assert from "node:assert/strict";
import { seatsRemaining } from "./events.js";

test("calculates available seats", () => {
  const event = { capacity: 80, registered: 62 };
  assert.equal(seatsRemaining(event), 18);
});
```

`test(...)` registers a named case. The callback performs the case. `assert.equal(actual, expected)` compares an observed value with the required value using strict equality. When it fails, read the test name, actual value, expected value, operator, and first relevant application frame. Do not change the expected value merely to make the failure disappear; first decide whether the implementation or the stated requirement is wrong.

Run the supplied Week 4 tests from the coursework repository root.

```
npm test
```

A passing test is evidence for the tested examples, not proof that every possible input is correct. Include normal cases, boundary cases, and invalid inputs. A test name should state behavior, and a failure should make the disagreement easy to locate.

Guided laboratory - Model Campus Hub event availability

Goal and evidence

Complete `labs/week-04/events.js` so event records can be validated, classified, filtered, and summarized. Required evidence is a passing test run, the correct browser summary, a clean Console, and a Git commit containing only the intended Week 4 changes.

Step 1 - Establish the baseline

From the personal coursework repository root, prove `package.json` is present, run `npm test`, then run `npm run serve:week04`. Open `http://localhost:8000/`. Keep the server terminal open and use a second terminal for tests and Git commands.

Step 2 - Read the event shape

Open `labs/week-04/events.js`. For each property, state its type and purpose. Predict `80 - 62` before executing code. Open Console and verify the starter table.

Step 3 - Write one function at a time

Implement `seatsRemaining(event)`, then test it. Add `eventStatus(event)` with the boundaries `full`, `almost full`, and `open`. Do not write all functions before collecting evidence; one small failing-to-passing cycle localizes mistakes.

Step 4 - Transform the collection

Implement `filterEvents(allEvents, category)` and `summarizeEvents(allEvents)`. Preserve the input array. Use names that describe domain meaning rather than temporary mechanics.

Step 5 - Validate the boundary

Reject missing or non-numeric capacity and registration values with a useful `TypeError`. Observe one deliberate failure, read its stack, repair the input, and rerun the tests.

Step 6 - Compare output with code

Reload the browser. The completed reference result appears in Figure 4.10. Check all three event titles, status words, and remaining-seat counts. A visual match is not enough; the automated tests must also pass.

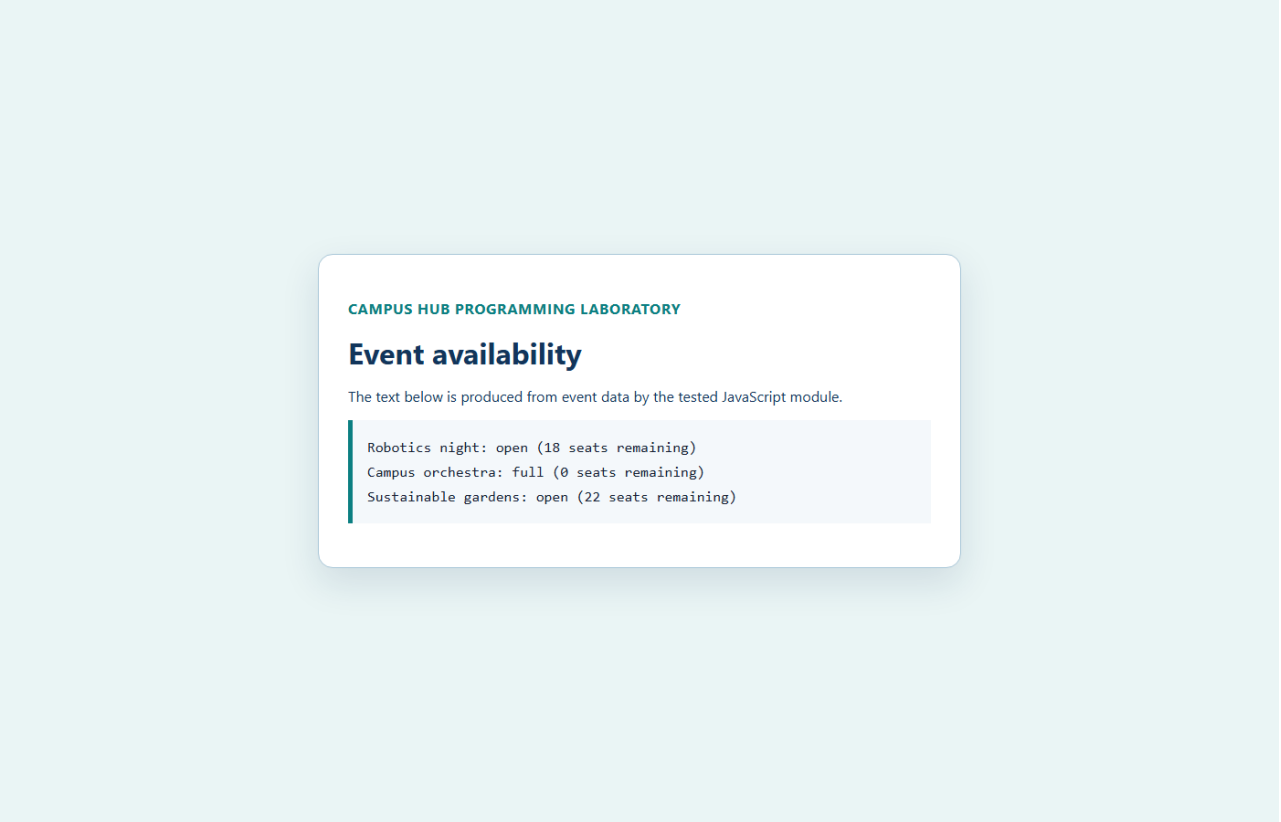


Figure 4.10. The tested JavaScript module transforms three event objects into visible availability summaries in the browser.

Step 7 - Commit evidence

Run `git status`, review `git diff`, then commit the completed Week 4 files with a message that describes the behavior, such as `Complete event availability logic`.

Use Table 4.6 as the laboratory completion record. Each row requires observable evidence rather than a statement that the task was attempted.

Table 4.6 - Week 4 laboratory completion record

Check	Expected evidence
Repository root	`package.json` is visible before npm commands
Automated tests	Normal, boundary, and invalid-input cases pass
Browser	Three correct event summaries replace the loading message
Console	No uncaught exception or failed module request
Data integrity	Filtering and summarizing do not mutate the source array
Git	One focused commit records the completed behavior

Exercises

Foundation

Exercise 1 - Values and types

State the type and value produced by `typeof 42`, `typeof "42"`, `typeof null`, `Array.isArray([])`, and `typeof (() => 1)`.

Exercise 2 - Binding or object change

Explain which operation fails and why: `const event = { capacity: 80 };`, then `event.capacity = 90;`, then `event = { capacity: 100 };`.

Exercise 3 - Strict comparison

Predict `0 === false`, `0 == false`, `"" || "Untitled"`, and `"" ?? "Untitled"`. Explain the conversion or short-circuit rule.

Exercise 4 - Trace a branch

For remaining-seat values `0`, `8`, and `18`, trace the `eventStatus` branch and give the returned status.

Application

Exercise 5 - Write a total function

Write `occupancyPercent(event)` that returns a rounded percentage and rejects a non-positive capacity. Explain the chosen error.

Exercise 6 - Array pipeline

From the laboratory data, produce titles of featured events with more than 15 remaining seats. Do not mutate the source array.

Exercise 7 - Scope diagnosis

Explain why a `const remaining` declared inside a function cannot be read after the function returns. Show how returning it repairs the data flow.

Exercise 8 - Identity diagnosis

Two bindings refer to the same event object. A change through one appears through the other. Explain why and show a shallow-copy update.

Challenge

Exercise 9 - Boundary table

Design cases for `eventStatus`: ordinary open, exactly 10 seats, one seat, zero seats, over-registration, and invalid numeric data. State the expected result.

Exercise 10 - Debug a failing calculation

The Console shows `"8062"` after adding capacity and registrations obtained from form fields. Diagnose the result and repair the calculation without relying on implicit coercion.

Exercise 11 - Choose loop or transformation

Choose between a loop, `filter()`, `map()`, and `find()` for four tasks: logging every title, selecting technology events, constructing summary objects, and locating the first full event. Explain each choice.

Exercise 12 - Test quality

Critique a test that only calls `seatsRemaining()` with one ordinary valid event. Propose a compact set that provides stronger evidence.

Exercise 13 - Prove the environment before debugging code

A student says “JavaScript is broken.” Their terminal prompt shows `C:\Users\Student>`, `dir package.json` reports that the file is absent, and `npm run serve:week04` reports `ENOENT`. Explain what the evidence proves, what it does not prove, and give the next three commands or actions in order.

Exercise 14 - Separate source, command, and output

Classify each item as source code, a shell command, or runtime output: `console.log(2 + 3);`, `node labs/week-04/hello.js`, and `5`. Explain why typing the output line back into the shell is a category mistake.

Exercise 15 - Trace browser script loading

The HTML page appears, but the Console has no greeting. Network shows the HTML request with status 200 and `hello-brower.js` with status 404. Identify the most likely defect, the exact evidence that supports the diagnosis, and the repair.

Worked answers

Answer 1

The results are `"number"`, `"string"`, `"object"`, `true`, and `"function"`. The `null` result is a historical `typeof` behavior, not evidence that null has ordinary object properties. `Array.isArray()` is the intentional array check.

Answer 2

Changing `event.capacity` succeeds because the object is mutable. Assigning a new object to `event` fails because `const` prevents rebinding. The misconception is that `const` recursively freezes the value; it protects the binding.

Answer 3

`0 === false` is false because the types differ. `0 == false` is true after loose equality conversion. `"" || "Untitled"` returns `"Untitled"` because the empty string is falsy. `"" ?? "Untitled"` returns the empty string because it is neither null nor undefined. Use `??` when an empty string is valid data.

Answer 4

Zero enters the first branch and returns `"full"`. Eight fails the zero test but satisfies `<= 10`, returning `"almost full"`. Eighteen reaches the final branch and returns `"open"`. Branch order makes the boundary behavior explicit.

Answer 5

```
function occupancyPercent(event) {
  if (!Number.isFinite(event.capacity) || event.capacity <= 0) {
    throw new RangeError("Capacity must be a positive finite number.");
  }
  if (!Number.isFinite(event.registered)) {
    throw new TypeError("Registered must be a finite number.");
  }
  return Math.round((event.registered / event.capacity) * 100);
}
```

The range error identifies a numeric value outside the accepted domain; the type error identifies unsuitable numeric data. A project may standardize differently, but the contract and tests must agree.

Answer 6

```
const titles = events
  .filter((event) => event.featured && seatsRemaining(event) > 15)
  .map((event) => event.title);
```

Both methods return new arrays. The existing event objects remain shared, but this pipeline reads rather than mutates them.

Answer 7

The local binding belongs to the function's lexical scope and is unavailable to the caller. Return the value and bind the call result:

```
function remainingFor(event) {
  const remaining = event.capacity - event.registered;
  return remaining;
}
const remaining = remainingFor(event);
```

Answer 8

Assignment copies the object reference, not a separate object. Use `const updated = { ...original, registered: 63 };` to create a new outer object. This is shallow: nested objects would still be shared and require deliberate copying when independent mutation is necessary.

Answer 9

Expect `open` for 18, `almost full` for exactly 10 and for 1, and `full` for zero. The supplied implementation clamps over-registration to zero and therefore also returns `full`. Non-numeric data must throw `TypeError`. This set covers both branch boundaries and the validation contract.

Answer 10

Form values are strings, so `+` concatenates them. Convert and validate first:

```
const capacity = Number(capacityInput.value);
const registered = Number(registeredInput.value);
if (!Number.isFinite(capacity) || !Number.isFinite(registered)) {
  throw new TypeError("Capacity and registrations must be numbers.");
}
const total = capacity + registered;
```

Answer 11

Use `for...of` for logging because the purpose is a side effect for every value; `filter()` for selecting technology events; `map()` for constructing one summary per event; and `find()` for the first full event. A loop could implement all four, but the named methods communicate the result shape directly.

Answer 12

One ordinary case misses zero, over-registration, boundary values, missing properties, strings, and non-finite numbers. A compact stronger suite includes an ordinary subtraction, exactly full, over-registered clamping, each status boundary, and invalid inputs that must throw. Tests should verify both returned values and the promised error type.

Answer 13

The evidence proves that npm searched `C:\Users\Student` and did not find the project manifest there. It does not prove that Node.js, npm, or the JavaScript source is defective. First open the authorized coursework folder in VS Code or run `cd` with its exact path. Second run `dir package.json` and stop unless the file is listed. Third run `npm run serve:week04` from that verified root. The reasoning follows the boundary named by `ENOENT`: a missing path in the current directory, not a language failure.

Answer 14

`console.log(2 + 3);` is JavaScript source code stored in a `.js` file. `node labs/week-04/hello.js` is a shell command that asks Node.js to execute a named file. `5` is runtime output produced after the expression is evaluated. Typing `5` into a shell treats evidence as though it were a command name; the shell then searches for a program named `5`, which is unrelated to the original calculation.

Answer 15

The spelling `hello-brower.js` is missing the `s` in `browser`. Status 200 for the HTML proves that the server and document request worked. Status 404 for the script proves that the URL requested by `src` did not match an available file, so no JavaScript source was delivered for execution. Change the HTML attribute to `src="hello-browser.js"`, save, reload, and verify a 200 script response before checking the Console message.

Debugging clinic

Symptom 1 - `ReferenceError: remaining is not defined`

Locate the first application frame and inspect the spelling and lexical scope. The name may be declared inside another block or function. Repair the data flow with a parameter or return value rather than creating an unexplained global.

Symptom 2 - `TypeError` mentions reading a property of undefined

Trace the expression left of the property access. An array lookup may not have found an item, or a function may not have returned a value. Verify the producer before adding optional chaining; `?.` can hide a violated required-data contract.

Symptom 3 - The calculation concatenates digits

Inspect each input with `typeof`. Convert at the input boundary, validate the converted number, and keep domain functions numeric. Do not scatter conversions through later calculations.

Symptom 4 - A test changed another test's data

Look for shared mutable objects or arrays. Construct fresh fixtures per test or copy the changed level. Test isolation is a design property, not a matter of test execution order.

Retrieval quiz

1 What is the difference between ECMAScript and a browser API?

2 What does an expression produce?

3 When should a binding use `let` rather than `const`?

4 Why can a property of a `const` object change?

5 Why is `typeof null` not a sufficient null test?

6 What is the difference between `===` and `==`?

7 When is `??` safer than `||`?

8 What must change during a loop to support termination?

9 What happens when a function reaches its end without `return`?

10 Where is a function parameter binding available?

11 Why does `map()` not guarantee deep immutability?

12 What is the difference between dot and bracket property access?

13 Why can two bindings observe the same object mutation?

14 When should a function throw rather than invent a result?

15 What do normal, boundary, and invalid-input tests contribute?

16 What evidence proves that Node.js is available in the current terminal?

17 Why must `package.json` be visible before an npm project command?

18 What is the difference between saving a file and running it?

19 Why can an HTML page appear even when its JavaScript request fails?

20 Which browser panel proves whether `hello-browser.js` returned status 200 or 404?

Quiz answers and misconception notes

1 ECMAScript defines the language; a browser API is supplied by the browser host.

2 An expression evaluates to a value; a statement controls or performs execution.

3 Use `let` when the algorithm deliberately reassigns that binding.

4 `const` prevents rebinding, not mutation of the referenced object.

5 It returns the historical string `"object"`; use `value === null`.

6 Strict equality compares without coercive type conversion; loose equality may convert.

7 Use `??` when falsy values such as `0`, `false`, or `""` remain valid.

8 State relevant to the condition must progress toward termination.

9 The call returns `undefined`.

10 A parameter is local to that function call's scope.

11 It creates a new array but may retain references to the same nested objects.

12 Dot notation uses a source-level property name; brackets evaluate a key expression.

13 Both contain references to one object identity.

14 Throw when the input violates a contract and no honest domain result exists.

15 They check ordinary behavior, decision edges, and rejection of invalid contracts.

16 `node --version` must print the expected version in that terminal; an installer file alone is not runtime evidence.

17 npm reads that manifest for named scripts and project metadata; absence usually means the terminal is in the wrong directory.

18 Saving writes editor contents to disk; running asks a runtime to read and execute the saved source.

19 The HTML and JavaScript are separate requests; the first can succeed while the second returns 404 or contains an error.

20 The Network panel records the request URL and Hypertext Transfer Protocol (HTTP) status; the Console reports execution messages and errors.

Session summary

- JavaScript is the course's primary programming language; hosts supply additional APIs.
- Values have types, while bindings give values names within lexical scopes.
- Prefer `const`; use `let` for deliberate reassignment.
- Strict equality and explicit conversion keep data boundaries visible.
- Branches choose paths, loops repeat with progress, and functions define contracts.
- Arrays represent ordered collections; objects represent named properties.
- Object assignment shares identity, while spread creates only a shallow copy.
- Errors and failing assertions are structured evidence.
- Small automated tests make expectations repeatable but prove only tested behavior.

Glossary

Table 4.7 defines the chapter's working vocabulary. These definitions are deliberately concise; the numbered sections provide the mechanisms and examples.

Table 4.7 - Session 4 glossary

Term	Working definition
Argument	A value supplied at a function call site.
Array	An indexed, ordered object commonly used as a collection.
Binding	An association between an identifier and a value.
Boolean	The primitive logical type containing `true` and `false`.
Branch	One possible path selected by control flow.
Callback	A function supplied to another operation to be called later or for each value.
Coercion	Conversion performed by language rules, sometimes implicitly.
ECMAScript	The standardized language specification implemented as JavaScript.
Expression	Source code that evaluates to a value.
Function	A callable object that packages behavior behind parameters and a result.
Mutation	A change to an existing object's state.
Identifier	A legal source-code name used for a binding, property, or other named construct.
Literal	A value written directly in source code, such as `18` or `"open"`.
Object	A value with identity and keyed properties.
Parameter	A local binding declared by a function for an incoming argument.
Primitive	An immutable non-object value such as a string, number, or boolean.
Scope	The region of source code in which a binding is available.
Statement	Source code that performs or controls an execution step.
Test assertion	A comparison between observed and expected behavior.
Runtime	Software and host facilities that execute program instructions.
Source code	Human-authored program text stored in files.
Terminal	A text interface for running shell commands and reading their output.

Primary references and further reading

1 Ecma International, ECMAScript 2026 Language Specification. Use it as the normative authority for language semantics.

2 MDN, JavaScript Guide. Use its approachable chapters for a second explanation and reference links.

3 MDN, What is JavaScript?. Use it to revisit the relationship among HTML, CSS, JavaScript, and browser APIs.

4 Node.js, Official download page. Use it to obtain the latest supported patch release rather than copying a stale installer link.

5 Visual Studio Code, Getting started. Use it for current installation, workspace, and editor-interface guidance.

6 Node.js, Test runner documentation. Use it for the tested command and test lifecycle.

7 Node.js, Strict assertion documentation. Use it to interpret equality failures and diffs.

8 Marijn Haverbeke, Eloquent JavaScript, 4th edition. Use Chapters 1-6 for additional language practice.

9 Axel Rauschmayer, Exploring JavaScript. Use it as an optional precise language companion after the first reading.

What changes often?

Core value, control-flow, function, array, and object semantics are stable. New ECMAScript editions add capabilities, Node.js download versions, diagnostics, and test-runner features evolve, and browser or editor interfaces change. Always use the latest available Node.js 24 LTS patch compatible with the course engine constraint. Technical review date: **21 October 2026**.

Instructor teaching guide

Suggested eight-hour sequence

1 Tool roles, installation verification, folders, and terminals, 45 minutes.

2 First Node.js and browser programs with evidence, 45 minutes.

3 Runtime boundary, values, bindings, and expressions, 55 minutes.

4 Operators, equality, conversion, branches, and trace tables, 60 minutes.

5 Loops, functions, returns, and scope, 60 minutes.

6 Arrays, objects, identity, and transformations, 60 minutes.

7 Errors, tests, and debugging demonstration, 35 minutes.

8 Guided laboratory and evidence review, 120 minutes.

Demonstrations that expose misconceptions

- Run `typeof document` in browser and Node.js to separate language from host.
- Reassign a `const`, then mutate a property of a `const` object.
- Compare `0`, `"0"`, `false`, an empty string, and an empty array under strict comparison and conditions.
- Step through `eventStatus()` at 0, 10, and 11 seats.
- Copy an array with spread, mutate a nested object, and observe shallow sharing.
- Break one assertion and read actual, expected, operator, and stack evidence.

Assessment and inclusion

Accept a trace table, spoken walkthrough, or debugger-step recording as formative evidence when these demonstrate the same reasoning. Provide code as text rather than images, allow additional time for syntax entry, and grade contract reasoning separately from typing speed. A complete submission must still pass the same behavioral tests and explain one failure repair.

Appendix A - Tested laboratory reference solutions

These files are generated into the PDF from the same tested sources used by the repository checks. Compare it with your committed attempt only after completing the laboratory.

Week 01 - index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Campus Hub - Events and Communities</title>
  </head>
  <body>
    <header>
      <h1>Campus Hub</h1>
      <nav aria-label="Primary navigation">
        <ul>
          <li><a href="#events">Events</a></li>
          <li><a href="#communities">Communities</a></li>
        </ul>
      </nav>
    </header>

    <main>
      <section id="events">
        <h2>Upcoming events</h2>
        <article>
          <h3>Robotics laboratory open evening</h3>
          <p><time datetime="2026-09-18T17:00">18 September at 17:00</time></p>
          <p>Meet the robotics team and see current student projects.</p>
        </article>
      </section>

      <section id="communities">
        <h2>Student communities</h2>
        <p>Explore groups for engineering, design, research, and volunteering.</p>
      </section>
    </main>

    <footer>
      <p>Campus Hub is a fictional student learning project.</p>
    </footer>
  </body>
</html>
```

Week 02 - index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="description" content="Submit a student event to Campus Hub.">
    <title>Campus Hub - Submit an event</title>
  </head>
  <body>
    <a href="#main-content">Skip to event form</a>
    <header>
      <p><a href="/">Campus Hub</a></p>
      <nav aria-label="Primary navigation">
        <ul>
          <li><a href="/events" aria-current="page">Events</a></li>
          <li><a href="/communities">Communities</a></li>
        </ul>
      </nav>
    </header>

    <main id="main-content">
      <h1>Submit a campus event</h1>
      <p>Required fields are marked in their labels.</p>

      <form action="/events" method="post">
        <p>
          <label for="event-title">Event title (required)</label>
          <input id="event-title" name="title" type="text" maxlength="120" required>
        </p>

        <p>
          <label for="organizer-email">Organizer email (required)</label>
          <input id="organizer-email" name="organizerEmail" type="email"
autocomplete="email" required>
        </p>

        <p>
          <label for="event-date">Event date (required)</label>
          <input id="event-date" name="date" type="date" required>
        </p>

        <p>
          <label for="start-time">Start time (required)</label>
          <input id="start-time" name="startTime" type="time" required>
        </p>

        <p>
          <label for="capacity">Capacity (required)</label>
          <input id="capacity" name="capacity" type="number" min="1" max="300" required>
        </p>

        <fieldset>
          <legend>Event format (required)</legend>
          <label><input type="radio" name="format" value="campus" required> On campus</
label>
          <label><input type="radio" name="format" value="online"> Online</label>
          <label><input type="radio" name="format" value="hybrid"> Hybrid</label>
        </fieldset>

        <p>
          <label for="description">Description</label>
          <textarea id="description" name="description" rows="6" maxlength="1000" aria-
describedby="description-help"></textarea>
          <span id="description-help">Describe the audience, location, and accessibility
arrangements. Do not include private student data.</span>
        </p>

        <button type="submit">Submit event</button>
      </form>
    </main>
```

```
    <footer>
      <p>Campus Hub is a fictional student learning project.</p>
    </footer>
  </body>
</html>
```

Week 03 - index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Campus Hub - Responsive events</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <a class="skip-link" href="#main-content">Skip to events</a>
    <div class="page-shell">
      <header class="site-header">
        <a class="brand" href="/">Campus Hub</a>
        <nav class="site-navigation" aria-label="Primary navigation">
          <ul><li><a href="#events">Events</a></li><li><a href="#communities">Communities</
a></li></ul>
        </nav>
      </header>
      <main id="main-content">
        <h1>Explore campus events</h1>
        <form class="event-filter" action="/" method="get">
          <div class="event-filter__field">
            <label for="event-type">Event type</label>
            <select id="event-type" name="type">
              <option value="all">All events</option>
              <option value="workshop">Workshops</option>
              <option value="seminar">Seminars</option>
              <option value="community">Community events</option>
            </select>
          </div>
          <button type="submit">Apply filter</button>
        </form>
        <section id="events" aria-labelledby="events-heading">
          <h2 id="events-heading">Upcoming events</h2>
          <div class="event-grid">
            <article class="event-card">
              <div class="event-card__content">
                <h3 class="event-card__title">Robotics laboratory open evening</h3>
                <p>Meet the robotics team and test current student projects.</p>
                <a class="event-card__action" href="/events/robotics">Event details</a>
              </div>
            </article>
            <article class="event-card">
              <div class="event-card__content">
                <h3 class="event-card__title">Accessible interfaces workshop with an
intentionally long title</h3>
                <p>Practice inclusive design and keyboard testing.</p>
                <a class="event-card__action" href="/events/accessibility">Event details</a>
              </div>
            </article>
            <article class="event-card">
              <div class="event-card__content">
                <h3 class="event-card__title">Sustainable systems seminar</h3>
                <p>Discuss measurement, energy, and responsible engineering.</p>
                <a class="event-card__action" href="/events/systems">Event details</a>
              </div>
            </article>
          </div>
        </section>
        <section id="communities"><h2>Student communities</h2><p>Find groups for
engineering, research, design, and volunteering.</p></section>
      </main>
      <footer><p>Campus Hub is a fictional student learning project.</p></footer>
    </div>
  </body>
</html>
```

Week 03 - styles.css

```
@layer reset, base, layout, components;

@layer reset {
  *, *::before, *::after { box-sizing: border-box; }
  body, h1, h2, h3, p, ul { margin-block-start: 0; }
  img, svg, video { max-inline-size: 100%; block-size: auto; }
  button, input, select, textarea { font: inherit; }
}

@layer base {
  :root {
    --color-ink: #172033;
    --color-accent: #146c94;
    --color-surface: #f7fafc;
    --color-border: #cad4df;
    --color-focus: #f0a202;
    --color-button-text: #ffffff;
    --space-2: 0.5rem;
    --space-4: 1rem;
    --space-6: 1.5rem;
    --radius: 0.65rem;
    --measure: 68ch;
  }
  body { margin: 0; color: var(--color-ink); background: var(--color-surface); font: 1rem/1.
6 system-ui, sans-serif; }
  h1 { font-size: clamp(2rem, 6vw, 4rem); line-height: 1.1; max-inline-size: 16ch; }
  h2, h3 { line-height: 1.25; }
  p { max-inline-size: var(--measure); }
  a { color: var(--color-accent); text-underline-offset: 0.18em; }
  :focus-visible { outline: 3px solid var(--color-focus); outline-offset: 3px; }
}

@layer layout {
  .page-shell { width: min(100% - 2rem, 72rem); margin-inline: auto; }
  .site-header { display: flex; flex-wrap: wrap; align-items: center; justify-content:
space-between; gap: var(--space-4); padding-block: var(--space-6); }
  .site-navigation ul { display: flex; flex-wrap: wrap; gap: var(--space-2) var(--space-4);
padding: 0; list-style: none; }
  .event-filter { display: flex; flex-wrap: wrap; align-items: end; gap: var(--space-4);
margin-block: var(--space-6); padding: var(--space-4); border: 1px solid var(--color-
border); border-radius: var(--radius); background: white; }
  .event-filter__field { display: grid; gap: var(--space-2); flex: 1 1 15rem; }
  .event-grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(min(100%,
17rem), 1fr)); gap: var(--space-4); }
  main, footer { padding-block: var(--space-6); }
}

@layer components {
  .skip-link { position: absolute; inset-inline-start: 1rem; inset-block-start: -5rem;
padding: 0.75rem 1rem; background: white; z-index: 1; }
  .skip-link:focus { inset-block-start: 1rem; }
  .brand { color: var(--color-ink); font-weight: 750; font-size: 1.25rem; }
  button, select { min-block-size: 2.75rem; padding-block: 0.6rem; padding-inline: 0.9rem;
border: 1px solid var(--color-accent); border-radius: calc(var(--radius) / 1.5); }
  button { color: var(--color-button-text); background: var(--color-accent); font-weight:
700; cursor: pointer; }
  select { color: var(--color-ink); background: white; }
  .event-card { min-inline-size: 0; padding: var(--space-6); border: 1px solid var(--color-
border); border-radius: var(--radius); background: white; box-shadow: 0 0.25rem 1rem rgb(18
53 91 / 8%); container-name: event-card; container-type: inline-size; }
  .event-card__content { display: grid; gap: var(--space-2); }
  .event-card__content > * { margin-block-end: 0; }
  .event-card h3 { overflow-wrap: anywhere; }
}

@container event-card (min-width: 20rem) {
  .event-card__content { grid-template-columns: 1fr auto; align-items: end; }
  .event-card__title, .event-card__content p { grid-column: 1 / -1; }
  .event-card__action { justify-self: end; }
}
```

```
@media (min-width: 48rem) {
  .site-header { padding-block: 2rem; }
}

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { scroll-behavior: auto; animation-duration: 0.01ms; animation-
iteration-count: 1; transition-duration: 0.01ms; }
}

@media (forced-colors: active) {
  .event-card, .event-filter { border-color: CanvasText; box-shadow: none; }
  button { border: 2px solid ButtonText; }
}
```

Week 04 - events.js

```
export const events = [
  { title: "Robotics night", category: "technology", capacity: 80, registered: 62, featured:
 true },
  { title: "Campus orchestra", category: "arts", capacity: 120, registered: 120, featured:
false },
  { title: "Sustainable gardens", category: "community", capacity: 40, registered: 18,
featured: true },
];

export function seatsRemaining(event) {
  if (!event || !Number.isFinite(event.capacity) || !Number.isFinite(event.registered)) {
    throw new TypeError("An event must contain numeric capacity and registered values.");
  }
  return Math.max(0, event.capacity - event.registered);
}

export function eventStatus(event) {
  const remaining = seatsRemaining(event);
  if (remaining === 0) return "full";
  if (remaining <= 10) return "almost full";
  return "open";
}

export function filterEvents(allEvents, category = "all") {
  if (!Array.isArray(allEvents)) throw new TypeError("allEvents must be an array.");
  return category === "all"
    ? [...allEvents]
    : allEvents.filter((event) => event.category === category);
}

export function summarizeEvents(allEvents) {
  return allEvents.map((event) => ({
    title: event.title,
    remaining: seatsRemaining(event),
    status: eventStatus(event),
  }));
}

const summary = summarizeEvents(events);
console.table(summary);

const output = typeof document === "undefined" ? null : document.querySelector("#program-
output");
if (output) {
  output.textContent = summary
    .map(({ title, remaining, status }) => `${title}: ${status} (${remaining} seats
remaining)`)
    .join("\n");
}
```

Week 04 - events.test.js

```
import test from "node:test";
import assert from "node:assert/strict";
import { eventStatus, filterEvents, seatsRemaining, summarizeEvents } from "./events.js";

globalThis.document = { querySelector: () => null };

const sample = { title: "Robotics night", category: "technology", capacity: 80, registered:
62 };

test("seatsRemaining subtracts registrations without returning a negative value", () => {
  assert.equal(seatsRemaining(sample), 18);
  assert.equal(seatsRemaining({ ...sample, registered: 90 }), 0);
});

test("eventStatus applies the documented boundaries", () => {
  assert.equal(eventStatus(sample), "open");
  assert.equal(eventStatus({ ...sample, registered: 72 }), "almost full");
  assert.equal(eventStatus({ ...sample, registered: 80 }), "full");
});

test("filterEvents returns a new array and respects the category", () => {
  const source = [sample, { ...sample, title: "Concert", category: "arts" }];
  assert.deepEqual(filterEvents(source, "technology"), [sample]);
  assert.notEqual(filterEvents(source), source);
});

test("summarizeEvents returns the public summary shape", () => {
  assert.deepEqual(summarizeEvents([sample]), [
    { title: "Robotics night", remaining: 18, status: "open" },
  ]);
});

test("invalid event data produces a useful TypeError", () => {
  assert.throws(() => seatsRemaining({ capacity: "80", registered: 10 }), TypeError);
});
```

Week 04 - hello-browser.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Campus Hub - First browser JavaScript</title>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <main>
      <p class="eyebrow">Campus Hub JavaScript orientation</p>
      <h1>The first browser program is running</h1>
      <p>Open Developer Tools, select <strong>Console</strong>, and compare its two
messages with <code>hello-browser.js</code>.</p>
    </main>
    <script src="hello-browser.js"></script>
  </body>
</html>
```

Week 04 - hello-browser.js

```
const courseName = "Web Programming";
const sum = 2 + 3;

console.log(`Hello from ${courseName}!`);
console.log(`Two plus three is ${sum}.`);
```

Week 04 - hello.js

```
const courseName = "Web Programming";
const sum = 2 + 3;

console.log(`Hello from ${courseName}!`);
console.log(`Two plus three is ${sum}.`);
```

Week 04 - index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Campus Hub - JavaScript foundations</title>
    <link rel="stylesheet" href="styles.css">
    <script type="module" src="events.js"></script>
  </head>
  <body>
    <main>
      <p class="eyebrow">Campus Hub programming laboratory</p>
      <h1>Event availability</h1>
      <p>The text below is produced from event data by the tested JavaScript module.</p>
      <pre id="program-output" aria-live="polite">Loading event summary...</pre>
    </main>
  </body>
</html>
```

Week 04 - styles.css

```
:root { color-scheme: light; font-family: system-ui, sans-serif; }
* { box-sizing: border-box; }
body { margin: 0; min-block-size: 100vh; display: grid; place-items: center; background:
#eaf5f5; color: #12355b; }
main { inline-size: min(44rem, calc(100% - 2rem)); padding: 2rem; border: 1px solid #a9c7d8;
 border-radius: 1rem; background: white; box-shadow: 0 1rem 2.5rem rgb(18 53 91 / 12%); }
.eyebrow { color: #0b7f82; font-weight: 700; letter-spacing: .04em; text-transform:
uppercase; }
h1 { margin-block: .25rem 1rem; }
pre { padding: 1rem; overflow: auto; border-inline-start: .35rem solid #0b7f82; background:
#f4f8fb; color: #172033; font: 1rem/1.7 ui-monospace, monospace; white-space: pre-wrap; }
```

Appendix B – Quick commands

In the student coursework repository, verify the supplied setup:

```
npm test
```

Serve Week 1 at http://localhost:8000/:

```
npm run serve:week01
```

In the course-book source repository, verify, build, and verify this PDF edition:

```
npm test
npm run pdf
npm run pdf:verify
```